

```
const shopName = "우리편의점";  
  
let price = 1200;  
  
console.log(price);  
  
const cart = ["아메리카노", "케이크"];  
  
cart.forEach((item) => console.log(item));  
  
document.querySelector("#list").textContent = "완료";  
  
const res = await fetch(url);
```

1 권 · 기 초

자바스크립트 기초

값을 화면에 찍는 것부터 시작해서, 브라우저에서 도는 할 일 목록을 직접 만드는 데까지 갑니다.

읽고, 따라 치고, 출력이 같은지 맞춰 보세요.

8

단원

41

개념

184

직접 해보기

170

문제

차례

01	변수와 자료형	13
01	<code>console.log</code> 로 출력하기	14
02	변수: 값에 이름 붙이기 (<code>let</code> , <code>const</code>)	18
03	자료형 (1) 숫자와 문자열	23
04	자료형 (2) 불리언, <code>undefined</code> , <code>null</code>	30
05	형변환: 자료형 바꾸기	36
02	연산자	53
01	산술 연산자: 계산하기	54
02	대입 연산자와 증감 연산자	62
03	비교 연산자	69
04	논리 연산자	76
05	템플릿 리터럴 (백틱 문자열)	83
03	조건문	99
01	if 문: 만약 ~라면	100
02	else 와 else if: 갈림길 만들기	107
03	조건 조합하기	115
04	switch 문	122
05	삼항 연산자	124
04	반복문	141
01	for 문	142
02	while 과 do...while	150
03	break 와 continue	157
04	중첩 반복문	163
05	함수	181
01	왜 함수인가	182

02	매개변수와 인자: 함수에 값 넘기기	188
03	<code>return</code> : 결과를 돌려주기	195
04	함수도 값이다 (함수 표현식)	204
05	화살표 함수	210
06	스코프: 변수가 사는 범위	217
06	배열	235
01	배열 만들기과 인덱스	236
02	배열에 추가하고 삭제하기	243
03	자르기, 합치기, 순서 바꾸기	251
04	배열에서 찾기	259
05	배열 반복하기	267
07	객체와 메소드	287
01	객체 만들기과 접근	288
02	객체 추가·수정·삭제	296
03	메소드와 <code>this</code>	303
04	객체 순회하기	311
05	객체와 배열 섞어 쓰기	319
08	콜백과 내장메소드	341
01	콜백이란	342
02	<code>forEach</code> : 하나씩 처리하기	349
03	<code>map</code> : 모양을 바꿔 새 배열 만들기	357
04	<code>filter</code> 와 <code>find</code> : 골라내기	365
05	<code>reduce</code> : 하나의 값으로 접기	374
06	<code>sort</code> : 정렬하기	382
부록 가	연습문제·종합연습 정답	401
부록 나	심화문제 정답	457

여는 글

시작하기 전에

다 배우고 나면 이런 것을 직접 만듭니다. 지금은 무슨 코드인지 하나도 몰라도 됩니다.

이 책의 표시

블록마다 왼쪽 가장자리 표시가 다릅니다. 표시만 보고 “지금 내가 무엇을 해야 하는지” 알 수 있습니다.

```
console.log("안녕하세요");
```

— 직접 쳐 볼 코드입니다. 파일을 열고 그대로 따라 치세요.

출력 **안녕하세요**

— 실행하면 컴퓨터가 내놓는 값입니다. 내 화면과 같은지 맞춰 보세요.

▢ 직접 해보기 **1**

자기 이름을 출력해 보세요.

— **여러분 차례입니다.** 이 표시가 보이면 손을 움직이세요. 정답은 그 개념 맨 끝에 있습니다.

여기에 코드를 쓰세요

— 연필로 직접 적는 자리입니다.

이런 실수를 합니다

따옴표를 빠뜨림

```
console.log(안녕하세요);
```

따옴표가 없으면 컴퓨터는 ‘안녕하세요’라는 이름의 변수를 찾습니다.

— 여기서 많이 넘어집니다. 미리 보고 피해 가세요.

RUN node 개념01_console_log로_출력하기.js

— 개념마다 맨 위에 있습니다. 이대로 실행하면 됩니다.

시작 전 준비 (처음 한 번만)

1. Node.js 설치

01~09단원은 **Node.js**라는 프로그램으로 실행합니다. 없으면 아무것도 안 돌아갑니다.

1. <https://nodejs.org> 접속
2. 왼쪽의 **LTS** 버튼을 눌러 내려받기 (오른쪽 Current 말고 왼쪽입니다)
3. 받은 파일을 실행 → 계속 "다음"만 누르면 됩니다
4. 설치가 끝나면 **컴퓨터를 켜다 켜거나, 열려 있던 터미널을 모두 닫고 다시 여세요**

설치가 됐는지 확인하는 방법 — 터미널에 이렇게 칩니다.

```
node -v
```

v22.14.0 처럼 **v로 시작하는 숫자**가 나오면 성공입니다.

'node'은(는) 내부 또는 외부 명령... 이 아닙니다 가 나오면 아직 설치가 안 됐거나 터미널을 다시 열지 않은 것입니다. 터미널을 닫았다가 새로 열어 보세요.

2. VS Code 설치

코드를 쓰는 편집기입니다. <https://code.visualstudio.com> 에서 받아 설치하세요.

설치 후 VS Code 를 열고 **파일 → 폴더 열기**로 이 **JS자료** 폴더를 통째로 여세요. 파일 하나만 여는 것보다 폴더째 여는 편이 훨씬 편합니다.

3. 알아두면 좋은 것

- **HTML과 CSS는 몰라도 됩니다.** 10~12단원 파일 위쪽에 있는 `<style>`, `<body>` 같은 코드는 화면 모양을 만드는 부분입니다. 이 자료에서는 가르치지 않습니다. `<script>` 안부터 읽으세요.
- **12단원은 인터넷 연결이 필요합니다.** 인터넷에서 데이터를 받아오는 내용이라 그렇습니다. 학교나 회사 네트워크가 막아 두어 안 되는 경우가 있는데, 그때 쓰는 방법을 아래에 적어 두었습니다.

4. 인터넷이 막혀 12단원이 안 될 때

12단원과 13단원 종합04는 `jsonplaceholder.typicode.com` 이라는 연습용 서버에서 데이터를 받아옵니다. 이 주소가 막혀 있으면 **인터넷 없이 돌리는 대체본**을 켤 수 있습니다.

해당 파일(12단원 개념03·04·05·연습문제·연습문제_정답, 13단원 종합04 두 개) 위쪽에 이런 줄이 있습니다.

```
<!-- <script src="오프라인_대체.js"></script> -->
```

여기서 앞의 `<!--` 와 뒤의 `-->` 만 지우면 켜집니다.

```
<script src="오프라인_대체.js"></script>
```

켜지면 콘솔(F12) 맨 위에 **[오프라인 대체본이 켜져 있습니다]** 가 찍힙니다. **진짜 서버가 주는 데이터를 그대로 담아 두었기 때문에, 자료에 적힌 결과가 똑같이 나옵니다.** 없는 번호를 요청했을 때 404가 나오는 것까지 똑같습니다.

인터넷이 되는 곳으로 옮기면 `<!--` 와 `-->` 를 다시 붙여 꺼 두세요.

실행 방법

01~09단원 (.js 파일) — node로 실행

VS Code에서 터미널을 엽니다. `Ctrl + `` (숫자 1 왼쪽 키) 입니다.

```
cd 01_변수와_자료형
node 개념01_console_log로_출력하기.js
```

`cd` 폴더이름 은 그 폴더로 들어가라는 뜻입니다. 다시 나오려면 `cd ..` 를 칩니다.

```
cd ..
cd 02_연산자
```

더 편한 방법: VS Code 왼쪽 파일 목록에서 폴더를 마우스 오른쪽 클릭 → **"통합 터미널에서 열기"** 를 하면 그 폴더에서 터미널이 바로 열립니다. `cd` 를 안 쳐도 됩니다.

파일 이름이 길어서 치기 힘들면, `node` 개념01 까지만 치고 `Tab` 키를 누르세요. 나머지가 자동으로 채워집니다.

10~12단원 (.html 파일) — 브라우저로 실행

파일을 더블클릭하면 브라우저가 열립니다. 화면에 결과가 보이고, 개발자도구(`F12`) → **Console** 탭에도 결과가 찍힙니다.

10~12단원은 **Console** 탭을 반드시 같이 보세요. 화면에 안 나오는 결과가 거기 찍힙니다.

더블클릭했는데 **VS Code** 가 열린다면 — 파일 우클릭 → 연결 프로그램 → 크롬(또는 엣지) → "항상 이 앱으로 열기" 체크. VS Code 를 설치하면 자주 이렇게 됩니다.

`F12` 를 눌러도 아무 일이 없다면 — 노트북은 `Fn + F12` 이거나, `Ctrl + Shift + J` 로도 열립니다. 화면 아무 곳이나 우클릭 → 검사 도 됩니다.

코드를 고친 뒤에는 (10~12단원에서 제일 많이 하는 실수)

.html 파일은 저장한다고 화면이 저절로 바뀌지 않습니다.

VS Code 에서 고친다 → `Ctrl + S` 로 저장 → 브라우저로 가서 `F5` (새로고침)

이 세 단계를 매번 해야 합니다. "코드는 고쳤는데 화면이 그대로예요" 의 1순위 원인이 **저장을 안 했거나 새로고침을 안 한 것**입니다. 10~12단원의  **직접 해보기** 는 전부 이 과정을 요구합니다.

파일 읽는 법

개념 파일 (개념01_..., 개념02_...)

설명이 주석으로 들어있는 코드입니다. 읽지만 말고 직접 쳐 보세요.

```
// — 섹션 1: 변수에 값 담기 —  
  
let drink = "아메리카노";  
console.log(drink);  
// 출력: 아메리카노
```


변수 이름은 영어로 씁니다. 한글로도 만들 수는 있지만, 나중에 배울 React·서버 코드는 전부 영어 이름을 쓰기 때문에 지금부터 습관을 들입니다. 대신 **주석과 화면에 보이는 글자는 한글**입니다.

파일은 항상 같은 순서로 되어 있습니다.

1. **맨 위 헤더** — 이 파일이 무엇을 다루는지, 어떻게 실행하는지
2. — **섹션 N: 제목** — 주제가 바뀌는 지점
3.  **직접 해보기** · 섹션마다 나오는 짧은 과제. **정답은 파일 맨 아래 직접 해보기 정답 블록**에 주석으로 있습니다. 먼저 스스로 해 보세요.
4. **섹션 N: 자주 하는 실수** · 학생들이 실제로 자주 하는 실수 모음. // [실수 1] 처럼 번호가 붙어 있고 **주석 처리되어 있습니다**. 주석을 풀고 실행해서 결과를 눈으로 본 다음, **반드시 다시 주석 처리하세요**.

단, **SyntaxError** 라고 적힌 것은 **눈으로만 보세요**. 다른 예러는 주석을 풀어도 그 줄에서만 나고 앞의 출력은 그대로 나옵니다. 그런데 **SyntaxError** 는 자바스크립트가 파일을 아예 못 읽는 상태라 **출력이 한 줄도 안 나옵니다**. 여러분이 망가뜨린 게 아니니 놀라지 마세요. 실수로 풀었다면 다시 // 를 붙이면 그대로 돌아옵니다. 각 파일의 실수 섹션 맨 위에도 같은 안내를 적어 두었습니다.
5. — **정리** — 그 파일의 핵심 5~6줄. 복습할 때 여기만 봐도 됩니다.

결과를 적어 둔 주석 3가지

주석	뜻	나오는 곳
// 출력:	터미널에 찍히는 결과	01~09단원, 13단원 .js
// 콘솔:	개발자도구 Console 탭에 찍히는 결과	10~12단원
// 화면:	브라우저 화면이 어떻게 바뀌는지	10~12단원

전부 **"실행하면 이렇게 나온다"** 는 뜻입니다. 직접 실행해서 같은지 확인하세요. 다르게 나오면 어딘가 잘못된 것입니다.

연습 파일 (연습문제.js / 연습문제.html)

TODO가 적힌 빈칸을 채우는 파일입니다. 각 문제는 이렇게 생겼습니다.

```
// —— 문제 1 ——
// 삼각김밥 가격 1200을 변수 riceBallPrice 에 담고 출력하세요.
//
// 기대 출력:
// 1200

// TODO: 여기에 코드를 쓰세요
```

- **기대 출력**과 똑같이 나오면 정답입니다.
- 막히면 **연습문제_정답.js**를 보세요. 다만 **먼저 10분은 혼자 고민하고** 보세요.
- 문제는 **기본 → 응용 → [도전]** 순서로 어려워집니다. **[도전]**은 못 풀어도 괜찮습니다.
- 맨 마지막 문제는 **에러를 직접 보는 문제**입니다. 주석을 풀어 에러를 확인하고 다시 주석 처리하세요.

10~12단원 연습문제는 화면을 바꾸는 문제가 많아서, 기대 출력 대신 문제 문장 안에 "이렇게 되어야 합니다" 가 적혀 있는 경우가 있습니다.

13단원 종합 실습도 같은 방식입니다. 문제 파일의 TODO를 채우고, 막히면 **_정답** 파일을 보세요.

심화 파일 (심화문제.js / 심화문제.html) — 선택

01~12단원에 **연습문제와 별개로** 심화문제가 3개씩 들어 있습니다. 총 36문제입니다.

- **안 풀어도 됩니다.** 다음 단원으로 넘어가는 데 아무 지장이 없습니다.
- 연습문제를 **다 풀 뒤에** 하세요. 실습 시간이 남을 때 쓰라고 만든 것입니다.
- 연습문제보다 확실히 어렵습니다. **답이 바로 안 보이는 게 정상입니다.**
- 정답 파일은 답보다 **"왜 그런지" 설명이 훨씬 깁니다.** 그게 심화의 목적입니다.

막혀도 좌절하지 마세요. 개념 파일을 다시 뒤져 보게 만드는 것이 이 문제들의 의도입니다.

자주 나오는 에러 4가지

에러 메시지	뜻	해결
'node'은(는) 내부 또는 외부 명령...	Node.js 가 없다	맨 위 "시작 전 준비" 를 다시 하세요. 설치 후 터미널을 새로 열어야 합니다

ReferenceError: x is not defined	없는 이름을 불렀다	변수 이름 오타 확인, 만들기 전에 쓰지 않았는지 확인
SyntaxError: Unexpected token	문법이 깨졌다	괄호 (), 중괄호 { }, 따옴표 쌍이 맞는지 확인
TypeError: x is not a function	함수가 아닌 걸 함수처럼 불렀다	이름 오타 확인 (lenght → length)

브라우저(10~12단원)에서는 에러가 화면이 아니라 **개발자도구 Console 탭**에 빨간 글씨로 나옵니다. **F12**를 눌러 확인하세요.

에러가 나면 **빨간 글씨의 맨 윗줄과 몇 번째 줄인지**를 먼저 보세요. 에러는 실패가 아니라 안내문입니다.

혼자 공부할 때

- 개념 파일을 위에서 아래로 읽으면서 **직접 타이핑**합니다. 복사·붙여넣기 하지 마세요.
- // **출력**: 이 나오면 **실행하기 전에 결과를 먼저 예상**해 보고, 실행해서 맞는지 확인합니다.
-  **직접 해보기** 는 건너뛰지 마세요. 읽는 것과 쓰는 것은 완전히 다릅니다.
- 자주 하는 실수 는 주석을 풀어 에러를 **눈으로 본 뒤** 다시 닫습니다. **SyntaxError** 짜리만 예외입니다. 그건 풀지 말고 읽기만 하세요 (위 설명 참고).
- 단원이 끝나면 **연습문제** 를 풀고, 그다음 **연습문제_정답** 과 비교합니다.

단 원 01

변수와 자료형

OPEN 01_변수와_자료형

```
console.log("안녕하세요");  
console.log('오늘도 좋은 하루');  
console.log("첫 번째 줄");  
console.log("두 번째 줄");
```

01 console.log 로 출력하기

02 변수: 값에 이름 붙이기 (let, const)

03 자료형 (1) 숫자와 문자열

04 자료형 (2) 불리언, undefined, null

05 형변환: 자료형 바꾸기

- 연습문제

- 심화문제

console.log 로 출력하기

RUN node 개념01_console_log로_출력하기.js

프로그래밍의 첫걸음은 "컴퓨터가 한 일을 눈으로 확인하는 것"입니다. `console.log()` 는 괄호 안의 것을 화면에 보여 달라는 명령입니다.

```
console.log( "안녕" );
```

└──┬──┘ └──┬──┘ └──┬──┘
 명령 이름 보여줄 것 문장이 끝났다는 표시(세미콜론)

이 파일을 실행하면 아래 결과들이 위에서부터 차례로 찍힙니다.

글자 출력하기

섹션 1

글자(문자)는 반드시 따옴표로 감쌉니다.

```
console.log("안녕하세요");
```

출력 안녕하세요

큰따옴표 " " 든 작은따옴표 ' ' 든 상관없습니다. 짝만 맞으면 됩니다.

```
console.log('오늘도 좋은 하루');
```

출력 오늘도 좋은 하루

`console.log` 를 여러 번 쓰면 한 줄씩 차례로 찍힙니다.

```
console.log("첫 번째 줄");
console.log("두 번째 줄");
```

출력 첫 번째 줄
 두 번째 줄

▸ 직접 해보기 1

자기 이름을 출력해 보세요. (정답은 파일 맨 아래)

숫자 출력하기

섹션 2

숫자는 따옴표 없이 그냥 씁니다.

```
console.log(1200);
```

출력 1200

따옴표를 씌우면 "숫자 모양의 글자"가 됩니다. 화면에는 똑같아 보이지만 컴퓨터는 둘을 다르게 봅니다. (자세한 차이는 개념03에서 배웁니다)

```
console.log("1200");
```

출력 1200

계산식을 넣으면 계산 결과가 찍힙니다.

```
console.log(1200 + 800);
```

출력 2000

▸ 직접 해보기 2

삼각김밥 1200원 두 개의 값을 계산해서 출력해 보세요.

한 번에 여러 개 출력하기

섹션 3

쉼표(,)로 나열하면 한 줄에 띄어쓰기로 이어서 찍힙니다.

```
console.log("삼각김밥", 1200, "원");
```

출력 삼각김밥 1200 원

글자와 숫자를 섞어도 됩니다.

```
console.log("총", 3, "개 구매");
```

출력 총 3 개 구매

▸ 직접 해보기 3

"라떼" 4500 "원" 을 한 줄에 출력해 보세요.

줄 앞에 // 를 붙이면 그 줄은 실행되지 않습니다. 이것을 '주석'이라고 합니다.

```
console.log("이 줄은 실행되지 않습니다");
```

여러 줄을 한꺼번에 주석 처리할 때는 /* 로 열고 */ 로 닫습니다.

```
/*
console.log("이것도");
console.log("실행되지 않습니다");
*/
```

코드 뒤에 붙일 수도 있습니다.

```
console.log("실행됩니다"); // ← 이 뒤부터는 설명글
```

```
출력      실행됩니다
```

주석은 남에게 남기는 메모이자, 미래의 나에게 남기는 메모입니다. 이 자료의 설명도 전부 주석으로 쓰여 있습니다.

자주 하는 실수

섹션 5

★ 아래에서 SyntaxError 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. 다른 에러 (ReferenceError, TypeError)는 주석을 풀어도 그 줄에서만 나고 그 앞의 출력은 그대로 다 나옵니다. 그런데 SyntaxError 는 다릅니다. 자바스크립트가 파일을 아예 못 읽어서 출력이 한 줄도 안 나옵니다. 여러분이 망가뜨린 것이 아닙니다. 실수로 풀었다면 다시 // 를 붙이면 그대로 돌아옵니다.

아래 코드들은 전부 주석 처리해 두었습니다. 하나씩 주석을 풀고 실행해서 "어떤 에러가 나는지" 눈으로 본 다음, 다시 주석 처리하세요. 에러 메시지를 읽는 연습이 실력입니다.

이런 실수를 합니다

따옴표를 빠뜨림 → ReferenceError

```
console.log(안녕하세요);
```

따옴표가 없으면 컴퓨터는 '안녕하세요'라는 이름의 변수를 찾습니다. 그런 이름이 없으니 ReferenceError: 안녕하세요 is not defined

이런 실수를 합니다

따옴표 짝이 안 맞음 → SyntaxError

```
console.log("안녕하세요');
```

큰따옴표로 열었으면 큰따옴표로 닫아야 합니다.

이런 실수를 합니다

이름 오타 → **TypeError**

```
console.Log("안녕");
```

대문자 L 이 아니라 소문자 l 입니다. `console.log` 는 전부 소문자.

이런 실수를 합니다

괄호를 안 닫음 → **SyntaxError**

```
console.log("안녕");
```

여는 괄호 (하나에 닫는 괄호) 하나가 짝을 이뤄야 합니다.

정리

- 1 `console.log(내용);` → 내용을 화면에 출력
- 2 글자는 따옴표로 감싼다. 숫자는 따옴표 없이 쓴다.
- 3 쉼표로 나열하면 한 줄에 여러 개를 출력한다.
- 4 `//` 뒤는 주석. 컴퓨터가 무시한다.

직접 해보기 정답

- 1 `console.log("홍길동");`
- 2 `console.log(1200 + 1200);` // 출력: 2400
`console.log(1200 * 2);` // 이렇게 써도 같습니다
- 3 `console.log("라떼", 4500, "원");` // 출력: 라떼 4500 원

```
RUN node 개념02_변수_let과_const.js
```

```
console.log(1200);
```

```
let price = 1200;
```

└─┬──┐ ^ └─┬──┐
| | | └─ 담을 값
| | └─ 대입 기호 (= 는 "같다"가 아니라 "넣어라"입니다)
| └─ 이름
└─ 만드는 방법 (let 또는 const)

섹션 1

출력 1200

```
console.log("price");
```

출력 1300

18

직접 해보기

1

count 라는 변수에 3을 담고 출력한 뒤, 5로 바꿔서 다시 출력해 보세요.

01

const — 바뀌지 않는 값

섹션 2

```
const shopName = "우리편의점";
console.log(shopName);
```

출력 우리편의점

const 로 만든 변수는 값을 바꿀 수 없습니다. 아래 주석을 풀고 실행하면 TypeError 가 납니다.

이런 실수를 합니다

```
shopName = "다른편의점";
```

Assignment to constant variable. const 는 "이 값은 안 바뀝니다"라고 약속하는 것입니다.

그럼 언제 무엇을 쓸까요? 기본은 const 로 만듭니다. "이건 나중에 바뀌겠는데?" 싶을 때만 let 으로 바꿉니다. 이렇게 하면 실수로 값이 바뀌는 사고를 컴퓨터가 대신 막아 줍니다.

직접 해보기

2

const 로 taxRate 에 0.1 을 담고 출력해 보세요.

변수를 계산에 쓰기

섹션 3

```
const riceBallPrice = 1200;
const drinkPrice = 1800;

console.log(riceBallPrice + drinkPrice);
```

출력 3000

계산 결과도 변수에 담을 수 있습니다.

```
const total = riceBallPrice + drinkPrice;
console.log("합계", total, "원");
```

출력 합계 3000 원

여기서 riceBallPrice 값을 1200 한 곳만 고치면 total 도 자동으로 따라 바뀝니다. 이것이 변수를 쓰는 이유입니다.

▹ 직접 해보기 3

라떼 4500원, 케이크 6000원을 각각 변수에 담고 합계를 출력해 보세요.

이름 짓는 규칙

섹션 4

— 지켜야 하는 규칙 — 어기면 에러

- 1 숫자로 시작할 수 없습니다.
- 2 띄어쓰기를 넣을 수 없습니다.
- 3 `let`, `const` 같은 예약어는 이름으로 쓸 수 없습니다.

```
const 2ndPrice = 100;    // 실수: 숫자로 시작 → SyntaxError
const my price = 100;    // 실수: 띄어쓰기 → SyntaxError
const let = 100;         // 실수: 예약어 → SyntaxError
```

[지키면 좋은 약속 — 안 지켜도 돌아가지만 안 지키면 욱먹습니다] 1. 무엇이 담겼는지 알 수 있는 이름을 씁니다.

```
const userAge = 20; // 좋음
const a = 20; // 나쁨 - 20이 뭔지 알 수 없음
console.log(userAge, a);
```

출력 20 20

2. 두 단어 이상이면 뒷 단어의 첫 글자를 대문자로 씁니다. (카멜 표기법)

```
const userName = "김민준"; // userName (0)
```

```
const username = "김민준"; // 이것도 되지만 읽기 불편
const user_name = "김민준"; // 파이썬 스타일. JS에서는 잘 안 씁니다
```

```
console.log(userName);
```

출력 김민준

- 3 대소문자를 구별합니다. `userName` 과 `username` 은 다른 변수입니다.
- 4 영문자, 숫자, `_`, `$` 로만 짓습니다.

한글로도 만들 수는 있고 문법상 예러도 아닙니다. 그런데도 안 쓰는 이유는, 나중에 배울 React·서버 코드가 전부 영어 이름이라 지금부터 습관을 들이는 것입니다. (주석과 화면에 보이는 글자는 계속 한글로 씁니다)

자주 하는 실수

섹션 5

★ 아래에서 SyntaxError 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. 다른 에러 (ReferenceError, TypeError)는 주석을 풀어도 그 줄에서만 나고 그 앞의 출력은 그대로 다 나옵니다. 그런데 SyntaxError 는 다릅니다. 자바스크립트가 파일을 아예 못 읽어서 출력이 한 줄도 안 나옵니다. 여러분이 망가뜨린 것이 아닙니다. 실수로 풀었다면 다시 // 를 붙이면 그대로 돌아옵니다.

이런 실수를 합니다

만들기 전에 사용 → ReferenceError

```
console.log(notMadeYet);
const notMadeYet = 10;
```

변수는 만든 다음 줄부터 쓸 수 있습니다.

이런 실수를 합니다

같은 이름을 두 번 만들기 → SyntaxError

```
const price2 = 100;
const price2 = 200;
```

값을 바꾸고 싶은 거라면 let 으로 만들고 price2 = 200; 이라고만 씁니다.

이런 실수를 합니다

let 없이 처음부터 이름만 쓰기

```
score = 90;
```

지금은 우연히 돌아가지만, 어디에도 없던 변수가 몰래 만들어져 나중에 찾기 힘든 버그가 됩니다. 반드시 const 나 let 을 붙이세요.

이런 실수를 합니다

= 를 "같다"로 착각

수학에서 = 는 "같다"지만, 프로그래밍에서 = 는 "오른쪽 값을 왼쪽에 넣어라"입니다.

```
let x = 5;
x = x + 1; // "x에 x+1을 넣어라" → 6
console.log(x);
```

수학이었다면 $x = x + 1$ 은 말이 안 되는 식입니다.

정리

- 1 **let** 이름 = 값; → 나중에 바꿀 수 있는 변수
- 2 **const** 이름 = 값; → 바꿀 수 없는 변수. 기본은 **const** 를 쓴다.
- 3 = 는 “같다”가 아니라 “넣어라”
- 4 이름은 영어 카멜 표기법(userName), 무엇이 담겼는지 알 수 있게

직접 해보기 정답

```
1 let count = 3;
  console.log(count); // 출력: 3
  count = 5;
  console.log(count); // 출력: 5

2 const taxRate = 0.1;
  console.log(taxRate); // 출력: 0.1

3 const latte = 4500;
  const cake = 6000;
  console.log(latte + cake); // 출력: 10500
```

자료형 (1) 숫자와 문자열

```
RUN node 개념03_숫자와_문자열.js
```

변수에는 아무 값이나 담을 수 있습니다. 그런데 담긴 값이 '숫자'나 '글자'냐에 따라 컴퓨터가 다르게 행동합니다. 값의 종류를 자료형(type)이라고 합니다.

이번 파일에서는 가장 많이 쓰는 두 가지, 숫자와 문자열을 봅니다.

typeof — 자료형을 확인하는 법

섹션 1

typeof 를 앞에 붙이면 그 값의 종류를 글자로 알려 줍니다.

코드	출력
<code>console.log(typeof 1200);</code>	<code>number</code>
<code>console.log(typeof "1200");</code>	<code>string</code>

화면에 보이는 모습은 비슷해도 종류가 다릅니다.

<code>console.log(1200, "1200");</code>
출력 1200 1200

그래서 앞으로 "이게 숫자야 글자야?" 헛갈리면 typeof 로 확인하면 됩니다. 값과 종류를 같이 찍는 습관을 들이세요.

<code>console.log(1200, typeof 1200);</code>
출력 1200 number

직접 해보기

1

"안녕" 의 자료형을 확인해 출력해 보세요.

number — 숫자

섹션 2

정수도, 소수도, 음수도 전부 number 하나입니다.

```
const count = 3;
const price = 1250.5;
const temperature = -7;

console.log(count, typeof count);
```

출력 3 number

코드

```
console.log(price, typeof price);

console.log(temperature, typeof temperature);
```

▶ 출력

▶ 1250.5 number

▶ -7 number

숫자에는 따옴표를 씌우지 않습니다. 쉼표도 넣지 않습니다.

```
const wrong = 1,250;   ← 이렇게 쓰면 안 됩니다
```

```
const bigNumber = 1250000;
console.log(bigNumber);
```

출력 1250000

알아두기 · 소수 계산은 아주 작은 오차가 생깁니다.

```
console.log(0.1 + 0.2);
```

출력 0.30000000000000004

컴퓨터가 소수를 2진법으로 저장하기 때문입니다. 버그가 아닙니다. 돈 계산을 할 때 소수점을 피하고 '원' 단위 정수로 다루는 이유가 이것입니다.

▸ 직접 해보기 2

자기 나이를 변수에 담고, 값과 자료형을 같이 출력해 보세요.

string — 문자열

섹션 3

따옴표로 감싼 글자를 문자열이라고 합니다. 감싸는 방법은 세 가지입니다.

```
const a1 = "큰따옴표";
const a2 = '작은따옴표';
const a3 = `백틱`; // 키보드에서 숫자 1 왼쪽, ~ 와 같은 키 console.log(a1, a2, a3);
```

출력 큰따옴표 작은따옴표 백틱

셋 다 자료형은 똑같이 string 입니다.

```
console.log(typeof a1, typeof a2, typeof a3);
```

출력 string string string

따옴표 안에 따옴표를 넣고 싶으면 다른 종류로 감싸면 됩니다.

```
console.log("그는 '안녕'이라고 말했다");
```

출력 그는 '안녕'이라고 말했다

```
console.log('그는 "안녕"이라고 말했다');
```

출력 그는 "안녕"이라고 말했다

빈 문자열도 있습니다. 따옴표 사이에 아무것도 없는 상태입니다.

```
const empty = "";
console.log(empty, typeof empty);
```

출력 string

(앞에 아무것도 안 찍히고 공백만 보이는 게 정상입니다)

▹ 직접 해보기

3

좋아하는 음식 이름을 문자열로 담고 출력해 보세요.

문자열 이어붙이기

섹션 4

+ 기호로 문자열을 이어 붙일 수 있습니다.

```
const userName = "김민준";
console.log("안녕하세요, " + userName + "님");
```

출력 안녕하세요, 김민준님

띄어쓰기는 자동으로 안 들어갑니다. 따옴표 안에 직접 넣어야 합니다.

```
console.log("안녕하세요, " + userName);
```

출력 안녕하세요, 김민준

`console.log` 의 쉼표(,)와는 다릅니다. 쉼표 → 각각을 띄어쓰기로 구분해서 출력 (출력할 때만 쓸 수 있음)
더하기 → 하나의 문자열로 합침 (변수에 담을 수 있음)

```
const greeting = "안녕하세요, " + userName + "님";
console.log(greeting);
```

출력 안녕하세요, 김민준님

+ 가 길어지면 읽기 어렵습니다. 더 편한 방법(백틱)은 02단원에서 배웁니다.

▹ 직접 해보기 4

“우리편의점” 과 “에 오신 것을 환영합니다” 를 이어 붙여 한 문장으로 출력해 보세요.

문자열의 길이

섹션 5

여기서 ‘점(.)’ 이 처음 나옵니다. 앞으로 계속 쓰니 지금 한 번 짚고 갑시다.

값 . 이름

점은 “이 값이 가지고 있는 것을 꺼내 쓴다” 는 뜻입니다. 왼쪽에 있는 값에게 물어보는 것이라고 생각하면 쉽습니다. “안녕하세요”.length → “너 길이가 몇이야?” 라고 문자열에게 묻는 것

사실 첫 파일부터 쓰고 있었습니다. console.log 의 점도 같은 것입니다. console 이라는 값에게서 log 를 꺼내 쓴 것이었습니다.

.length 를 붙이면 글자 수를 알려 줍니다.

```
const message = "안녕하세요";
console.log(message.length);
```

출력 5

찍어쓰기도 한 글자로 셉니다.

```
console.log("안녕 하세요".length);
```

출력 6

숫자에는 .length 가 없습니다.

```
console.log((12345).length);
```

출력 undefined

(undefined 가 무엇인지는 다음 파일 개념04에서 배웁니다)

12345 를 괄호로 감싼 이유가 궁금하셨을 겁니다. 숫자에 점을 바로 찍으면(12345.length)

자바스크립트가 그 점을 소수점으로 착각해서 에러가 납니다. 그래서 괄호로 한 번 묶어 준 것입니다. 변수에 담아서 쓰면 괄호가 필요 없습니다. const n = 12345; n.length

직접 해보기

5

자기 이름의 글자 수를 출력해 보세요.

숫자 모양의 문자열은 숫자가 아니다

섹션 6

이 단원에서 가장 중요한 내용입니다.

```
const numberValue = 100;
const stringValue = "100";

console.log(numberValue, typeof numberValue);
```

출력

100 number

```
console.log(stringValue, typeof stringValue);
```

출력

100 string

겉보기엔 같지만 + 를 했을 때 결과가 완전히 달라집니다.

코드	출력
<code>console.log(numberValue + 50);</code>	150
<code>console.log(stringValue + 50);</code>	10050

문자열에 + 를 하면 '더하기'가 아니라 '이어붙이기'가 됩니다.

나중에 화면의 입력창에서 값을 받아오면 숫자를 입력해도 전부 문자열로 옵니다. 그래서 이 차이를 모르면 "왜 10 + 5 가 105 가 되지?" 하고 한참 헤맬니다. 해결 방법은 다음다음 파일 개념05(형변환)에서 배웁니다.

직접 해보기

6

"3" + 4 와 3 + 4 를 각각 출력해서 결과를 비교해 보세요.

자주 하는 실수

섹션 7

★ 아래에서 SyntaxError 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. 다른 에러 (ReferenceError, TypeError)는 주석을 풀어도 그 줄에서만 나고 그 앞의 출력은 그대로 다 나옵니다. 그런데 SyntaxError 는 다릅니다. 자바스크립트가 파일을 아예 못 읽어서 출력이 한 줄도 안 나옵니다. 여러분이 망가뜨린 것이 아닙니다. 실수로 풀었다면 다시 // 를 붙이면 그대로 돌아옵니다.

이런 실수를 합니다

length 오타 → **undefined** (에러도 안 나서 더 위험합니다)

```
console.log("안녕하세요".lenght);
```

출력 undefined

lenght 가 아니라 length 입니다. g 와 t 순서를 자주 틀립니다.

이런 실수를 합니다

숫자에 쉼표 넣기

```
const money = 1,250;
```

SyntaxError: Unexpected number

자바스크립트는 쉼표를 만나면 "숫자가 여기서 끝났다" 고 봅니다. 그래서 뒤에 남은 250 을 어떻게 해야 할지 몰라 문법이 깨집니다. 숫자에는 쉼표를 쓰지 마세요. 1250000 처럼 붙여서 씁니다.

이런 실수를 합니다

따옴표 종류를 섞어서 닫기 → SyntaxError

```
console.log("안녕');
```

연 따옴표와 닫는 따옴표는 같은 종류여야 합니다.

이런 실수를 합니다

한글 따옴표 사용 → SyntaxError

```
console.log("안녕");
```

위드나 한글에서 코드를 복사하면 " 가 " 로 바뀌어 있습니다. 반드시 코드 편집기에서 직접 치세요.

정리

- 1 **typeof** 값 → 자료형 확인. 헛갈리면 무조건 찍어 본다.
- 2 **number** : 정수·소수·음수 전부. 따옴표와 쉼표를 쓰지 않는다.
- 3 **string** : 따옴표로 감싼 글자. ' ' ` 셋 다 가능.
- 4 문자열 + 문자열 = 이어붙이기 / .length = 글자 수
- 5 "100" 은 숫자가 아니다. "100" + 50 은 10050.

직접 해보기 정답

- 1 `console.log(typeof "안녕");` // 출력: string
- 2 `const age = 20;`
`console.log(age, typeof age);` // 출력: 20 number
- 3 `const food = "김치찌개";`
`console.log(food);` // 출력: 김치찌개
- 4 `console.log("우리편의점" + "에 오신 것을 환영합니다");`
// 출력: 우리편의점에 오신 것을 환영합니다
- 5 `const myName = "김민준";`
`console.log(myName.length);` // 출력: 3
- 6 `console.log("3" + 4);` // 출력: 34 (이어붙이기)
`console.log(3 + 4);` // 출력: 7 (더하기)

자료형 (2) 불리언, undefined, null

RUN node 개념04_불리언과_undefined_null.js

숫자와 문자열 말고도 자주 만나는 값이 세 가지 더 있습니다.

boolean	참이나 거짓이나	→ true / false
undefined	아직 값이 없음	(컴퓨터가 알아서 넣어 준 '없음')
null	비어 있음	(사람이 일부러 넣은 '없음')

boolean — 참과 거짓

섹션 1

true(참)와 false(거짓), 딱 두 가지 값만 있는 자료형입니다. 따옴표를 씌우지 않습니다. 전부 소문자입니다.

```
const isOpen = true;
const isClosed = false;

console.log(isOpen, typeof isOpen);
```

출력 true boolean

```
console.log(isClosed, typeof isClosed);
```

출력 false boolean

따옴표를 씌우면 그냥 글자가 됩니다. 완전히 다른 값입니다.

```
console.log("true", typeof "true");
```

출력 true string

참/거짓을 담는 변수는 이름을 is~, has~ 로 시작하면 읽기 좋습니다.

```
const isMember = true;
const hasCoupon = false;
console.log(isMember, hasCoupon);
```

출력 true false

▸ 직접 해보기 1

isStudent 라는 변수에 true 를 담고, 값과 자료형을 같이 출력해 보세요.

비교하면 불리언이 나온다

섹션 2

값을 비교하면 그 결과가 `true` 나 `false` 로 나옵니다.

코드	출력
<code>console.log(10 > 5);</code>	<code>true</code>
<code>console.log(10 < 5);</code>	<code>false</code>
<code>console.log(10 === 10);</code>	<code>true</code>

`===` 는 “같다”인지 묻는 기호입니다. `=` 하나는 “넣어라”였죠.

`=` 넣어라
`===` 같으냐?

비교 기호는 02단원에서 자세히 배웁니다. 여기서는 “비교하면 불리언이 나온다”만 기억하면 됩니다.

결과를 변수에 담을 수도 있습니다.

```
const stock = 3;
const isSoldOut = stock === 0;
console.log(isSoldOut);
```

출력 `false`

이 `true/false` 가 있어야 “재고가 0이면 품절이라고 띄워라” 같은 판단을 할 수 있습니다. 그 판단문(조건문)은 03단원에서 배웁니다.

▸ 직접 해보기

2

내 나이가 19보다 큰지 비교한 결과를 출력해 보세요.

undefined — 아직 값이 없음

섹션 3

변수를 만들어 놓고 값을 안 넣으면 `undefined` 가 들어갑니다.

```
let orderMemo;
console.log(orderMemo, typeof orderMemo);
```

출력 `undefined undefined`

나중에 값을 넣으면 그때부터 그 값이 됩니다.

```
orderMemo = "얼음 적게";
console.log(orderMemo, typeof orderMemo);
```

출력 얼음 적게 string

`undefined` 는 “내가 안 넣었다”는 신호입니다. 직접 넣는 값이 아닙니다. 앞 파일에서 봤던 이 결과도 `undefined` 였습니다.

```
console.log("안녕".length); // length 오타
```

출력 undefined

없는 것을 꺼내려 하면 에러 대신 `undefined` 가 나옵니다. 그래서 `undefined` 가 찍히면 “이름을 잘못 썼나?” 부터 의심하세요.

`const` 는 만들 때 값을 반드시 넣어야 합니다.

이런 실수를 합니다

```
const something;
```

SyntaxError: Missing initializer in const declaration

바꿀 수 없는 변수를 비워 두면 영원히 비어 있으니 애초에 막아 둔 것입니다.

▸ 직접 해보기 3

`let` 으로 nickname 변수를 만들되 값은 넣지 말고 출력해 보세요.

null — 일부러 비워 둔 값

섹션 4

`null` 은 “여기에 값이 없다”고 사람이 직접 넣는 값입니다.

```
let coupon = null;
console.log(coupon);
```

출력 null

예를 들어 쿠폰을 아직 안 골랐다는 뜻으로 `null` 을 넣어 두었다가

```
coupon = "3000원 할인";
console.log(coupon);
```

출력 3000원 할인

다시 취소하면 `null` 로 되돌립니다.


```
coupon = null;
console.log(coupon);
```

출력 null

undefined 와 null 의 차이를 한 줄로 정리하면 이렇습니다. undefined : 컴퓨터가 넣은 '없음' → 아직 아무도 손대지 않았다 null : 사람이 넣은 '없음' → 일부러 비웠다 실무에서는 "값 없음"을 표현할 때 주로 null 을 씁니다.

▹ 직접 해보기

4

selectedSeat 변수에 null 을 담아 출력한 뒤, "A12" 로 바꿔 다시 출력해 보세요.

typeof null 은 "object" — 유명한 함정

섹션 5

코드	▶ 출력
console.log(typeof undefined);	▶ undefined
console.log(typeof null);	▶ object

null 인데 왜 object 일까요? 자바스크립트가 처음 만들어질 때 생긴 버그인데, 지금 고치면 세상의 수많은 코드가 깨져서 그냥 두고 있습니다. "typeof null 은 object 다"는 그냥 외우세요. 면접 단골 질문입니다.

null 인지 확인하려면 typeof 말고 값을 직접 비교합니다.

```
const value = null;
console.log(value === null);
```

출력 true

지금까지 나온 자료형 한눈에 보기

섹션 6

코드	▶ 출력
console.log(typeof 1200);	▶ number
console.log(typeof "안녕");	▶ string
console.log(typeof true);	▶ boolean
console.log(typeof undefined);	▶ undefined
console.log(typeof null);	▶ object

↑ 이 한 줄만 함정입니다. 나머지는 예상대로 나옵니다.

이 다섯 가지를 원시 자료형이라고 부릅니다. “값 하나”를 담는 종류입니다. (원시 자료형은 사실 두 가지가 더 있지만 아주 특수한 용도로 이 자료에서는 다루지 않습니다) 값을 여러 개 묶어 담는 배열과 객체는 06·07단원에서 배웁니다.

자주 하는 실수

섹션 7

★ 아래에서 `SyntaxError` 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. 다른 에러 (`ReferenceError`, `TypeError`)는 주석을 풀어도 그 줄에서만 나고 그 앞의 출력은 그대로 다 나옵니다. 그런데 `SyntaxError` 는 다릅니다. 자바스크립트가 파일을 아예 못 읽어서 출력이 한 줄도 안 나옵니다. 여러분이 망가뜨린 것이 아닙니다. 실수로 풀었다면 다시 `//` 를 붙이면 그대로 돌아옵니다.

이런 실수를 합니다

`true` 를 대문자로 쓰기 → `ReferenceError`

```
console.log(True);
```

자바스크립트는 전부 소문자 `true` / `false` 입니다. (파이썬은 `True`)

이런 실수를 합니다

불리언에 따옴표 씌우기

```
console.log(typeof "false");
```

출력 `string`

“false” 는 글자입니다. 조건문에서 참으로 취급되어 버그가 됩니다.

이런 실수를 합니다

`undefined` 를 직접 넣기

```
let x = undefined;
```

문법 오류는 아니지만, “비었다”를 표현할 땐 `null` 을 쓰는 게 약속입니다.

이런 실수를 합니다

`null` 과 `undefined` 를 같다고 생각하기

```
console.log(null === undefined);
```

출력 `false`

둘 다 ‘없음’이지만 서로 다른 값입니다.

정리

- 1 `boolean` : `true` / `false` 두 가지. 소문자, 따옴표 없음.
- 2 비교 (`>`, `===`)의 결과는 불리언이다.
- 3 `undefined` : 값을 안 넣었을 때 자동으로 들어가는 '없음'
- 4 `null` : 사람이 일부러 넣는 '없음'
- 5 `typeof null` 은 "`object`" — 언어의 오래된 버그. 외울 것.

직접 해보기 정답

```

1  const isStudent = true;
   console.log(isStudent, typeof isStudent); // 출력: true boolean

2  const myAge = 20;
   console.log(myAge > 19);                  // 출력: true

3  let nickname;
   console.log(nickname);                    // 출력: undefined

4  let selectedSeat = null;
   console.log(selectedSeat);                // 출력: null
   selectedSeat = "A12";
   console.log(selectedSeat);                // 출력: A12

```

형변환: 자료형 바꾸기

RUN node 개념05_형변환.js

개념03에서 이런 문제를 봤습니다.

```
"100" + 50 → "10050" (더하기가 아니라 이어붙이기)
```

화면 입력창에서 받아온 값은 숫자를 입력해도 전부 문자열로 옵니다. 그래서 계산하기 전에 '숫자로 바꾸는' 작업이 반드시 필요합니다. 자료형을 바꾸는 것을 형변환이라고 합니다.

문자열 → 숫자: Number()

섹션 1

```
const inputValue = "100"; // 입력창에서 받아온 값이라고 생각하세요
console.log(inputValue + 50);
```

출력 10050

원하는 결과가 아닙니다.

Number() 로 감싸면 숫자로 바뀝니다.

```
const numberValue = Number(inputValue);
console.log(numberValue, typeof numberValue);
```

출력 100 number

```
console.log(numberValue + 50);
```

출력 150

이제 제대로 더해집니다.

소수도 됩니다.

```
console.log(Number("3.14"));
```

출력 3.14

앞뒤 공백은 알아서 무시합니다.

```
console.log(Number(" 42 "));
```

출력 42

빈 문자열은 0 이 됩니다. (직관적이지 않으니 기억해 두세요)

```
console.log(Number(""));
```

출력 0

⇒ 직접 해보기 1

"250" 을 숫자로 바꿔서 100 을 더한 결과를 출력해 보세요.

숫자로 못 바꾸면 NaN

섹션 2

숫자가 아닌 글자가 섞여 있으면 NaN 이 나옵니다.

코드

```
console.log(Number("1200원"));
```

```
console.log(Number("안녕"));
```

▶ 출력

▶ NaN

▶ NaN

NaN 은 Not a Number, "숫자가 아님"의 줄임말입니다. 그런데 자료형은 number 입니다. "고장 난 숫자" 라고 생각하세요.

```
console.log(typeof NaN);
```

출력 number

NaN 이 섞이면 그다음 계산도 전부 NaN 이 됩니다.

```
console.log(Number("1200원") + 500);
```

출력 NaN

이런 실수를 합니다

NaN 은 자기 자신과도 같지 않습니다.

```
console.log(NaN === NaN);
```

출력 false

그래서 NaN 인지 확인할 때는 === 가 아니라 Number.isNaN() 을 씁니다.

```
console.log(Number.isNaN(Number("안녕")));
```

출력 true

결과에 NaN 이 찍히면 “숫자로 못 바꾸는 값이 들어왔구나” 하고 그 값을 거슬러 올라가면서 찾으면 됩니다.

▹ 직접 해보기

2

“3개”를 Number 로 바꿔서 출력하고, 결과가 왜 그런지 생각해 보세요.

parseInt / parseFloat — 앞에서부터 읽히는 만큼만

섹션 3

Number 는 조금이라도 이상하면 NaN 이지만, parseInt 는 앞에서부터 읽을 수 있는 숫자까지만 가져옵니다.

코드	출력
<code>console.log(parseInt("1200원"));</code>	▶ 1200
<code>console.log(parseInt("30px"));</code>	▶ 30

단, 앞에서 숫자가 아니면 이것도 NaN 입니다.

<code>console.log(parseInt("원1200"));</code>
출력 NaN

parseInt 는 정수만 가져옵니다. 소수점 아래는 버립니다.

<code>console.log(parseInt("3.9"));</code>
출력 3

소수까지 살리려면 parseFloat 를 씁니다.

<code>console.log(parseFloat("3.9"));</code>
출력 3.9

정리하면 이렇습니다.

Number	: 값 전체가 깔끔한 숫자일 때 (입력창 값 대부분)
parseInt	: "30px" 처럼 단위가 붙어 있을 때

parseFloat : 위와 같은데 소수점까지 필요할 때

▹ 직접 해보기

3

“24.5도”에서 24.5 만 뽑아서 출력해 보세요.

숫자 → 문자열 : String()

섹션 4

```
const score = 95;
const scoreText = String(score);

console.log(scoreText, typeof scoreText);
```

출력 95 string

.toString() 을 붙여도 같습니다.

```
console.log((95).toString(), typeof (95).toString());
```

출력 95 string

문자열이 되면 .length 를 쓸 수 있습니다. 자릿수 세기에 씁니다.

```
console.log(String(12345).length);
```

출력 5

숫자를 문자열로 바꿔야 하는 경우는 사실 드뭅니다. + 로 이어붙일 때 자동으로 바뀌기 때문입니다. (다음 섹션)

▹ 직접 해보기 4

1234567 이 몇 자리 숫자인지 출력해 보세요.

자동 형변환 — 컴퓨터가 알아서 바꿔 버린다

섹션 5

우리가 시키지 않아도 자바스크립트가 알아서 형을 바꿀 때가 있습니다. 편할 때도 있지만 버그의 단골 원인입니다.

규칙 1 · + 는 한쪽이라도 문자열이면 '이어붙이기'가 된다

코드

```
console.log(1 + "2");
```

```
console.log("가격: " + 1200);
```

▶ 출력

▶ 12

▶ 가격: 1200

↑ 이 덕분에 문자열과 숫자를 섞어 쓸 수 있습니다.

규칙 2 · - * / % 는 문자열을 숫자로 바꿔서 계산한다

코드	출력
<code>console.log("10" - 5);</code>	▶ 5
<code>console.log("10" * 2);</code>	▶ 20
<code>console.log("10" / 2);</code>	▶ 5

같은 값인데 + 만 결과가 다릅니다. 이게 가장 헷갈리는 지점입니다.

<code>console.log("10" + 5, "10" - 5);</code>
출력 105 5

규칙 3 · 불리언은 숫자로 바뀌면 `true=1`, `false=0`

코드	출력
<code>console.log(true + 1);</code>	▶ 2
<code>console.log(false + 1);</code>	▶ 1

자동 형변환에 기대지 말고, 계산 전에 `Number()` 로 직접 바꾸는 습관을 들이세요.

▫ 직접 해보기

5

"7" + 3 과 "7" - 3 을 각각 출력해서 비교해 보세요.

불리언으로 바꾸기 : `Boolean()`

섹션 6

어떤 값이든 참/거짓으로 바꿀 수 있습니다.

코드	출력
<code>console.log(Boolean(1));</code>	▶ true
<code>console.log(Boolean("안녕"));</code>	▶ true

거짓이 되는 값은 아래 여섯 가지입니다. 이것만 외우면 됩니다.

코드	▶ 출력
<code>console.log(Boolean(0));</code>	▶ false
<code>console.log(Boolean(""));</code>	▶ false
<code>console.log(Boolean(null));</code>	▶ false
<code>console.log(Boolean(undefined));</code>	▶ false
<code>console.log(Boolean(NaN));</code>	▶ false
<code>console.log(Boolean(false));</code>	▶ false

이 여섯 가지를 falsy(거짓 같은 값)라고 부르고, 나머지는 전부 truthy 입니다.

주의 · 글자가 하나라도 있으면 true 입니다.

코드	▶ 출력
<code>console.log(Boolean("false"));</code>	▶ true
<code>console.log(Boolean("0"));</code>	▶ true
<code>console.log(Boolean(" "));</code>	▶ true

빈 문자열 "" 만 false 이고, 공백 한 칸 " " 은 true 입니다.

이 falsy 목록은 03단원 조건문에서 “값이 비었으면” 을 판단할 때 그대로 씁니다.

▹ 직접 해보기

6

빈 문자열과 공백 한 칸을 각각 Boolean 으로 바꿔 비교해 보세요.

자주 하는 실수

섹션 7

이런 실수를 합니다

입력값을 안 바꾸고 더하기

```
const price1 = "3000"; // 입력창에서 온 값
const price2 = "2000";
console.log(price1 + price2);
```

출력 30002000

숫자로 바꿔야 합니다.

```
console.log(Number(price1) + Number(price2));
```

출력 5000

이런 실수를 합니다

Number 를 number 로 쓰기 → ReferenceError

```
console.log(number("100"));
```

첫 글자가 대문자인 Number 입니다. String, Boolean 도 마찬가지입니다.

이런 실수를 합니다

NaN 을 === 로 비교

```
if (result === NaN) ...
```

항상 false 라서 절대 걸리지 않습니다. Number.isNaN(result) 를 쓰세요.

이런 실수를 합니다

parseInt 가 소수를 반올림한다고 착각

```
console.log(parseInt("3.9"));
```

출력 3

반올림이 아니라 그냥 버립니다. 4가 아니라 3입니다.

정리

- 1 Number(값) : 문자열 → 숫자. 못 바꾸면 NaN
- 2 parseInt(값) : "30px" 처럼 단위가 붙었을 때 앞의 숫자만
- 3 String(값) : 숫자 → 문자열
- 4 + 는 한쪽이 문자열이면 이어붙이기, - * / 는 숫자로 바뀌어서 계산
- 5 falsy 6개 : 0, "", null, undefined, NaN, false — 나머지는 전부 true

직접 해보기 정답

- 1 console.log(Number("250") + 100); // 출력: 350
- 2 console.log(Number("3개")); // 출력: NaN

'개'라는 글자 때문에 전체를 숫자로 못 바꿉니다. 숫자만 뽑으려면 parseInt("3개") → 3

```
3) console.log(parseFloat("24.5도"));      // 출력: 24.5
```

parseInt 를 쓰면 24 만 나옵니다.

4	console.log(String(1234567).length);	// 출력: 7
5	console.log("7" + 3);	// 출력: 73 (이어붙이기)
	console.log("7" - 3);	// 출력: 4 (뺄셈)
6	console.log(Boolean(""));	// 출력: false
	console.log(Boolean(" "));	// 출력: true

변수와 자료형

RUN node 연습문제.js

- 1 TODO 자리에 코드를 씁니다.
- 2 저장하고 node 연습문제.js 로 실행합니다.
- 3 '기대 출력'과 똑같이 나오면 정답입니다.
- 4 10분 고민해도 안 되면 연습문제_정답.js 를 보세요.

순서대로 푸세요. 문제 1~13은 기본, 14~15는 응용, 16은 [도전]입니다. 문제 17은 에러를 직접 보는 문제라 맨 뒤에 있습니다.

문제 1 (개념01)

다음 문장을 화면에 출력하세요.

기대 출력 자바스크립트 시작!

여기에 코드를 쓰세요

문제 2 (개념01)

console.log 를 딱 한 번만 써서 아래처럼 한 줄에 출력하세요. 힌트: 심표(,)로 나열하면 됩니다.

기대 출력 삼각김밥 1200 원

여기에 코드를 쓰세요

문제 3 (개념02)

cafeName 이라는 변수에 “봄날카페” 를 담고 출력하세요. 카페 이름은 바뀌지 않으니 `const` 로 만드세요.

기대 출력

봄날카페

여기에 코드를 쓰세요

문제 4 (개념02)

stock 이라는 변수에 10 을 담아 출력한 뒤, 값을 7로 바꿔서 다시 출력하세요. 값이 바뀌므로 `let` 으로 만들어야 합니다.

기대 출력

10

7

여기에 코드를 쓰세요

문제 5 (개념02)

아래 두 변수를 더한 값을 출력하세요.

```
const bread = 3200;
const milk = 1500;
```

기대 출력

4700

여기에 코드를 쓰세요

문제 6 (개념03)

아래 세 값의 '값'과 '자료형'을 한 줄씩 같이 출력하세요. 42 / "42" / true 힌트: `console.log(값, typeof 값)`

기대 출력 42 number
 42 string
 true boolean

여기에 코드를 쓰세요

문제 7 (개념03)

아래 변수를 + 로 이어 붙여서 한 문장으로 출력하세요.

```
const city = "부산";
```

기대 출력 부산 여행 가자

여기에 코드를 쓰세요

문제 8 (개념03)

"대한민국만세" 가 몇 글자인지 출력하세요.

기대 출력

6

여기에 코드를 쓰세요

문제 9 (개념04)

myScore 가 60보다 큰지 비교한 결과를 isPass 라는 변수에 담고 출력하세요.

```
const myScore = 85;
```

기대 출력

true

여기에 코드를 쓰세요

문제 10 (개념04)

selectedMenu 라는 변수를 만들되, 아직 아무것도 안 골랐다는 뜻으로 null 을 담아 출력하세요. 그다음 "김밥" 으로 바뀌서 다시 출력하세요.

기대 출력

null

김밥

여기에 코드를 쓰세요

문제 11 (개념05)

아래 두 값은 입력창에서 받아온 값이라 문자열입니다. 두 값을 더한 결과가 7500 이 나오게 출력하세요.

```
const input1 = "3000";
const input2 = "4500";
```

기대 출력 7500

여기에 코드를 쓰세요

문제 12 (개념05)

아래 값에서 숫자 부분만 뽑아서 출력하세요.

```
const height = "180cm";
```

힌트: Number("180cm")는 NaN 이 됩니다. 글자가 섞여 있으면 통째로는 못 바꿉니다.

앞에서부터 읽히는 만큼만 가져오는 방법을 개념05 섹션 3에서 배웠습니다.

기대 출력 180

여기에 코드를 쓰세요

문제 13 (개념05)

아래 세 값을 불리언으로 바꿔서 한 줄씩 출력하세요. 0 / "" / "0" 셋 중 하나만 true 입니다. 왜 그런지 생각해 보세요.

기대 출력 false
false
true

여기에 코드를 쓰세요

문제 14 [응용]

아래 숫자가 몇 자리 숫자인지 출력하세요. 힌트: 숫자에는 .length 가 없습니다. 먼저 문자열로 바꾸세요.

const bigNumber2 = 987654321;

기대 출력 9

여기에 코드를 쓰세요

문제 15 [응용]

아래 변수로 두 줄을 출력하세요. 첫째 줄: before 에 5 를 그냥 + 한 결과 둘째 줄: before 를 숫자로 바꾼 뒤 5 를 더한 결과 두 결과가 왜 다른지 주석으로 한 줄 적어 보세요.

const before = "10";

기대 출력 105
 15

여기에 코드를 쓰세요

문제 16 [도전]

편의점 영수증 한 줄을 만듭니다. 지금까지 배운 것을 한 번에 씁니다. 문자열을 숫자로 바꾸고, 곱하고, 한 줄로 이어 붙여야 합니다. `itemPrice` 는 문자열이라는 점에 주의하세요.

```
const itemName = "우유";
const itemPrice = "1500";
const itemCount = 3;
```

기대 출력 우유 3개 = 4500원

여기에 코드를 쓰세요

문제 17 (에러 확인 - 맨 마지막)

아래 두 줄의 주석을 풀고 실행해서 어떤 에러가 나는지 확인하세요. 확인했으면 반드시 다시 주석 처리하세요. (에러가 나면 스크립트가 거기서 멈춰 버립니다. 그래서 맨 뒤에 있습니다.)

```
const fixedValue = "고정";
fixedValue = "변경";
```

무슨 에러가 났나요? 왜 그럴까요?

답

변수와 자료형

```
RUN node 심화문제.js
```

— 이걸 선택 과제입니다

연습문제를 다 푼 사람만 하세요. 안 풀어도 다음 단원에 지장 없습니다. 실습 시간이 남을 때 쓰라고 만든 것입니다.

연습문제보다 어렵습니다. 답이 바로 안 보이는 게 정상입니다. 막히면 개념 파일을 다시 뒤져 보세요. 그게 이 문제의 목적입니다.

심화 1 형변환 함정 네 가지

아래 네 값을 Number 로 바꾸면 각각 무엇이 나올까요? 먼저 예상을 적어 보고, 그다음 코드를 써서 확인하세요.

"" (빈 문자열) " " (공백 하나) "12abc" null

내 예상: _____

기대 출력	0
	0
	NaN
	0

여기에 코드를 쓰세요

심화 2 parseInt 는 어디까지 읽나

같은 네 값을 이번엔 parseInt 로 바꿔서 출력하세요. Number 와 결과가 다른 것이 몇 개인지도 세어 보세요.

힌트: parseInt 는 “앞에서부터 읽을 수 있는 만큼만” 가져옵니다. (개념05 섹션 3)

기대 출력	NaN
	NaN
	12
	NaN

여기에 코드를 쓰세요

심화 3 falsy 여섯 개를 한 번에 확인

개념05에서 “false 로 취급되는 값이 여섯 개” 라고 배웠습니다. 그 여섯 개를 Boolean 으로 바꿔 전부 false 가 나오는지 확인하세요. 그리고 헷갈리기 쉬운 세 가지도 확인하세요. 이건 전부 true 입니다.

falsy 여섯 개 : 0 "" null undefined NaN false 헷갈리는 세 개 : "0" " " "false"

힌트: 세 개가 왜 true 인지 생각해 보세요. 글자가 하나라도 들어 있으면 어떻게 될까요?

기대 출력	false false false false false false
	true true true

여기에 코드를 쓰세요

(console.log 두 번으로 끝내세요. 심표로 나열하면 한 줄에 나옵니다)

01단원 마무리 변수와 자료형

자주 넘어지는 자리

- 1 “10” + 5 가 105 인 것. 형변환을 여기서 못 잡으면 계속 따라옵니다

연산자

OPEN 02_연산자

```
console.log(7 + 3);  
console.log(7 - 3);  
console.log(7 * 3);  
console.log(7 / 3);
```

01 산술 연산자: 계산하기

02 대입 연산자와 증감 연산자

03 비교 연산자

04 논리 연산자

05 템플릿 리터럴 (백틱 문자열)

- 연습문제

- 심화문제

산술 연산자: 계산하기

RUN node 개념01_산술연산자.js

01단원에서는 값을 담고 보여 주기만 했습니다. 하지만 프로그램이 하는 일의 대부분은 '계산'입니다. 장바구니 총액, 할인가, 평균 점수, 남은 시간, 몇 페이지인지... 계산에 쓰는 기호를 산술 연산자라고 합니다.

```
+   더하기
-   빼기
*   곱하기      (× 가 아니라 별표)
/   나누기      (÷ 가 아니라 슬래시)
%   나머지      (이건 처음 볼 겁니다)
**  거듭제곱
```

사칙연산

섹션 1

코드	출력
<code>console.log(7 + 3);</code>	10
<code>console.log(7 - 3);</code>	4
<code>console.log(7 * 3);</code>	21
<code>console.log(7 / 3);</code>	
출력	2.3333333333333335

나누기 결과에 주의하세요. 초등학교 때처럼 “몫 3, 나머지 1” 로 나뉘지 않습니다. 자바스크립트의 / 는 소수까지 그대로 나옵니다. 몫만 필요하면 따로 처리해야 합니다. (섹션 4)

변수를 넣어도 똑같습니다.

<code>const price = 4500;</code> <code>const count = 3;</code> <code>console.log(price * count);</code>
출력 13500

➤ 직접 해보기

1

라떼 4500원 2잔과 케이크 6000원 1개의 총액을 출력해 보세요.

% — 나머지

섹션 2

% 는 “나누고 남은 것”입니다. 백분율(퍼센트)이 아닙니다.

```
console.log(10 % 3);
```

출력 1

$10 \div 3 = 3 \dots$ 나머지 1

```
console.log(10 % 5);
```

출력 0

딱 나누어떨어지면 나머지가 0입니다.

나머지는 이런 데 씁니다.

쓰임 1 · 짝수/홀수 판별 — 2로 나눈 나머지가 0이면 짝수

코드

```
console.log(8 % 2);
```

```
console.log(7 % 2);
```

▶ 출력

▶ 0

▶ 1

쓰임 2 · 초를 분과 초로 나누기

```
const totalSeconds = 125;
console.log(totalSeconds % 60);
```

출력 5

125초 = 2분 5초 → 남는 초가 5

쓰임 3 · 여러 개를 순서대로 돌려 쓰기 (0,1,2,0,1,2...)

```
console.log(0 % 3, 1 % 3, 2 % 3, 3 % 3, 4 % 3);
```

출력 0 1 2 0 1

▸ 직접 해보기 **2**

17을 5로 나눈 나머지를 출력해 보세요.

```
console.log(2 ** 3);
```

출력 8

$2 \times 2 \times 2$

```
console.log(10 ** 2);
```

출력 100

별표 하나(*)와 둘(**)은 완전히 다릅니다.

```
console.log(2 * 3, 2 ** 3);
```

출력 6 8

▫ 직접 해보기

3

한 변이 5인 정사각형의 넓이를 ** 로 구해 출력해 보세요.

몫 구하기와 소수 다듬기

/ 는 소수까지 나오므로, 몫만 필요하면 소수점을 버립니다.

```
console.log(7 / 2);
```

출력 3.5

여기서 Math 가 처음 나옵니다. Math 는 자바스크립트가 미리 만들어 둔 '계산 도구 상자' 의 이름입니다.

우리가 만든 것이 아니라 처음부터 들어 있습니다. 상자 안의 도구를 꺼낼 때 01단원에서 배운 점(.)을 씁니다.

Math . floor (7 / 2)

도구상자 도구 재료

도구에게 일을 시킬 때는 괄호를 붙이고, 그 안에 재료를 넣습니다. (.length 처럼 괄호 없이 쓰는 것도 있습니다. 괄호가 붙는 것은 "일을 시키는 것", 안 붙는 것은 "이미 가지고 있는 값" 이라고 보면 대체로 맞습니다)

Math.floor() : 소수점 아래를 버림 (내림)


```
console.log(Math.floor(7 / 2));
```

출력 3

`Math.ceil()` : 무조건 올림

```
console.log(Math.ceil(7 / 2));
```

출력 4

`Math.round()` : 반올림

코드

```
console.log(Math.round(3.4));
```

```
console.log(Math.round(3.5));
```

▶ 출력

▶ 3

▶ 4

`.toFixed(자릿수)` : 소수점 아래 자릿수를 정해서 다듬기

```
console.log((3.14159).toFixed(2));
```

출력 3.14

주의 · `toFixed` 의 결과는 숫자가 아니라 문자열입니다!

```
console.log(typeof (3.14159).toFixed(2));
```

출력 string

계산에 다시 쓰려면 `Number()` 로 되돌려야 합니다.

```
console.log(Number((3.14159).toFixed(2)), typeof Number((3.14159).toFixed(2)));
```

출력 3.14 number

125초를 "2분 5초"로 나누는 완성형

```
const sec = 125;
console.log(Math.floor(sec / 60), "분", sec % 60, "초");
```

출력 2 분 5 초

몫은 `Math.floor(/)`, 나머지는 `%` — 이 조합을 자주 씁니다.

◀ 직접 해보기 4

200초가 몇 분 몇 초인지 출력해 보세요.

수학과 똑같습니다. * / % 를 + - 보다 먼저 계산합니다.

```
console.log(2 + 3 * 4);
```

출력 14

3*4=12 를 먼저 하고 2를 더합니다. (20 이 아닙니다)

괄호로 순서를 바꿀 수 있습니다.

```
console.log((2 + 3) * 4);
```

출력 20

헛갈리면 괄호를 쓰세요. 괄호는 많이 써도 손해가 없습니다.

```
const basePrice = 12900;
const discount = 2000;
const taxRate = 0.1;
console.log((basePrice - discount) * (1 + taxRate));
```

출력 11990.000000000002

10900 × 1.1 은 11990 이어야 하는데 뒤에 이상한 꼬리가 붙었습니다. 소수 오차 때문입니다. 돈 계산은 이렇게 마무리합니다.

```
console.log(Math.round((basePrice - discount) * (1 + taxRate)));
```

출력 11990

▢ 직접 해보기 5

(5 + 5) 나누기 2 를 계산해 출력해 보세요.

특이한 결과들

0으로 나누면 에러가 아니라 Infinity(무한대)가 나옵니다.

코드	출력
<code>console.log(5 / 0);</code>	Infinity
<code>console.log(-5 / 0);</code>	-Infinity

0을 0으로 나누면 NaN 입니다.

```
console.log(0 / 0);
```

출력 NaN

소수 계산은 아주 작은 오차가 생깁니다. (01단원에서 봤죠)

```
console.log(0.1 + 0.2);
```

출력 0.30000000000000004

돈처럼 정확해야 하는 값은 '원' 단위 정수로 다루거나, 마지막에 `Math.round` 나 `toFixed` 로 다듬습니다.

```
console.log((0.1 + 0.2).toFixed(1));
```

출력 0.3

자주 하는 실수

섹션 7

★ 아래에서 `SyntaxError` 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. 다른 에러 (`ReferenceError`, `TypeError`)는 주석을 풀어도 그 줄에서만 나고 그 앞의 출력은 그대로 다 나옵니다. 그런데 `SyntaxError` 는 다릅니다. 자바스크립트가 파일을 아예 못 읽어서 출력이 한 줄도 안 나옵니다. 여러분이 망가뜨린 것이 아닙니다. 실수로 풀었다면 다시 // 를 붙이면 그대로 돌아옵니다.

이런 실수를 합니다

x ÷ 기호 사용 → `SyntaxError`

```
console.log(3 ÷ 4);
```

곱하기는 *, 나누기는 / 입니다. 수학 기호는 쓸 수 없습니다.

이런 실수를 합니다

% 를 퍼센트로 착각

```
console.log(50 % 100);
```

출력 50

"50퍼센트"가 아니라 "50을 100으로 나눈 나머지"입니다. 퍼센트 계산은 곱하기로 합니다.

```
console.log(200 * 0.5);
```

출력 100

이런 실수를 합니다

문자열에 + 를 써 놓고 덧셈이라고 생각하기

```
const inputA = "10";
console.log(inputA + 5);
```

출력 105

문자열이면 이어붙이기입니다. `Number(inputA) + 5` 로 고쳐야 합니다.

이런 실수를 합니다

`toFixed` 결과를 숫자로 착각

```
const fixed = (10.567).toFixed(1);
console.log(fixed + 1);
```

출력 10.61

`fixed` 는 문자열 "10.6" 이라 이어붙였습니다.

```
console.log(Number(fixed) + 1);
```

출력 11.6

정리

- 1 `+` `-` `*` `/` `%` `**` — 곱하기는 `*`, 나누기는 `/`, 나머지는 `%`
- 2 `/` 는 소수까지 나온다. 뭇만 필요하면 `Math.floor(a / b)`
- 3 `Math.floor`(내림) / `Math.ceil`(올림) / `Math.round`(반올림)
- 4 `toFixed` 의 결과는 문자열이다.
- 5 계산 순서는 수학과 같다. 헷갈리면 괄호를 쓴다.

직접 해보기 정답

1 `console.log(4500 * 2 + 6000);` // 출력: 15000

2 `console.log(17 % 5);` // 출력: 2

3 `console.log(5 ** 2);` // 출력: 25

4 `const s = 200;`
`console.log(Math.floor(s / 60), "분", s % 60, "초");`
// 출력: 3 분 20 초

5 `console.log((5 + 5) / 2);` // 출력: 5

대입 연산자와 증감 연산자

RUN node 개념02_대입연산자와_증감연산자.js

프로그램에서 가장 자주 하는 일은 “지금 값을 조금 바꿔서 다시 넣기”입니다.

장바구니 합계에 상품값 더하기
남은 재고에서 1 빼기
점수 누적하기

이걸 짧게 쓰는 기호들을 배웁니다.

= 는 “같다”가 아니라 “넣어라”

섹션 1

다시 한 번 강조합니다. = 는 오른쪽 값을 왼쪽 변수에 넣으라는 명령입니다.

```
let total = 100;
console.log(total);
```

출력 100

그래서 이런 식이 성립합니다. 수학이었다면 말이 안 되는 식입니다.

```
total = total + 50;
console.log(total);
```

출력 150

읽는 법: “지금 total(100)에 50을 더해서, 그 결과를 다시 total에 넣어라”

▸ 직접 해보기 1

score 에 70을 담고, 여기에 15를 더해서 다시 담은 뒤 출력해 보세요.

+= -= *= /= %= — 줄여 쓰기

섹션 2

total = total + 50; 은 total 을 두 번 쓰니 길고 오타가 나기 쉽습니다.

그래서 이렇게 줄여 씁니다.

```
let cart = 100;
```

— 기본

```
cart = cart + 50;
console.log(cart);
```

출력 150

줄여쓰기 · 위와 완전히 같은 뜻입니다.

```
cart += 50;
console.log(cart);
```

출력 200

```
cart -= 30;
console.log(cart);
```

출력 170

```
cart *= 2;
console.log(cart);
```

출력 340

```
cart /= 4;
console.log(cart);
```

출력 85

```
cart %= 7;
console.log(cart);
```

출력 1

$85 \div 7 = 12 \dots$ 나머지 1

문자열에도 += 를 쓸 수 있습니다. 이어붙이기가 됩니다.

```
let message = "안녕";
message += "하세요";
console.log(message);
```

출력 안녕하세요

이건 나중에 반복문(04단원)에서 글자를 차곡차곡 쌓을 때 아주 많이 씁니다.

▹ 직접 해보기 2

money 에 10000을 담고, += 로 3000을 더한 뒤 출력해 보세요.

++ 와 -- — 1씩 늘리고 줄이기

섹션 3

1을 더하는 일이 워낙 잦아서 전용 기호가 있습니다.

```
let visitor = 5;

visitor++; // visitor += 1 과 같습니다
console.log(visitor);
```

출력 6

```
visitor--; // visitor -= 1 과 같습니다
console.log(visitor);
```

출력 5

값이 숫자일 때 세 가지는 전부 같은 뜻입니다.

```
visitor = visitor + 1;
visitor += 1;
visitor++;
```

(값이 문자열이면 셋의 결과가 달라집니다. 섹션 6 [실수 4]에서 확인합니다)

▹ 직접 해보기 3

like 에 0을 담고, ++ 를 세 번 써서 3이 되게 만든 뒤 출력해 보세요.

++ 를 앞에 붙일 때와 뒤에 붙일 때

섹션 4

변수의 값만 바꿀 때는 앞뒤 어디에 붙여도 결과가 같습니다. 하지만 “그 자리에서 바로 사용”하면 달라집니다.

뒤에 붙이기 — 후위 · 먼저 쓰고, 그다음 증가

```
let a = 5;
console.log(a++);
```

출력 5

↑ 5를 먼저 내보내고, 그 뒤에 6이 됩니다


```
console.log(a);
```

출력 6

앞에 붙이기 — 전위 · 먼저 증가하고, 그다음 씀

```
let b = 5;
console.log(++b);
```

출력 6

↑ 먼저 6으로 만들고, 그 6을 내보냅니다

```
console.log(b);
```

출력 6

둘 다 최종 값은 6으로 같습니다. 다른 건 '그 줄에서 나온 값'뿐입니다.

헷갈리죠? 그래서 실무에서는 이렇게 씁니다. → 값을 쓰면서 동시에 증가시키지 마세요. 줄을 나누면 헷갈릴 일이 없습니다.

```
let c = 5;
console.log(c);
```

출력 5

```
c++; // 그다음 줄에서 따로 증가
console.log(c);
```

출력 6

⇒ 직접 해보기 4

let n = 10; 일 때 console.log(n++) 과 console.log(++n) 을 차례로 실행하면 무엇이 찍힐지 먼저 예상하고, 실행해 확인해 보세요.

누적 — 실전에서 쓰는 모습

섹션 5

값을 담아 두는 변수를 하나 만들고, 거기에 계속 더해 나가는 방식입니다.

```
let orderTotal = 0; // 반드시 0으로 시작합니다
```

```
orderTotal += 4500; // 라떼
console.log(orderTotal);
```

출력 4500

```
orderTotal += 6000; // 케이크  
console.log(orderTotal);
```

출력 10500

```
orderTotal += 3000; // 쿠키  
console.log(orderTotal);
```

출력 13500

시작값을 0이 아니라 안 넣으면 어떻게 될까요?

```
let wrongTotal;  
wrongTotal += 4500;  
console.log(wrongTotal);
```

출력 NaN

undefined 에 숫자를 더하면 NaN 입니다. 그래서 누적 변수는 반드시 0 으로 시작해야 합니다.

글자를 누적할 때는 빈 문자열 "" 로 시작합니다.

```
let receipt = "";  
receipt += "라떼 ";  
receipt += "케이크 ";  
receipt += "쿠키";  
console.log(receipt);
```

출력 라떼 케이크 쿠키

▸ 직접 해보기 5

sum 을 0으로 시작해서 10, 20, 30 을 차례로 더한 뒤 출력해 보세요.

자주 하는 실수

섹션 6

이런 실수를 합니다

const 에 += 쓰기 → TypeError

```
const fixedCount = 1;  
fixedCount += 1;
```

값이 바뀌는 변수는 let 으로 만들어야 합니다.

이런 실수를 합니다

=+ 로 잘못 쓰기 (기호 순서 바꿔 쓰기)

```
let d = 10;
d = +5; // += 가 아니라 += 로 썼습니다
console.log(d);
```

출력 5

15가 아니라 5가 됩니다. "5를 넣어라"가 되어 버렸습니다. 에러가 안 나서 찾기 어려운 실수입니다.

이런 실수를 합니다

누적 변수 초기값을 안 줌 → NaN (섹션 5에서 봤습니다)

이런 실수를 합니다

++ 를 문자열에 쓰기

```
let word = "5";
word++;
console.log(word, typeof word);
```

출력 6 number

실수라기보다 함정입니다. ++ 는 값을 숫자로 바꿔 버립니다. 그래서 문자열이었던 게 슬그머니 숫자가 됩니다.

정리

- 1 = 는 "넣어라". total = total + 50 이 성립하는 이유.
- 2 += -= *= /= %= 로 줄여 쓴다. 문자열 += 는 이어붙이기.
- 3 ++ 는 1 증가, -- 는 1 감소.
- 4 a++ 는 쓰고 나서 증가, ++a 는 증가하고 나서 씀. 헷갈리면 줄을 나눠라.
- 5 누적 변수는 숫자면 0, 글자면 "" 으로 시작한다. 안 그러면 NaN.

- 1 `let score = 70;`
`score = score + 15;`
`console.log(score);` // 출력: 85

- 2 `let money = 10000;`
`money += 3000;`
`console.log(money);` // 출력: 13000

- 3 `let like = 0;`
`like++;`
`like++;`
`like++;`
`console.log(like);` // 출력: 3

- 4 `let n = 10;`
`console.log(n++);` // 출력: 10 (쓰고 나서 11이 됨)
`console.log(++n);` // 출력: 12 (12로 만들고 나서 씀)

- 5 `let sum = 0;`
`sum += 10;`
`sum += 20;`
`sum += 30;`
`console.log(sum);` // 출력: 60

비교 연산자

RUN node 개념03_비교연산자.js

두 값을 견주어 보고 `true` / `false` 를 내놓는 기호들입니다. 03단원 조건문("만약 ~라면")의 재료가 되므로 아주 중요합니다.

```
> 보다 크다
< 보다 작다
>= 크거나 같다 (이상)
<= 작거나 같다 (이하)
=== 같다
!== 다르다
```

크기 비교

섹션 1

코드	출력
<code>console.log(10 > 5);</code>	<code>true</code>
<code>console.log(10 < 5);</code>	<code>false</code>

‘이상’과 ‘초과’를 구별하세요. 실무 버그의 단골입니다.

코드	출력
<code>console.log(10 >= 10);</code>	<code>true</code>
<code>console.log(10 > 10);</code>	<code>false</code>
<code>console.log(10 <= 10);</code>	<code>true</code>

기호 순서에 주의하세요. `=>` 가 아니라 `>=` 입니다. (`=>` 는 나중에 배울 화살표 함수의 기호입니다)

<code>const age = 20;</code> <code>console.log(age >= 19);</code>
출력 <code>true</code>

▫ 직접 해보기 1

재고가 3개일 때 "5개 이상 남았는가"를 판단해 출력해 보세요.

=== 같다 / !== 다르다

섹션 2

코드	▶ 출력
<code>console.log(5 === 5);</code>	▶ true
<code>console.log(5 === 3);</code>	▶ false
<code>console.log(5 !== 3);</code>	▶ true
<code>console.log(5 !== 5);</code>	▶ false

= 기호가 몇 개인지가 전부 다른 뜻입니다. 이것만은 꼭 구별하세요.

= 넣어라 (대입)
== 같으냐? (느슨한 비교 - 쓰지 마세요, 섹션 3)
=== 같으냐? (엄격한 비교 - 항상 이걸 쓰세요)

```
const menu = "아메리카노";  
console.log(menu === "아메리카노");
```

출력 true

```
console.log(menu === "라떼");
```

출력 false

문자열은 한 글자라도 다르면 false 입니다. 띄어쓰기도 글자입니다.

```
console.log("아메리카노" === "아메리카노 ");
```

출력 false

▫ 직접 해보기 2

아이디가 "admin" 인지 비교한 결과를 출력해 보세요.

== 를 쓰면 안 되는 이유

섹션 3

== 는 자료형이 달라도 "대충 비슷하면" true 를 줍니다.

```
console.log("5" == 5);
```

출력 true

문자열 "5" 와 숫자 5 를 같다고 합니다.

```
console.log("5" === 5);
```

```
출력      false
```

=== 는 자료형까지 봅니다. 이게 맞는 동작입니다.

== 의 이상한 결과들을 보세요.

코드	출력
<code>console.log(0 == false);</code>	▶ true
<code>console.log("" == false);</code>	▶ true
<code>console.log("0" == false);</code>	▶ true
<code>console.log(null == undefined);</code>	▶ true

반면 === 는 예상대로 동작합니다.

코드	출력
<code>console.log(0 === false);</code>	▶ false
<code>console.log("" === false);</code>	▶ false
<code>console.log("0" === false);</code>	▶ false
<code>console.log(null === undefined);</code>	▶ false

결론: 비교는 언제나 === 와 !== 로 하세요. == 는 “이런 게 있다” 정도만 알아 두면 됩니다. 남의 코드에서 만날 수는 있으니까요.

▫ 직접 해보기

3

“0” == 0 과 “0” === 0 을 각각 출력해 비교해 보세요.

문자열끼리 비교하면 사전 순

섹션 4

문자열에 > < 를 쓰면 사전(순서)으로 비교합니다.

```
console.log("apple" < "banana");
```

```
출력      true
```

a 가 b 보다 앞이라서 apple 이 ‘작습니다’

```
console.log("가" < "나");
```

```
출력      true
```

이런 실수를 합니다
대문자가 소문자보다 앞입니다.

```
console.log("Z" < "a");
```

출력 true

큰 함정 · 숫자처럼 생긴 문자열은 숫자로 비교되지 않습니다.

```
console.log("10" < "9");
```

출력 true

"1" 과 "9" 를 한 글자씩 비교해서 1이 앞이라 "10"이 작다고 나옵니다.

```
console.log(10 < 9);
```

출력 false

숫자로 비교하면 당연히 false 입니다.

그래서 입력창에서 받은 값을 크기 비교하기 전에는 반드시 Number() 로 바꿔야 합니다. (01단원 개념05)

```
const inputAge = "9"; // 입력창에서 온 값  
console.log(Number(inputAge) < 10);
```

출력 true

▹ 직접 해보기 4
"100" > "99" 와 100 > 99 를 각각 출력해 비교해 보세요.

비교 결과는 값이다

섹션 5

비교 결과 true / false 는 그냥 값이라 변수에 담을 수 있습니다.

```
const stock = 0;  
const isSoldOut = stock === 0;  
console.log(isSoldOut, typeof isSoldOut);
```

출력 true boolean


```
const userAge = 17;
const isAdult = userAge >= 19;
console.log(isAdult);
```

출력 false

이렇게 이름을 붙여 두면 코드가 문장처럼 읽힙니다. 조건문(03단원)에서 이 변수를 그대로 쓰게 됩니다.

▫ 직접 해보기

5

온도가 28도일 때 isHot 이라는 변수에 "27도 초과인가"를 담아 출력해 보세요.

자주 하는 실수

섹션 6

★ 아래에서 SyntaxError 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. 다른 에러 (ReferenceError, TypeError)는 주석을 풀어도 그 줄에서만 나고 그 앞의 출력은 그대로 다 나옵니다. 그런데 SyntaxError 는 다릅니다. 자바스크립트가 파일을 아예 못 읽어서 출력이 한 줄도 안 나옵니다. 여러분이 망가뜨린 것이 아닙니다. 실수로 풀었다면 다시 // 를 붙이면 그대로 돌아옵니다.

이런 실수를 합니다

= 와 === 를 헷갈리기

```
let count = 5;
count = 3; // "3을 넣어라" ← 비교가 아닙니다
console.log(count);
```

출력 3

비교하려면 count === 3 이라고 써야 합니다.

```
console.log(count === 3);
```

출력 true

이런 실수를 합니다

⇒ 로 잘못 쓰기 → SyntaxError

```
console.log(10 => 5);
```

이상은 >= 입니다. = 가 뒤에 옵니다.

이런 실수를 합니다

범위를 수학처럼 쓰기

```
const temp = 25;
console.log(10 < temp < 30);
```

출력 true

맞은 것처럼 보이지만 우연입니다. $10 < 25 \rightarrow \text{true}$ 가 되고, 그다음 $\text{true} < 30$ 을 계산합니다. true 는 숫자로 1이라 $1 < 30 \rightarrow \text{true}$ 입니다.

```
console.log(10 < 100 < 30);
```

출력 true

100은 30보다 큰데도 true 가 나옵니다. 완전히 틀린 코드입니다. 범위는 $\&\&$ 로 이어야 합니다. (다음 파일 개념04)

이런 실수를 합니다

소수 비교

```
console.log(0.1 + 0.2 === 0.3);
```

출력 false

소수 오차 때문입니다. 소수는 $\&\&\&$ 로 비교하지 마세요.

정리

- 1 $> < >= <=$ — '이상/이하'와 '초과/미만'을 구별할 것
- 2 비교는 언제나 $\&\&\&$ 와 $\&\&\&=$, $\&\&\&$ 는 쓰지 않는다.
- 3 문자열끼리 비교하면 사전 순. "10" < "9" 가 true 다.
- 4 크기를 비교할 값이 입력창에서 왔다면 먼저 `Number()` 로 바꾼다.
- 5 $10 < x < 30$ 은 틀린 코드다. $\&\&$ 로 이어야 한다.

직접 해보기 정답

- 1 `const stockCount = 3;`
`console.log(stockCount >= 5);` // 출력: false
- 2 `const userId = "admin";`
`console.log(userId === "admin");` // 출력: true
- 3 `console.log("0" == 0);` // 출력: true
`console.log("0" === 0);` // 출력: false
- 4 `console.log("100" > "99");` // 출력: false (사전 순: "1"<"9")
`console.log(100 > 99);` // 출력: true
- 5 `const temperature = 28;`
`const isHot = temperature > 27;`
`console.log(isHot);` // 출력: true

논리 연산자

RUN node 개념04_논리연산자.js

조건 하나로는 부족할 때가 많습니다.

"19살 이상이고, 회원인 사람" → 둘 다 만족
 "쿠폰이 있거나, 5만원 이상 샀으면" → 하나만 만족해도 됨
 "품절이 아니면" → 반대로 뒤집기

이걸 이어 주는 기호가 논리 연산자입니다.

&& 그리고 (and) - 둘 다 true 여야 true || 또는 (or) - 하나라도 true 면 true
 ! 아니다 (not) - true 를 false 로, false 를 true 로

&& — 둘 다 참이어야 참

섹션 1

코드	출력
<code>console.log(true && true);</code>	▶ true
<code>console.log(true && false);</code>	▶ false
<code>console.log(false && true);</code>	▶ false
<code>console.log(false && false);</code>	▶ false

"둘 다 만족해야 통과" — 하나라도 어긋나면 false 입니다.

```
const age = 22;
const isMember = true;
console.log(age >= 19 && isMember);
```

출력 true

22 >= 19 는 true, isMember 도 true → 둘 다 true 라서 true

```
const age2 = 17;
console.log(age2 >= 19 && isMember);
```

출력 false

나이 조건이 false 라서 전체가 false

키보드에서 & 는 숫자 7 위에 있습니다. 두 개를 붙여 씁니다.

▹ 직접 해보기 1

점수 85점이 “60점 이상이고 90점 미만”인지 판단해 출력해 보세요.

범위 판단 — && 의 대표 쓰임

섹션 2

개념03에서 $10 < x < 30$ 은 틀린 코드라고 했습니다. 이렇게 써야 합니다.

```
const temp = 25;
console.log(temp > 10 && temp < 30);
```

출력 true

아까 틀렸던 예도 이제 제대로 나옵니다.

```
const temp2 = 100;
console.log(temp2 > 10 && temp2 < 30);
```

출력 false

조건을 세 개 이상 이을 수도 있습니다.

```
const score = 85;
console.log(score >= 0 && score <= 100 && score >= 80);
```

출력 true

▹ 직접 해보기 2

나이가 13 이상 19 이하인지 판단하는 식을 만들어 출력해 보세요. (나이는 15로)

|| — 하나라도 참이면 참

섹션 3

코드	▶ 출력
<code>console.log(true true);</code>	▶ true
<code>console.log(true false);</code>	▶ true
<code>console.log(false true);</code>	▶ true
<code>console.log(false false);</code>	▶ false

“둘 중 하나만 만족해도 통과”

```
const hasCoupon = false;
const totalPrice = 60000;
console.log(hasCoupon || totalPrice >= 50000);
```

출력 true

쿠폰은 없지만 5만원 이상이라 무료배송

키보드에서 | 는 백스페이스 아래, 역슬래시(\)와 같은 키입니다. Shift 를 누르고 칩니다. 대문자 I 나 소문자 i 이 아닙니다.

▫ 직접 해보기 3

요일이 "토" 이거나 "일" 인지 판단해 출력해 보세요. (요일은 "일"로)

! — 뒤집기

섹션 4

코드

▶ 출력

```
console.log(!true);
```

▶ false

```
console.log(!false);
```

▶ true

```
const isSoldOut = false;
console.log(!isSoldOut);
```

출력 true

"품절이 아니다" → 살 수 있다

비교식 앞에 붙일 때는 반드시 괄호로 감싸세요.

```
const temperature = -5;
```

맞는 코드 · "영상이 아니다" = 영하다

```
console.log(!(temperature > 0));
```

출력 true

이런 실수를 합니다

괄호를 빼면 뜻이 완전히 달라집니다.

```
console.log(!temperature > 0);
```

출력 false

1 !temperature → !(-5) → -5 는 truthy 라서 false

2 false > 0 → false 는 숫자로 0 → 0 > 0 → false 영하 5도인데 “영하가 아니다”라는 답이 나왔습니다. ! 는 바로 뒤에 오는 것 하나에만 걸린다고 기억하세요.

▹ 직접 해보기

4

isLoggedIn 이 false 일 때 “로그인하지 않았다”를 ! 로 판단해 출력해 보세요.

계산 순서 — ! → && → ||

섹션 5

셋이 섞이면 ! 이 먼저, 그다음 && , 마지막이 || 입니다.

```
console.log(true || false && false);
```

출력 true

&& 를 먼저 계산합니다. false && false = false → true || false = true

```
console.log((true || false) && false);
```

출력 false

괄호로 순서를 바꾸면 결과가 달라집니다.

헷갈리면 괄호를 쓰세요. 읽는 사람도 편합니다.

```
const isWeekend = true;
const isHoliday = false;
const isWorkday = !(isWeekend || isHoliday);
console.log(isWorkday);
```

출력 false

▹ 직접 해보기

5

(나이가 19 이상) 그리고 (회원이거나 쿠폰이 있음) 을 괄호를 써서 판단해 보세요. (나이 20, 회원 false, 쿠폰 true)

값이 비었을 때 기본값 채우기 (|| 와 ??)

섹션 6

지금까지 배운 true / false 이야기는 전부 맞습니다. 여기에 하나만 더합니다. && 와 || 는 true/false 가 아니라 ‘값’ 을 그대로 내놓습니다. || 는 왼쪽이 falsy 면 오른쪽을 내놓습니다. (falsy 6개: 0, "", null,

undefined, NaN, false — 01단원 개념05)

```
const inputName = ""; // 이름을 안 적고 보냈다고 가정
console.log(inputName || "손님");
```

출력 손님

```
const inputName2 = "김민준";
console.log(inputName2 || "손님");
```

출력 김민준

“값이 없으면 기본값을 쓴다”를 한 줄로 쓰는 방법입니다. 아주 자주 씁니다.

주의 · 0 도 falsy 라서 기본값으로 밀려납니다.

```
const count = 0;
console.log(count || 10);
```

출력 10

재고가 진짜 0인데 10으로 바뀌어 버립니다. 이건 버그입니다.

그래서 ?? 라는 기호가 새로 생겼습니다. ?? 는 null 과 undefined 일 때만 오른쪽을 씁니다.

```
console.log(count ?? 10);
```

출력 0

0은 그대로 살아남습니다.

```
const notChosen = null;
console.log(notChosen ?? 10);
```

출력 10

정리:

```
|| 값이 falsy(0, "", null, undefined, NaN, false)면 기본값
?? 값이 null 이나 undefined 일 때만 기본값   ← 대부분 이게 안전합니다
```

▸ 직접 해보기 6

nickname 이 빈 문자열일 때 || 로 “익명” 이 나오게 해 보세요.

자주 하는 실수

섹션 7

이런 실수를 합니다

& 나 | 를 하나만 쓰기

```
console.log(true & true);
```

출력 1

& 하나는 완전히 다른 기호(비트 연산자)입니다. 결과도 숫자로 나옵니다. 반드시 두 개씩 &&, || 로 쓰세요.

이런 실수를 합니다

범위를 수학처럼 쓰기 (개념03에서 봤습니다)

 $10 < x < 30 \leftarrow$ 틀림 $x > 10 \ \&\& \ x < 30 \leftarrow$ 맞음

이런 실수를 합니다

"또는"을 말로 옮기다 틀리기

```
const day = "일";
console.log(day === "토" || "일");
```

출력 일

true/false 가 아니라 "일" 이 나옵니다. || 오른쪽의 "일" 은 비교식이 아니라 그냥 글자입니다. 비교를 양쪽에 다 써야 합니다.

```
console.log(day === "토" || day === "일");
```

출력 true

이런 실수를 합니다

부정을 이중으로 써서 헷갈리기

```
const isNotClosed = true;
console.log(!isNotClosed);
```

출력 false

실수는 아니지만, 변수 이름에 부정을 넣으면 !isNotClosed 처럼 부정의 부정이 되어 읽기 힘듭니다. isOpen 처럼 긍정으로 이름 지으세요.

정리

- 1 && 둘 다 true 여야 true. 범위 판단에 쓴다.
- 2 || 하나라도 true 면 true.
- 3 ! 뒤집기. 비교식 앞에 붙일 땐 괄호로 감쌀 것.
- 4 순서는 ! → && → ||. 헷갈리면 괄호.
- 5 값 || 기본값 / 값 ?? 기본값 — 0을 살리려면 ?? 를 쓴다.
- 6 day === "토" || day === "일" — 비교는 양쪽에 다 쓴다.

직접 해보기 정답

- 1

```
const s = 85;
console.log(s >= 60 && s < 90);
```

 // 출력: true
- 2

```
const a = 15;
console.log(a >= 13 && a <= 19);
```

 // 출력: true
- 3

```
const d = "일";
console.log(d === "토" || d === "일");
```

 // 출력: true
- 4

```
const isLoggedIn = false;
console.log(!isLoggedIn);
```

 // 출력: true
- 5

```
const myAge = 20;
const member = false;
const coupon = true;
console.log(myAge >= 19 && (member || coupon));
```

 // 출력: true
- 6

```
const nickname = "";
console.log(nickname || "익명");
```

 // 출력: 익명

템플릿 리터럴 (백틱 문자열)

RUN node 개념05_템플릿리터럴.js

지금까지 문자열에 변수를 끼워 넣을 때 + 를 썼습니다. 짧을 땐 괜찮지만 길어지면 금방 지옥이 됩니다. 그래서 나온 게 백틱(`)을 쓰는 템플릿 리터럴입니다. ('리터럴'은 코드에 값을 그대로 적어 넣은 것을 부르는 말입니다. "안녕"도 1200도 전부 리터럴입니다. 어려운 말이 아닙니다)

+ 방식의 불편함

섹션 1

```
const userName = "김민준";
const userAge = 20;
const city = "부산";
```

기본 + 로 이어붙이기

```
console.log("안녕하세요, " + userName + "님. 나이는 " + userAge + "세, 사는 곳은 " + city + "입니다.");
```

출력 안녕하세요, 김민준님. 나이는 20세, 사는 곳은 부산입니다.

잘 나오긴 합니다. 하지만 - 따옴표와 + 가 번갈아 나와서 눈이 아픕니다 - 띄어쓰기를 어디에 넣었는지 세기 어렵습니다 - + 하나만 빠뜨려도 에러입니다

백틱과 \${}

섹션 2

백틱은 키보드 숫자 1 왼쪽, 물결(~)과 같은 키입니다. 백틱으로 감싼 문자열 안에서는 \${변수이름}으로 값을 바로 끼워 넣습니다.

```
console.log(`안녕하세요, ${userName}님. 나이는 ${userAge}세, 사는 곳은 ${city}입니다.`);
```

출력 안녕하세요, 김민준님. 나이는 20세, 사는 곳은 부산입니다.

결과는 위와 똑같은데 훨씬 읽기 쉽습니다. 문장이 문장 그대로 보이고, 값이 들어갈 자리만 \${}로 뚫려 있습니다.

짧은 것도 봅시다.

```
const price = 4500;
console.log(`라떼는 ${price}원입니다`);
```

출력 라떼는 4500원입니다

변수가 없어도 백틱을 쓸 수 있습니다. 그냥 문자열입니다.

```
console.log(`그냥 문자열`);
```

출력 그냥 문자열

자료형도 그대로 string 입니다.

```
console.log(typeof `안녕`);
```

출력 string

▹ 직접 해보기

1

백틱을 써서 “부산 여행 가자” 를 출력해 보세요. (city 변수 사용)

`${}` 안에는 계산식도 들어간다

섹션 3

변수 이름만 넣을 수 있는 게 아닙니다. 값이 나오는 식이면 뭐든 됩니다.

```
const itemPrice = 4500;
const itemCount = 3;

console.log(`총액: ${itemPrice * itemCount}원`);
```

출력 총액: 13500원

비교식도 됩니다. (결과인 true/false 가 글자로 들어갑니다)

```
console.log(`성인인가? ${userAge >= 19}`);
```

출력 성인인가? true

다른 값을 바로 만들어 넣을 수도 있습니다.

```
console.log(`내년이면 ${userAge + 1}살`);
```

출력 내년이면 21살

다만 `${}` 안이 길어지면 오히려 읽기 나빠집니다. 계산은 미리 변수에 담고, `${}` 에는 변수만 넣는 편이 좋습니다.

```
const totalPrice = itemPrice * itemCount;
console.log(`총액: ${totalPrice}원`);
```

출력 총액: 13500원

▸ 직접 해보기 2

가로 8, 세로 5 인 직사각형의 넓이를 백틱 안에서 계산해 "넓이는 40 입니다" 로 출력해 보세요.

여러 줄 문자열

섹션 4

작은따옴표·큰따옴표는 줄을 바꾸면 에러가 납니다. 백틱은 엔터를 친 그대로 줄바꿈이 됩니다.

```
const receipt = `주문 내역
- 라떼 4500원
- 케이크 6000원`;

console.log(receipt);
```

출력 주문 내역
 - 라떼 4500원
 - 케이크 6000원

들여쓰기도 그대로 들어가니 주의하세요. 코드를 예쁘게 맞추려고 넣은 공백까지 출력에 포함됩니다.

변수와 섞어도 됩니다.

```
const orderName = "김민준";
const orderTotal = 10500;
console.log(`${orderName}님 주문서
합계: ${orderTotal}원
감사합니다`);
```

출력 김민준님 주문서
 합계: 10500원
 감사합니다

▸ 직접 해보기 3

자기 이름과 좋아하는 음식을 두 줄로 출력해 보세요.

백틱 안에서 따옴표 쓰기

섹션 5

백틱 안에서는 " 와 ' 를 그냥 쓸 수 있습니다. 신경 쓸 게 없습니다.

```
console.log(`그는 "안녕"이라고 말하며 '손'을 흔들었다`);
```

```
출력      그는 "안녕"이라고 말하며 '손'을 흔들었다
```

백틱 자체를 넣고 싶다면 앞에 역슬래시(\)를 붙입니다. (드문 경우입니다)

```
console.log(`백틱은 \` 이렇게 생겼습니다`);
```

```
출력      백틱은 ` 이렇게 생겼습니다
```

언제 무엇을 쓸까

섹션 6

실무 기준은 간단합니다. 변수를 끼워 넣거나 줄을 바꾼다 → 백틱 그냥 고정된 글자 하나 → 작은따옴표나 큰따옴표

헛갈리면 그냥 백틱을 쓰세요. 손해 볼 게 없습니다.

```
const label = "합계"; // 고정 글자 - 따옴표
console.log(`${label}: ${totalPrice}원`); // 조합 - 백틱
```

```
출력      합계: 13500원
```

자주 하는 실수

섹션 7

★ 아래에서 SyntaxError 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. 다른 에러 (ReferenceError, TypeError)는 주석을 풀어도 그 줄에서만 나고 그 앞의 출력은 그대로 다 나옵니다. 그런데 SyntaxError 는 다릅니다. 자바스크립트가 파일을 아예 못 읽어서 출력이 한 줄도 안 나옵니다. 여러분이 망가뜨린 것이 아닙니다. 실수로 풀었다면 다시 // 를 붙이면 그대로 돌아옵니다.

이런 실수를 합니다

백틱 대신 작은따옴표를 쓰기

```
const wrong1 = '가격은 ${price}원';
console.log(wrong1);
```

```
출력      가격은 ${price}원
```

`${}` 가 그대로 글자로 나옵니다. 에러가 안 나서 더 헛갈립니다. `${}` 를 쓰려면 반드시 백틱이어야 합니다.

이런 실수를 합니다

\$를 빠뜨리기

```
console.log(`가격은 {price}원`);
```

출력 가격은 {price}원

중괄호만 쓰면 그냥 글자입니다. \$가 있어야 합니다.

이런 실수를 합니다

백틱을 작은따옴표로 착각해서 닫기 → SyntaxError

```
console.log(`안녕`);
```

백틱으로 열었으면 백틱으로 닫아야 합니다. 글꼴에 따라 `와 '가 비슷해 보이니 주의하세요.

이런 실수를 합니다

\${ } 안에 세미콜론이나 문장 넣기 → SyntaxError

```
console.log(`값: ${const x = 1;}`);
```

\${ } 안에는 '값이 되는 식'만 들어갑니다. 변수 만들거나 문장은 넣을 수 없습니다.

정리

- 1 백틱 안에서 \${변수} 로 값을 끼워 넣는다.
- 2 \${ } 안에는 계산식·비교식도 들어간다. 단, 길어지면 변수로 빼라.
- 3 백틱은 엔터를 친 그대로 여러 줄이 된다.
- 4 백틱 안에서는 " 와 ' 를 그냥 써도 된다.
- 5 작은따옴표에 \${ } 를 쓰면 글자 그대로 나온다. 에러도 안 난다.

```

1 console.log(`${city} 여행 가자`);
  // 출력: 부산 여행 가자

2 const w = 8;
  const h = 5;
  console.log(`넓이는 ${w * h} 입니다`);
  // 출력: 넓이는 40 입니다

3 const me = "김민준";
  const food = "김치찌개";
  console.log(`이름: ${me}
  좋아하는 음식: ${food}`);

```

이렇게 쓰면 둘째 줄 앞에 공백 4칸이 붙어서 출력됩니다. 코드를 보기 좋게 하려고 넣은 들여쓰기까지 글자로 들어가기 때문입니다. 공백 없이 출력하려면 백틱 안의 둘째 줄을 왼쪽 끝에 붙여 쓰세요.

```

const text = `이름: ${me}
  좋아하는 음식: ${food}`;
console.log(text);
// 출력: 이름: 김민준
// 출력: 좋아하는 음식: 김치찌개

```


연산자

RUN node 연습문제.js

TODO 자리에 코드를 쓰고, '기대 출력'과 같은지 확인하세요. 문제 1~14는 기본, 15는 응용, 16은 [도전], 17은 에러 확인입니다.

문제 1 (개념01 산술)

사과 한 개 값과 개수가 아래에 있습니다. 총액을 출력하세요.

```
const applePrice = 1200;
const appleCount = 3;
```

기대 출력 3600

여기에 코드를 쓰세요

문제 2 (개념01 산술)

25를 4로 나눈 나머지를 출력하세요.

기대 출력 1

여기에 코드를 쓰세요

문제 3 (개념01 산술)

아래 초를 "몇 분 몇 초" 형태로 출력하세요. 힌트: 몫은 `Math.floor(a / 60)`, 나머지는 `a % 60`

```
const totalSec = 350;
```

기대 출력 5분 50초

여기에 코드를 쓰세요

문제 4 (개념01 산술)

아래 값을 소수점 첫째 자리까지만 남겨서 출력하세요.

```
const rawValue = 12345.678;
```

기대 출력 12345.7

여기에 코드를 쓰세요

문제 5 (개념02 대입)

sumTotal 을 0으로 시작해서 += 로 1200, 800, 500 을 차례로 더한 뒤 출력하세요.

기대 출력 2500

여기에 코드를 쓰세요

문제 6 (개념02 증감)

visitorCount 를 99로 시작해서 ++ 로 1 늘린 뒤 출력하세요.

기대 출력 100

여기에 코드를 쓰세요

문제 7 (개념03 비교)

아래 나이가 19세 이상인지 판단해 출력하세요.

```
const checkAge = 17;
```

기대 출력 false

여기에 코드를 쓰세요

문제 8 (개념03 비교)

"7" 과 7 을 === 로 비교한 결과와 == 로 비교한 결과를 차례로 출력하세요. 두 결과가 왜 다른지 생각해 보세요.

기대 출력 false
true

여기에 코드를 쓰세요

문제 9 (개념03 비교)

아래 두 줄을 출력하세요. 첫째 줄: 문자열 "80" 이 문자열 "100" 보다 큰지 둘째 줄: 숫자 80 이 숫자 100 보다 큰지 결과가 왜 반대인지 생각해 보세요.

기대 출력 true
 false

여기에 코드를 쓰세요

문제 10 (개념04 논리)

아래 점수가 "70 이상이고 80 미만"인지 판단해 출력하세요.

```
const score2 = 75;
```

기대 출력 true

여기에 코드를 쓰세요

문제 11 (개념04 논리)

닉네임이 비어 있으면 "익명" 이 나오도록 출력하세요. 힌트: ||

```
const inputNick = "";
```

기대 출력 익명

여기에 코드를 쓰세요

문제 12 (개념04 논리)

“비가 오지 않는다”를 ! 를 써서 판단해 출력하세요.

```
const isRaining = false;
```

기대 출력 true

여기에 코드를 쓰세요

문제 13 (개념05 템플릿 리터럴)

아래 두 변수를 써서 백틱으로 한 문장을 만들어 출력하세요.

```
const name3 = "이서연";
const age3 = 22;
```

기대 출력 안녕하세요, 이서연님! 22살이시군요.

여기에 코드를 쓰세요

문제 14 (개념05 템플릿 리터럴)

백틱 안에서 곱셈을 해서 아래처럼 출력하세요.

```
const numA = 3;
const numB = 4;
```

기대 출력 3 곱하기 4 는 12

여기에 코드를 쓰세요

문제 15 [응용]

아래 값으로 영수증 한 줄을 만들어 출력하세요. 총액은 직접 계산해야 합니다.

```
const itemName15 = "샌드위치";
const itemPrice15 = 4800;
const itemCount15 = 2;
```

기대 출력 샌드위치 2개 9600원

여기에 코드를 쓰세요

문제 16 [도전]

아래 값으로 세 줄짜리 주문서를 출력하세요. `console.log` 는 한 번만 쓰세요. 힌트: 백틱 안에서 엔터를 치면 줄이 바뀝니다.

```
const menu16 = "아메리카노";
const price16 = 4500;
const count16 = 2;
```

기대 출력 === 주문서 ===
아메리카노 x 2
합계 9000원

여기에 코드를 쓰세요

문제 17 (에러 확인 - 맨 마지막)

아래 두 줄의 주석을 풀고 실행해서 어떤 에러가 나는지 확인하세요. 확인했으면 반드시 다시 주석 처리하세요.

```
const clickCount = 0;
clickCount++;
```

무슨 에러가 났나요? ++ 는 결국 무엇을 하는 것이길래 에러가 날까요?

답

연산자

RUN node 심화문제.js

— 이걸 선택 과제입니다

연습문제를 다 푼 사람만 하세요. 안 풀어도 다음 단원에 지장 없습니다. 답이 바로 안 보이는 게 정상입니다. 개념 파일을 다시 뒤져 보세요.

심화1 소수 오차를 실무에서 어떻게 넘기나

개념01에서 $0.1 + 0.2$ 가 0.3 이 아니라고 배웠습니다. 그럼 “두 소수를 더한 값이 0.3 인가” 를 어떻게 확인해야 할까요?

- 1 $0.1 + 0.2$ 의 값
- 2 그것이 0.3 과 `===` 로 같은지
- 3 100 을 곱해 반올림한 뒤 다시 100 으로 나눈 값
- 4 그 값이 0.3 과 `===` 로 같은지

힌트: 소수를 정수로 만들어 계산하고 다시 되돌리는 것이 실무에서 쓰는 방법입니다.

돈 계산을 '원' 단위 정수로 하는 이유가 이것입니다.

기대 출력	<code>0.30000000000000004</code>
	<code>false</code>
	<code>0.3</code>
	<code>true</code>

여기에 코드를 쓰세요

심화 2 == 는 왜 위험한가

개념03에서 “== 대신 === 를 쓰라” 고 배웠습니다. 왜 그런지 직접 보세요. 아래 다섯 가지를 순서대로 출력하세요.

1 0 == ""

2 0 === ""

3 null == undefined

4 null === undefined

5 NaN === NaN

5번은 특히 이상합니다. 자기 자신과도 같지 않습니다. 왜 그럴까요?

```
기대 출력  true
           false
           true
           false
           false
```

여기에 코드를 쓰세요

심화 3 || 와 ?? 는 언제 갈리나

개념04에서 둘 다 “기본값 채우기” 라고 배웠습니다. 결과가 갈리는 경우를 찾으세요.

아래 두 값에 각각 || 와 ?? 를 써서 네 줄을 출력하세요. count 는 0, nickname 은 빈 문자열입니다.

대괄호로 감싸서 출력하세요. 빈 문자열이 나오면 눈에 안 보이니까요. 예: `console.log([`${...}`])`

```
const count = 0;
const nickname = "";
```

기대 출력

[100] [0]

— 손님

[]

여기에 코드를 쓰세요

(순서: count||100, count??100, nickname||"손님", nickname??"손님")

02단원 마무리 연산자

자주 넘어지는 자리

1 $0.1 + 0.2$, 그리고 `toFixed()` 결과가 문자열이라는 것

자주 나오는 질문

Q **"**(제공)는 언제 써요?"**

A 넓이·복리 계산. %(나머지)는 짝수 판별·나누어 담기에 계속 씁니다

Q **"Math.round랑 toFixed 중 뭐요?"**

A 계산에 다시 쓸 값은 round(숫자), 화면에만 보여 줄 값은 toFixed(문자열)

조건문

OPEN 03_조건문

```
const age = 22;  
if (age >= 19) {  
  console.log("성인입니다");  
}  
const age2 = 15;
```

01 if 문: 만약 ~라면

02 else 와 else if: 갈림길 만들기

03 조건 조합하기

04 switch 문

05 삼항 연산자

- 연습문제

- 심화문제

if 문: 만약 ~라면

RUN node 개념01_if문.js

지금까지 코드는 위에서 아래로 무조건 전부 실행됐습니다. 이제 “조건이 맞을 때만 실행”을 배웁니다.

```
if (조건) {
  조건이 true 일 때 실행할 코드
}
^^ ^^ ^
|   |   ↳ 중괄호로 묶은 '실행할 덩어리'
|   ↳ true 나 false 가 나오는 식 (02단원 비교·논리 연산자)
↳ 만약
```

가장 기본 형태

섹션 1

```
const age = 22;

if (age >= 19) {
  console.log("성인입니다");
}
```

출력 성인입니다

조건이 `false` 면 중괄호 안이 통째로 건너뛰어집니다.

```
const age2 = 15;

if (age2 >= 19) {
  console.log("이 줄은 실행되지 않습니다");
}
```

(아무것도 출력되지 않습니다)

if 가 끝난 다음 줄은 조건과 상관없이 실행됩니다.

```
console.log("검사 끝");
```

출력 검사 끝

▹ 직접 해보기 1

점수가 90점 이상이면 "우수" 를 출력하는 if 문을 써 보세요. (점수는 95)

중괄호와 들여쓰기

섹션 2

중괄호 { } 는 "여기서부터 여기까지가 한 덩어리"라는 표시입니다. 안에 여러 줄을 넣을 수 있습니다.

```
const stock = 0;

if (stock === 0) {
  console.log("품절되었습니다");
  console.log("재입고를 기다려 주세요");
  console.log("알림 신청이 가능합니다");
}
```

출력	품절되었습니다
	재입고를 기다려 주세요
	알림 신청이 가능합니다

중괄호 안의 코드는 스페이스 두 칸(또는 탭)만큼 들여쓰입니다. 들여쓰기는 컴퓨터에겐 상관없지만 사람에게겐 아주 중요합니다. "어디까지가 if 안인지"를 눈으로 보여 주기 때문입니다.

실행할 줄이 하나면 중괄호를 생략할 수도 있습니다.

```
if (stock === 0) console.log("품절");
```

출력	품절
----	----

하지만 생략하지 마세요. 나중에 줄을 하나 추가할 때 사고가 납니다. (섹션 5 실수 1에서 확인합니다)

▹ 직접 해보기 2

잔액이 0이면 "잔액이 부족합니다" 와 "충전해 주세요" 를 두 줄로 출력하는 if 문을 써 보세요. (잔액은 0)

조건 자리에 들어갈 수 있는 것

섹션 3

if 의 괄호 안에는 true / false 가 나오는 식이면 뭐든 들어갑니다.

— 비교식

```
const temperature = 31;
if (temperature > 30) {
  console.log("폭염 주의보");
}
```

출력 폭염 주의보

— 논리식

```
const isMember = true;
const totalPrice = 60000;
if (isMember && totalPrice >= 50000) {
  console.log("무료 배송");
}
```

출력 무료 배송

— 불리언 변수 그대로

```
const isLoggedIn = true;
if (isLoggedIn) {
  console.log("환영합니다");
}
```

출력 환영합니다

`isLoggedIn === true` 라고 써도 되지만 굳이 그럴 필요 없습니다. 이미 `true` / `false` 인 값이니깐요.

▸ 직접 해보기 3

`isOpen` 이 `true` 일 때 “영업 중” 을 출력해 보세요.

조건에 불리언이 아닌 값이 오면

섹션 4

조건 자리에 숫자나 문자열을 넣으면 자동으로 `true`/`false` 로 바뀝니다. 01단원에서 배운 falsy 6개(0, "", `null`, `undefined`, `NaN`, `false`)면 `false`, 나머지는 전부 `true` 입니다.

```
const userName = "김민준";
if (userName) {
  console.log("이름이 있습니다");
}
```

출력 이름이 있습니다

```
const emptyName = "";
if (emptyName) {
  console.log("이 줄은 실행되지 않습니다");
}
```

(빈 문자열은 falsy 라서 건너뛰니다)

이 성질을 이용해 “값이 비었는지” 를 짧게 검사합니다.

```
const memo = null;
if (!memo) {
  console.log("메모가 없습니다");
}
```

출력 메모가 없습니다

주의 · 숫자 0 도 falsy 입니다.

```
const count = 0;
if (count) {
  console.log("이 줄은 실행되지 않습니다");
}
```

재고가 진짜 0개일 때도 “값이 없다”로 취급되어 버립니다. 숫자를 검사할 때는 뜻을 분명히 쓰세요.

```
if (count === 0) {
  console.log("재고가 0개입니다");
}
```

출력 재고가 0개입니다

▸ 직접 해보기

4

nickname 이 빈 문자열일 때 “닉네임을 입력하세요” 를 출력해 보세요.

자주 하는 실수

섹션 5

이런 실수를 합니다

중괄호를 생략했다가 줄을 추가

```
const isVip = false;
if (isVip) console.log("VIP 인사");
console.log("이 줄은 if 와 상관없이 항상 실행됩니다");
```

출력 이 줄은 if 와 상관없이 항상 실행됩니다

들여쓰기를 해 봐도 소용없습니다. 중괄호가 없으면 if 는 바로 다음 한 줄에만 걸립니다. 그래서 중괄호는 항상 쓰는 게 안전합니다.

이런 실수를 합니다

= 와 === 혼동

```
let level = 1;
```

level 에 5를 넣어 버리고, 5는 truthy 라서 항상 실행됩니다. 비교는 === 입니다.

```
if (level === 5) {  
  console.log("레벨 5");  
}
```

(level 은 1이라 실행되지 않습니다)

```
console.log(level);
```

출력 1

이런 실수를 합니다

if 뒤에 세미콜론 붙이기

```
const price = 100;  
if (price > 1000); // ← 여기 세미콜론  
{  
  console.log("이 줄은 조건과 상관없이 실행됩니다");  
}
```

출력 이 줄은 조건과 상관없이 실행됩니다

세미콜론이 "if 문 끝"이 되어 버려서, 아래 종괄호는 if 와 무관한 그냥 덩어리가 됩니다. if 뒤에 세미콜론을 찍지 마세요.

이런 실수를 합니다

범위를 수학처럼 쓰기 (02단원에서 배웠죠)

```
const score = 100;  
if (10 < score < 30) {  
  console.log("범위 안이라고 잘못 판단합니다");  
}
```

출력 범위 안이라고 잘못 판단합니다

100은 30보다 큰데도 실행됩니다. score > 10 && score < 30 이라고 써야 합니다.

정리

- 1 **if** (조건) { 실행할 코드 } — 조건이 true 일 때만 실행
- 2 중괄호는 생략하지 말 것. 나중에 줄을 추가할 때 사고가 난다.
- 3 조건 자리에는 비교식·논리식·불리언 변수가 들어간다.
- 4 불리언이 아닌 값은 falsy 6개면 **false**, 나머지는 **true**.
- 5 if 뒤에 세미콜론을 찍지 말 것.

직접 해보기 정답

```
1) const myScore = 95;
   if (myScore >= 90) {
```

```
   console.log("우수");
```

```
   }
   // 출력: 우수
```

```
2) const balance = 0;
   if (balance === 0) {
```

```
   console.log("잔액이 부족합니다");
   console.log("충전해 주세요");
```

```
   }
```

```
3) const isOpen = true;
   if (isOpen) {
```

```
   console.log("영업 중");
```

```
   }
```

```
4) const nickname = "";
   if (!nickname) {
```

```
   console.log("닉네임을 입력하세요");
```

```
}  
// nickname === "" 라고 써도 됩니다. 뜻이 더 분명해서 이쪽이 나을 때도 많습니다.
```

else 와 else if: 갈림길 만들기

RUN node 개념02_else와_else_if.js

if 만으로는 “조건이 맞을 때”만 처리할 수 있습니다. “아니면 이렇게” 를 붙이려면 else 가 필요합니다.

```
if (조건) {
  조건이 true 일 때
} else {
  아니면
}
```

else — 아니면

섹션 1

```
const age = 15;

if (age >= 19) {
  console.log("성인입니다");
} else {
  console.log("미성년자입니다");
}
```

출력 미성년자입니다

if 와 else 중 하나는 반드시 실행됩니다. 둘 다 실행되는 일은 없습니다.

else 에는 조건을 쓰지 않습니다. “나머지 전부”라는 뜻이니까요.

```
} else (age < 19) { ← 이렇게 쓰면 SyntaxError
```

```
const stock = 5;
if (stock > 0) {
  console.log("구매 가능");
} else {
  console.log("품절");
}
```

출력 구매 가능

▸ 직접 해보기 1

점수가 60점 이상이면 “합격”, 아니면 “불합격” 을 출력해 보세요. (55점)

else if 를 쓰면 조건을 여러 개 늘어놓을 수 있습니다.

```
const score = 85;

if (score >= 90) {
  console.log("A 등급");
} else if (score >= 80) {
  console.log("B 등급");
} else if (score >= 70) {
  console.log("C 등급");
} else {
  console.log("F 등급");
}
```

출력 B 등급

동작 방식이 중요합니다. 위에서부터 하나씩 검사합니다. 처음으로 true 가 되는 곳을 실행하고, 나머지는 전부 건너뛩니다. 그래서 85점은 "B 등급"만 출력되고 "C 등급"은 나오지 않습니다.

마지막 else 는 생략할 수 있습니다. 그러면 아무것도 안 맞을 때 그냥 넘어갑니다.

```
const temp = 15;
if (temp >= 28) {
  console.log("덥다");
} else if (temp <= 5) {
  console.log("춥다");
}
```

(15도는 어느 조건에도 안 맞아서 아무것도 출력되지 않습니다)

```
console.log("판정 끝");
```

출력 판정 끝

▸ 직접 해보기 2

나이를 "어린이(13 미만) / 청소년(19 미만) / 성인" 을 구분해 출력해 보세요. (나이는 16)

순서가 결과를 바꾼다

else if 는 위에서부터 검사하므로 순서를 잘못 쓰면 틀립니다.

— 틀린 순서

```
const score2 = 95;

if (score2 >= 70) {
  console.log("C 등급"); // 95점인데 여기서 걸려 버립니다
} else if (score2 >= 80) {
  console.log("B 등급");
} else if (score2 >= 90) {
  console.log("A 등급");
}
```

출력 C 등급

95는 70 이상이기도 하므로 첫 조건에서 걸리고 끝나 버립니다.

맞는 순서 · 좁은 조건(큰 수)부터 씁니다.

```
if (score2 >= 90) {
  console.log("A 등급");
} else if (score2 >= 80) {
  console.log("B 등급");
} else if (score2 >= 70) {
  console.log("C 등급");
}
```

출력 A 등급

규칙: 범위를 나눌 때는 조건을 '한 방향으로' 정렬하세요.

큰 수에서 작은 수로 내려오거나, 작은 수에서 큰 수로 올라가거나
한 방향으로 정렬하면 됩니다.

▸ 직접 해보기 3

배송비를 정해 출력해 보세요. (금액 30000) 5만원 이상 무료 / 3만원 이상 1500원 / 그 외 3000원

if 를 여러 개 쓰는 것과 else if 의 차이

섹션 4

이 둘은 완전히 다릅니다. 아주 헷갈리는 지점입니다.

```
const point = 95;
```

— else if — 하나만 실행

```
if (point >= 90) {  
  console.log("else if: 90점 이상");  
} else if (point >= 80) {  
  console.log("else if: 80점 이상");  
}
```

출력 else if: 90점 이상

— if 여러 개 — 맞는 것마다 전부 실행

```
if (point >= 90) {  
  console.log("if: 90점 이상");  
}  
if (point >= 80) {  
  console.log("if: 80점 이상");  
}
```

출력 if: 90점 이상
 if: 80점 이상

언제 무엇을 쓰나? 등급처럼 '하나만 골라야' 하면 → else if 조건마다 '따로따로 처리'해야 하면 → if 여러 개
예를 들어 "90점 이상이면 상장, 개근이면 개근상"은 서로 무관하므로 if 를 따로 씁니다.

▹ 직접 해보기 4

나이가 65 이상이면 "경로 할인", 회원이면 "회원 할인" 을 각각 출력해 보세요. (나이 70, 회원 true — 둘 다 나와야 합니다)

조건이 길어질 때 정리하는 법

섹션 5

조건이 길어지면 변수에 이름을 붙여서 빼세요. 코드가 문장처럼 읽힙니다.

```
const userAge = 20;  
const hasTicket = true;  
const isWeekend = false;
```

— 읽기 나쁜 코드

```
if (userAge >= 19 && hasTicket && !isWeekend) {  
  console.log("입장 가능");  
}
```

출력 입장 가능

— 읽기 좋은 코드

```
const isAdult = userAge >= 19;
const canEnterToday = !isWeekend;

if (isAdult && hasTicket && canEnterToday) {
  console.log("입장 가능 (정리된 버전)");
}
```

출력 입장 가능 (정리된 버전)

▫ 직접 해보기 5

“재고가 있고, 판매중이면” 구매 가능하다는 조건을 변수 두 개로 정리해서 써 보세요. (재고 3, 판매중 true)

자주 하는 실수

섹션 6

★ 아래에서 SyntaxError 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. 다른 에러 (ReferenceError, TypeError)는 주석을 풀어도 그 줄에서만 나고 그 앞의 출력은 그대로 다 나옵니다. 그런데 SyntaxError 는 다릅니다. 자바스크립트가 파일을 아예 못 읽어서 출력이 한 줄도 안 나옵니다. 여러분이 망가뜨린 것이 아닙니다. 실수로 풀었다면 다시 // 를 붙이면 그대로 돌아옵니다.

이런 실수를 합니다

else 에 조건 붙이기 → SyntaxError

```
if (a > 1) { } else (a < 1) { }
```

조건을 붙이려면 else if 입니다.

이런 실수를 합니다

else if 를 elseif 로 붙여 쓰기 → SyntaxError

```
}elseif (score >= 80) {
```

else 와 if 사이에 반드시 띄어쓰기가 필요합니다.

이런 실수를 합니다

범위 순서를 거꾸로 (섹션 3에서 봤습니다)

이런 실수를 합니다

중괄호 짝을 놓치기

중괄호가 많아지면 짝이 안 맞아 SyntaxError 가 자주 납니다. VS Code 에서 여는 중괄호에 커서를 두면 짝이 되는 닫는 중괄호가 표시됩니다. 그리고 저장할 때마다 자동 정렬(Shift + Alt + F)을 하면 들여쓰기가 어긋난 곳이 눈에 보여서 짝이 안 맞는 걸 금방 찾을 수 있습니다.

정리

- 1 if / else — 둘 중 하나만 실행된다.
- 2 else if — 위에서부터 검사해 처음 맞는 하나만 실행하고 끝난다.
- 3 범위를 나눌 때는 좁은 조건(큰 수)부터 위에 쓴다.
- 4 else if 는 하나만, if 여러 개는 맞는 것 전부 실행된다.
- 5 조건이 길면 변수로 빼서 이름을 붙인다.

직접 해보기 정답

```
1) const s = 55;
if (s >= 60) {

  console.log("합격");

} else {

  console.log("불합격");

}
// 출력: 불합격
```

```
2) const a = 16;
if (a < 13) {

  console.log("어린이");

} else if (a < 19) {

  console.log("청소년");

} else {
```



```
console.log("성인");
```

```
}
// 출력: 청소년
// 작은 수부터 올라가는 순서입니다. 이것도 한 방향 정렬이라 맞습니다.
```

```
3) const total = 30000;
if (total >= 50000) {
```

```
console.log("배송비 0원");
```

```
} else if (total >= 30000) {
```

```
console.log("배송비 1500원");
```

```
} else {
```

```
console.log("배송비 3000원");
```

```
}
// 출력: 배송비 1500원
```

```
4) const myAge = 70;
const member = true;
if (myAge >= 65) {
```

```
console.log("경로 할인");
```

```
}
if (member) {
```

```
console.log("회원 할인");
```

```
}
// 출력: 경로 할인
// 출력: 회원 할인
// else if 로 쓰면 "경로 할인" 하나만 나옵니다.
```

```
5) const stockCount = 3;
const onSale = true;
const hasStock = stockCount > 0;
if (hasStock && onSale) {
```

```
console.log("구매 가능");
```

```
}  
// 출력: 구매 가능
```

조건 조합하기

RUN node 개념03_조건_조합하기.js

실제 프로그램의 조건은 하나로 끝나지 않습니다.

회원이면서 5만원 이상 샀으면 무료배송
주말이거나 공휴일이면 휴무
로그인했고, 그중에서도 관리자면 관리 메뉴 표시

if 안에 if 를 넣는 방법과, && || 로 퍼는 방법을 비교해 봅니다.

중첩 if — if 안의 if

섹션 1

```
const isLoggedIn = true;
const isAdmin = true;

if (isLoggedIn) {
  console.log("환영합니다");

  if (isAdmin) {
    console.log("관리 메뉴를 표시합니다");
  }
}
```

출력 환영합니다
 관리 메뉴를 표시합니다

바깥 조건이 false 면 안쪽은 아예 검사하지 않습니다.

```
const isLoggedIn2 = false;

if (isLoggedIn2) {
  console.log("이 줄도");
  if (isAdmin) {
    console.log("이 줄도 실행되지 않습니다");
  }
}
```

(아무것도 출력되지 않습니다)

중첩은 “바깥 조건일 때만 안쪽을 따진다”는 뜻이 분명해서 좋습니다. 다만 깊어지면 읽기 힘듭니다. 두 단계까지만 쓰세요.

▫ 직접 해보기 1

회원이고(true) 포인트가 1000 이상이면(1500) "포인트 사용 가능" 을 중첩 if 로 출력해 보세요.

&& 로 펴기

섹션 2

안쪽에서만 뭔가 한다면 중첩할 필요 없이 && 로 이으면 됩니다.

— 중첩 버전

```
const hasStock = true;
const onSale = true;

if (hasStock) {
  if (onSale) {
    console.log("중첩: 구매 가능");
  }
}
```

출력 중첩: 구매 가능

펼친 버전 · — 위와 완전히 같은 뜻입니다

```
if (hasStock && onSale) {
  console.log("&&: 구매 가능");
}
```

출력 &&: 구매 가능

언제 무엇을 쓰나? 바깥 조건일 때 따로 할 일이 있다 → 중첩 (섹션 1의 "환영합니다") 안쪽에서만 할 일이 있다 → && 로 펴기

▫ 직접 해보기 2

위 직접 해보기 1을 && 로 펴서 다시 써 보세요.

|| 로 여러 값 중 하나 검사

섹션 3

```
const day = "일";

if (day === "토" || day === "일") {
  console.log("주말입니다");
}
```

출력 주말입니다

02단원에서 배운 실수를 다시 확인합니다. 비교는 양쪽에 다 써야 합니다. `if (day === "토" || "일")` ← 항상 실행됩니다. 틀린 코드입니다.

값이 많아지면 `||` 가 길어집니다. 이럴 땐 06단원의 배열을 쓰면 짧아집니다. 지금은 `||` 로 충분합니다.

```
const grade = "B";
if (grade === "A" || grade === "B" || grade === "C") {
  console.log("합격 등급");
}
```

출력 합격 등급

▹ 직접 해보기

3

결제수단이 "카드" 이거나 "간편결제" 면 "수수료 없음" 을 출력해 보세요. (결제수단은 "간편결제")

조건 순서로 에러를 막는다

섹션 4

`&&` 는 왼쪽이 `false` 면 오른쪽을 아예 계산하지 않습니다. (단축 평가) 이 성질로 에러를 막을 수 있습니다.

```
const memo = null;
```

위험한 코드 · `memo` 가 `null` 인데 `.length` 를 꺼내려 하면 에러

이런 실수를 합니다

```
if (memo.length > 0) { }
```

```
TypeError: Cannot read properties of null (reading 'length')
```

안전한 코드 · 먼저 값이 있는지 확인하고, 있을 때만 `.length` 를 본다

```
if (memo && memo.length > 0) {
  console.log("메모가 있습니다");
} else {
  console.log("메모가 없습니다");
}
```

출력 메모가 없습니다

`memo` 가 `null` 이라 왼쪽에서 `false` 가 되고, 오른쪽은 실행조차 안 됩니다. 그래서 에러가 나지 않습니다.

```
const memo2 = "장보기";
if (memo2 && memo2.length > 0) {
  console.log("메모가 있습니다:", memo2);
}
```

출력 메모가 있습니다: 장보기

이 “먼저 있는지 확인” 패턴은 앞으로 계속 나옵니다. 꼭 익혀 두세요.

▸ 직접 해보기

4

nickname 이 `null` 일 때 에러 없이 “닉네임 없음” 을 출력해 보세요.

값이 비었을 때 기본값 채우기

섹션 5

if 로 쓰면 이렇습니다.

```
const inputName = "";
let displayName;

if (inputName) {
  displayName = inputName;
} else {
  displayName = "손님";
}
console.log(displayName);
```

출력 손님

|| 를 쓰면 한 줄로 끝납니다. (02단원 개념04)

```
const displayName2 = inputName || "손님";
console.log(displayName2);
```

출력 손님

숫자 0 을 살려야 하면 ?? 를 씁니다. (?? 도 02단원 개념04에서 배웠습니다)

```
const pointValue = 0;
console.log(pointValue || 100);
```

출력 100

```
console.log(pointValue ?? 100);
```

출력 0

➤ 직접 해보기 5

profileImage 가 빈 문자열일 때 “기본이미지.png” 가 나오도록 한 줄로 써 보세요.

조건을 뒤집어서 간단하게

섹션 6

조건이 복잡하면 뒤집는 게 더 읽기 쉬울 때가 있습니다.

```
const age = 15;
```

— 원래

```
if (age >= 19) {
  console.log("입장 가능");
} else {
  console.log("입장 불가");
}
```

출력 입장 불가

뒤집기 · — 걸러낼 것을 먼저 처리

```
if (age < 19) {
  console.log("입장 불가 (뒤집은 버전)");
} else {
  console.log("입장 가능 (뒤집은 버전)");
}
```

출력 입장 불가 (뒤집은 버전)

어느 쪽이든 맞습니다. “읽었을 때 자연스러운 쪽”을 고르세요. 보통은 ‘정상 경우’를 if 에, ‘예외’를 else 에 두는 편이 읽기 좋습니다.

자주 하는 실수

섹션 7

이런 실수를 합니다
|| 로 비교를 한쪽만 쓰기 (섹션 3에서 봤습니다)

이런 실수를 합니다
&& 를 써야 하는데 || 를 씀

```
const isMember = false;
const isVip = false;
if (isMember || isVip) {
  console.log("할인 대상");
} else {
  console.log("할인 없음");
}
```

출력 할인 없음

“회원이면서 VIP” 를 뜻하려 했다면 && 여야 합니다. 말로 먼저 읽어 보세요. “~이고” 면 && , “~이거나” 면 || 입니다.

이런 실수를 합니다

값 검사 순서를 반대로 쓰기

```
if (memo.length > 0 && memo) { }
```

순서가 반대라 여전히 에러가 납니다. 있는지 먼저 확인해야 합니다.

이런 실수를 합니다

중첩을 너무 깊게

```
if (a) { if (b) { if (c) { if (d) { ... } } } }
```

세 단계만 넘어가도 읽을 수 없습니다. && 로 퍼거나, 걸러낼 것을 먼저 처리해서 깊이를 줄이세요.

정리

- 1 중첩 if 는 “바깥 조건일 때만 안쪽을 따진다”. 두 단계까지만.
- 2 안쪽에서만 할 일이 있으면 && 로 퍼라.
- 3 여러 값 중 하나면 || . 비교는 양쪽에 다 쓴다.
- 4 값 && 값.length — 값이 있는지 먼저 확인해서 에러를 막는 패턴
- 5 값 || 기본값 / 값 ?? 기본값
- 6 “~이고” 면 && , “~이거나” 면 || . 말로 읽어 보고 고른다.

직접 해보기 정답

```
1) const member = true;
const point = 1500;
if (member) {
```



```
if (point >= 1000) {
  console.log("포인트 사용 가능");
}
```

```
}
// 출력: 포인트 사용 가능
```

```
2) if (member && point >= 1000) {
```

```
  console.log("포인트 사용 가능");
```

```
}
```

```
3) const pay = "간편결제";
if (pay === "카드" || pay === "간편결제") {
```

```
  console.log("수수료 없음");
```

```
}
// 출력: 수수료 없음
```

```
4) const nickname = null;
if (nickname && nickname.length > 0) {
```

```
  console.log(nickname);
```

```
} else {
```

```
  console.log("닉네임 없음");
```

```
}
// 출력: 닉네임 없음
```

```
5) const profileImage = "";
console.log(profileImage || "기본이미지.png");
// 출력: 기본이미지.png
```

switch 문

RUN node 개념04_switch문.js

“값이 A면 이거, B면 저거, C면 그거” 처럼 하나의 값을 여러 값과 하나씩 맞춰 볼 때 쓰는 문법입니다.

```
switch (검사할값) {
  case 값1:
    실행할 코드;
    break;
  case 값2:
    실행할 코드;
    break;
  default:
    아무것도 안 맞을 때;
}
```

기본 형태

섹션 1

```
const menu = "라떼";

switch (menu) {
  case "아메리카노":
    console.log("4000원입니다");
    break;
  case "라떼":
    console.log("4500원입니다");
    break;
  case "케이크":
    console.log("6000원입니다");
    break;
  default:
    console.log("메뉴에 없습니다");
}
```

출력 4500원입니다

읽는 법 switch 괄호 안의 값을 위에서부터 case 값과 하나씩 비교합니다. 같은 case 를 만나면 거기서부터 실행합니다. break 를 만나면 switch 를 빠져나옵니다. 끝까지 못 만나면 default 를 실행합니다.

▹ 직접 해보기 1

요일이 "월" 이면 "한 주 시작", "금" 이면 "불금", 그 외엔 "평범한 날" 을 출력하는 switch 를 써 보세요.
(요일은 "금")

break 를 빠뜨리면 아래로 흘러내린다

섹션 2

break 가 없으면 다음 case 로 그냥 넘어가서 계속 실행됩니다. 이것 fall-through(흘러내림)라고 합니다.

```
const grade = "B";

switch (grade) {
  case "A":
    console.log("A 실행");
  case "B":
    console.log("B 실행");
  case "C":
    console.log("C 실행");
  default:
    console.log("default 실행");
}
```

```
출력      B 실행
          C 실행
          default 실행
```

B 에서 시작했지만 break 가 없어서 아래까지 전부 실행됐습니다. 대부분 이건 실수입니다. case 마다 break 를 꼭 쓰세요.

▹ 직접 해보기 2

위 코드에 break 를 넣어 "B 실행" 만 나오게 고쳐 보세요.

흘러내림을 일부러 쓰기 — 여러 case 묶기

섹션 3

여러 값을 똑같이 처리하고 싶을 때는 일부러 break 를 생략합니다.

```
const day = "토";

switch (day) {
  case "토":
  case "일":
    console.log("주말입니다");
    break;
  case "월":
```

삼항 연산자

RUN node 개념05_삼항연산자.js

if / else 를 한 줄로 줄여 쓰는 문법입니다.

```
조건 ? 참일때값 : 거짓일때값
   ^         ^
   물음표     콜론
```

“조건이 참이야? 그럼 앞의 값, 아니면 뒤의 값”

기본 형태

섹션 1

```
const age = 22;
```

— if 버전

```
if (age >= 19) {
  console.log("성인");
} else {
  console.log("미성년자");
}
```

출력 성인

삼항 버전 · — 위와 완전히 같은 뜻입니다

```
console.log(age >= 19 ? "성인" : "미성년자");
```

출력 성인

```
const age2 = 15;
console.log(age2 >= 19 ? "성인" : "미성년자");
```

출력 미성년자

▸ 직접 해보기 1

점수 55점이 60 이상인지로 “합격”/“불합격” 을 삼항 연산자로 출력해 보세요.

값을 골라 변수에 담기

섹션 2

삼항 연산자의 진짜 쓸모는 여기입니다. '값'이라서 변수에 담을 수 있습니다.

```
const stock = 0;
```

if 버전 · — 변수를 먼저 만들고 나중에 채워야 해서 `let` 이 됩니다

```
let statusText;
if (stock > 0) {
  statusText = "구매 가능";
} else {
  statusText = "품절";
}
console.log(statusText);
```

출력 품절

삼항 버전 · — 만들면서 바로 채우니 `const` 로 쓸 수 있습니다

```
const statusText2 = stock > 0 ? "구매 가능" : "품절";
console.log(statusText2);
```

출력 품절

`const` 로 쓸 수 있다는 게 생각보다 큰 장점입니다. 중간에 값이 바뀔 걱정을 안 해도 되니까요.

숫자를 고를 수도 있습니다.

```
const isMember = true;
const shippingFee = isMember ? 0 : 3000;
console.log(shippingFee);
```

출력 0

▸ 직접 해보기 2

나이 15로 `ticketPrice` 를 정해 출력해 보세요. (19세 이상이면 12000, 아니면 8000)

템플릿 리터럴 안에서 쓰기

섹션 3

`${ }` 안에는 '값이 나오는 식'이 들어갑니다. 삼항 연산자가 딱 맞습니다. (if 는 문장이라 `${ }` 안에 넣을 수 없습니다)

```
const userName = "김민준";
const point = 1500;

console.log(`${userName}님은 ${point >= 1000 ? "VIP" : "일반"} 회원입니다`);
```

출력 김민준님은 VIP 회원입니다

```
const count = 1;
console.log(`장바구니에 ${count}개${count > 1 ? "들" : ""} 담겼습니다`);
```

출력 장바구니에 1개 담겼습니다

```
const count2 = 3;
console.log(`장바구니에 ${count2}개${count2 > 1 ? "들" : ""} 담겼습니다`);
```

출력 장바구니에 3개들 담겼습니다

이 패턴은 나중에 화면에 글자를 찍을 때(10단원 DOM) 아주 많이 씁니다.

▹ 직접 해보기

3

재고가 0이면 "품절", 아니면 "재고 N개" 가 나오도록 백틱 안에서 삼항 연산자를 써 보세요. (재고 5)

언제 삼항을 쓰고 언제 if 를 쓸까

섹션 4

삼항이 좋을 때 · 둘 중 하나의 '값'을 고를 때

```
const temperature = 31;
const weatherText = temperature >= 28 ? "덥다" : "괜찮다";
console.log(weatherText);
```

출력 덥다

if 가 좋을 때 · 실행할 '동작'이 여러 줄일 때

```
if (temperature >= 28) {
  console.log("에어컨을 켵니다");
  console.log("창문을 닫습니다");
  console.log("선풍기도 켵니다");
}
```

출력 에어컨을 켵니다
창문을 닫습니다
선풍기도 켵니다

한 줄로 쓸 수 있다고 항상 좋은 게 아닙니다. 기준: "값을 고르는가(삼항), 일을 시키는가(if)"

▹ 직접 해보기 4

아래 if 문을 삼항으로 바꿔 보세요. `if (isRaining) { umbrella = "필요"; } else { umbrella = "불필요"; }`

중첩 삼항 — 되지만 하지 마세요

섹션 5

```
const score = 85;
```

삼항 안에 삼항을 넣을 수 있습니다.

```
const grade = score >= 90 ? "A" : score >= 80 ? "B" : score >= 70 ? "C" : "F";
console.log(grade);
```

출력 B

동작은 합니다. 하지만 읽기가 매우 나쁩니다. 콜론이 어디에 걸리는지 눈으로 따라가기 어렵고, 조건이 하나만 늘어도 무너집니다.

갈림길이 셋 이상이면 else if 를 쓰세요.

```
let grade2;
if (score >= 90) {
  grade2 = "A";
} else if (score >= 80) {
  grade2 = "B";
} else if (score >= 70) {
  grade2 = "C";
} else {
  grade2 = "F";
}
console.log(grade2);
```

출력 B

규칙: 삼항은 갈림길이 '둘'일 때만.

자주 하는 실수

섹션 6

★ 아래에서 `SyntaxError` 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. 다른 에러 (`ReferenceError`, `TypeError`)는 주석을 풀어도 그 줄에서만 나고 그 앞의 출력은 그대로 다 나옵니다. 그런데 `SyntaxError` 는 다릅니다. 자바스크립트가 파일을 아예 못 읽어서 출력이 한 줄도 안 나옵니다. 여러분이 망가뜨린 것이 아닙니다. 실수로 풀었다면 다시 // 를 붙이면 그대로 돌아옵니다.

이런 실수를 합니다

콜론을 빠뜨리기 → **SyntaxError**

```
const x = age >= 19 ? "성인";
```

삼항은 ? 와 : 가 반드시 짝입니다. "아니면"을 생략할 수 없습니다. 거짓일 때 아무것도 안 하고 싶다면 if 를 쓰세요.

이런 실수를 합니다

삼항 안에서 **console.log** 를 실행하기

```
const isVip = true;
isVip ? console.log("VIP 인사") : console.log("일반 인사");
```

출력 VIP 인사

문법 오류는 아니고 동작도 합니다. 하지만 삼항은 '값을 고르는' 문법입니다. 동작을 시킬 거면 if / else 로 쓰세요. 이렇게 쓰면 읽는 사람이 헷갈립니다.

이런 실수를 합니다

조건 자리에 대입을 쓰기

```
let level = 1;
```

level 에 5를 넣어 버립니다. 비교는 === 입니다.

```
console.log(level);
```

출력 1

이런 실수를 합니다

중첩 삼항을 한 줄에 길게 (섹션 5에서 봤습니다)

정리

- 1 조건 ? 참일때값 : 거짓일때값
- 2 '값'이라서 변수에 담을 수 있고 **const** 로 쓸 수 있다.
- 3 템플릿 리터럴 `${ }` 안에는 if 를 못 넣지만 삼항은 넣을 수 있다.
- 4 값을 고르면 삼항, 일을 시키면 if.
- 5 갈림길이 셋 이상이면 else if. 중첩 삼항은 쓰지 말 것.

직접 해보기 정답

- 1

```
const s = 55;
console.log(s >= 60 ? "합격" : "불합격");
// 출력: 불합격
```
- 2

```
const a = 15;
const ticketPrice = a >= 19 ? 12000 : 8000;
console.log(ticketPrice);
// 출력: 8000
```
- 3

```
const stockCount = 5;
console.log(`${stockCount === 0 ? "품절" : `재고 ${stockCount}개`}`);
// 출력: 재고 5개
// 백틱 안에 백틱을 또 쓸 수 있습니다. 헛갈리면 이렇게 나눠 쓰세요.
// const text = stockCount === 0 ? "품절" : `재고 ${stockCount}개`;
// console.log(text);
```
- 4

```
umbrella = isRaining ? "필요" : "불필요";
```

조건문

RUN node 연습문제.js

TODO 자리에 코드를 쓰고, '기대 출력'과 같은지 확인하세요. 1~15는 기본, 16은 응용, 17은 [도전], 18은 에러 확인입니다.

문제 1 (개념01 if)

기온이 33도 이상이면 "폭염 주의보"를 출력하세요. ('이상'은 그 값을 포함합니다. 33도도 폭염 주의보입니다 → $\geq$)

```
const temp1 = 33;
```

기대 출력 폭염 주의보

여기에 코드를 쓰세요

문제 2 (개념01 if)

잔액이 0이면 두 줄을 출력하세요.

```
const balance1 = 0;
```

기대 출력 잔액이 부족합니다
충전해 주세요

여기에 코드를 쓰세요

문제 3 (개념01 falsy)

메모가 비어 있으면 "메모 없음" 을 출력하세요. 힌트: 빈 문자열은 falsy 입니다.

```
const memo1 = "";
```

기대 출력 메모 없음

여기에 코드를 쓰세요

문제 4 (개념02 else)

나이가 19세 이상이면 "성인", 아니면 "미성년자" 를 출력하세요.

```
const age4 = 15;
```

기대 출력 미성년자

여기에 코드를 쓰세요

문제 5 (개념02 else if)

점수로 등급을 출력하세요. 90 이상 A / 80 이상 B / 70 이상 C / 그 외 F

```
const score5 = 72;
```

기대 출력 C

여기에 코드를 쓰세요

문제 6 (개념02 else if)

주문 금액으로 배송비를 출력하세요. 5만원 이상 → 배송비 0원 3만원 이상 → 배송비 1500원 그 외 → 배송비 3000원

const total6 = 30000;

기대 출력 배송비 1500원

여기에 코드를 쓰세요

문제 7 (개념02 if 여러 개)

65세 이상이면 "경로 할인", 회원이면 "회원 할인" 을 출력하세요. 둘 다 해당하면 둘 다 나와야 합니다. (else if 를 쓰면 안 됩니다)

const age7 = 70;
const member7 = true;

기대 출력 경로 할인
 회원 할인

여기에 코드를 쓰세요

문제 8 (개념03 &&)

회원이면서 5만원 이상 샀으면 "무료 배송" 을 출력하세요.

```
const member8 = true;
const total8 = 60000;
```

기대 출력 무료 배송

여기에 코드를 쓰세요

문제 9 (개념03 ||)

요일이 "토" 이거나 "일" 이면 "주말입니다" 를 출력하세요.

```
const day9 = "일";
```

기대 출력 주말입니다

여기에 코드를 쓰세요

문제 10 (개념03 안전 검사)

닉네임이 있으면 닉네임을, 없으면 "닉네임 없음" 을 출력하세요. nickname10 은 null 입니다. .length 를 그냥 꺼내면 에러가 나니 주의하세요.

```
const nickname10 = null;
```

기대 출력 닉네임 없음

여기에 코드를 쓰세요

문제 11 (개념03 기본값)

프로필 이미지가 비어 있으면 "기본이미지.png" 가 나오도록 한 줄로 출력하세요.

```
const profile11 = "";
```

기대 출력 기본이미지.png

여기에 코드를 쓰세요

문제 12 (개념04 switch)

메뉴에 따라 가격을 출력하는 switch 를 쓰세요. 아메리카노 4000원 / 라떼 4500원 / 케이크 6000원 / 그 외 "메뉴에 없습니다"

```
const menu12 = "케이크";
```

기대 출력 6000원

여기에 코드를 쓰세요

문제 13 (개념04 switch 묶기)

등급이 "A" 나 "B" 면 "합격", "C" 나 "D" 면 "재시험" 을 출력하세요. case 를 묶어서 쓰세요.

`const grade13 = "C";`

기대 출력 재시험

여기에 코드를 쓰세요

문제 14 (개념05 삼항)

19세 이상이면 "성인", 아니면 "미성년자" 를 삼항 연산자로 출력하세요.

`const age14 = 20;`

기대 출력 성인

여기에 코드를 쓰세요

문제 15 (개념05 삼항 + 템플릿)

재고가 0이면 "품절", 아니면 "재고 N개" 가 나오도록 출력하세요.

`const stock15 = 0;`

기대 출력 품절

여기에 코드를 쓰세요

문제 16 [응용]

편의점 계산입니다. 총액 = 가격 × 개수 회원이면 10% 할인 최종 금액을 출력하세요. 소수가 나오면 `Math.round` 로 다듬으세요.

```
const price16 = 1200;  
const count16 = 5;  
const member16 = true;
```

기대 출력 합계 5400원

여기에 코드를 쓰세요

문제 17 [도전]

아래 조건으로 회원 등급을 출력하세요. 회원이고 총구매액이 10만원 이상 → VIP 회원이기만 하면 → 일반회원 회원이 아니면 → 비회원

```
const age17 = 25;  
const member17 = true;  
const total17 = 120000;
```

기대 출력 VIP

여기에 코드를 쓰세요

(age17 은 이 문제에서 쓰지 않습니다. 조건에 필요 없는 값도 섞여 있을 수 있습니다)

문제 18 (에러 확인 - 맨 마지막)

아래 두 줄의 주석을 풀고 실행해서 어떤 에러가 나는지 확인하세요. 확인했으면 반드시 다시 주석 처리하세요.


```
const nullMemo = null;  
console.log(nullMemo.length);
```

무슨 에러가 났나요? 문제 10처럼 에러 없이 처리하려면 어떻게 해야 할까요?

답

조건문

RUN node 심화문제.js

— 이걸 선택 과제입니다

연습문제를 다 푼 사람만 하세요. 안 풀어도 다음 단원에 지장 없습니다.

심화 1 조건 순서가 틀리면 어떻게 되나

아래는 '일부러 순서를 틀리게' 쓴 등급 판정입니다. 먼저 이 코드가 무엇을 출력할지 예상해 보고, 실행해서 확인하세요. 그다음 순서를 고쳐서 올바른 등급이 나오게 하세요.

```
const score1 = 95;
```

이런 실수를 합니다

— 이대로 두고 먼저 실행해 보세요

```
if (score1 >= 60) {
  console.log("틀린 결과: D");
} else if (score1 >= 70) {
  console.log("틀린 결과: C");
} else if (score1 >= 80) {
  console.log("틀린 결과: B");
} else if (score1 >= 90) {
  console.log("틀린 결과: A");
}
```

출력 틀린 결과: D

95점인데 D가 나왔습니다. 왜 그럴까요? 아래에 순서를 고친 코드를 써서 "올바른 결과: A" 가 나오게 하세요.

기대 출력 올바른 결과: A

여기에 코드를 쓰세요

심화 2 switch 흘러내림을 일부러 쓰기

월(1~12)을 받아 계절을 출력하는 코드를 switch 로 만드세요. if 를 쓰지 말고, case 를 묶는 방식(개념04 섹션 3)을 쓰세요.

12, 1, 2 → 겨울 3, 4, 5 → 봄 6, 7, 8 → 여름 9, 10, 11 → 가을

아래 세 값으로 각각 확인하세요. 같은 코드를 세 번 쓰지 말고 month 값만 바꿔 가며 확인해도 되고, 03·04단원을 배운 뒤라면 다르게 해도 됩니다. 여기서는 세 번 써도 괜찮습니다.

```
const month1 = 1;
const month2 = 7;
const month3 = 10;
```

기대 출력 겨울
 여름
 가을

여기에 코드를 쓰세요

심화 3 안전하게 확인하기

아래 세 값이 각각 “쓸 수 있는 글자인가” 를 판정하세요. 조건은 세 가지를 모두 만족해야 합니다. ① 값이 있어야 하고 (null·undefined 가 아니어야 하고) ② 문자열이어야 하고 ③ 공백만 있는 것도 아니어야 합니다

힌트: 개념03의 && 안전 검사와 01단원의 typeof 를 함께 씁니다.

공백만 있는지는 trim() 을 쓰면 되는데, 아직 안 배웠습니다.
대신 " " 와 직접 비교해도 이 문제는 풀립니다.

```
const input1 = "안녕하세요";
const input2 = " ";
const input3 = null;
```

기대 출력 사용 가능
 사용 불가
 사용 불가

여기에 코드를 쓰세요

03단원 마무리 조건문

자주 넘어지는 자리

1

= 와 === 혼동, falsy 값 목록

반복문

OPEN 04_반복문

04

```
console.log("1번 출석");  
console.log("2번 출석");  
console.log("3번 출석");  
for (let i = 1; i <= 3; i++) {
```

01 for 문

02 while 과 do...while

03 break 와 continue

04 중첩 반복문

- 연습문제

- 심화문제

for 문

RUN node 개념01_for문.js

같은 일을 여러 번 해야 할 때 복사·붙여넣기를 하면 안 됩니다. 반복문을 쓰면 몇 번을 하든 코드는 세 줄입니다.

```
for (시작값; 조건; 증감) {
  반복할 코드
}
```

~~~~ ~~~~ ~~~~  
어디서    언제까지    얼마씩

## 왜 반복문인가

섹션 1

반복문 없이 · 출석을 3번 부른다면

```
console.log("1번 출석");
console.log("2번 출석");
console.log("3번 출석");
```

출력            1번 출석  
                 2번 출석  
                 3번 출석

30명이면 30줄을 써야 합니다. 게다가 숫자를 하나씩 손으로 바꿔야 하죠.

## — 반복문으로

```
for (let i = 1; i <= 3; i++) {
  console.log(`${i}번 출석 (for)`);
}
```

출력            1번 출석 (for)  
                 2번 출석 (for)  
                 3번 출석 (for)

30명으로 바꾸려면 3 을 30 으로 고치기만 하면 됩니다.

▸ 직접 해보기 1

for 문으로 "안녕하세요" 를 3번 출력해 보세요.

## for 문의 세 부분

섹션 2

```
for (let i = 0; i < 3; i++) 을 뜯어봅시다.
```

`let i = 0` 시작값. 반복을 세는 변수를 만듭니다. 보통 `i` 라고 짓습니다. `i < 3` 조건. 이게 `true` 인 동안 계속 반복합니다. `i++` 증감. 한 바퀴가 끝날 때마다 실행됩니다.

1 `let i = 0` (딱 한 번)

2 `i < 3` 검사 → `true` 면 중괄호 안 실행

3 `i++`

4 다시 2번으로

5 조건이 `false` 가 되면 끝

```
for (let i = 0; i < 3; i++) {
  console.log("i 값:", i);
}
```

```
출력      i 값: 0
          i 값: 1
          i 값: 2
```

`i` 가 3이 되면 `3 < 3` 이 `false` 라서 멈춥니다. 그래서 0, 1, 2 세 번입니다.

`i` 는 중괄호 안에서만 쓸 수 있습니다. 밖에서 쓰면 에러입니다.

이런 실수를 합니다

```
console.log(i);
```

```
ReferenceError: i is not defined
```

▹ 직접 해보기 2

0부터 4까지 다섯 개의 숫자를 출력해 보세요.

## 0부터? 1부터?

섹션 3

**0부터** ·  $i < \text{개수}$  — 0, 1, 2 ... 프로그래밍의 기본입니다

```
for (let i = 0; i < 3; i++) {
  console.log("0부터:", i);
}
```

```
출력      0부터: 0
          0부터: 1
          0부터: 2
```

**1부터** ·  $i \leq \text{개수}$  — 사람이 세는 방식입니다

```
for (let i = 1; i <= 3; i++) {
  console.log("1부터:", i);
}
```

```
출력      1부터: 1
          1부터: 2
          1부터: 3
```

둘 다 세 번 반복합니다. 짝을 기억하세요. 0부터 시작하면  $<$  (미만) 1부터 시작하면  $\leq$  (이하) 이 짝을  
어기면 한 번 더 돌거나 한 번 덜 돌립니다. (섹션 6 실수 1)

**거꾸로** · 감소시키기

```
for (let i = 3; i >= 1; i--) {
  console.log("거꾸로:", i);
}
```

```
출력      거꾸로: 3
          거꾸로: 2
          거꾸로: 1
```

**2씩** · 증감은  $++$  말고 다른 것도 됩니다

```
for (let i = 0; i <= 6; i += 2) {
  console.log("2씩:", i);
}
```

```
출력      2씩: 0
          2씩: 2
          2씩: 4
          2씩: 6
```



### ▹ 직접 해보기 3

10부터 1까지 거꾸로 세는 for 문을 써 보세요. (출력이 길어지니 5부터 1까지로 해도 됩니다)

## 반복하면서 누적하기

섹션 4

반복문의 가장 흔한 쓰임입니다. 02단원에서 배운 += 를 씁니다. 누적 변수는 반드시 반복문 '밖'에서 만들고 0으로 시작합니다.

```
let sum = 0;

for (let i = 1; i <= 5; i++) {
  sum += i;
}

console.log("1부터 5까지의 합:", sum);
```

출력          1부터 5까지의 합: 15

$1+2+3+4+5 = 15$

중간 과정을 눈으로 보고 싶으면 반복문 안에서 찍어 보세요.

```
let sum2 = 0;
for (let i = 1; i <= 3; i++) {
  sum2 += i;
  console.log(`i=${i} 일 때 sum2=${sum2}`);
}
```

출력          i=1 일 때 sum2=1  
                i=2 일 때 sum2=3  
                i=3 일 때 sum2=6

**중요** · 누적 변수를 반복문 안에서 만들면 매번 초기화되어 버립니다.

```
for (let i = 1; i <= 3; i++) {
  let wrongSum = 0; // ← 매 바퀴 0으로 되돌아갑니다
  wrongSum += i;
  console.log("잘못된 누적:", wrongSum);
}
```

출력          잘못된 누적: 1  
                잘못된 누적: 2  
                잘못된 누적: 3

1, 3, 6 이 나와야 하는데 그냥 i 값이 나왔습니다.

1부터 10까지의 합을 구해 출력해 보세요. (정답은 55)

## 글자를 누적하기

섹션 5

숫자만 누적하는 게 아닙니다. 문자열도 += 로 쌓을 수 있습니다.

```
let stars = "";

for (let i = 1; i <= 5; i++) {
  stars += "*";
}

console.log(stars);
```

출력 \*\*\*\*\*

한 줄씩 늘려 가며 출력하면 이런 모양이 됩니다.

```
let line = "";
for (let i = 1; i <= 4; i++) {
  line += "*";
  console.log(line);
}
```

출력 \*  
\*\*  
\*\*\*  
\*\*\*\*

"-" 를 7개 이어붙인 줄을 만들어 출력해 보세요.

## 자주 하는 실수

섹션 6

★ 아래에서 SyntaxError 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. 다른 에러 (ReferenceError, TypeError)는 주석을 풀어도 그 줄에서만 나고 그 앞의 출력은 그대로 다 나옵니다. 그런데 SyntaxError 는 다릅니다. 자바스크립트가 파일을 아예 못 읽어서 출력이 한 줄도 안 나옵니다. 여러분이 망가뜨린 것이 아닙니다. 실수로 풀었다면 다시 // 를 붙이면 그대로 돌아옵니다.

이런 실수를 합니다

시작값과 조건의 짝이 안 맞음

```
for (let i = 1; i < 3; i++) {
  console.log("한 번 덜 둘:", i);
}
```

출력            한 번 덜 둘: 1  
                  한 번 덜 둘: 2

1부터 3번 돌리려 했는데 두 번만 돌았습니다. 1부터면  $i \leq 3$  이어야 합니다.

이런 실수를 합니다

무한 루프 — 조건이 영원히 true

```
for (let i = 0; i < 3; i - ) { }
```

$i$ 가 계속 줄어들어  $i < 3$ 이 영원히 true입니다. 프로그램이 멈추지 않습니다. 터미널에서 Ctrl + C로 강제 종료하세요. 증감 방향과 조건 방향이 맞는지 항상 확인하세요.

이런 실수를 합니다

세미콜론을 쉼표로 쓰기 → SyntaxError

```
for (let i = 0, i < 3, i++) { }
```

for 괄호 안의 구분자는 세미콜론(;)입니다.

이런 실수를 합니다

for 뒤에 세미콜론

```
for (let i = 0; i < 3; i++);
{
  console.log("이 줄은 한 번만 실행됩니다");
}
```

출력            이 줄은 한 번만 실행됩니다

세미콜론이 "for 문 끝"이 되어서 반복은 빈 채로 3번 돌고, 아래 중괄호는 반복과 무관한 덩어리가 됩니다.

이런 실수를 합니다

반복문 밖에서  $i$  쓰기 (섹션 2에서 봤습니다)

## 정리

- 1 **for** (**let** i = 0; i < 횟수; i++) { 반복할 코드 }
- 2 0부터면 < , 1부터면 <= . 짝을 기억할 것.
- 3 감소는 i-- , 여러 칸씩은 i += 2
- 4 누적 변수는 반복문 '밖'에서 만들고 0(또는 "")으로 시작한다.
- 5 조건이 영원히 **true** 면 무한 루프. Ctrl + C 로 멈춘다.

## 직접 해보기 정답

```
1) for (let i = 0; i < 3; i++) {
```

```
  console.log("안녕하세요");
```

```
}
```

```
2) for (let i = 0; i < 5; i++) {
```

```
  console.log(i);
```

```
}
```

```
// 출력: 0 1 2 3 4 (한 줄씩)
```

```
3) for (let i = 5; i >= 1; i--) {
```

```
  console.log(i);
```

```
}
```

```
// 출력: 5 4 3 2 1 (한 줄씩)
```

```
4) let total = 0;
```

```
for (let i = 1; i <= 10; i++) {
```

```
  total += i;
```

```
}
```

```
console.log(total); // 출력: 55
```

```
5) let dash = "";  
for (let i = 1; i <= 7; i++) {  
  
dash += "-";  
  
}  
console.log(dash);    // 출력: -----
```

# while 과 do...while

RUN node 개념02\_while과\_do\_while.js

for 는 “몇 번 반복할지 아는” 경우에 좋습니다. while 은 “언제 끝날지 모르지만 조건이 맞는 동안” 반복할 때 씁니다.

```
while (조건) {
  반복할 코드
}
```

## while 기본

섹션 1

```
let i = 1;

while (i <= 3) {
  console.log("while:", i);
  i++; // ← 이걸 빠뜨리면 무한 루프입니다
}
```

```
출력      while: 1
          while: 2
          while: 3
```

for 와 비교하면 세 부분이 흩어져 있을 뿐 하는 일은 같습니다.

```
for : for (let i = 1; i <= 3; i++) { ... }
```

^시작값    ^조건    ^증감

```
while : let i = 1;                    ← 시작값은 밖에서
```

```
while (i <= 3) {                    ← 조건만 괄호 안에
  ...
  i++;                              ← 증감은 안에서 직접
}
```

그래서 while 은 증감을 깜빡하기 쉽습니다. 이게 가장 흔한 사고입니다.

## ▫ 직접 해보기

1

while 로 1부터 5까지 출력해 보세요.

## 언제 for 를 쓰고 언제 while 을 쓸까

섹션 2

횟수를 안다 · → for

```
for (let n = 1; n <= 3; n++) {
  console.log("3번 반복:", n);
}
```

```
출력      3번 반복: 1
          3번 반복: 2
          3번 반복: 3
```

횟수를 모른다 · → while

예: "잔액이 0보다 큰 동안 1000원씩 빼기"

```
let balance = 3500;
let usedCount = 0;

while (balance >= 1000) {
  balance -= 1000;
  usedCount++;
  console.log(`${usedCount}번 사용, 남은 잔액 ${balance}원`);
}
```

```
출력      1번 사용, 남은 잔액 2500원
          2번 사용, 남은 잔액 1500원
          3번 사용, 남은 잔액 500원
```

```
console.log("최종 잔액:", balance);
```

```
출력      최종 잔액: 500
```

3500원으로 몇 번 쓸 수 있는지 미리 세지 않아도 됩니다. "조건이 안 맞을 때까지" 알아서 돕니다.

## ▫ 직접 해보기

2

10000원에서 3000원씩 빼면서 남은 돈을 출력해 보세요.

```
let count = 10;

while (count < 5) {
  console.log("이 줄은 실행되지 않습니다");
  count++;
}
```

(`10 < 5` 가 처음부터 `false` 라 한 번도 실행되지 않습니다)

```
console.log("반복 끝, count =", count);
```

```
출력      반복 끝, count = 10
```

## do...while — 일단 한 번은 실행

do...while 은 조건 검사를 '나중에' 합니다. 그래서 조건이 `false` 여도 최소 한 번은 실행됩니다.

```
let j = 10;

do {
  console.log("do...while 은 일단 한 번 실행됩니다. j =", j);
  j++;
} while (j < 5);
```

```
출력      do...while 은 일단 한 번 실행됩니다. j = 10
```

마지막 while 뒤에 세미콜론이 붙는다는 점에 주의하세요.

같은 조건을 while 로 쓰면 아예 실행되지 않습니다.

```
let k = 10;
while (k < 5) {
  console.log("이 줄은 실행되지 않습니다");
  k++;
}
```

정상 동작도 봅시다.

```
let m = 1;
do {
  console.log("do...while 반복:", m);
  m++;
} while (m <= 3);
```



```
출력      do...while 반복: 1
          do...while 반복: 2
          do...while 반복: 3
```

do...while 은 “입력을 일단 한 번 받고, 잘못됐으면 다시 받기” 같은 데 씁니다. 실무에서 자주 쓰이지는 않습니다. “이런 게 있다” 정도로 알아 두세요.

▹ 직접 해보기

3

do...while 로 1부터 3까지 출력해 보세요.

## 무한 루프

섹션 5

while 에서 가장 흔한 사고입니다. 증감을 빠뜨리면 영원히 돌립니다.

```
let n = 1;
while (n <= 3) {
```

```
  console.log(n);    ← n 이 영원히 1
```

```
}
```

이러면 터미널이 멈추지 않습니다. Ctrl + C 를 눌러 강제 종료하세요.

안전장치를 두는 방법도 있습니다. 최대 횟수를 정해 두는 것입니다.

```
let tries = 0;
let found = false;

while (!found) {
  tries++;
  console.log(`${tries}번째 시도`);

  if (tries >= 3) {
    found = true; // 조건을 바꿔서 빠져나갑니다
  }
}
```

```
출력      1번째 시도
          2번째 시도
          3번째 시도
```

```
console.log("총 시도 횟수:", tries);
```

출력

총 시도 횟수: 3

while (true) 로 두고 안에서 빠져나오는 방법은 다음 파일(개념03 break)에서 배웁니다.

▸ 직접 해보기

4

위 코드를 5번 시도 후 멈추도록 고쳐 보세요.

## 자주 하는 실수

섹션 6

이런 실수를 합니다

증감 빠뜨리기 → 무한 루프 (섹션 5)

이런 실수를 합니다

시작값을 while 안에서 만들기

```
while (x < 3) { let x = 0; ... }
```

ReferenceError: x is not defined let x

는 while 의 중괄호 안에서만 살기 때문에 조건식에서는 아예 보이지 않습니다. 시작값은 반드시 while 밖에서 만듭니다. (이 규칙은 05단원 개념06에서 자세히)

이런 실수를 합니다

while 뒤에 세미콜론

```
let p = 0;
```

```
while (p < 3);
```

세미콜론이 "반복할 내용"이 되어 아무것도 안 하며 영원히 돌립니다. do...while 은 세미콜론이 필요하지만, while 은 필요 없습니다.

```
console.log("p =", p);
```

출력

p = 0

이런 실수를 합니다

조건이 절대 **false** 가 되지 않게 만들기

```
let q = 1;  
while (q !== 10) { q += 3; }
```

1, 4, 7, 10 ... 이 경우엔 우연히 맞지만 q += 4 였다면 1, 5, 9, 13 으로 10을 건너뛰어 무한 루프입니다. !== 보다 < 나 >= 같은 범위 조건이 안전합니다.

## 정리

- 1 **while** (조건) { ... } — 조건이 true 인 동안 반복
- 2 시작값은 밖에서, 증감은 안에서 직접. 증감을 빠뜨리면 무한 루프.
- 3 횟수를 알면 for, 모르면 while.
- 4 do { ... } while (조건); — 조건이 **false** 여도 최소 한 번 실행
- 5 무한 루프에 걸리면 터미널에서 Ctrl + C

## 직접 해보기 정답

```
1) let n = 1;  
while (n <= 5) {
```

```
  console.log(n);  
  n++;
```

```
}
```

```
2) let money = 10000;  
while (money >= 3000) {
```

```
  money -= 3000;  
  console.log(money);
```

```
}
```

// 출력: 7000 / 4000 / 1000 (한 줄씩)

```
3) let d = 1;  
do {
```

```
console.log(d);  
d++;
```

```
} while (d <= 3);
```

```
4) if (tries >= 5) { found = true; }
```

3 을 5 로 바꾸기만 하면 됩니다.

# break 와 continue

RUN node 개념03\_break와\_continue.js

반복 도중에 흐름을 바꾸는 두 가지 명령입니다.

**break**      반복을 아예 그만둔다 (탈출)  
**continue**   이번 바퀴만 건너뛰고 다음 바퀴로 (통과)

## break — 그만두기

섹션 1

```
for (let i = 1; i <= 10; i++) {
  if (i > 3) {
    break; // 여기서 for 문을 완전히 빠져나갑니다
  }
  console.log("break 예제:", i);
}
```

출력      break 예제: 1  
             break 예제: 2  
             break 예제: 3

```
console.log("반복문 밖입니다");
```

출력      반복문 밖입니다

10까지 돌게 써 놔지만 i 가 4가 되는 순간 멈춥니다. break 아래의 코드는 실행되지 않고, 반복문 다음 줄로 건너뛵니다.

## continue — 건너뛰기

섹션 2

```
for (let i = 1; i <= 5; i++) {
  if (i === 3) {
    continue; // 이번 바퀴의 나머지를 건너뛰고 i++ 로 갑니다
  }
  console.log("continue 예제:", i);
}
```

```
출력      continue 예제: 1
          continue 예제: 2
          continue 예제: 4
          continue 예제: 5
```

3만 빠졌습니다. 반복 자체는 끝까지 돌았습니다.

바로 위 코드에서 continue 를 break 로 바꾸면 이렇게 달라집니다.

```
if (i === 3) continue; → 1, 2, 4, 5 (3만 빠짐) ← 방금 본 결과
if (i === 3) break;    → 1, 2 에서 끝
```

▹ 직접 해보기

1

1부터 10까지 중 짝수만 출력해 보세요. (홀수일 때 continue 를 쓰면 됩니다. 힌트: `i % 2 === 1`)

## 찾으면 멈추기

섹션 3

break 의 가장 흔한 쓰임입니다. 원하는 것을 찾으면 더 볼 필요가 없으니까요.

```
const target = 7;
let foundAt = -1; // 못 찾았을 때를 뜻하는 값으로 시작합니다
for (let i = 1; i <= 10; i++) {
  console.log(`${i} 확인 중`);

  if (i === target) {
    foundAt = i;
    break; // 찾았으니 나머지는 볼 필요가 없습니다
  }
}
```

```
출력      1 확인 중
          2 확인 중
          3 확인 중
          4 확인 중
          5 확인 중
          6 확인 중
          7 확인 중
```

```
console.log("찾은 위치:", foundAt);
```

```
출력      찾은 위치: 7
```

break 가 없었다면 8, 9, 10 까지 쓸데없이 확인했을 겁니다. 데이터가 수만 개라면 큰 차이가 됩니다.

1부터 20까지 세다가 처음으로 3의 배수를 만나면 그 수를 출력하고 멈춰 보세요. (힌트:  $i \% 3 == 0$ )

## while (true) 와 break

## 섹션 4

"언제 끝날지 모르지만 어떤 일이 생기면 끝낸다" 를 이렇게 씁니다.

```
let attempt = 0;

while (true) {
  attempt++;
  console.log(`${attempt}번째 시도`);

  if (attempt === 3) {
    console.log("성공! 반복을 끝냅니다");
    break;
  }
}
```

출력

```
1번째 시도
2번째 시도
3번째 시도
성공! 반복을 끝냅니다
```

`while (true)` 는 조건이 영원히 참이라 그 자체로 무한 루프입니다. 반드시 안에 `break` 가 있어야 합니다.

break 를 빠뜨리면 프로그램이 멈추지 않습니다. Ctrl + C 로 강제 종료하세요.

## continue 로 걸러내기

## 섹션 5

"조건에 안 맞는 건 건너뛰고, 맞는 것만 처리" 패턴입니다.

```
let sum = 0;

for (let i = 1; i <= 10; i++) {
  if (i % 2 === 1) {
    continue; // 홀수는 건너뛴
  }
  sum += i; // 짝수만 더함
}

console.log("1~10 짝수의 합:", sum);
```

출력 1~10 짝수의 합: 30

2+4+6+8+10 = 30

continue 없이 if 로 감싸도 똑같습니다.

```
let sum2 = 0;
for (let i = 1; i <= 10; i++) {
  if (i % 2 === 0) {
    sum2 += i;
  }
}
console.log("if 로 쓴 같은 결과:", sum2);
```

출력      if 로 쓴 같은 결과: 30

어느 쪽이든 맞습니다. 걸러낼 게 많고 처리 코드가 길면 → continue 로 먼저 걸러내는 편이 읽기 좋음  
처리 코드가 짧으면 → if 로 감싸는 게 단순함

▹ 직접 해보기

3

1부터 10까지 중 3의 배수만 더한 값을 출력해 보세요. (정답 18)

## 자주 하는 실수

섹션 6

이런 실수를 합니다

**break** 를 써야 하는데 **continue** 를 씀

```
for (let i = 1; i <= 5; i++) {
  if (i === 3) {
    continue;
  }
  console.log("멈추려 했지만 계속 돕니다:", i);
}
```

출력      멈추려 했지만 계속 돕니다: 1  
          멈추려 했지만 계속 돕니다: 2  
          멈추려 했지만 계속 돕니다: 4  
          멈추려 했지만 계속 돕니다: 5

멈추려면 break 입니다.



이런 실수를 합니다

**while** 에서 **continue** 앞에 증감을 안 씀 → 무한 루프

```
let n = 0;
while (n < 5) {
  if (n === 2) continue;
  n++;
}
// n 이 2에서 영원히 멈춥니다
```

while 에서 continue 는 아래 줄(증감 포함)을 건너뛰고 곧장 조건 검사로 돌아갑니다. for 문은 continue 를 만나도 증감식이 실행되므로 괜찮습니다. while 을 쓸 때는 증감을 continue 보다 '위'에 두거나 직접 챙기세요.

이런 실수를 합니다

**break** 를 조건문 없이 쓰기

```
for (let i = 1; i <= 5; i++) {
  console.log("한 번만 돕니다:", i);
  break;
}
```

출력      한 번만 돕니다: 1

조건 없이 break 를 쓰면 첫 바퀴에서 바로 끝납니다. break 는 거의 항상 if 안에 들어갑니다.

이런 실수를 합니다

**break** 로 여러 겹을 한 번에 빠져나올 수 있다고 생각하기

break 는 자기가 속한 반복문 하나만 빠져나옵니다. 중첩 반복문에서 어떻게 되는지는 다음 파일(개념04)에서 확인합니다.

## 정리

- 1 break 반복을 완전히 끝낸다.
- 2 continue 이번 바퀴만 건너뛴다.
- 3 찾으면 break — 뒤를 안 봐도 되니 훨씬 빠르다.
- 4 while (true) 안에는 반드시 break 가 있어야 한다.
- 5 while 에서 continue 를 쓸 땐 증감을 언제 하는지 꼭 확인할 것.

## 직접 해보기 정답

```
1) for (let i = 1; i <= 10; i++) {
```

```
  if (i % 2 === 1) continue;  
  console.log(i);
```

```
}  
// 출력: 2 4 6 8 10 (한 줄씩)
```

```
2) for (let i = 1; i <= 20; i++) {
```

```
  if (i % 3 === 0) {  
    console.log(i);  
    break;  
  }
```

```
}  
// 출력: 3
```

```
3) let total = 0;  
for (let i = 1; i <= 10; i++) {
```

```
  if (i % 3 !== 0) continue;  
  total += i;
```

```
}  
console.log(total); // 출력: 18 (3+6+9)
```

## 중첩 반복문

RUN node 개념04\_중첩반복문.js

반복문 안에 반복문을 넣는 것을 중첩 반복문이라고 합니다. 표(가로 × 세로)를 만들 때 씁니다. 구구단이 대표적입니다.

### 어떻게 도는가

섹션 1

바깥이 한 바퀴 돌 때마다, 안쪽은 처음부터 끝까지 다 돌립니다.

```
for (let outer = 1; outer <= 2; outer++) {
  console.log(`바깥 ${outer} 시작`);

  for (let inner = 1; inner <= 3; inner++) {
    console.log(`  안쪽 ${inner}`);
  }

  console.log(`바깥 ${outer} 끝`);
}
```

```
출력      바깥 1 시작
          안쪽 1
          안쪽 2
          안쪽 3
          바깥 1 끝
          바깥 2 시작
          안쪽 1
          안쪽 2
          안쪽 3
          바깥 2 끝
```

전체 실행 횟수 = 바깥 횟수 × 안쪽 횟수 = 2 × 3 = 6번 이 곱셈을 꼭 기억하세요. 바깥이 100, 안쪽이 100이면 1만 번입니다.

변수 이름은 보통 바깥부터 i, j 를 씁니다. 처음엔 outer / inner 처럼 뜻이 보이는 이름을 쓰는 게 헛갈리지 않습니다.

▸ 직접 해보기 1

바깥 3번, 안쪽 2번 도는 중첩 반복문을 만들고 안쪽에서 "\*" 를 출력해 보세요. (총 6번 찍힙니다)

먼저 반복문 하나로 2단을 만들어 봅시다.

```
const dan = 2;

for (let n = 1; n <= 9; n++) {
  console.log(`${dan} x ${n} = ${dan * n}`);
}
```

출력

```
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
```

▸ 직접 해보기 2

위 코드로 5단을 출력해 보세요.

## 구구단 여러 단 (중첩)

단이 바깥, 곱하는 수가 안쪽입니다. 2~9단 전체는 72줄이라 길어서, 여기서는 2~3단을 3까지만 봅시다.

```
for (let d = 2; d <= 3; d++) {
  console.log(`--- ${d}단 ---`);

  for (let n = 1; n <= 3; n++) {
    console.log(`${d} x ${n} = ${d * n}`);
  }
}
```

출력

```
--- 2단 ---
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
--- 3단 ---
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
```

전체 구구단이 필요하면 d 를 9까지, n 을 9까지로 바꾸면 됩니다. (출력이 72줄이라 이 파일에서는 줄여 두었습니다)

▹ 직접 해보기

3

4단과 5단을 1~3까지 출력해 보세요.

## 별 찍기

섹션 4

중첩 반복문의 단골 문제입니다. 바깥은 '줄', 안쪽은 '한 줄의 글자'입니다.

**직각삼각형** · 첫 줄 1개, 둘째 줄 2개 ...

```
for (let row = 1; row <= 4; row++) {
  let line = ""; // 줄마다 새로 비웁니다
  for (let col = 1; col <= row; col++) {
    line += "*";
  }

  console.log(line);
}
```

출력

```

*
**
***
****
```

핵심은 안쪽 조건이 `col <= row` 라는 것입니다. 안쪽 반복 횟수가 바깥 변수에 따라 달라집니다.

**사각형** · 안쪽을 고정하면 사각형이 됩니다.

```
for (let row = 1; row <= 3; row++) {
  let line = "";
  for (let col = 1; col <= 4; col++) {
    line += "#";
  }
  console.log(line);
}
```

출력

```

####
####
####
```

문자열을 한 줄로 모아 두었다가 한 번에 출력하는 게 요령입니다. 안쪽에서 바로 `console.log` 를 하면 한 글자씩 줄이 바뀝니다.

거꾸로 된 삼각형을 만들어 보세요. `** / * ** / *` (힌트: 안쪽 조건을 바꾸거나 바깥을 감소)

## 중첩에서 break 는 안쪽만 빠져나온다

섹션 5

```
for (let i = 1; i <= 3; i++) {
  for (let j = 1; j <= 3; j++) {
    if (j === 2) {
      break; // 안쪽 for 만 빠져나옵니다
    }
    console.log(`i=${i}, j=${j}`);
  }
}
```

출력            i=1, j=1  
                  i=2, j=1  
                  i=3, j=1

j 가 2가 되면 안쪽이 끝나지만, 바깥은 계속 돕니다.

바깥까지 한 번에 빠져나오려면 표시용 변수를 씁니다.

```
let stop = false;

for (let i = 1; i <= 3; i++) {
  for (let j = 1; j <= 3; j++) {
    if (i === 2 && j === 2) {
      stop = true;
      break; // 안쪽 탈출
    }
    console.log(`전체탈출 예제 i=${i}, j=${j}`);
  }

  if (stop) {
    break; // 바깥 탈출
  }
}
```

출력            전체탈출 예제 i=1, j=1  
                  전체탈출 예제 i=1, j=2  
                  전체탈출 예제 i=1, j=3  
                  전체탈출 예제 i=2, j=1

왜 i=2, j=1 에서 멈췄는지 한 번만 따라가 봅시다.

i=1 일 때 j 가 1, 2, 3 을 다 돌며 세 줄이 찍힙니다. 조건에 안 걸립니다. i=2, j=1 아직 조건(i가 2이고 j도 2)에 안 맞으니 찍힙니다. ← 마지막 출력 i=2, j=2 조건에 맞습니다. stop 에 true 를 넣고 break 로 안쪽을 빠져나옵니다.

그래서 "i=2, j=2" 는 찍히지 않습니다.

안쪽을 나오자마자 if (stop) 을 만나 바깥 for 도 break 합니다. i=3 은 아예 시작하지 않습니다.

break 하나로는 안쪽만 나옵니다. 그래서 "밖으로도 나가라" 는 쪽지(stop)를 남겨 두는 것입니다.

이런 코드가 필요해지면 대개 구조가 복잡해졌다는 신호입니다. 05단원에서 함수를 배우면 return 으로 훨씬 깔끔하게 빠져나올 수 있습니다.

## 자주 하는 실수

섹션 6

이런 실수를 합니다

안쪽과 바깥에 같은 변수 이름 쓰기

```
for (let i = 1; i <= 3; i++) {
  for (let i = 1; i <= 3; i++) { ... }
}
```

문법 에러는 아니지만 안쪽 i 가 바깥 i 를 가려서 바깥 값을 쓸 수 없게 됩니다. 반드시 다른 이름(i, j)을 쓰세요.

이런 실수를 합니다

줄 변수를 바깥에서 만들기

```
let wrongLine = "";
for (let row = 1; row <= 3; row++) {
  for (let col = 1; col <= 2; col++) {
    wrongLine += "*";
  }
  console.log(wrongLine);
}
```

출력        \*\*  
             \*\*\*\*  
             \*\*\*\*\*

줄마다 비우지 않아서 계속 쌓입니다. 줄 변수는 바깥 반복문 '안'에서 만들어야 합니다. (섹션 4 참고)

이런 실수를 합니다

반복 횟수를 알아보기

바깥 1000, 안쪽 1000 이면 100만 번입니다. 중첩을 세 겹 이상 쓰면 금방 느려집니다. 두 겹까지만 쓰세요.

이런 실수를 합니다

**break** 가 바깥까지 나갈 거라고 착각 (섹션 5에서 봤습니다)

## 정리

- 1 바깥 한 바퀴마다 안쪽이 처음부터 끝까지 돈다.
- 2 전체 횟수 = 바깥 × 안쪽
- 3 표(구구단)는 바깥이 단, 안쪽이 곱하는 수.
- 4 별 찍기는 바깥이 줄, 안쪽이 글자. 줄 변수는 바깥 반복문 안에서 만든다.
- 5 `break` 는 자기가 속한 반복문 하나만 빠져나온다.

## 직접 해보기 정답

```
1) for (let i = 1; i <= 3; i++) {
```

```
  for (let j = 1; j <= 2; j++) {  
    console.log("*");  
  }  
}
```

```
}
```

```
2) const d = 5;  
for (let n = 1; n <= 9; n++) {
```

```
  console.log(`${d} x ${n} = ${d * n}`);
```

```
}
```

```
3) for (let d = 4; d <= 5; d++) {
```

```
  for (let n = 1; n <= 3; n++) {  
    console.log(`${d} x ${n} = ${d * n}`);  
  }  
}
```

```
}
```

```
4) for (let row = 4; row >= 1; row--) {
```



```
let line = "";
for (let col = 1; col <= row; col++) {
  line += "*";
}
console.log(line);
```

```
}
// 출력: **** / *** / ** / *
```

# 반복문

RUN node 연습문제.js

TODO 자리에 코드를 쓰고, '기대 출력'과 같은지 확인하세요. 1~14는 기본, 15는 응용, 16은 [도전], 17은 에러 확인입니다.

⚠ 반복문은 잘못 쓰면 프로그램이 안 멈춥니다(무한 루프). 터미널이 멈춘 것 같으면 Ctrl + C 를 누르세요.

## 문제 1 (개념01 for)

for 문으로 1부터 5까지 한 줄에 하나씩 출력하세요.

기대 출력    1  
              2  
              3  
              4  
              5

여기에 코드를 쓰세요

## 문제 2 (개념01 for)

for 문으로 "안녕하세요" 를 3번 출력하세요.

기대 출력    안녕하세요  
              안녕하세요  
              안녕하세요

여기에 코드를 쓰세요

문제 3 (개념01 for 거꾸로)

5부터 1까지 거꾸로 출력하세요.

|       |   |
|-------|---|
| 기대 출력 | 5 |
|       | 4 |
|       | 3 |
|       | 2 |
|       | 1 |

여기에 코드를 쓰세요

문제 4 (개념01 누적)

1부터 100까지 모두 더한 값을 출력하세요. 누적 변수는 반복문 밖에서 0으로 시작해야 합니다.

|       |      |
|-------|------|
| 기대 출력 | 5050 |
|-------|------|

여기에 코드를 쓰세요

문제 5 (개념01 증감 조절)

2부터 10까지 짝수만 출력하세요. `i += 2` 를 쓰세요.

|       |    |
|-------|----|
| 기대 출력 | 2  |
|       | 4  |
|       | 6  |
|       | 8  |
|       | 10 |

여기에 코드를 쓰세요

## 문제 6 (개념01 문자열 누적)

"=" 를 10개 이어붙인 줄을 만들어 한 번에 출력하세요.

기대 출력

여기에 코드를 쓰세요

## 문제 7 (개념02 while)

while 문으로 1부터 5까지 출력하세요. 증감을 빠뜨리지 마세요.

| 기대 출력 |  |
|-------|--|
| 1     |  |
| 2     |  |
| 3     |  |
| 4     |  |
| 5     |  |

여기에 코드를 쓰세요

## 문제 8 (개념02 while)

10000원에서 2500원씩 빼면서, 뺄 때마다 남은 돈을 출력하세요. 2500원 미만이 남으면 멈춥니다.

기대 출력

7500  
5000  
2500  
0

여기에 코드를 쓰세요

문제 9 (개념02 do...while)

do...while 로 “일단 한 번은 실행됩니다” 를 출력하세요. 조건은 처음부터 false 인 것으로 쓰세요. (예: 1 > 10)

기대 출력

일단 한 번은 실행됩니다

여기에 코드를 쓰세요

문제 10 (개념03 break)

1부터 100까지 세다가 처음으로 7의 배수를 만나면 그 수를 출력하고 반복을 멈추세요.

기대 출력

7

여기에 코드를 쓰세요

문제 11 (개념03 continue)

1부터 10까지 중 홀수만 출력하세요. continue 를 쓰세요.

기대 출력    1  
                  3  
                  5  
                  7  
                  9

여기에 코드를 쓰세요

## 문제 12 (개념03 응용)

1부터 20까지 중 3의 배수만 더한 값을 출력하세요.

기대 출력    63

여기에 코드를 쓰세요

## 문제 13 (개념04 중첩 준비)

3단 구구단을 1부터 9까지 출력하세요.

기대 출력    3 x 1 = 3  
                  3 x 2 = 6  
                  3 x 3 = 9  
                  3 x 4 = 12  
                  3 x 5 = 15  
                  3 x 6 = 18  
                  3 x 7 = 21  
                  3 x 8 = 24  
                  3 x 9 = 27

여기에 코드를 쓰세요

문제 14 (개념04 중첩)

별로 삼각형을 5줄 그리세요. 줄 변수는 바깥 반복문 '안'에서 만들어야 합니다.

기대 출력     \*  
              \*\*  
             \*\*\*  
            \*\*\*\*  
           \*\*\*\*\*

여기에 코드를 쓰세요

문제 15 [응용]

첫날 저축액이 100원이고, 매일 두 배로 늘어납니다. 저축액이 10000원을 넘는 날이 며칠째이고 그때 얼마인지 출력하세요. 힌트: while 로 “10000 이하인 동안 두 배로 늘리기”

날짜는 1일째부터 셉니다. 두 배로 만든 다음에 날짜를 1 올리세요.  
순서를 바꾸면 7일째나 9일째가 나옵니다.

기대 출력     8일째에 12800원

여기에 코드를 쓰세요

## 문제 16 [도전]

아래 모양을 중첩 반복문으로 출력하세요. 각 줄에는 1부터 그 줄 번호까지의 숫자가 이어져 있습니다.

힌트: 줄을 담을 변수를 숫자 0 이 아니라 빈 문자열 "" 로 시작하세요.

0 으로 시작하면 더하기가 되어 1, 3, 6, 10 이 나옵니다.  
바깥 반복문이 줄 번호, 안쪽 반복문이 그 줄에 이어 붙일 숫자입니다.

기대 출력     1  
                 12  
                123  
               1234

여기에 코드를 쓰세요

## 문제 17 (에러 확인 - 맨 마지막)

아래 두 줄의 주석을 풀고 실행해서 어떤 에러가 나는지 확인하세요. 확인했으면 반드시 다시 주석 처리하세요.

```
const idx = 0;  
for (idx = 0; idx < 3; idx++) { console.log(idx); }
```

무슨 에러가 났나요? for 문의 카운터 변수는 왜 `let` 이어야 할까요?

답



## 반복문

RUN node 심화문제.js

### — 이걸 선택 과제입니다

연습문제를 다 푼 사람만 하세요. 안 풀어도 다음 단원에 지장 없습니다.

### 심화 1 구구단을 가로로

2단부터 4단까지를 아래처럼 한 줄에 하나씩 출력하세요.

힌트: 줄을 담을 변수를 빈 문자열 "" 로 시작해서 안쪽 반복문으로 이어 붙입니다.

숫자 0 으로 시작하면 더하기가 되어 버립니다.  
바깥이 단, 안쪽이 곱하는 수입니다.

기대 출력    2단: 2 4 6 8 10 12 14 16 18  
              3단: 3 6 9 12 15 18 21 24 27  
              4단: 4 8 12 16 20 24 28 32 36

여기에 코드를 쓰세요

### 심화 2 3의 배수이면서 5의 배수

1부터 100까지 중 3으로도 나누어떨어지고 5로도 나누어떨어지는 수를 한 줄에 모아서 출력하고, 마지막에 몇 개인지 출력하세요.

조건: 반드시 continue 를 써서 “아닌 것은 건너뛰는” 방식으로 만드세요.

if 로 감싸는 방식이 아니라, 조건에 안 맞으면 continue 로 넘기는 방식입니다.

기대 출력    15 30 45 60 75 90  
              총 6개

여기에 코드를 쓰세요

### 심화 3 숫자 뒤집기

12345 를 54321 로 뒤집어 출력하세요. 문자열로 바꾸지 말고, 숫자 계산만으로 하세요.

힌트: 두 가지를 반복하면 됩니다. ① 맨 뒷자리 꺼내기 → 숫자 % 10 ② 맨 뒷자리 떼어 버리기 → `Math.floor(숫자 / 10)`

결과를 담을 변수는 0으로 시작하고, 한 자리 꺼낼 때마다 기존 값에 10을 곱한 뒤 새 자리를 더합니다.

12345 → 1 자리씩: 5 / 4 / 3 / 2 / 1

출력            0 → 5 → 54 → 543 → 5432 → 54321

마지막에 자릿수도 함께 출력하세요.

```
const number = 12345;
```

기대 출력    54321  
              5자리

여기에 코드를 쓰세요

04단원 마무리    반복문

자주 넘어지는 자리

---

- 1    무한 루프를 일부러 한 번 내 보기 (Ctrl+C 로 멈추는 법까지)

자주 나오는 질문

---

- Q "i++ 말고 ++i는요?"

A 반복문의 증감 자리에서는 그 결과를 아무도 안 써서 둘이 똑같습니다
- Q "반복문 밖에서 i 쓰려면요?"

A 밖에 let last = 0 을 두고 안에서 넣어 두면 됩니다



# 함수

OPEN 05\_함수

```
console.log("    우리편의점입니다");  
function printHeader() {  
  printHeader();  
  function neverCalled() {
```

---

01 왜 함수인가

---

02 매개변수와 인자: 함수에 값 넘기기

---

03 return: 결과를 돌려주기

---

04 함수도 값이다 (함수 표현식)

---

05 화살표 함수

---

06 스코프: 변수가 사는 범위

---

- 연습문제

---

- 심화문제

## 왜 함수인가

RUN node 개념01\_왜\_함수인가.js

함수는 “코드에 이름을 붙여 두고, 필요할 때마다 불러 쓰는 것”입니다.

```
function 이름() {
  실행할 코드
}
```

이름(); ← 이렇게 부르면(호출하면) 그때 실행됩니다

### 함수 없이 쓰면

섹션 1

인사말을 세 군데에서 출력해야 한다고 해 봅시다.

```
console.log("=====");
console.log("  우리편의점입니다");
console.log("=====");

console.log("=====");
console.log("  우리편의점입니다");
console.log("=====");
```

출력

```
=====
  우리편의점입니다
=====

=====
  우리편의점입니다
=====
```

문제가 보이시나요? - 같은 코드를 계속 복사해야 합니다 - 가게 이름이 바뀌면 모든 곳을 찾아 고쳐야 합니다 - 한 군데를 빠뜨리면 화면마다 이름이 달라집니다

### 함수로 묶기

섹션 2

반복되는 코드에 이름을 붙입니다.

```
function printHeader() {
  console.log("=====");
  console.log("  우리편의점입니다");
  console.log("=====");
}
```

여기까지는 '만들기'입니다. 아직 아무것도 실행되지 않았습니다. 이름 뒤에 괄호를 붙여야 그때 실행됩니다.

```
printHeader();
```

```
출력      =====
          우리편의점입니다
          =====
```

```
printHeader();
```

```
출력      =====
          우리편의점입니다
          =====
```

이제 가게 이름을 바꾸려면 함수 안 한 줄만 고치면 됩니다. 100군데에서 부르고 있어도 마찬가지입니다.

▹ 직접 해보기

1

printGoodbye 라는 함수를 만들어 "이용해 주셔서 감사합니다" 를 출력하고, 두 번 호출해 보세요.

## 만들기와 부르는 다르다

섹션 3

함수를 만들기만 하고 부르지 않으면 아무 일도 일어나지 않습니다.

```
function neverCalled() {
  console.log("이 줄은 실행되지 않습니다");
}
```

(부르지 않았으므로 아무것도 출력되지 않습니다)

괄호를 빼먹으면 '실행'이 아니라 '함수 자체'를 가리키게 됩니다.

```
function sayHi() {
  console.log("안녕하세요");
}

sayHi();
```

```
출력      안녕하세요
```

```
console.log(typeof sayHi);
```

출력          function

괄호 없이 이름만 쓰면 함수라는 '값'입니다. 실행되지 않습니다.

▫ 직접 해보기 2

sayHi 를 괄호 없이 `console.log(sayHi)` 로 찍어 보고 무엇이 나오는지 확인해 보세요.

## 함수가 좋은 이유 세 가지

섹션 4

[1] 반복을 없앤다 — 섹션 2에서 봤습니다

[2] 이름이 곧 설명이 된다 아래 코드를 보세요. 주석이 없어도 무슨 일을 하는지 읽힙니다.

```
function printLine() {  
  console.log("-----");  
}  
  
function printMenu() {  
  console.log("아메리카노 4000원");  
  console.log("라떼 4500원");  
}  
  
printLine();  
printMenu();  
printLine();
```

출력          -----  
아메리카노 4000원  
라떼 4500원  
-----

[3] 고칠 곳이 한 군데다 메뉴가 바뀌면 printMenu 안만 고치면 됩니다.

▫ 직접 해보기 3

printLine 과 printMenu 를 순서만 바꿔 불러 보세요.

## 함수 안에서 다른 함수 부르기

섹션 5

함수 안에서 다른 함수를 부를 수 있습니다.



```
function printReceipt() {
  printLine();
  printMenu();
  printLine();
  console.log("감사합니다");
}
```

```
printReceipt();
```

```
출력      -----
          아메리카노  4000원
          라떼        4500원
          -----
          감사합니다
```

작은 함수를 여러 개 만들고, 그걸 조립해 큰 함수를 만드는 것이 프로그램을 짜는 기본 방식입니다.

▹ 직접 해보기

4

printHeader 와 printMenu 를 부르는 printAll 함수를 만들어 실행해 보세요.

## 이름 짓는 법

섹션 6

함수는 '일을 하는 것'이라 이름을 동사로 시작합니다.

|             |                                  |
|-------------|----------------------------------|
| printHeader | 출력한다                             |
| getTotal    | 가져온다 / 계산해서 돌려준다                 |
| isAdult     | ~인가? (참/거짓을 돌려주는 함수는 is~ / has~) |
| setName     | 설정한다                             |

변수는 명사(userName), 함수는 동사(getUserName). 이 구별만 지켜도 코드가 훨씬 잘 읽힙니다.

변수와 마찬가지로 카멜 표기법을 씁니다.

```
function showWelcomeMessage() {
  console.log("환영합니다!");
}
showWelcomeMessage();
```

```
출력      환영합니다!
```

## 자주 하는 실수

섹션 7

★ 아래에서 SyntaxError 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. 다른 에러 (ReferenceError, TypeError)는 주석을 풀어도 그 줄에서만 나고 그 앞의 출력은 그대로 다 나옵니다.

그런데 `SyntaxError` 는 다릅니다. 자바스크립트가 파일을 아예 못 읽어서 출력이 한 줄도 안 나옵니다. 여러분이 망가뜨린 것이 아닙니다. 실수로 풀었다면 다시 `//` 를 붙이면 그대로 돌아옵니다.

이런 실수를 합니다

괄호를 빼고 호출

```
sayHi;
```

에러도 안 나고 아무 일도 안 일어납니다. 가장 찾기 힘든 실수입니다. 반드시 `sayHi()` 로 괄호를 붙이세요.

이런 실수를 합니다

없는 함수 부르기 → `ReferenceError`

```
sayBye();
```

```
ReferenceError: sayBye is not defined
```

이름 오타이거나 아직 안 만든 함수입니다.

이런 실수를 합니다

함수가 아닌 것을 함수처럼 부르기 → `TypeError`

```
const price = 1000;
price();
```

```
TypeError: price is not a function
```

이런 실수를 합니다

함수 안에 함수 만들기를 깜빡하고 중괄호를 잘못 닫기

중괄호 짝이 안 맞으면 `SyntaxError` 가 납니다. VS Code 에서 `Shift + Alt + F` 로 자동 정렬하면 들여쓰기로 금방 찾을 수 있습니다.

## 정리

- 1 **function** 이름() { ... } 으로 만들고, 이름() 으로 부른다.
- 2 만들기만 하면 실행되지 않는다. 괄호를 붙여야 실행된다.
- 3 괄호 없이 이름만 쓰면 함수라는 '값'이다. (`typeof` 는 `function`)
- 4 함수 안에서 다른 함수를 부를 수 있다.
- 5 함수 이름은 동사로 시작한다. (`print~`, `get~`, `is~`)

## 직접 해보기 정답

```
1) function printGoodbye() {
```

```
    console.log("이용해 주셔서 감사합니다");
```

```
    }
    printGoodbye();
    printGoodbye();
```

```
2    console.log(sayHi);
    // 출력: [Function: sayHi]
    // 실행 결과가 아니라 함수 그 자체가 찍힙니다.
```

```
3    printMenu();
    printLine();
    // 부르는 순서대로 실행됩니다.
```

```
4    function printAll() {
```

```
        printHeader();
        printMenu();
```

```
    }
    printAll();
```

## 매개변수와 인자: 함수에 값 넘기기

RUN node 개념02\_매개변수와\_인자.js

앞 파일의 함수는 항상 똑같은 일만 했습니다. 값을 넘겨 주면 그 값에 따라 다르게 동작하게 만들 수 있습니다.

```
function greet(name) {      ← name 이 매개변수 (받는 쪽)
  console.log(name);
}

greet("김민준");            ← "김민준" 이 인자 (주는 쪽)
```

### 값 하나 넘기기

섹션 1

```
function greet(name) {
  console.log(`안녕하세요, ${name}님!`);
}

greet("김민준");
```

출력      안녕하세요, 김민준님!

```
greet("이서연");
```

출력      안녕하세요, 이서연님!

같은 함수인데 넘긴 값에 따라 결과가 달라집니다.

용어 정리 (헷갈리면 이렇게 외우세요) 매개변수(parameter) : 함수를 '만들 때' 괄호 안에 적는 이름 — 받는 그릇 인자(argument) : 함수를 '부를 때' 괄호 안에 넣는 값 — 담기는 내용

매개변수 이름은 그 함수 안에서만 쓰는 변수입니다. 마음대로 지어도 됩니다.

변수를 넘길 수도 있습니다.

```
const userName = "박지훈";
greet(userName);
```

출력      안녕하세요, 박지훈님!

## ▹ 직접 해보기 1

음료 이름을 받아 "OOO 나왔습니다" 를 출력하는 함수를 만들어 "라떼" 로 불러 보세요.

## 값 여러 개 넘기기

섹션 2

쉼표로 나열하면 됩니다. 순서가 중요합니다.

```
function printOrder(menu, count) {
  console.log(`${menu} ${count}개 주문되었습니다`);
}

printOrder("아메리카노", 2);
```

출력      아메리카노 2개 주문되었습니다

```
printOrder("케이크", 1);
```

출력      케이크 1개 주문되었습니다

순서를 바꿔 넣으면 뜻이 뒤집힙니다.

```
printOrder(2, "아메리카노");
```

출력      2 아메리카노개 주문되었습니다

에러는 안 나지만 결과가 엉망입니다. 순서를 꼭 맞추세요.

계산도 함께 해 봅시다.

```
function printTotal(price, count) {
  const total = price * count;
  console.log(`합계 ${total}원`);
}

printTotal(4500, 3);
```

출력      합계 13500원

## ▹ 직접 해보기 2

가로와 세로를 받아 넓이를 출력하는 함수를 만들고 8, 5 로 불러 보세요.

```
function introduce(name, age) {
  console.log(`${name} / ${age}살`);
}
```

```
introduce("김민준", 20);
```

출력      김민준 / 20살

두 번째 값을 안 넣으면 age 는 undefined 가 됩니다.

```
introduce("이서연");
```

출력      이서연 / undefined살

에러가 나지 않는다는 점이 중요합니다. 조용히 undefined 가 들어갑니다. 그래서 “왜 화면에 undefined 가 나오지?” 하고 한참 찾게 됩니다.

반대로 더 많이 넣으면 남는 것은 그냥 버려집니다.

```
introduce("박지훈", 30, "부산", "개발자");
```

출력      박지훈 / 30살

▹ 직접 해보기 3

introduce 를 아무 인자 없이 불러 보고 무엇이 출력되는지 확인해 보세요.

## 기본값 정해 두기

매개변수에 = 로 기본값을 정하면, 안 넘겼을 때 그 값이 쓰입니다.

```
function order(menu, count = 1) {
  console.log(`${menu} ${count}개`);
}
```

```
order("아메리카노", 3);
```

출력      아메리카노 3개

```
order("라떼");
```

출력      라떼 1개

count 를 안 넘겨서 기본값 1이 쓰였습니다.

여러 개에 기본값을 줄 수도 있습니다.

```
function makeProfile(name = "손님", city = "서울") {
  console.log(`${name} (${city})`);
}

makeProfile();
```

출력      손님 (서울)

코드

makeProfile("김민준");

makeProfile("김민준", "부산");

▶ 출력

▶ 김민준 (서울)

▶ 김민준 (부산)

**주의** · 기본값이 있는 매개변수는 뒤쪽에 두세요.

앞쪽에 두면 건너될 방법이 없어서 의미가 없습니다.

```
function bad(count = 1, menu) {
  console.log(menu, count);
}

bad("라떼");
```

출력      undefined 라떼

“라떼”가 첫 번째 자리인 count로 들어가 버렸습니다.

▹ 직접 해보기

4

할인율 기본값이 0인 함수를 만들어 가격 10000에 할인율을 안 넘겼을 때 10000이 나오게 해 보세요.

## 매개변수는 함수 안에서만 산다

섹션 5

```
function showPrice(price) {
  console.log("함수 안:", price);
}

showPrice(5000);
```

출력      함수 안: 5000

함수 밖에서 price를 쓰면 에러입니다.

이런 실수를 합니다

```
console.log(price);
```

```
ReferenceError: price is not defined
```

매개변수는 함수가 실행되는 동안에만 존재합니다. 자세한 내용은 개념06(스코프)에서 배웁니다.

그래서 다른 함수에서 같은 이름을 써도 서로 상관없습니다.

```
function showTax(price) {  
  console.log("세금 계산용 price:", price);  
}  
showTax(1000);
```

출력      세금 계산용 price: 1000

## 자주 하는 실수

섹션 6

★ 아래에서 SyntaxError 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. 다른 에러 (ReferenceError, TypeError)는 주석을 풀어도 그 줄에서만 나고 그 앞의 출력은 그대로 다 나옵니다. 그런데 SyntaxError 는 다릅니다. 자바스크립트가 파일을 아예 못 읽어서 출력이 한 줄도 안 나옵니다. 여러분이 망가뜨린 것이 아닙니다. 실수로 풀었다면 다시 // 를 붙이면 그대로 돌아옵니다.

이런 실수를 합니다

매개변수에 값을 미리 넣으려 하기

```
function greet2(name = "김민준") { } ← 이걸 기본값이라 괜찮습니다  
function greet3("김민준") { }
```

```
SyntaxError.
```

만들 때 괄호 안에는 '이름'만 적습니다.

이런 실수를 합니다

인자 순서를 헷갈리기 (섹션 2에서 봤습니다)

매개변수가 세 개를 넘어가면 순서를 외우기 어렵습니다. 그럴 땐 07단원에서 배울 객체로 묶어서 넘깁니다.

이런 실수를 합니다

숫자를 넘겼는데 문자열로 계산됨



```
function addPrice(a, b) {
  console.log(a + b);
}
addPrice(1000, 2000);
```

출력 3000

```
addPrice("1000", 2000);
```

출력 10002000

입력창에서 온 값은 문자열입니다. 넘기기 전에 Number( ) 로 바꾸세요.

```
addPrice(Number("1000"), 2000);
```

출력 3000

이런 실수를 합니다

함수 안에서 매개변수 값을 바꾸고 밖에서도 바뀔 거라 기대하기

```
let outsideValue = 10;
function tryChange(value) {
  value = 999; // 함수 안의 value 만 바꿉니다
}
tryChange(outsideValue);
console.log(outsideValue);
```

출력 10

숫자·문자열 같은 값은 '복사해서' 넘어갑니다. 원본은 그대로입니다.

## 정리

- 1 **function** 이름(매개변수) { ... } / 이름(인자) 로 값을 넘긴다.
- 2 여러 개는 쉼표로. 순서가 중요하다.
- 3 인자를 덜 넣으면 **undefined**. 에러가 안 나서 더 위험하다.
- 4 매개변수 = 기본값 으로 기본값을 정할 수 있다. 기본값은 뒤쪽에.
- 5 매개변수는 함수 안에서만 존재한다.

## 직접 해보기 정답

```
1) function callDrink(drink) {
```

```
  console.log(`${drink} 나왔습니다`);
```

```
}
```

```
callDrink("라떼");      // 출력: 라떼 나왔습니다
```

```
2) function printArea(width, height) {
```

```
  console.log(width * height);
```

```
}
```

```
printArea(8, 5);        // 출력: 40
```

```
3) introduce();
```

```
  // 출력: undefined / undefined살
```

```
  // 인자를 하나도 안 넘기면 매개변수가 전부 undefined 가 됩니다.
```

```
4) function printDiscounted(price, rate = 0) {
```

```
  console.log(price - price * rate);
```

```
}
```

```
printDiscounted(10000);    // 출력: 10000
```

```
printDiscounted(10000, 0.1); // 출력: 9000
```

# return: 결과를 돌려주기

```
RUN node 개념03_return.js
```

지금까지 함수는 안에서 `console.log` 로 '출력'만 했습니다. 그런데 계산 결과를 다른 곳에서 다시 쓰려면 '돌려받아야' 합니다.

```
return 값; ← 이 값을 함수를 부른 자리로 돌려보냅니다
```

출력하는 함수 vs 돌려주는 함수

섹션 1

— 출력만 하는 함수

```
function printSum(a, b) {
  console.log(a + b);
}

printSum(3, 4);
```

출력7

화면에는 잘 나옵니다. 하지만 이 7을 다시 쓸 수는 없습니다.

```
const result1 = printSum(3, 4);
```

출력7

```
console.log("돌려받은 값:", result1);
```

출력돌려받은 값: undefined

함수가 아무것도 돌려주지 않아서 `undefined` 입니다.

## — 돌려주는 함수

```
function getSum(a, b) {  
  return a + b;  
}  
  
const result2 = getSum(3, 4);  
console.log("돌려받은 값:", result2);
```

출력      돌려받은 값: 7

돌려받았으니 다시 계산에 쓸 수 있습니다.

| 코드                                                 | ▶ 출력 |
|----------------------------------------------------|------|
| <code>console.log(getSum(3, 4) * 10);</code>       | ▶ 70 |
| <code>console.log(getSum(getSum(1, 2), 4));</code> | ▶ 7  |

함수의 결과를 다른 함수의 인자로 바로 넘길 수도 있습니다.

기억할 것: `console.log` 는 '사람에게 보여 주는' 것 `return` 은 '코드에게 돌려주는' 것 둘은 완전히 다릅니다.  
초보자가 가장 많이 헷갈리는 지점입니다.

### ▫ 직접 해보기

1

두 수를 곱해서 돌려주는 `getProduct` 함수를 만들고 결과를 변수에 담아 출력해 보세요.

## return 한 값을 바로 쓰기

섹션 2

```
function getTotal(price, count) {  
  return price * count;  
}
```

변수에 담아 쓰기

```
const total = getTotal(4500, 3);  
console.log(total);
```

출력      13500

바로 출력하기

```
console.log(getTotal(1200, 5));
```

출력      6000

템플릿 리터럴 안에서 쓰기

```
console.log(`합계는 ${getTotal(3000, 2)}원입니다`);
```

출력          합계는 6000원입니다

조건문에서 쓰기

```
if (getTotal(4500, 3) >= 10000) {
  console.log("무료 배송 대상입니다");
}
```

출력          무료 배송 대상입니다

▸ 직접 해보기

2

원의 넓이를 돌려주는 함수를 만들어 보세요. (반지름 × 반지름 × 3.14, 반지름 10으로 확인)

05

## return 을 만나면 함수가 끝난다

섹션 3

```
function testReturn() {
  console.log("첫 줄");
  return "돌려준 값";
  console.log("이 줄은 절대 실행되지 않습니다");
}

const r = testReturn();
```

출력          첫 줄

```
console.log(r);
```

출력          돌려준 값

return 아래의 코드는 실행되지 않습니다. 이 성질을 이용하면 “조건에 안 맞으면 즉시 끝내기”를 할 수 있습니다. (섹션 5)

## return 이 없으면 undefined

섹션 4

```
function noReturn() {
  console.log("일만 하고 아무것도 안 돌려줍니다");
}

const value = noReturn();
```

출력      일만 하고 아무것도 안 돌려줍니다

```
console.log(value);
```

출력      undefined

return 만 쓰고 값을 안 적어도 undefined 입니다.

```
function emptyReturn() {  
  return;  
}  
console.log(emptyReturn());
```

출력      undefined

그래서 "함수 결과가 자꾸 undefined 로 나온다" 면 십중팔구 return 을 안 썼거나 console.log 로 잘못 쓴 것입니다.

▹ 직접 해보기 3

console.log 만 있는 함수의 결과를 변수에 담아 찍어 보고 undefined 가 나오는 것을 직접 확인해 보세요.

## 조기 반환 — 조건에 안 맞으면 즉시 끝내기

섹션 5

### — 중첩이 깊은 코드

```
function checkAge1(age) {  
  if (age >= 0) {  
    if (age >= 19) {  
      return "성인";  
    } else {  
      return "미성년자";  
    }  
  } else {  
    return "잘못된 나이";  
  }  
}
```

```
console.log(checkAge1(20));
```

출력      성인

```
console.log(checkAge1(-5));
```

출력      잘못된 나이

조기 반환으로 편 코드 — 걸러낼 것을 먼저 처리하고 끝내 버립니다

```
function checkAge2(age) {
  if (age < 0) {
    return "잘못된 나이"; // 여기서 함수가 끝납니다
  }

  if (age >= 19) {
    return "성인";
  }

  return "미성년자";
}

console.log(checkAge2(20));
```

출력      성인

코드

console.log(checkAge2(-5));

console.log(checkAge2(15));

▶ 출력

▶ 잘못된 나이

▶ 미성년자

else 가 사라지고 들여쓰기가 얇아졌습니다. 훨씬 읽기 쉽습니다. 이 패턴을 '조기 반환' 또는 '가드 절'이라고 부릅니다. 실무에서 아주 많이 씁니다.

◀ 직접 해보기

4

나누기 함수를 만들되, 나누는 수가 0이면 "0으로 나눌 수 없습니다" 를 조기 반환하게 해 보세요.

## 참/거짓을 돌려주는 함수

섹션 6

판단 결과를 돌려주는 함수는 이름을 is~ / has~ 로 짓습니다.

```
function isAdult(age) {
  return age >= 19;
}

console.log(isAdult(20));
```

출력      true

console.log(isAdult(15));

출력      false

조건문에 그대로 넣어 쓸 수 있습니다.

```
if (isAdult(20)) {  
  console.log("입장 가능");  
}
```

출력      입장 가능

이렇게 쓰지 마세요. 불필요합니다.

```
function isAdultBad(age) {  
  if (age >= 19) {  
    return true;  
  } else {  
    return false;  
  }  
}  
  
console.log(isAdultBad(20));
```

출력      true

age >= 19 자체가 이미 true / false 입니다. 그대로 return 하면 됩니다.

▸ 직접 해보기 5

짝수인지 판단해 true/false 를 돌려주는 isEven 함수를 만들어 4와 7로 확인해 보세요.

## 자주 하는 실수

섹션 7

이런 실수를 합니다

**return 대신 console.log 를 씀 (섹션 1)**

가장 흔한 실수입니다. "값을 다시 쓸 건가?" 를 기준으로 고르세요.

이런 실수를 합니다

**return 뒤에서 줄을 바꿈**

```
function brokenReturn() {  
  return // ← 세미콜론을 안 찍고 줄을 바꿨습니다 1 + 2; // 자바스크립트가 return 뒤에  
  세미콜론을 자동으로 넣어 버려서
```

이 줄은 돌려주는 값이 아니라 그냥 버려지는 코드가 됩니다



```

}
console.log(brokenReturn());

```

출력      undefined

이런 실수를 합니다

`return` 과 값 사이에서 줄을 바꾸면 안 됩니다. 자바스크립트가 `return` 뒤에 세미콜론을 자동으로 넣어 버립니다. 값이 길면 괄호로 감싸세요.

```

function fixedReturn() {
  return 1 + 2;
}
console.log(fixedReturn());

```

출력      3

이런 실수를 합니다

`return` 을 반복문 안에 잘못 넣기

```

function sumTo(n) {
  let sum = 0;
  for (let i = 1; i <= n; i++) {
    sum += i;

```

`return sum;`   ← 여기에 두면 첫 바퀴에서 함수가 끝나 버립니다

```

  }
  return sum; // 반복이 끝난 뒤에 돌려줘야 합니다
}
console.log(sumTo(5));

```

출력      15

이런 실수를 합니다

여러 값을 `return` 하려고 심표로 나열

```
function getTwo() {  
  return 1, 2;  
}  
console.log(getTwo());
```

출력            2

마지막 값만 돌아옵니다. 여러 값을 돌려주려면 06단원의 배열이나 07단원의 객체로 묶어야 합니다.

## 정리

- 1 `return` 값; — 함수를 부른 자리로 값을 돌려준다.
- 2 `console.log` 는 사람에게 보여 주기, `return` 은 코드에게 돌려주기.
- 3 `return` 을 만나면 함수가 즉시 끝난다. 아래 코드는 실행되지 않는다.
- 4 `return` 이 없으면 결과는 `undefined`.
- 5 걸러낼 조건은 위에서 먼저 `return` 으로 끝내라 (조기 반환).
- 6 판단 함수는 `is~` / `has~` 이름에 비교식을 그대로 `return`.

## 직접 해보기 정답

1) `function getProduct(a, b) {`

`return a * b;`

```
}  
const p = getProduct(3, 4);  
console.log(p);            // 출력: 12
```

2) `function getCircleArea(radius) {`

`return radius * radius * 3.14;`

```
}  
console.log(getCircleArea(10));    // 출력: 314
```

3) `function onlyLog() {`

```
console.log("출력만 합니다");
```

```
}
const v = onlyLog();
console.log(v);           // 출력: undefined
```

```
4) function divide(a, b) {
```

```
  if (b === 0) {
    return "0으로 나눌 수 없습니다";
  }
  return a / b;
```

```
}
console.log(divide(10, 0)); // 출력: 0으로 나눌 수 없습니다
console.log(divide(10, 2)); // 출력: 5
```

```
5) function isEven(n) {
```

```
  return n % 2 === 0;
```

```
}
```

```
console.log(isEven(4)); // 출력: true
console.log(isEven(7)); // 출력: false
```

# 함수도 값이다 (함수 표현식)

RUN node 개념04\_함수도\_값이다.js

자바스크립트에서 함수는 특별한 존재가 아니라 '값'입니다. 숫자나 문자열처럼 변수에 담고, 넘기고, 돌려줄 수 있습니다. 이 성질이 08단원(콜백)의 바탕이 됩니다.

## 함수를 변수에 담기

섹션 1

### — 지금까지 쓰던 방법 — 함수 선언식

```
function add(a, b) {
  return a + b;
}
console.log(add(3, 4));
```

출력 7

### 함수 표현식 · — 이름 없는 함수를 만들어 변수에 담습니다

```
const subtract = function (a, b) {
  return a - b;
};

console.log(subtract(10, 3));
```

출력 7

부르는 방법은 똑같습니다. 변수 이름 뒤에 괄호를 붙입니다.

생김새를 비교해 보세요.

```
function add(a, b) { ... }           ← 이름이 function 뒤에
const subtract = function (a, b) { ... }; ← 이름이 변수 쪽에, 끝에 세미콜론
```

함수 표현식은 '값을 변수에 넣는 문장'이라 세미콜론으로 끝냅니다.

함수가 값이라는 증거를 봅시다.

| 코드                                         | 출력         |
|--------------------------------------------|------------|
| <code>console.log(typeof add);</code>      | ▶ function |
| <code>console.log(typeof subtract);</code> | ▶ function |

#### ▫ 직접 해보기 1

두 수를 곱하는 함수를 '함수 표현식'으로 만들어 multiply 라는 변수에 담고 불러 보세요.

## 다른 이름을 붙여 쓸 수도 있다

섹션 2

함수는 값이므로 다른 변수에 옮겨 담을 수 있습니다.

|                                                                             |    |
|-----------------------------------------------------------------------------|----|
| <code>const plus = add;</code><br><br><code>console.log(plus(5, 5));</code> |    |
| 출력                                                                          | 10 |

add 와 plus 는 같은 함수를 가리킵니다. 괄호 없이 대입했다는 점에 주의하세요. `add()` 라고 쓰면 '결과값' 이 담깁니다.

|                                                                                                  |   |
|--------------------------------------------------------------------------------------------------|---|
| <code>const addResult = add(1, 2); // 결과인 3이 담깁니다</code><br><code>console.log(addResult);</code> |   |
| 출력                                                                                               | 3 |

|                                             |        |
|---------------------------------------------|--------|
| <code>console.log(typeof addResult);</code> |        |
| 출력                                          | number |

|                                                                                                       |          |
|-------------------------------------------------------------------------------------------------------|----------|
| <code>const addFunction = add; // 함수 자체가 담깁니다</code><br><code>console.log(typeof addFunction);</code> |          |
| 출력                                                                                                    | function |

괄호 하나 차이로 완전히 달라집니다. 앞으로 계속 나오니 확실히 구분하세요.

## 만들기 전에 부를 수 있는가

섹션 3

이게 선언식과 표현식의 가장 큰 차이입니다.

선언식 · 만들기 전에 불러도 됩니다

```
console.log(sayHello());
```

출력      안녕하세요

```
function sayHello() {  
  return "안녕하세요";  
}
```

자바스크립트가 실행 전에 함수 선언을 먼저 훑어서 미리 등록해 두기 때문입니다. 이것을 '호이스팅 (끌어올리기)'이라고 합니다.

표현식 · 만들기 전에 부르면 에러입니다

```
console.log(sayBye());  
const sayBye = function () {  
  return "안녕히 가세요";  
};
```

이런 실수를 합니다

```
};
```

ReferenceError: Cannot access 'sayBye' before initialization

const 로 만든 변수는 그 줄에 도달해야 값이 들어가기 때문입니다.

```
const sayBye = function () {  
  return "안녕히 가세요";  
};  
console.log(sayBye());
```

출력      안녕히 가세요

그럼 뭘 써야 할까요? 호이스팅에 기대는 코드는 읽기 어렵습니다. 위에서 아래로 읽히지 않으니까요. 그래서 실무에서는 "만든 다음에 부른다"는 순서를 지키는 편이고, 표현식(특히 다음 파일의 화살표 함수)을 더 많이 씁니다.

## 함수를 다른 함수에 넘기기

섹션 4

함수가 값이라면 인자로도 넘길 수 있습니다.

```
function callTwice(fn) {
  fn(); // 넘겨받은 함수를 실행합니다 fn();
}

function printHi() {
  console.log("안녕!");
}

callTwice(printHi);
```

출력      안녕!  
          안녕!

넘길 때는 괄호를 붙이지 않습니다. printHi 라고만 씁니다. printHi() 라고 쓰면 '실행 결과(undefined)'가 넘어가 버립니다.

이런 실수를 합니다

```
callTwice(printHi());
```

```
TypeError: fn is not a function undefined
```

를 실행하려 했기 때문입니다.

계산 방법 자체를 넘길 수도 있습니다.

```
function calculate(a, b, operation) {
  return operation(a, b);
}

console.log(calculate(10, 3, add));
```

출력      13

```
console.log(calculate(10, 3, subtract));
```

출력      7

같은 calculate 인데 넘긴 함수에 따라 더하기도 하고 빼기도 합니다. 이렇게 '넘겨받아 나중에 실행되는 함수'를 콜백이라고 부릅니다. 08단원에서 본격적으로 배웁니다.

▹ 직접 해보기 2

calculate 에 곱하기 함수를 넘겨  $10 \times 3$  을 구해 보세요.

한 번만 쓸 함수라면 변수에 담지 않고 그 자리에서 만들어 넘깁니다.

```
console.log(
  calculate(10, 3, function (a, b) {
    return a * b;
  })
);
```

출력      30

괄호 안에 함수가 통째로 들어가 있어서 처음엔 눈이 어지럽습니다. 다음 파일에서 배울 화살표 함수를 쓰면 이게 훨씬 짧아집니다.

## 자주 하는 실수

이런 실수를 합니다

**함수를 넘길 때 괄호를 붙임 (섹션 4에서 봤습니다)**

callTwice(printHi) ← 함수를 넘김 (맞음) callTwice(**printHi()**) ← 실행 결과를 넘김 (틀림)

이런 실수를 합니다

**함수 표현식 끝에 세미콜론을 안 붙임**

```
const f = function () { } ← 세미콜론 빠짐
```

대부분 동작하지만, 다음 줄이 괄호로 시작하면 이어진 코드로 해석되어 이상한 에러가 납니다. 표현식은 세미콜론으로 끝내세요.

이런 실수를 합니다

**표현식을 만들기 전에 호출 (섹션 3에서 봤습니다)**

이런 실수를 합니다

**함수 안에서 자기 자신 이름을 잘못 부르기**

```
const greet = function () { greeting(); };
```

변수 이름은 greet 인데 greeting 을 불렀습니다. ReferenceError. 표현식은 변수 이름으로 부른다는 것을 기억하세요.



## 정리

- 1 함수는 값이다. `typeof` 는 "function".
- 2 `const` 이름 = `function` (매개변수) { ... }; ← 함수 표현식, 끝에 세미콜론
- 3 선언식은 만들기 전에도 부를 수 있고(호이스팅), 표현식은 안 된다.
- 4 함수를 넘길 때는 괄호를 붙이지 않는다. 붙이면 '결과'가 넘어간다.
- 5 넘겨받아 나중에 실행되는 함수를 콜백이라고 한다. (08단원)

## 직접 해보기 정답

```
1) const multiply = function (a, b) {
  return a * b;
};
console.log(multiply(3, 4));    // 출력: 12

2) console.log(calculate(10, 3, multiply));  // 출력: 30
```

또는 그 자리에서 만들어 넘기기:

```
console.log(calculate(10, 3, function (a, b) { return a * b; }));
```

## 화살표 함수

RUN node 개념05\_화살표함수.js

함수 표현식을 더 짧게 쓰는 문법입니다. 요즘 코드는 대부분 이걸 씁니다. React 를 배울 때도 계속 나오니 눈에 익혀 두세요.

```
const 이름 = (매개변수) => { 코드 };
                ^^
                화살표 (= 와 > 를 붙여서 씁니다)
```

### function 을 화살표로 바꾸기

섹션 1

#### — 함수 표현식

```
const add1 = function (a, b) {
  return a + b;
};
console.log(add1(3, 4));
```

출력 7

**화살표 함수** · — function 을 지우고, 괄호와 중괄호 사이에 => 를 넣습니다

```
const add2 = (a, b) => {
  return a + b;
};
console.log(add2(3, 4));
```

출력 7

1 function 이라는 글자를 지운다

2 매개변수 괄호와 중괄호 사이에 => 를 넣는다

▫ 직접 해보기 1

아래 함수를 화살표 함수로 바꿔 보세요. `const sub = function (a, b) { return a - b; };`

## 줄여 쓰기 3단계

섹션 2

화살표 함수는 조건이 맞으면 더 줄일 수 있습니다.

### 1단계 · 기본형

```
const square1 = (n) => {
  return n * n;
};
console.log(square1(5));
```

출력 25

### 2단계 · 몸통이 return 한 줄뿐이면 → 중괄호와 return 을 함께 생략

```
const square2 = (n) => n * n;
console.log(square2(5));
```

출력 25

중괄호를 없애면 “이 식의 결과를 돌려준다”는 뜻이 됩니다. 중괄호와 return 은 반드시 같이 지워야 합니다. 하나만 지우면 이렇게 됩니다.

(n) => return n \* n 중괄호만 지움 → SyntaxError. 바로 알 수 있습니다. (n) => { n \* n } return 만 지움 → 에러 없이 조용히 undefined ← 이쪽이 위험

아래쪽이 훨씬 무섭습니다. 에러가 안 나니 어디가 잘못됐는지 알 수가 없습니다. 섹션 4에서 실제로 확인합니다.

### 3단계 · 매개변수가 딱 하나면 → 괄호도 생략 가능

```
const square3 = n => n * n;
console.log(square3(5));
```

출력 25

셋 다 완전히 같은 함수입니다. 다만 3단계는 취향이 갈립니다. 매개변수 괄호는 남겨 두는 편이 나중에 매개변수를 추가할 때 편하고 보기에다 일관돼서 권장됩니다.

▸ 직접 해보기

2

숫자를 두 배로 만드는 함수를 2단계 형태로 써 보세요.

## 0개 · 빈 괄호가 반드시 필요합니다

```
const sayHi = () => {
  console.log("안녕하세요");
};
sayHi();
```

출력      안녕하세요

```
const getPi = () => 3.14;
console.log(getPi());
```

출력      3.14

## 1개 · 괄호 생략 가능

```
const double = (n) => n * 2;
console.log(double(7));
```

출력      14

## 2개 이상 · 괄호 필수

```
const sum = (a, b) => a + b;
console.log(sum(1, 2));
```

출력      3

```
const sum3 = (a, b, c) => a + b + c;
console.log(sum3(1, 2, 3));
```

출력      6

## — 기본값도 그대로 쓸 수 있습니다

```
const greet = (name = "손님") => `${name}, 어서 오세요`;
console.log(greet());
```

출력      손님, 어서 오세요

```
console.log(greet("김민준"));
```

출력 김민준, 어서 오세요

▹ 직접 해보기

3

매개변수가 없고 “오늘도 화이팅” 을 돌려주는 화살표 함수를 만들어 보세요.

## 여러 줄이면 중괄호와 return 을 쓴다

섹션 4

몸통이 두 줄 이상이면 줄여 쓸 수 없습니다.

```
const getDiscountPrice = (price, rate) => {
  const discount = price * rate;
  const final = price - discount;
  return final;
};

console.log(getDiscountPrice(10000, 0.2));
```

출력 8000

중괄호를 썼다면 return 을 반드시 써야 합니다.

```
const broken = (price) => {
  price * 2; // return 이 없습니다
};

console.log(broken(100));
```

출력 undefined

아주 흔한 실수입니다. 중괄호를 쓰는 순간 return 이 필요합니다.

▹ 직접 해보기

4

가격과 개수를 받아 “총액에 10% 세금을 더한 값” 을 돌려주는 화살표 함수를 여러 줄로 써 보세요.  
(4500원 2개 → 9900)

## 조건과 함께 쓰기

섹션 5

삼항 연산자와 화살표 함수는 궁합이 좋습니다.

```
const isAdult = (age) => age >= 19;

console.log(isAdult(20));
```

출력 true

```
const getGrade = (score) => (score >= 90 ? "A" : "B");
console.log(getGrade(95));
```

출력 A

```
console.log(getGrade(80));
```

출력 B

갈림길이 셋 이상이면 중괄호를 쓰고 조기 반환을 씁니다.

```
const getGrade2 = (score) => {
  if (score >= 90) return "A";
  if (score >= 80) return "B";
  if (score >= 70) return "C";
  return "F";
};
console.log(getGrade2(85));
```

출력 B

▹ 직접 해보기

5

나이를 받아 19 이상이면 "성인", 아니면 "미성년자" 를 돌려주는 화살표 함수를 삼항 연산자로 써 보세요.

## 화살표 함수의 주의점

섹션 6

[1] 호이스팅이 안 됩니다 화살표 함수는 함수 표현식이라 만들기 전에 부를 수 없습니다.

```
console.log(later());
```

이런 실수를 합니다

```
const later = () => "나중";
```

ReferenceError: Cannot access 'later' before initialization

[2] this 가 다릅니다 화살표 함수는 자기만의 this 를 갖지 않습니다. this 는 07단원(객체의 메소드)에서 배웁니다. 그때 다시 비교합니다. 지금은 "객체의 메소드를 만들 때는 화살표 함수를 쓰지 않는다" 만 기억하세요.

[3] 객체를 바로 돌려줄 때는 괄호가 필요합니다 07단원에서 객체를 배우고 나면 다시 만날 규칙입니다.

```
const makeUser = (name) => ({ name: name }); ← 중괄호를 소괄호로 감쌉니다
```

## 자주 하는 실수

섹션 7

★ 아래에서 `SyntaxError` 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. 다른 에러 (`ReferenceError`, `TypeError`)는 주석을 풀어도 그 줄에서만 나고 그 앞의 출력은 그대로 다 나옵니다. 그런데 `SyntaxError` 는 다릅니다. 자바스크립트가 파일을 아예 못 읽어서 출력이 한 줄도 안 나옵니다. 여러분이 망가뜨린 것이 아닙니다. 실수로 풀었다면 다시 // 를 붙이면 그대로 돌아옵니다.

이런 실수를 합니다

중괄호를 쓰고 `return` 을 빠뜨림 (섹션 4에서 봤습니다)(n) ⇒ { n \* 2 } ← `undefined` (n) ⇒ n \* 2 ← 맞음 (n) ⇒ { `return` n \* 2; } ← 맞음

이런 실수를 합니다

화살표를 =< 나 → 로 씀 → `SyntaxError`

```
const f = (n) → n * 2;
```

반드시 = 다음에 &gt; 입니다. ⇒ 입니다.

이런 실수를 합니다

비교 연산자 &gt;= 와 헷갈림

```
const check = (n) => n >= 10;
console.log(check(15));
```

출력      true

⇒ 는 화살표, &gt;= 는 '크거나 같다'. 순서가 반대입니다.

이런 실수를 합니다

매개변수가 0개인데 괄호를 뺌 → `SyntaxError`

```
const f2 = => "안녕";
```

매개변수가 없어도 빈 괄호 ( ) 는 있어야 합니다.

## 정리

- 1 **const** 이름 = (매개변수) => { 코드 };
- 2 **return** 한 줄이면 중괄호와 **return** 을 같이 생략할 수 있다.
- 3 매개변수 0개면 빈 괄호 필수, 1개면 괄호 생략 가능, 2개 이상은 필수.
- 4 중괄호를 쓰면 **return** 을 반드시 써야 한다. 안 쓰면 **undefined**.
- 5 화살표 함수는 호이스팅되지 않는다. 만든 다음에 부를 것.

## 직접 해보기 정답

1) **const** sub = (a, b) => {

**return** a - b;

};

또는 줄여서: **const** sub = (a, b) => a - b;

2 **const** twice = (n) => n \* 2;  
console.log(twice(7)); // 출력: 14

3 **const** cheer = () => "오늘도 화이팅";  
console.log(cheer()); // 출력: 오늘도 화이팅

4 **const** getTotalWithTax = (price, count) => {

**const** total = price \* count;  
**return** total + total \* 0.1;

};  
console.log(getTotalWithTax(4500, 2)); // 출력: 9900

5) **const** checkAge = (age) => (age >= 19 ? "성인" : "미성년자");  
console.log(checkAge(20)); // 출력: 성인



## 스코프: 변수가 사는 범위

RUN node 개념06\_스코프.js

변수는 아무 데서나 쓸 수 있는 게 아닙니다. “이 변수를 어디서부터 어디까지 쓸 수 있는가”를 스코프라고 합니다.

핵심 규칙은 딱 하나입니다. 나머지는 전부 여기서 따라 나옵니다.

중괄호 { } 안에서 만든 변수는 그 중괄호 안에서만 쓸 수 있다.

### 함수 안의 변수는 밖에서 못 쓴다

섹션 1

05

```
function makeCoffee() {
  const beans = "에티오피아 원두"; // 함수 안에서 만든 변수 console.log("함수 안:",
  beans);
}

makeCoffee();
```

출력      함수 안: 에티오피아 원두

함수 밖에서 beans 를 쓰면 에러입니다.

이런 실수를 합니다

```
console.log(beans);
```

```
ReferenceError: beans is not defined
```

함수가 끝나면 그 안의 변수는 사라집니다.

이건 불편한 게 아니라 좋은 것입니다. 함수마다 자기 변수를 마음껏 만들어도 다른 함수와 부딪히지 않으니까요.

```
function makeTea() {
  const beans = "녹차 잎"; // 위와 같은 이름이지만 완전히 다른
  변수입니다 console.log("다른 함수 안:", beans);
}

makeTea();
```

출력      다른 함수 안: 녹차 잎

용어 두 개만 알고 갑시다. 앞으로 계속 나옵니다. 지역 변수 : 함수(또는 중괄호) 안에서 만든 변수. 그 안에서만 산다. ← beans 전역 변수 : 함수 밖, 파일의 맨 바깥에서 만든 변수. 어디서나 보인다. (섹션 2) '지역' 은 동네, '전역' 은 나라 전체라고 생각하면 쉽습니다.

#### ▹ 직접 해보기 1

함수 안에서 변수를 만들고, 함수 밖에서 출력해 보세요. (에러가 나는 것을 직접 확인하고 다시 주석 처리하세요)

## 밖의 변수는 안에서 쓸 수 있다

섹션 2

```
const shopName = "봄날카페"; // 함수 밖 - 전역 변수
function printShop() {
  console.log("가게 이름:", shopName); // 밖의 변수를 그대로 씁니다
}

printShop();
```

출력      가게 이름: 봄날카페

안에서 밖은 보이고, 밖에서 안은 안 보입니다. 선텩한 자동차 유리를 생각하세요. 안에서는 밖이 보이지만 밖에서는 안이 안 보입니다.

중첩된 함수도 마찬가지입니다. 바깥으로는 계속 찾아 나갑니다.

```
function outer() {
  const outerValue = "바깥 값";

  function inner() {
    console.log("안쪽에서 본 바깥:", outerValue);
    console.log("안쪽에서 본 전역:", shopName);
  }

  inner();
}

outer();
```

출력      안쪽에서 본 바깥: 바깥 값  
          안쪽에서 본 전역: 봄날카페

#### ▹ 직접 해보기 2

함수 밖에 taxRate 를 만들고, 함수 안에서 써 보세요.

## 같은 이름이면 가까운 쪽이 이긴다

섹션 3

```
const message = "전역 메시지";

function printMessage() {
  const message = "함수 안 메시지"; // 같은 이름을 다시
  만들었습니다 console.log(message);
}

printMessage();
```

출력      함수 안 메시지

```
console.log(message);
```

출력      전역 메시지

함수 안의 message 가 전역 message 를 '가린' 것입니다. 전역 변수가 바뀐 건 아닙니다. 함수가 끝나면 원래대로 보입니다.

매개변수도 마찬가지입니다.

```
const price = 1000;

function showPrice(price) {
  console.log("매개변수 price:", price);
}

showPrice(9999);
```

출력      매개변수 price: 9999

```
console.log("전역 price:", price);
```

출력      전역 price: 1000

▹ 직접 해보기 3

전역에 count 를 10으로 만들고, 함수 안에서 같은 이름으로 99를 만들어 각각 출력해 보세요.

## 중괄호면 다 스코프다 (if, for)

섹션 4

함수만이 아니라 if 와 for 의 중괄호도 스코프를 만듭니다.

```
if (true) {
  const insideIf = "if 안의 변수";
  console.log(insideIf);
}
```

출력      if 안의 변수

이런 실수를 합니다

```
console.log(insideIf);
```

```
ReferenceError: insideIf is not defined
```

```
for (let i = 0; i < 2; i++) {
  const insideFor = `for 안 ${i}`;
  console.log(insideFor);
}
```

출력      for 안 0  
          for 안 1

이런 실수를 합니다

```
console.log(i);
```

```
ReferenceError: i is not defined for
```

의 `let i` 도 중괄호 안에서만 삽니다. (04단원에서 봤죠)

그래서 반복문 결과를 밖에서 쓰려면 변수를 밖에 만들어야 합니다.

```
let total = 0; // 밖에서 만들기 for (let i = 1; i <= 3; i++) {
  total += i; // 안에서 밖의 변수를 고치는 건 됩니다
}

console.log("합계:", total);
```

출력      합계: 6

▸ 직접 해보기 4

for 문 안에서만 쓸 변수와, 밖에서도 쓸 변수를 각각 만들어 차이를 확인해 보세요.

## 전역 변수를 함부로 쓰면 안 되는 이유

섹션 5

전역 변수는 어디서든 고칠 수 있습니다. 편해 보이지만 위험합니다.

```
let sharedCount = 0; // 전역 function addOne() {
  sharedCount++;
}

function reset() {
  sharedCount = 0;
}

addOne();
addOne();
console.log("두 번 더한 뒤:", sharedCount);
```

출력      두 번 더한 뒤: 2

```
reset();
console.log("리셋 뒤:", sharedCount);
```

출력      리셋 뒤: 0

지금은 함수가 두 개뿐이라 흐름이 보입니다. 함수가 50개가 되면 "이 값이 언제 누구에 의해 바뀌었는지" 알 수 없게 됩니다.

- 1 값은 매개변수로 받는다
- 2 결과는 `return` 으로 돌려준다
- 3 전역 변수는 정말 안 바뀌는 것(가게 이름, 서울)에만 쓴다

```
function addOneBetter(count) {
  return count + 1;
}

let myCount = 0;
myCount = addOneBetter(myCount);
myCount = addOneBetter(myCount);
console.log("매개변수/return 방식:", myCount);
```

출력      매개변수/return 방식: 2

이 함수는 밖의 무엇도 건드리지 않습니다. 같은 값을 넣으면 항상 같은 결과가 나오니 테스트하기도 쉽습니다.

이런 실수를 합니다

함수 안에서 만든 값을 밖에서 쓰려 함

```
function calc() {
  const answer = 42;
  console.log(answer);
}
calc();
```

출력 42

console.log(answer); ← ReferenceError

해결: return 으로 돌려주세요.

```
function calcBetter() {
  return 42;
}
const answer2 = calcBetter();
console.log(answer2);
```

출력 42

이런 실수를 합니다

let / const 없이 변수 만들기

```
function makeGlobal() {
  leaked = "실수로 만들어진 전역 변수"; // let, const 가 없습니다
}
makeGlobal();
console.log(leaked);
```

출력 실수로 만들어진 전역 변수

함수 안에서 만들었는데 밖에서도 보입니다. 어디서 만들어졌는지 추적할 수 없는 변수가 생겨 버그의 원인이 됩니다. 반드시 const 나 let 을 붙이세요.

이런 실수를 합니다

if 안에서 만든 변수를 밖에서 쓰기 (섹션 4에서 봤습니다)

해결: if 밖에 let 으로 만들고 안에서 값을 넣거나, 삼항 연산자를 쓰세요.

```
const grade = 85 >= 80 ? "B" : "C";
console.log(grade);
```

출력      B

이런 실수를 합니다

### 매개변수 이름과 전역 변수 이름이 같아서 헷갈리기 (섹션 3)

문법 문제는 아니지만, 이름을 다르게 지으면 읽기가 훨씬 편합니다.

## 정리

- 1 중괄호 { } 안에서 만든 변수는 그 안에서만 쓸 수 있다.
- 2 안에서 밖은 보이고, 밖에서 안은 안 보인다.
- 3 같은 이름이면 가까운(안쪽) 변수가 이긴다. 밖의 값이 바뀌는 건 아니다.
- 4 if, for 의 중괄호도 스코프를 만든다.
- 5 값은 매개변수로 받고 결과는 `return` 으로 돌려준다. 전역 변수에 기대지 말 것.
- 6 `const` / `let` 없이 변수를 만들면 전역이 새어 나온다.

## 직접 해보기 정답

```
1) function test1() {
```

```
const secret = "비밀";
```

```
}
test1();
// console.log(secret); ← ReferenceError: secret is not defined
```

```
2) const taxRate = 0.1;
function printTax(price) {
```

```
console.log(price * taxRate);
```

```
}
printTax(10000);      // 출력: 1000
```

```
3) const count = 10;
function showCount() {
```

```
const count = 99;
console.log(count); // 출력: 99
```

```
}
showCount();
console.log(count); // 출력: 10
```

```
4) let outside = 0;
for (let i = 1; i <= 3; i++) {
```

```
const inside = i * 10;
outside += inside;
console.log(inside); // 출력: 10 / 20 / 30
```

```
}
console.log(outside); // 출력: 60
// console.log(inside); ← ReferenceError
```



# 함수

RUN node 연습문제.js

TODO 자리에 코드를 쓰고, '기대 출력'과 같은지 확인하세요. 1~15는 기본, 16은 응용, 17은 [도전], 18은 에러 확인입니다.

## 문제 1 (개념01)

"환영합니다!" 를 출력하는 printWelcome 함수를 만들고 한 번 호출하세요.

기대 출력    환영합니다!

여기에 코드를 쓰세요

## 문제 2 (개념01)

아래 두 함수를 만들고, 세 번째 함수 printHeader 에서 순서대로 불러 아래 결과가 나오게 하세요.

printStar : "\*\*\*\*\*" 출력 printTitle : "MENU" 출력 printHeader: printStar → printTitle → printStar  
순서로 호출

기대 출력    \*\*\*\*\*  
                  MENU  
                  \*\*\*\*\*

여기에 코드를 쓰세요

### 문제 3 (개념02 매개변수)

이름을 받아 인사하는 callName 함수를 만들고 "이서연" 으로 호출하세요.

기대 출력 이서연님, 안녕하세요

여기에 코드를 쓰세요

### 문제 4 (개념02 매개변수 2개)

가로와 세로를 받아 넓이를 출력하는 printArea 함수를 만들고 8, 5 로 호출하세요.

기대 출력 40

여기에 코드를 쓰세요

### 문제 5 (개념02 기본값)

메뉴와 개수를 받는 order 함수를 만드세요. 개수의 기본값은 1입니다. order("라떼") 와 order("케이크", 3) 을 각각 호출하세요.

기대 출력 라떼 1개  
케이크 3개

여기에 코드를 쓰세요

## 문제 6 (개념03 return)

두 수를 더해서 '돌려주는' getSum 함수를 만드세요. `console.log` 를 함수 안에 쓰면 안 됩니다. `return` 을 쓰세요.

기대 출력     10

여기에 코드를 쓰세요 (getSum(3, 7) 을 출력)

## 문제 7 (개념03 return 재사용)

가격과 개수를 받아 총액을 돌려주는 getTotal 함수를 만들고, 결과를 변수에 담아 아래처럼 출력하세요. (4500원 4개)

기대 출력     합계 18000원

여기에 코드를 쓰세요

## 문제 8 (개념03 조기 반환)

나누기 함수 divide 를 만드세요. 나누는 수가 0이면 "0으로 나눌 수 없습니다" 를 돌려주고 즉시 끝냅니다. divide(10, 0) 과 divide(10, 2) 를 각각 출력하세요.

기대 출력     0으로 나눌 수 없습니다  
5

여기에 코드를 쓰세요

## 문제 9 (개념03 판단 함수)

점수가 60 이상인지 `true/false` 로 돌려주는 `isPass` 함수를 만드세요. `if` 를 쓰지 말고 비교식을 그대로 `return` 하세요. `isPass(45)` 와 `isPass(80)` 을 각각 출력하세요.

기대 출력     `false`  
                  `true`

여기에 코드를 쓰세요

## 문제 10 (개념04 함수 표현식)

두 수를 빼는 함수를 '함수 표현식'으로 만들어 `minus` 변수에 담고, `minus(10, 3)` 을 출력하세요.

기대 출력     `7`

여기에 코드를 쓰세요

## 문제 11 (개념04 함수를 인자로)

아래 `calculate` 함수는 이미 만들어져 있습니다. 곱하기 함수를 만들어 `calculate` 에 넘겨 `6 × 7` 을 구하세요. 넘길 때 괄호를 붙이지 않도록 주의하세요.

```
function calculate(a, b, operation) {
  return operation(a, b);
}
```

기대 출력 42

여기에 코드를 쓰세요

## 문제 12 (개념05 화살표 변환)

아래 함수를 화살표 함수로 바꿔서 arrowDouble 이라는 이름으로 만들고 arrowDouble(7) 을 출력하세요.

```
function double(n) {
  return n * 2;
}
```

기대 출력 14

여기에 코드를 쓰세요

## 문제 13 (개념05 줄여쓰기)

숫자를 제공해서 돌려주는 화살표 함수를 '중괄호와 return 없이' 한 줄로 쓰세요. square(9) 를 출력하세요.

기대 출력 81

여기에 코드를 쓰세요

## 문제 14 (개념05 여러 줄)

가격과 할인율을 받아 할인된 가격을 돌려주는 화살표 함수를 만드세요. 몸통이 여러 줄이므로 중괄호와 `return` 을 씁니다. `getDiscounted(10000, 0.3)` 을 출력하세요.

기대 출력 7000

여기에 코드를 쓰세요

## 문제 15 (개념06 스코프)

함수 밖에 `taxRate` 를 0.1 로 만들고, 가격을 받아 세금을 출력하는 `printTax` 함수를 만들어 10000 으로 호출하세요. `taxRate` 는 함수 안에서 그대로 쓸 수 있습니다.

기대 출력 1000

여기에 코드를 쓰세요

## 문제 16 [응용]

편의점 계산기입니다. 함수 두 개를 만드세요. `getRawTotal(price, count)` : 가격 × 개수를 돌려준다  
`applyDiscount(total, isMember)` : 회원이면 10% 할인한 값, 아니면 그대로

(소수가 나오면 `Math.round` 로 다듬을 것)

1200원짜리를 5개, 회원인 경우로 계산해 아래처럼 출력하세요.

기대 출력      합계 5400원

여기에 코드를 쓰세요

## 문제 17 [도전]

시작 수부터 끝 수까지 모두 더한 값을 돌려주는 `getSumRange` 함수를 만드세요. `getSumRange(1, 10)` 과 `getSumRange(5, 10)` 을 각각 출력하세요.

힌트: 반복문으로 시작 수부터 끝 수까지 더해 나가고, 그 값을 `return` 하세요.

더한 값을 담을 변수는 반복문 밖에 만들어야 합니다.  
`console.log` 로 찍지 말고 `return` 으로 돌려주어야 두 번 호출해 쓸 수 있습니다.

기대 출력      55  
                    45

여기에 코드를 쓰세요

## 문제 18 (에러 확인 - 맨 마지막)

아래 주석을 풀고 실행해서 어떤 에러가 나는지 확인하세요. 확인했으면 반드시 다시 주석 처리하세요.

```
function makeSecret() {
  const secret = "비밀번호";
}
makeSecret();
console.log(secret);
```

무슨 에러가 났나요? `secret` 을 함수 밖에서 쓰려면 어떻게 고쳐야 할까요?

답

# 함수

RUN node 심화문제.js

## — 이걸 선택 과제입니다

연습문제를 다 푼 사람만 하세요. 안 풀어도 다음 단원에 지장 없습니다.

## 심화 1 함수를 이어 붙이기

편의점 계산을 세 단계로 나눈 함수 세 개를 만드세요.

getSubtotal(price, count) 가격 × 개수 applyDiscount(total, isMember) 회원이면 10% 할인  
(Math.round 로 다듬기) addTax(total) 10% 부가세를 더함 (Math.round 로 다듬기)

그리고 세 함수를 이어서 최종 금액을 구하세요. 중간 결과를 변수에 담지 말고, 한 줄로 이어 쓰는 것에 도전해 보세요. 예: addTax(applyDiscount(getSubtotal(...), ...))

1200원짜리 5개, 회원인 경우로 계산하세요.

힌트:  $1200 \times 5 = 6000 \rightarrow 10\% \text{ 할인} = 5400 \rightarrow 10\% \text{ 부가세} = 5940$

|       |      |
|-------|------|
| 기대 출력 | 6000 |
|       | 5400 |
|       | 5940 |

여기에 코드를 쓰세요

(중간값 확인용으로 세 줄을 각각 출력하세요)

## 심화 2 함수가 함수를 돌려주기

개념04에서 “함수도 값이다” 를 배웠습니다. 그럼 함수가 함수를 돌려줄 수도 있겠죠.

makeGreeter(인사말) 을 만드세요. 이 함수는 ‘이름을 받아 인사하는 새 함수’ 를 돌려줍니다.



```
const 안녕 = makeGreeter("안녕하세요");
const 반가워 = makeGreeter("반갑습니다");
```

안녕("김민준") → "안녕하세요, 김민준님"  
반가워("이서연") → "반갑습니다, 이서연님"

힌트: makeGreeter 안에서 `return` 으로 '함수 자체' 를 돌려주면 됩니다.

돌려주는 함수는 화살표 함수로 쓰면 짧습니다.  
바깥 함수의 매개변수(인사말)를 안쪽 함수가 그대로 씁니다. (개념06 스코프)

기대 출력    안녕하세요, 김민준님  
             반갑습니다, 이서연님  
             안녕하세요, 박지훈님

여기에 코드를 쓰세요

### 심화 3 스코프 한정

아래 코드가 무엇을 출력할지 먼저 예상해 보고, 실행해서 확인하세요. 예상이 틀렸다면 왜 그런지 개념06을 다시 보세요.

```
let message = "바깥";

function test() {
  console.log(message);
  let message2 = "안쪽";
  console.log(message2);
}

test();
```

출력        바깥  
             안쪽

```
console.log(message);
```

출력        바깥

여기까지는 예상대로였을 겁니다. 그럼 아래 함수는 어떨까요? 이번엔 함수 '안에서' 같은 이름을 다시 만듭니다. 실행하기 전에 예상을 적어 보세요.

내 예상: \_\_\_\_\_

```
function test2() {  
  let message = "안에서 새로 만든 것";  
  console.log(message);  
}  
  
test2();
```

출력      안에서 새로 만든 것

```
console.log(message);
```

출력      바깥

예상과 맞았나요? 함수 안에서 같은 이름을 새로 만들었는데 바깥은 그대로입니다.

아래에 “왜 바깥 message 가 안 바뀌었는지” 를 주석으로 한 줄 적어 보세요

답

## 05단원 마무리    함수

### 자주 넘어지는 자리

- 1 `return` 뒤에서 줄바꿈하면 값이 안 넘어오는 것 (ASI). 눈으로 안 보이는 예러라 반드시 시연
- 2 화살표 함수에서 `{ }` 를 쓰고 `return` 을 빼면 **에러 없이 조용히** `undefined`

# 배열

OPEN 06\_배열

```
console.log(students);  
const scores = [90, 85, 70];  
console.log(scores);  
const emptyList = [];
```

01 배열 만들기과 인덱스

02 배열에 추가하고 삭제하기

03 자르기, 합치기, 순서 바꾸기

04 배열에서 찾기

05 배열 반복하기

- 연습문제

- 심화문제

## 배열 만들기과 인덱스

RUN node 개념01\_배열\_만들기과\_인덱스.js

지금까지 변수 하나에는 값 하나만 담았습니다.

```
const student1 = "김민준";
const student2 = "이서연";
const student3 = "박지훈";    ← 30명이면 30줄?
```

여러 값을 하나로 묶어 담는 것이 배열입니다.

```
const students = ["김민준", "이서연", "박지훈"];
                ^               ^
                대괄호로 감싸고 쉼표로 구분
```

### 배열 만들기

섹션 1

```
const students = ["김민준", "이서연", "박지훈"];
console.log(students);
```

출력 [ '김민준', '이서연', '박지훈' ]

숫자 배열도 됩니다.

```
const scores = [90, 85, 70];
console.log(scores);
```

출력 [ 90, 85, 70 ]

빈 배열로 시작할 수도 있습니다. 나중에 채워 넣을 때 씁니다.

```
const emptyList = [];
console.log(emptyList);
```

출력 []

자료형을 섞어도 됩니다. (다만 실무에서는 같은 종류만 넣는 게 좋습니다)

```
const mixed = ["김민준", 20, true];
console.log(mixed);
```

출력 [ '김민준', 20, true ]

배열의 자료형은 object 입니다. 값 하나가 아니라 값들의 묶음이니깐요.

```
console.log(typeof students);
```

출력 object

배열인지 확인하려면 `Array.isArray` 를 씁니다.

```
console.log(Array.isArray(students));
```

출력 true

#### ▹ 직접 해보기

1

좋아하는 음식 3개를 담은 배열을 만들어 출력해 보세요.

## 인덱스 — 몇 번째인지

섹션 2

06

배열의 각 자리에는 번호가 붙어 있습니다. 이 번호를 인덱스라고 합니다. 중요한 것은 0부터 시작한다는 점입니다.

```
students = [ "김민준", "이서연", "박지훈" ]
인덱스      0 1 2
```

코드

▶ 출력

```
console.log(students[0]);
```

▶ 김민준

```
console.log(students[1]);
```

▶ 이서연

```
console.log(students[2]);
```

▶ 박지훈

“첫 번째”가 0번인 게 이상하죠? 이유가 있습니다. 컴퓨터는 인덱스를 “맨 앞에서 몇 칸 떨어져 있나”로 셉니다. 첫 번째 값은 맨 앞에서 0칸 떨어져 있어서 0번입니다. 프로그래밍 전반의 약속이니 익숙해지세요.

없는 인덱스를 꺼내면 에러가 아니라 `undefined` 입니다.

```
console.log(students[3]);
```

출력 undefined

에러가 안 나기 때문에 “왜 `undefined` 가 나오지?” 하고 헤매게 됩니다.

#### ▹ 직접 해보기

2

위에서 만든 음식 배열의 두 번째 값을 출력해 보세요.

| 코드                                          | ▶ 출력 |
|---------------------------------------------|------|
| <code>console.log(students.length);</code>  | ▶ 3  |
| <code>console.log(emptyList.length);</code> | ▶ 0  |

인덱스는 0부터인데 개수는 1부터 셉니다. 그래서 항상 이렇게 됩니다.

|                                                          |
|----------------------------------------------------------|
| <code>마지막 인덱스 = length - 1</code>                        |
| <code>console.log(students[students.length - 1]);</code> |
| 출력      박지훈                                              |

이건 개수가 몇 개든 항상 마지막을 가리킵니다.

|                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------|
| <code>const longList = ["a", "b", "c", "d", "e"];</code><br><code>console.log(longList[longList.length - 1]);</code> |
| 출력      e                                                                                                            |

최근에 생긴 더 편한 방법도 있습니다.

| 코드                                         | ▶ 출력 |
|--------------------------------------------|------|
| <code>console.log(longList.at(-1));</code> | ▶ e  |
| <code>console.log(longList.at(-2));</code> | ▶ d  |

at(-1) 은 뒤에서 첫 번째라는 뜻입니다.

▸ 직접 해보기

3

음식 배열의 개수와 마지막 값을 출력해 보세요.

|                                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------|
| <code>const menu = ["아메리카노", "라떼", "케이크"];</code><br><br><code>menu[1] = "카푸치노";</code><br><code>console.log(menu);</code> |
| 출력      [ '아메리카노', '카푸치노', '케이크' ]                                                                                         |

`const` 로 만들었는데 왜 바뀔까요? `const` 는 “menu 라는 이름이 다른 배열을 가리키지 못하게” 막는 것입니다. 배열 '안의 내용'은 바꿀 수 있습니다.

이건 안 됩니다.

이런 실수를 합니다

```
menu = ["새 배열"];
```

```
TypeError: Assignment to constant variable.
```

그래서 배열은 대부분 `const` 로 만듭니다. 내용은 어차피 바꿀 수 있으니까요.

없는 자리에 값을 넣으면 배열이 늘어납니다.

```
menu[5] = "쿠키";  
console.log(menu);
```

```
출력      [ '아메리카노', '카푸치노', '케이크', <2 empty items>, '쿠키' ]
```

```
console.log(menu.length);
```

```
출력      6
```

중간에 빈 자리가 생겨 버립니다. 이렇게 넣지 말고 `push` 를 쓰세요. (개념02)

▸ 직접 해보기 4

음식 배열의 첫 번째 값을 다른 음식으로 바꿔 보세요.

## 배열 안의 배열

섹션 5

배열 안에 배열을 넣을 수 있습니다. 표(행과 열)를 표현할 때 씁니다.

```
const classScores = [  
  [90, 85, 70], // 1모둠  
  [60, 95, 88], // 2모둠  
];  
  
console.log(classScores[0]);
```

```
출력      [ 90, 85, 70 ]
```

대괄호를 두 번 붙이면 안쪽 값까지 꺼냅니다.

```
console.log(classScores[0][1]);
```

출력 85

classScores[0] 이 [90, 85, 70] 이고, 그중 [1] 이 85 입니다.

```
console.log(classScores[1][2]);
```

출력 88

👉 직접 해보기 5

2모둠의 첫 번째 점수를 출력해 보세요.

## 자주 하는 실수

섹션 6

★ 아래에서 SyntaxError 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. 다른 에러 (ReferenceError, TypeError)는 주석을 풀어도 그 줄에서만 나고 그 앞의 출력은 그대로 다 나옵니다. 그런데 SyntaxError 는 다릅니다. 자바스크립트가 파일을 아예 못 읽어서 출력이 한 줄도 안 나옵니다. 여러분이 망가뜨린 것이 아닙니다. 실수로 풀었다면 다시 // 를 붙이면 그대로 돌아옵니다.

이런 실수를 합니다

인덱스를 1부터라고 생각하기

```
const fruits = ["사과", "바나나", "포도"];  
console.log(fruits[1]);
```

출력 바나나

“첫 번째”를 꺼내려고 [1] 을 쓰면 두 번째가 나옵니다.

이런 실수를 합니다

length 를 인덱스로 쓰기

```
console.log(fruits[fruits.length]);
```

출력 undefined

length 는 3인데 인덱스는 0,1,2 뿐입니다. length - 1 이어야 합니다.

이런 실수를 합니다

length 를 함수처럼 부르기 → TypeError

```
console.log(fruits.length());
```

length 는 괄호를 붙이지 않습니다. push() 같은 것과 헷갈리지 마세요.



이런 실수를 합니다

대괄호 대신 중괄호

```
const wrong = {"사과", "바나나"};

SyntaxError.
```

중괄호는 07단원에서 배울 객체입니다. 배열은 대괄호입니다.

이런 실수를 합니다

배열끼리 === 로 비교

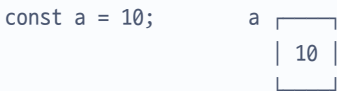
```
console.log([1, 2] === [1, 2]);

출력      false
```

내용이 같아도 '다른 배열'이라 false 입니다. 배열은 값이 아니라 '어디에 있는지'로 비교되기 때문입니다.

왜 그런지 그림으로 봐 주세요. 앞으로 여러 번 나옵니다.

숫자·글자를 담을 때 - 변수 안에 '값' 이 그대로 들어갑니다



배열·객체를 담을 때 - 변수 안에는 '주소' 만 들어갑니다



집 주소를 적어 준 것이지 집을 새로 지어 준 게 아닙니다.  
=== 는 주소끼리 비교하므로, 똑같이 생긴 집이 두 채 있어도 주소가 달라 false 입니다.  
거꾸로 주소가 같으면 한쪽에서 물건을 넣었을 때 다른 쪽에서도 그게 보입니다.  
(07단원 개념02, 09단원 개념03에서 이 그림을 다시 씁니다)

정리

- 1 **const** 이름 = [값1, 값2, 값3]; — 대괄호와 쉼표
- 2 인덱스는 0부터. 배열[0] 이 첫 번째.
- 3 배열.length 는 개수. 마지막 인덱스는 length - 1 (또는 at(-1))
- 4 없는 인덱스를 꺼내면 **undefined**. 에러가 안 나서 더 위험하다.
- 5 **const** 배열도 내용은 바꿀 수 있다. 배열 자체를 갈아끼우는 것만 막는다.

## 직접 해보기 정답

- 1 `const foods = ["김치찌개", "제육볶음", "냉면"];`  
`console.log(foods);`  
`// 출력: [ '김치찌개', '제육볶음', '냉면' ]`
- 2 `console.log(foods[1]);`                      `// 출력: 제육볶음`
- 3 `console.log(foods.length);`              `// 출력: 3`  
`console.log(foods[foods.length - 1]);`    `// 출력: 냉면`
- 4 `foods[0] = "된장찌개";`  
`console.log(foods);`  
`// 출력: [ '된장찌개', '제육볶음', '냉면' ]`
- 5 `console.log(classScores[1][0]);`    `// 출력: 60`

## 배열에 추가하고 삭제하기

RUN node 개념02\_추가와\_삭제.js

배열은 만든 뒤에도 계속 늘리고 줄일 수 있습니다. 앞뒤로 넣고 빼는 네 가지 ‘메소드’ 를 배웁니다.

‘메소드’ 라는 말을 여기서 처음 씁니다

값 뒤에 점을 찍고 붙여서 일을 시키는 것을 ‘메소드’ 라고 부릅니다.

```
cart.push("우유") → "장바구니야, 우유 좀 넣어 줘"
```

01단원에서 배운 그 점입니다. 왼쪽 값에게 시키는 것입니다. 지금까지 쓴 `console.log()` 도, `Math.floor()` 도 전부 같은 모양이었습니다. 앞으로 “배열의 메소드”, “문자열의 메소드” 라는 말이 자주 나오는데, “그 값 뒤에 점 찍고 쓰는 것” 이라고 읽으시면 됩니다.

|    | 앞(0번)   | 뒤(마지막) |
|----|---------|--------|
| 넣기 | unshift | push   |
| 빼기 | shift   | pop    |

뒤쪽(push / pop)을 훨씬 많이 씁니다. 이것부터 확실히 익히세요.

### push — 뒤에 추가

섹션 1

```
const cart = [];
console.log(cart);
```

출력 []

```
cart.push("아메리카노");
console.log(cart);
```

출력 [ '아메리카노' ]

```
cart.push("케이크");
console.log(cart);
```

출력 [ '아메리카노', '케이크' ]

한 번에 여러 개를 넣을 수도 있습니다.

```
cart.push("쿠키", "머핀");
console.log(cart);
```

출력 [ '아메리카노', '케이크', '쿠키', '머핀' ]

push 는 '넣은 뒤의 개수'를 돌려줍니다.

```
const newLength = cart.push("샌드위치");
console.log(newLength);
```

출력 5

```
console.log(cart.length);
```

출력 5

반복문과 함께 쓰면 배열을 자동으로 채울 수 있습니다.

```
const numbers = [];
for (let i = 1; i <= 5; i++) {
  numbers.push(i * 10);
}
console.log(numbers);
```

출력 [ 10, 20, 30, 40, 50 ]

▹ 직접 해보기

1

빈 배열을 만들고 push 로 과일 3개를 넣어 출력해 보세요.

## pop — 뒤에서 빼기

섹션 2

```
const stack = ["첫째", "둘째", "셋째"];

const removed = stack.pop();
console.log(removed);
```

출력 셋째

```
console.log(stack);
```

출력 [ '첫째', '둘째' ]

pop 은 '빼낸 값'을 돌려줍니다. push 와 돌려주는 것이 다릅니다.

push → 넣은 뒤의 개수  
pop → 빼낸 값

빈 배열에서 pop 하면 `undefined` 입니다. 에러는 안 납니다.

```
const emptyArr = [];
console.log(emptyArr.pop());
```

출력          undefined

▸ 직접 해보기 2

과일 배열에서 마지막 과일을 빼내어 출력하고, 남은 배열도 출력해 보세요.

## unshift / shift — 앞쪽 다루기

섹션 3

```
const queue = ["둘째", "셋째"];
```

unshift : 앞에 추가

```
queue.unshift("첫째");
console.log(queue);
```

출력          [ '첫째', '둘째', '셋째' ]

shift : 앞에서 빼기

```
const first = queue.shift();
console.log(first);
```

출력          첫째

```
console.log(queue);
```

출력          [ '둘째', '셋째' ]

이름을 외우는 요령:

push(밀어 넣다) / pop(툭 튀어나오다) - 둘 다 뒤쪽. 자주 씀  
shift 는 맨 앞을 빼고 뒤를 한 칸씩 '밀어' 당기는 것,  
unshift 는 그 반대(un-)라서 맨 앞에 넣는 것입니다. 가끔 씀  
헛갈리면 "un 붙은 게 넣기" 하나만 기억하세요.

앞쪽 작업은 뒤의 값들을 전부 한 칸씩 밀어야 해서 느립니다. 데이터가 수만 개라면 차이가 납니다. 웬만하면 push / pop 을 쓰세요.

▹ 직접 해보기

3

대기줄 배열을 만들고 맨 앞 사람을 빼내어 출력해 보세요.

## splice — 중간에 넣고 빼기

섹션 4

splice 는 원하는 위치에서 원하는 개수만큼 다룹니다.

```
배열.splice(시작위치, 지울개수, 넣을값들...)
```

### — 중간 삭제

```
const menu = ["아메리카노", "라떼", "카푸치노", "케이크"];

menu.splice(1, 1); // 1번 자리에서 1개 삭제
console.log(menu);
```

출력 [ '아메리카노', '카푸치노', '케이크' ]

여러 개 삭제 · 삭제한 값들을 배열로 돌려줍니다.

```
const list = ["a", "b", "c", "d", "e"];
const cut = list.splice(1, 3);
console.log(cut);
```

출력 [ 'b', 'c', 'd' ]

```
console.log(list);
```

출력 [ 'a', 'e' ]

중간 삽입 · 지울 개수를 0으로 하면 넣기만 합니다.

```
const seats = ["A", "C"];
seats.splice(1, 0, "B");
console.log(seats);
```

출력 [ 'A', 'B', 'C' ]

교체 · 지우면서 동시에 넣습니다.

```
const colors = ["빨강", "노랑", "파랑"];
colors.splice(1, 1, "초록");
console.log(colors);
```

출력 [ '빨강', '초록', '파랑' ]

⇒ 직접 해보기 4

["월", "수", "목"]에서 1번 자리에 "화"를 끼워 넣어 보세요.

## delete를 쓰면 안 되는 이유

섹션 5

```
const wrongWay = ["a", "b", "c"];
delete wrongWay[1];

console.log(wrongWay);
```

출력 [ 'a', <1 empty item>, 'c' ]

```
console.log(wrongWay.length);
```

출력 3

값만 사라지고 자리는 그대로 남습니다. length도 안 줄어듭니다. 배열에 구멍이 뚫려서 나중에 반복할 때 문제가 생깁니다. 배열에서 삭제할 때는 반드시 splice(또는 pop / shift)를 쓰세요.

```
const rightWay = ["a", "b", "c"];
rightWay.splice(1, 1);
console.log(rightWay);
```

출력 [ 'a', 'c' ]

```
console.log(rightWay.length);
```

출력 2

## 전부 비우기

섹션 6

```
const toClear = [1, 2, 3];
```

## 방법 1 · length 를 0으로

```
toClear.length = 0;  
console.log(toClear);
```

출력 []

## 방법 2 · splice 로 전부 삭제

```
const toClear2 = [1, 2, 3];  
toClear2.splice(0, toClear2.length);  
console.log(toClear2);
```

출력 []

주의 · 이걸 비우는 게 아닙니다.

```
let toClear3 = [1, 2, 3];  
toClear3 = [];  
console.log(toClear3);
```

출력 []

겉보기엔 같지만 '새 빈 배열로 갈아끼운' 것입니다. 다른 곳에서 원래 배열을 보고 있었다면 그쪽은 그대로 [1,2,3] 입니다. `const` 로 만든 배열에는 이 방법을 쓸 수도 없습니다.

## 자주 하는 실수

섹션 7

이런 실수를 합니다

**push** 의 결과를 배열이라고 생각하기

```
const arr1 = [1, 2];  
const result1 = arr1.push(3);  
console.log(result1);
```

출력 3

배열이 아니라 개수입니다. 배열은 `arr1` 자체가 이미 바뀌었습니다.

```
console.log(arr1);
```

출력 [ 1, 2, 3 ]



이런 실수를 합니다

**배열을 push 하면 통째로 들어감**

```
const arr2 = [1, 2];
arr2.push([3, 4]);
console.log(arr2);
```

출력 [ 1, 2, [ 3, 4 ] ]

안의 값들을 하나씩 넣으려면 다른 방법이 필요합니다. (09단원 스프레드)

이런 실수를 합니다

**splice 와 slice 를 헷갈리기**

splice : 원본을 바꾼다 (자르기, 넣기) slice : 원본은 그대로 두고 복사본을 돌려준다 (다음 파일 개념03)  
이름이 한 글자 차이로 정말 자주 헷갈립니다.

이런 실수를 합니다

**없는 인덱스에 직접 값 넣기**

```
const arr3 = ["a"];
arr3[3] = "d";
console.log(arr3);
```

출력 [ 'a', <2 empty items>, 'd' ]

구멍이 생깁니다. 항상 push 를 쓰세요.

## 정리

- 1 push(값) 뒤에 추가 → 넣은 뒤의 개수를 돌려줌
- 2 pop() 뒤에서 빼기 → 빼낸 값을 돌려줌
- 3 unshift/shift 앞쪽. 느리니 꼭 필요할 때만.
- 4 splice(시작, 지울개수, 넣을값) 중간 처리. 원본을 바꾼다.
- 5 delete 로 배열 안의 값을 지우지 말 것. 구멍이 남는다.

```

1  const fruits = [];
   fruits.push("사과");
   fruits.push("바나나", "포도");
   console.log(fruits);
   // 출력: [ '사과', '바나나', '포도' ]

2  const last = fruits.pop();
   console.log(last);      // 출력: 포도
   console.log(fruits);    // 출력: [ '사과', '바나나' ]

3  const line = ["김민준", "이서연", "박지훈"];
   console.log(line.shift()); // 출력: 김민준
   console.log(line);        // 출력: [ '이서연', '박지훈' ]

4  const days = ["월", "수", "목"];
   days.splice(1, 0, "화");
   console.log(days);
   // 출력: [ '월', '화', '수', '목' ]

```

# 자르기, 합치기, 순서 바꾸기

RUN node 개념03\_자르기와\_합치기.js

이번 파일의 메소드들은 두 부류로 나뉩니다. 이 구분이 제일 중요합니다.

원본을 바꾸지 않는다 (새 값을 돌려줌) : slice, concat, join  
 ↳ slice·concat 은 새 '배열' 을, join 은 '문자열' 을 돌려줍니다  
 원본을 바꾼다 : reverse, sort, splice

## slice — 잘라서 복사하기

섹션 1

배열.slice(시작인덱스, 끝인덱스) 끝 인덱스는 포함되지 않습니다. "시작 이상, 끝 미만"입니다.

```
const week = ["월", "화", "수", "목", "금"];

console.log(week.slice(1, 3));
```

출력 [ '화', '수' ]

인덱스 1, 2 만 가져옵니다. 3은 포함되지 않습니다.

끝을 생략하면 마지막까지입니다.

```
console.log(week.slice(2));
```

출력 [ '수', '목', '금' ]

둘 다 생략하면 통째로 복사합니다.

```
console.log(week.slice());
```

출력 [ '월', '화', '수', '목', '금' ]

음수를 쓰면 뒤에서부터 썹니다.

```
console.log(week.slice(-2));
```

출력 [ '목', '금' ]

가장 중요한 점 — 원본은 그대로입니다.

```
console.log(week);
```

출력 [ '월', '화', '수', '목', '금' ]

slice 와 splice 를 헛갈리지 마세요.

slice 자른 '복사본'을 돌려준다. 원본 그대로.  
splice 원본을 '직접' 자른다.

▸ 직접 해보기 1

week 에서 앞 3개만 잘라 출력해 보세요.

## concat — 배열 합치기

섹션 2

```
const group1 = ["김민준", "이서연"];  
const group2 = ["박지훈", "최유진"];  
  
const allMembers = group1.concat(group2);  
console.log(allMembers);
```

출력 [ '김민준', '이서연', '박지훈', '최유진' ]

원본은 둘 다 그대로입니다.

```
console.log(group1);
```

출력 [ '김민준', '이서연' ]

값을 바로 붙일 수도 있습니다.

```
console.log(group1.concat("정하늘"));
```

출력 [ '김민준', '이서연', '정하늘' ]

여러 개를 한 번에 합칠 수도 있습니다.

```
console.log(group1.concat(group2, ["정하늘"]));
```

출력 [ '김민준', '이서연', '박지훈', '최유진', '정하늘' ]

+ 로는 합칠 수 없습니다. 이상한 결과가 나옵니다.

```
console.log(group1 + group2);
```

출력 김민준,이서연박지훈,최유진

배열이 문자열로 바뀌어 이어붙어 버립니다. 반드시 `concat` 을 쓰세요. (09단원에서 배울 스프레드를 쓰면 더 짧게 합칠 수 있습니다)

#### ▹ 직접 해보기 2

배열 두 개를 만들어 `concat` 으로 합쳐 보세요.

## join — 배열을 문자열로

섹션 3

```
const fruits = ["사과", "바나나", "포도"];
```

구분자를 넣으면 그것으로 이어 붙입니다.

```
console.log(fruits.join(", "));
```

출력      사과, 바나나, 포도

```
console.log(fruits.join(" / "));
```

출력      사과 / 바나나 / 포도

```
console.log(fruits.join(""));
```

출력      사과바나나포도

아무것도 안 넣으면 쉼표로 이어집니다.

```
console.log(fruits.join());
```

출력      사과,바나나,포도

결과는 문자열입니다.

```
console.log(typeof fruits.join(", "));
```

출력      string

화면에 목록을 보여 줄 때 아주 많이 씁니다.

```
console.log(`오늘의 과일: ${fruits.join(", ")}`);
```

출력      오늘의 과일: 사과, 바나나, 포도

#### ▹ 직접 해보기 3

과일 배열을 " - " 로 이어 붙여 출력해 보세요.

## split — 문자열을 배열로 (join 의 반대)

섹션 4

split 은 문자열의 메소드입니다. 배열이 아니라 문자열에 붙입니다.

```
const csv = "김민준,이서연,박지훈";
const nameList = csv.split(",");

console.log(nameList);
```

출력 [ '김민준', '이서연', '박지훈' ]

```
console.log(nameList.length);
```

출력 3

공백으로 나누기

```
const sentence = "오늘 날씨 좋다";
console.log(sentence.split(" "));
```

출력 [ '오늘', '날씨', '좋다' ]

빈 문자열로 나누면 한 글자씩 쪼개집니다.

```
console.log("안녕".split(""));
```

출력 [ '안', '녕' ]

입력창에 "사과,바나나" 처럼 받은 값을 배열로 만들 때 씁니다.

⇒ 직접 해보기

4

"월-화-수" 를 "-" 로 나눠 배열로 만들어 출력해 보세요.

## reverse — 순서 뒤집기 (원본이 바뀝니다)

섹션 5

```
const numbers = [1, 2, 3, 4, 5];
numbers.reverse();

console.log(numbers);
```

출력 [ 5, 4, 3, 2, 1 ]

원본이 바뀌었습니다. 돌려받을 필요도 없이 numbers 자체가 뒤집혔습니다.

원본을 지키고 싶으면 slice 로 복사한 뒤 뒤집습니다.

```
const original = ["a", "b", "c"];
const reversed = original.slice().reverse();

console.log(reversed);
```

출력 [ 'c', 'b', 'a' ]

```
console.log(original);
```

출력 [ 'a', 'b', 'c' ]

▸ 직접 해보기 5

배열을 원본 그대로 두고 뒤집은 복사본을 만들어 보세요.

## sort — 정렬 (원본이 바뀝니다, 그리고 함정이 있습니다)

섹션 6

문자열 정렬은 기대대로 동작합니다.

```
const names = ["박지훈", "김민준", "이서연"];
names.sort();
console.log(names);
```

출력 [ '김민준', '박지훈', '이서연' ]

```
const alphabet = ["banana", "apple", "cherry"];
alphabet.sort();
console.log(alphabet);
```

출력 [ 'apple', 'banana', 'cherry' ]

이런 실수를 합니다

숫자를 정렬하면 이상해집니다.

```
const scores = [10, 9, 100, 1];
scores.sort();
console.log(scores);
```

출력 [ 1, 10, 100, 9 ]

왜 이럴까요? sort 는 기본적으로 값을 '문자열로 바꿔서' 사전 순으로 비교합니다. "10" 과 "9" 를 비교하면 첫 글자 "1" 이 "9" 보다 앞이라 10이 먼저 옵니다. (02단원에서 본 "10" < "9" 가 true 였던 것과 같은

이유입니다)

제대로 정렬하려면 '비교하는 방법'을 함수로 넘겨야 합니다. 함수를 넘기는 방식(콜백)은 08단원에서 배웁니다. 미리 보기만 하면 이렇습니다.

```
const scores2 = [10, 9, 100, 1];
scores2.sort((a, b) => a - b);
console.log(scores2);
```

출력 [ 1, 9, 10, 100 ]

지금은 "숫자 배열에 `sort()` 를 그냥 쓰면 안 된다" 만 기억하세요.

▫ 직접 해보기 6

[5, 30, 7] 을 `sort()` 로 정렬해 보고 결과가 왜 그런지 생각해 보세요.

## 자주 하는 실수

섹션 7

이런 실수를 합니다

**slice 와 splice 헷갈리기**

```
const arr1 = [1, 2, 3, 4];
console.log(arr1.slice(1, 3));
```

출력 [ 2, 3 ]

```
console.log(arr1);
```

출력 [ 1, 2, 3, 4 ]

slice 는 원본이 그대로입니다.

```
const arr2 = [1, 2, 3, 4];
console.log(arr2.splice(1, 3));
```

출력 [ 2, 3, 4 ]

```
console.log(arr2);
```

출력 [ 1 ]

splice 는 원본이 잘려 나갔습니다. 두 번째 인자의 뜻도 다릅니다. slice(시작, 끝) — 끝 '인덱스'  
splice(시작, 개수) — 지울 '개수'



이런 실수를 합니다

**reverse** 의 결과를 새 배열이라고 생각하기

```
const arr3 = [1, 2, 3];
const arr4 = arr3.reverse();
arr4.push(0);
console.log(arr3);
```

출력 [ 3, 2, 1, 0 ]

arr3 과 arr4 는 같은 배열입니다. 하나를 고치면 둘 다 바뀝니다.

이런 실수를 합니다

**join** 의 결과에 배열 메소드 쓰기 → **TypeError**

```
console.log(fruits.join(",").push("귤"));
```

join 의 결과는 문자열입니다. push 는 배열에만 있습니다.

이런 실수를 합니다

**split** 을 배열에 쓰기 → **TypeError**

```
console.log(fruits.split(","));
```

split 은 문자열의 메소드입니다. 배열에는 join 을 씁니다. 외우는 요령: 배열 --join→ 문자열 --split→ 배열

## 정리

- 1 slice(시작, 끝) 잘라낸 복사본. 원본 그대로. 끝은 포함 안 됨.
- 2 concat(배열) 합친 새 배열. + 로 합치면 안 된다.
- 3 join(구분자) 배열 → 문자열 / split(구분자) 문자열 → 배열
- 4 reverse(), sort() 는 원본을 바꾼다. 지키려면 slice() 로 복사 후 사용.
- 5 숫자 배열에 sort() 를 그냥 쓰면 사전 순으로 정렬되어 틀린다.

- 1 `console.log(week.slice(0, 3));`  
// 출력: [ '월', '화', '수' ]

---

- 2 `const a = [1, 2];`  
`const b = [3, 4];`  
`console.log(a.concat(b));`      // 출력: [ 1, 2, 3, 4 ]

---

- 3 `console.log(fruits.join(" - "));`  
// 출력: 사과 - 바나나 - 포도

---

- 4 `console.log("월-화-수".split("-"));`  
// 출력: [ '월', '화', '수' ]

---

- 5 `const origin = [1, 2, 3];`  
`const copy = origin.slice().reverse();`  
`console.log(copy);`      // 출력: [ 3, 2, 1 ]  
`console.log(origin);`      // 출력: [ 1, 2, 3 ]

---

- 6 `const nums = [5, 30, 7];`  
`nums.sort();`  
`console.log(nums);`      // 출력: [ 30, 5, 7 ]  
// 문자열로 바꿔 비교하기 때문에 "30" 의 첫 글자 "3" 이 "5" 보다 앞섭니다.

## 배열에서 찾기

RUN node 개념04\_찾기.js

“이 값이 배열에 있나?” “몇 번째에 있나?” 를 알아내는 방법입니다.

```
includes(값)  있으면 true, 없으면 false
indexOf(값)  몇 번째인지. 없으면 -1
```

### includes — 있는지 없는지

섹션 1

```
const menu = ["아메리카노", "라떼", "케이크"];

console.log(menu.includes("라떼"));
```

출력        true

```
console.log(menu.includes("녹차"));
```

출력        false

결과가 true / false 라서 조건문에 바로 쓸 수 있습니다.

```
if (menu.includes("케이크")) {
  console.log("케이크는 판매 중입니다");
}
```

출력        케이크는 판매 중입니다

```
if (!menu.includes("녹차")) {
  console.log("녹차는 취급하지 않습니다");
}
```

출력        녹차는 취급하지 않습니다

03단원에서 || 를 길게 이어 쓰던 것을 이렇게 줄일 수 있습니다. before: day === "토" || day === "일"  
after: ["토", "일"].includes(day)

```
const day = "일";
console.log(["토", "일"].includes(day));
```

출력        true

값이 많아질수록 훨씬 깔끔합니다.

```
const holiday = "5월 5일";
const holidays = ["1월 1일", "3월 1일", "5월 5일", "8월 15일"];
console.log(holidays.includes(holiday));
```

출력      true

includes 는 === 로 비교합니다. 자료형이 다르면 못 찾습니다.

```
console.log([1, 2, 3].includes("2"));
```

출력      false

▸ 직접 해보기 1

과일 배열을 만들고 "포도" 가 있는지 확인해 보세요.

## indexOf — 몇 번째인지

섹션 2

```
const fruits = ["사과", "바나나", "포도", "바나나"];
console.log(fruits.indexOf("바나나"));
```

출력      1

앞에서부터 찾아 처음 만난 위치를 돌려줍니다.

```
console.log(fruits.indexOf("포도"));
```

출력      2

없으면 -1 입니다. 0이 아니라는 점이 중요합니다.

```
console.log(fruits.indexOf("귤"));
```

출력      -1

왜 -1 일까요? 0은 '첫 번째'라는 진짜 위치이기 때문입니다. 그래서 "없음"을 표현할 다른 값이 필요했고 -1 을 쓰기로 한 것입니다.

lastIndexOf 는 뒤에서부터 찾습니다.

```
console.log(fruits.lastIndexOf("바나나"));
```

출력      3

▢ 직접 해보기 2

과일 배열에서 “사과”의 위치를 출력해 보세요.

indexOf로 존재 확인하기 (옛날 방식)

섹션 3

includes가 생기기 전에는 이렇게 썼습니다. 오래된 코드에서 자주 봅니다.

```
if (fruits.indexOf("포도") !== -1) {
  console.log("포도가 있습니다");
}
```

출력      포도가 있습니다

이 코드는 자주 틀립니다.

```
if (fruits.indexOf("사과")) {
  console.log("이 줄은 실행되지 않습니다");
}
```

이런 실수를 합니다

“사과”는 0번에 있는데, 0은 falsy라서 조건이 false가 됩니다. 배열에 있는데도 없다고 판단해 버립니다.

그래서 반드시 !== -1을 붙여야 합니다.

```
if (fruits.indexOf("사과") !== -1) {
  console.log("사과가 있습니다");
}
```

출력      사과가 있습니다

결론: 있는지만 알면 되면 includes를 쓰세요. 실수할 일이 없습니다.

위치가 필요할 때만 indexOf를 씁니다.

▢ 직접 해보기 3

indexOf로 “귤”이 없다는 것을 확인하는 if문을 써 보세요.

찾아서 지우기

섹션 4

indexOf로 위치를 찾고, splice로 지우는 조합을 자주 씁니다.

```
const cart = ["아메리카노", "케이크", "쿠키"];
const targetItem = "케이크";
const targetIndex = cart.indexOf(targetItem);

console.log(targetIndex);
```

출력      1

```
if (targetIndex !== -1) {
  cart.splice(targetIndex, 1);
}

console.log(cart);
```

출력      [ '아메리카노', '쿠키' ]

-1 검사를 빼먹으면 큰일 납니다.

```
const cart2 = ["아메리카노", "케이크"];
const notFound = cart2.indexOf("녹차"); // -1
cart2.splice(notFound, 1); // splice(-1, 1) → 뒤에서 첫 번째를 지웁니다!
console.log(cart2);
```

출력      [ '아메리카노' ]

없는 것을 지우려 했는데 엉뚱한 것이 지워졌습니다. 반드시 !== -1 을 확인하고 지우세요.

▹ 직접 해보기    4

배열에서 "쿠키" 를 찾아 안전하게 지워 보세요.

## 문자열에서도 똑같이 쓴다

섹션 5

includes 와 indexOf 는 문자열에도 있습니다. 사용법이 같습니다.

```
const message = "오늘 날씨가 참 좋습니다";

console.log(message.includes("날씨"));
```

출력      true

코드

```
console.log(message.includes("비"));
```

```
console.log(message.indexOf("날씨"));
```

▶ 출력

▶ false

▶ 3

3번째 글자(0부터 세서)부터 "날씨"가 시작한다는 뜻입니다.

검색 기능을 만들 때 이렇게 씁니다.

```
const keyword = "날씨";
if (message.includes(keyword)) {
  console.log(`${keyword} 를 찾았습니다`);
}
```

출력      "날씨" 를 찾았습니다

시작·끝 확인용 메소드도 있습니다.

| 코드                                                  | ▶ 출력   |
|-----------------------------------------------------|--------|
| <code>console.log(message.startsWith("오늘"));</code> | ▶ true |
| <code>console.log(message.endsWith("좋습니다"));</code> | ▶ true |

대소문자를 바꾸는 메소드도 있습니다. 영어에만 효과가 있습니다.

| 코드                                               | ▶ 출력    |
|--------------------------------------------------|---------|
| <code>console.log("Hello".toUpperCase());</code> | ▶ HELLO |
| <code>console.log("Hello".toLowerCase());</code> | ▶ hello |

검색 기능을 만들 때 아주 요긴합니다. 사용자가 "APPLE" 이라고 쳐도 "Apple" 을 찾아 줘야 하니까요.

```
const productName = "Apple iPhone";
const searchWord = "APPLE";

console.log(productName.includes(searchWord));
```

출력      false

대소문자가 달라서 못 찾았습니다.

양쪽을 다 소문자로 만들어 비교하는 것이 요령입니다.

```
console.log(productName.toLowerCase().includes(searchWord.toLowerCase()));
```

출력      true

원본은 바뀌지 않습니다. 바뀐 새 문자열을 돌려줄 뿐입니다.

```
console.log(productName);
```

출력      Apple iPhone

이메일 주소 문자열에 "@"가 들어 있는지 확인해 보세요.

## 조건으로 찾으려면

섹션 6

includes와 indexOf는 '값이 정확히 같은 것'만 찾습니다. "80점 넘는 점수를 찾아라" 같은 조건 검색은 못 합니다.

```
const scores = [55, 90, 72, 88];
```

지금 배운 것으로 하려면 반복문을 써야 합니다.

```
let found = -1;
for (let i = 0; i < scores.length; i++) {
  if (scores[i] > 80) {
    found = scores[i];
    break;
  }
}
console.log(found);
```

출력      90

08단원에서 배운 find를 쓰면 이게 한 줄이 됩니다.

```
console.log(scores.find((score) => score > 80));
```

출력      90

지금은 "정확히 같은 값은 includes / indexOf,

조건으로 찾는 건 반복문(나중엔 find)"로 정리해 두세요.

## 자주 하는 실수

섹션 7

이런 실수를 합니다

없을 때 0이 나온다고 생각하기

```
console.log(fruits.indexOf("없는과일"));
```

출력      -1

0은 '첫 번째'입니다. 없음은 -1입니다.



이런 실수를 합니다

**indexOf** 결과를 그대로 조건에 쓰기 (섹션 3에서 봤습니다)

이런 실수를 합니다

**includes** 에 괄호를 빼먹기

```
console.log(typeof fruits.includes);
```

출력                  function

괄호 없이 쓰면 함수 자체가 나옵니다. fruits.includes("사과") 로 쓰세요.

이런 실수를 합니다

**자료형이 달라서 못 찾기**

```
const nums = [1, 2, 3];
console.log(nums.includes("1"));
```

출력                  false

입력창에서 온 값은 문자열입니다. Number( ) 로 바꾸고 찾으세요.

```
console.log(nums.includes(Number("1")));
```

출력                  true

정리

- 1 includes(값) 있으면 true — 존재 확인은 이걸 쓴다.
- 2 indexOf(값) 위치. 없으면 -1. 0이 아니다.
- 3 indexOf 결과는 반드시 !== -1 로 비교할 것.
- 4 indexOf + splice 로 '찾아서 지우기'. -1 검사를 꼭 할 것.
- 5 문자열에도 includes / indexOf / startsWith / endsWith 가 있다.
- 6 조건으로 찾으려면 반복문(또는 08단원의 find).

## 직접 해보기 정답

```
1  const myFruits = ["사과", "포도", "귤"];  
   console.log(myFruits.includes("포도"));    // 출력: true
```

```
2  console.log(fruits.indexOf("사과"));        // 출력: 0
```

```
3  if (fruits.indexOf("귤") === -1) {
```

```
   console.log("귤이 없습니다");
```

```
   }  
   // 출력: 귤이 없습니다
```

```
4) const box = ["사탕", "쿠키", "젤리"];  
   const idx = box.indexOf("쿠키");  
   if (idx !== -1) {
```

```
     box.splice(idx, 1);
```

```
   }  
   console.log(box);    // 출력: [ '사탕', '젤리' ]
```

```
5) const email = "test@example.com";  
   console.log(email.includes("@"));    // 출력: true
```

# 배열 반복하기

```
RUN node 개념05_배열_반복하기.js
```

배열의 값을 하나씩 꺼내 처리하는 것이 배열을 쓰는 진짜 이유입니다. 04단원의 for 문과 배열이 만나는 지점입니다.

for 문으로 하나씩 꺼내기

섹션 1

배열을 처음부터 끝까지 하나씩 훑는 것을 '순회한다' 고 말합니다. 앞으로 이 말이 종종 나오니 기억해 두세요.

```
const students = ["김민준", "이서연", "박지훈"];

for (let i = 0; i < students.length; i++) {
  console.log(students[i]);
}
```

출력

김민준  
이서연  
박지훈

조건이 왜 `i < students.length` 인지 확인해 보세요. 인덱스는 0, 1, 2 까지입니다. `length` 는 3입니다. `i < 3` 이면 0, 1, 2 에서 멈춥니다. 정확히 맞습니다. `i <= 3` 으로 쓰면 `students[3]` 즉 `undefined` 까지 나옵니다.

번호를 붙여 출력하려면 `i + 1` 을 씁니다.

```
for (let i = 0; i < students.length; i++) {
  console.log(`${i + 1}번: ${students[i]}`);
}
```

출력

1번: 김민준  
2번: 이서연  
3번: 박지훈

▸ 직접 해보기 1

숫자 배열 `[10, 20, 30]` 을 for 문으로 하나씩 출력해 보세요.

인덱스가 필요 없다면 for...of 가 훨씬 간단합니다.

```
for (const student of students) {
  console.log(student);
}
```

출력          김민준  
                이서연  
                박지훈

읽는 법: “students 안의 각 student 에 대해”

```
for (const 하나씩담을변수 of 배열) { ... }
```

인덱스 계산도, length 도 신경 쓸 필요가 없습니다. 매 바퀴 새 변수가 만들어지므로 const 를 쓸 수 있습니다.

for 와 for...of 비교 for : 인덱스가 필요하다 / 거꾸로 돌아야 한다 / 건너뛰어야 한다 for...of : 값만 순서대로 필요하다 ← 대부분 이쪽

▹ 직접 해보기

2

위 for 문 예제를 for...of 로 바꿔 보세요.

## 합계와 평균

```
const scores = [90, 85, 70, 95];

let sum = 0; // 누적 변수는 반복문 '밖'에서 0으로 시작
for (const score of scores) {
  sum += score;
}

console.log("합계:", sum);
```

출력          합계: 340

```
const average = sum / scores.length;
console.log("평균:", average);
```

출력          평균: 85

평균이 소수로 나오면 다듬습니다.

```
const scores2 = [90, 85, 70];
let sum2 = 0;
for (const score of scores2) {
  sum2 += score;
}
console.log(sum2 / scores2.length);
```

출력            81.66666666666667

```
console.log((sum2 / scores2.length).toFixed(1));
```

출력            81.7

▫ 직접 해보기 3

가격 배열 [1200, 4500, 3000]의 합계를 구해 보세요.

## 최댓값과 최솟값

섹션 4

06

```
const temps = [23, 31, 18, 27];
```

첫 번째 값을 일단 최댓값으로 두고, 더 큰 게 나오면 바꿉니다.

```
let max = temps[0];
for (const temp of temps) {
  if (temp > max) {
    max = temp;
  }
}
console.log("최고 기온:", max);
```

출력            최고 기온: 31

```
let min = temps[0];
for (const temp of temps) {
  if (temp < min) {
    min = temp;
  }
}
console.log("최저 기온:", min);
```

출력            최저 기온: 18

주의 · 최댓값의 시작을 0으로 두면 음수 배열에서 틀립니다.

```
const cold = [-5, -12, -3];
let wrongMax = 0; // 잘못된 시작값
for (const c of cold) {
  if (c > wrongMax) {
    wrongMax = c;
  }
}
console.log("잘못된 최댓값:", wrongMax);
```

출력          잘못된 최댓값: 0

배열에 없는 0이 답으로 나옵니다. 반드시 첫 번째 값으로 시작하세요.

간단한 방법도 있습니다.

```
console.log(Math.max(23, 31, 18));
```

출력          31

다만 `Math.max` 는 배열을 통째로 못 받습니다. 배열을 펼쳐 넣는 방법은 09단원(스프레드)에서 배웁니다.

▸ 직접 해보기

4

[45, 88, 62] 의 최솟값을 구해 보세요.

## 조건에 맞는 것만 세기 / 모으기

섹션 5

```
const ages = [15, 22, 34, 17, 40];
```

— 세기

```
let adultCount = 0;
for (const age of ages) {
  if (age >= 19) {
    adultCount++;
  }
}
console.log("성인 수:", adultCount);
```

출력          성인 수: 3

모으기 · 새 배열에 push 합니다.

```
const adults = [];
for (const age of ages) {
  if (age >= 19) {
    adults.push(age);
  }
}
console.log(adults);
```

출력 [ 22, 34, 40 ]

— 바꿔서 모으기

```
const doubled = [];
for (const age of ages) {
  doubled.push(age * 2);
}
console.log(doubled);
```

출력 [ 30, 44, 68, 34, 80 ]

이 세 가지 패턴(세기 / 거르기 / 바꾸기)은 프로그래밍에서 끝없이 반복됩니다. 08단원에서 배울 filter 와 map 을 쓰면 각각 한 줄로 줄어듭니다.

```
console.log(ages.filter((age) => age >= 19));
```

출력 [ 22, 34, 40 ]

```
console.log(ages.map((age) => age * 2));
```

출력 [ 30, 44, 68, 34, 80 ]

지금은 반복문으로 직접 해 보세요. 그래야 filter / map 이 무엇을 대신해 주는지 압니다.

▫ 직접 해보기 5

점수 배열 [55, 90, 72, 88] 에서 80점 이상만 모아 보세요.

## 배열을 반복하며 화면 만들기

섹션 6

실무에서 가장 많이 하는 일입니다. 목록을 글자로 만들어 냅니다.

```
const cart = ["아메리카노", "케이크", "쿠키"];

let receipt = "";
for (let i = 0; i < cart.length; i++) {
  receipt += `${i + 1}. ${cart[i]}\n`;
}
console.log(receipt);
```

```
출력      1. 아메리카노
          2. 케이크
          3. 쿠키
```

\n 은 '줄바꿈'을 뜻하는 특별한 표시입니다. 마지막에도 붙어서 빈 줄이 하나 생깁니다.

join 을 쓰면 훨씬 간단합니다. (개념03)

```
console.log(cart.join("\n"));
```

```
출력      아메리카노
          케이크
          쿠키
```

## 자주 하는 실수

섹션 7

이런 실수를 합니다

조건에 <= 를 씀

```
const arr = ["a", "b"];
for (let i = 0; i <= arr.length; i++) {
  console.log(arr[i]);
}
```

```
출력      a
          b
          undefined
```

인덱스 2는 없습니다. i < arr.length 여야 합니다.

이런 실수를 합니다

for...of 와 for...in 을 헷갈리기

```
for (const idx in arr) {
  console.log("for...in 은 인덱스가 나옵니다:", idx);
}
```



출력

for...in 은 인덱스가 나옵니다: 0  
for...in 은 인덱스가 나옵니다: 1

for...in 은 07단원에서 객체를 순회할 때 쓰는 문법입니다. 배열에는 for...of 를 쓰세요.

이런 실수를 합니다

누적 변수를 반복문 안에서 만들기 (04단원에서 봤습니다)

이런 실수를 합니다

반복 도중에 배열 길이를 바꾸기

```
const items = ["a", "b", "c"];
for (let i = 0; i < items.length; i++) {
  if (items[i] === "b") {
    items.splice(i, 1); // 지우면 뒤가 한 칸씩 당겨집니다
  }
}
console.log(items);
```

출력

[ 'a', 'c' ]

이번엔 맞았지만, 연속으로 지울 값이 있으면 하나를 건너뛵니다. 반복 중에는 원본을 건드리지 말고, 새 배열에 모으는 방식을 쓰세요.

정리

- 1
- for (let i = 0; i < 배열.length; i++) — 인덱스가 필요할 때
- 2
- for (const 값 of 배열) — 값만 필요할 때. 대부분 이쪽.
- 3
- 합계는 sum 을 밖에서 0으로 시작. 최댓값은 배열[0] 으로 시작.
- 4
- 세기 / 거르기 / 바꾸기 — 세 가지 기본 패턴을 몸에 익힐 것.
- 5
- 반복 도중에 원본 배열의 길이를 바꾸지 말 것.

직접 해보기 정답

```
1) const nums = [10, 20, 30];
for (let i = 0; i < nums.length; i++) {

  console.log(nums[i]);

}
```

```
2) for (const n of nums) {
```

```
  console.log(n);
```

```
}
```

```
3) const prices = [1200, 4500, 3000];
```

```
let priceSum = 0;
```

```
for (const p of prices) {
```

```
  priceSum += p;
```

```
}
```

```
console.log(priceSum);      // 출력: 8700
```

```
4) const list = [45, 88, 62];
```

```
let minValue = list[0];
```

```
for (const v of list) {
```

```
  if (v < minValue) minValue = v;
```

```
}
```

```
console.log(minValue);      // 출력: 45
```

```
5) const sc = [55, 90, 72, 88];
```

```
const high = [];
```

```
for (const s of sc) {
```

```
  if (s >= 80) high.push(s);
```

```
}
```

```
console.log(high);          // 출력: [ 90, 88 ]
```

# 배열

RUN node 연습문제.js

TODO 자리에 코드를 쓰고, '기대 출력'과 같은지 확인하세요. 1~16은 기본, 17은 응용, 18은 [도전], 19는 에러 확인입니다.

배열을 `console.log` 로 찍으면 [ '값', '값' ] 처럼 따옴표가 함께 나옵니다. 기대 출력에 적힌 모양 그대로 나와야 정답입니다.

## 문제 1 (개념01)

"빨강", "파랑", "노랑" 을 담은 배열 colors1 을 만들어 출력하세요.

기대 출력

— '빨강', '파랑', '노랑'

여기에 코드를 쓰세요

## 문제 2 (개념01 인덱스)

아래 배열에서 "고양이" 를 꺼내 출력하세요.

```
const animals2 = ["강아지", "고양이", "토끼"];
```

기대 출력    고양이

여기에 코드를 쓰세요

### 문제 3 (개념01 length)

아래 배열의 개수와 마지막 값을 차례로 출력하세요. 마지막 값은 length 를 이용해 꺼내세요. (직접 [4] 라고 쓰지 마세요) 개념01에서 배운 at(-1) 을 써도 정답입니다. 둘 다 해 보세요.

```
const week3 = ["월", "화", "수", "목", "금"];
```

기대 출력     5  
                  금

여기에 코드를 쓰세요

### 문제 4 (개념01 값 바꾸기)

아래 배열의 두 번째 값을 "우동" 으로 바꾸고 배열 전체를 출력하세요.

```
const menu4 = ["김밥", "라면", "돈까스"];
```

기대 출력

— '김밥', '우동', '돈까스'

여기에 코드를 쓰세요

### 문제 5 (개념02 push)

빈 배열 basket5 를 만들고 push 로 "사과", "바나나", "포도" 를 넣어 출력하세요.

기대 출력

## — '사과', '바나나', '포도'

여기에 코드를 쓰세요

**문제 6** (개념02 pop)

아래 배열에서 마지막 값을 빼내어 출력하고, 남은 배열도 출력하세요.

```
const stack6 = ["첫째", "둘째", "셋째"];
```

기대 출력     셋째

## — '첫째', '둘째'

여기에 코드를 쓰세요

**문제 7** (개념02 splice)

아래 배열의 1번 자리에 "화" 를 끼워 넣고 출력하세요. 지우면 안 됩니다.

```
const days7 = ["월", "수", "목"];
```

기대 출력

## — '월', '화', '수', '목'

여기에 코드를 쓰세요

## 문제 8 (개념03 slice)

아래 배열에서 20, 30, 40 만 잘라내어 출력하고, 그다음 원본이 그대로인지 확인하기 위해 원본도 출력하세요.

```
const nums8 = [10, 20, 30, 40, 50];
```

기대 출력

— 20, 30, 40

— 10, 20, 30, 40, 50

여기에 코드를 쓰세요

## 문제 9 (개념03 concat)

아래 두 배열을 합쳐서 출력하세요. + 를 쓰면 안 됩니다.

```
const a9 = [1, 2];  
const b9 = [3, 4];
```

기대 출력

— 1, 2, 3, 4

여기에 코드를 쓰세요

## 문제 10 (개념03 join)

아래 배열을 ", " 로 이어 붙인 문자열로 출력하세요.

```
const fruits10 = ["사과", "바나나", "포도"];
```

기대 출력     사과, 바나나, 포도

여기에 코드를 쓰세요

## 문제 11 (개념03 split)

아래 문자열을 "-" 기준으로 나눠 배열로 만들어 출력하세요.

```
const text11 = "월-화-수";
```

기대 출력

— '월', '화', '수'

여기에 코드를 쓰세요

## 문제 12 (개념04 includes)

아래 배열에 "라떼"가 있는지, "녹차"가 있는지 차례로 출력하세요.

```
const menu12 = ["아메리카노", "라떼", "케이크"];
```

기대 출력     true  
false

여기에 코드를 쓰세요

### 문제 13 (개념04 indexOf)

아래 배열에서 "포도"의 위치와 "귤"의 위치를 차례로 출력하세요.

```
const list13 = ["사과", "바나나", "포도"];
```

기대 출력     2  
                 -1

여기에 코드를 쓰세요

### 문제 14 (개념05 for...of)

아래 배열의 값을 for...of로 한 줄씩 출력하세요.

```
const scores14 = [90, 85, 70];
```

기대 출력     90  
                 85  
                 70

여기에 코드를 쓰세요

### 문제 15 (개념05 합계와 평균)

아래 배열의 합계와 평균을 차례로 출력하세요.

```
const prices15 = [1200, 4500, 3000];
```

기대 출력     8700  
                 2900



여기에 코드를 쓰세요

## 문제 16 (개념05 조건으로 모으기)

아래 나이 배열에서 19세 이상만 새 배열에 모아 출력하세요.

```
const ages16 = [15, 22, 34, 17, 40];
```

기대 출력

— 22, 34, 40

여기에 코드를 쓰세요

## 문제 17 [응용]

두 배열은 같은 순서로 짝을 이룹니다. (cart17[0]의 가격이 prices17[0]) 아래처럼 번호를 붙여 한 줄씩 출력하고, 마지막에 합계를 출력하세요.

```
const cart17 = ["아메리카노", "케이크", "쿠키"];
const prices17 = [4500, 6000, 3000];
```

힌트: 이 문제는 for...of 로 풀 수 없습니다. 번호(인덱스)가 필요하기 때문입니다.

for (let i = 0; i < cart17.length; i++) 형태를 쓰세요.  
같은 i 로 cart17[i] 와 prices17[i] 를 함께 꺼내면 짝이 맞습니다.  
화면에 보이는 번호는 i + 1 입니다.

기대 출력

1. 아메리카노 4500원
2. 케이크 6000원
3. 쿠키 3000원
- 합계 13500원

여기에 코드를 쓰세요

## 문제 18 [도전]

아래 기온 배열에서 가장 높은 기온과 그것이 몇 번째인지 출력하세요. 번호는 1부터 셉니다.

```
const temps18 = [23, 31, 18, 27];
```

힌트: 두 가지를 같이 기억해야 합니다. 가장 높은 값과, 그게 몇 번째였는지.

최고값을 담은 변수와 그 위치를 담은 변수를 각각 만들어 두고,  
반복문에서 더 큰 값을 만날 때마다 둘 다 함께 바꾸세요.  
(indexOf 로 위치를 다시 찾는 방법도 됩니다)

기대 출력    최고 기온 31도 (2번째)

여기에 코드를 쓰세요

## 문제 19 (에러 확인 - 맨 마지막)

아래 두 줄의 주석을 풀고 실행해서 어떤 에러가 나는지 확인하세요. 확인했으면 반드시 다시 주석 처리하세요.

```
const notArray = "안녕";  
notArray.push("!");
```

무슨 에러가 났나요? 문자열에는 왜 push 가 없을까요?

답

## 배열

RUN node 심화문제.js

### — 이걸 선택 과제입니다

연습문제를 다 푼 사람만 하세요. 안 풀어도 다음 단원에 지장 없습니다.

### 심화 1 중복 제거

아래 배열에서 중복을 없앤 새 배열을 만드세요. 원본은 그대로 두세요. 순서는 처음 나온 순서를 유지해야 합니다.

힌트: 빈 배열을 하나 만들고, 원본을 돌면서

"새 배열에 아직 없으면 넣는다" 를 반복합니다. (개념04의 includes)

```
const numbers = [3, 1, 4, 1, 5, 9, 2, 5, 3];
```

기대 출력

### — 3, 1, 4, 5, 9, 2

원본 9개 → 중복 제거 6개

여기에 코드를 쓰세요

### 심화 2 교집합과 차집합

두 배열을 비교해서 세 가지를 각각 배열로 만들어 출력하세요.

① 양쪽에 다 있는 것 (교집합) ② a에만 있는 것 ③ b에만 있는 것

힌트: includes 로 "상대편에 있는가" 를 물어보면 됩니다.

```
const a = ["사과", "바나나", "포도", "딸기"];
const b = ["바나나", "딸기", "수박"];
```

기대 출력     둘 다: [ '바나나', '딸기' ]  
                  a에만: [ '사과', '포도' ]  
                  b에만: [ '수박' ]

여기에 코드를 쓰세요

### 심화 3 배열 회전

배열을 오른쪽으로 2칸 밀어 보세요. 뒤로 밀려난 것은 앞으로 돌아옵니다.

**1, 2, 3, 4, 5** → 오른쪽 2칸 → **[4, 5, 1, 2, 3]**

원본은 바뀌면 안 됩니다. 마지막에 원본도 함께 출력해 확인하세요.

힌트: 두 가지 방법이 있습니다. 하나만 성공하면 됩니다. ① slice 로 뒤 2개와 앞 3개를 각각 잘라 이어 붙이기

(slice 는 음수를 넣을 수 있습니다. slice(-2) 는 뒤에서 2개)

② 복사본을 만든 뒤 pop 한 것을 unshift 로 앞에 넣기를 2번 반복

①이 짧지만, ②가 무슨 일이 일어나는지 더 잘 보입니다.

```
const items = [1, 2, 3, 4, 5];
```

기대 출력

— **4, 5, 1, 2, 3**

원본: [ 1, 2, 3, 4, 5 ]

여기에 코드를 쓰세요

## 06단원 마무리 배열

### 자주 넘어지는 자리

---

1 `slice`(새 배열) vs `splice`(원본 변경)

### 자주 나오는 질문

---

Q "원본이 바뀌면 뭐가 나쁜데요?"

A 장바구니를 정렬해 버리면 담은 순서로 돌아갈 방법이 없습니다

Q "`push`요 `concat`이요?"

A 내 배열을 늘릴 땐 `push`, 원본을 두고 새 배열이 필요할 땐 `concat`/스프레드



# 객체와 메소드

OPEN 07\_객체와\_메소드

```
const user = {  
  name: "김민준",  
  age: 20,  
  city: "부산",  
}
```

01 객체 만들기과 접근

02 객체 추가·수정·삭제

03 메소드와 this

04 객체 순회하기

05 객체와 배열 섞어 쓰기

- 연습문제

- 심화문제

## 객체 만들기과 접근

RUN node 개념01\_객체\_만들기과\_접근.js

배열은 값을 '순서'로 묶었습니다.

```
const user = ["김민준", 20, "부산"];
user[1] 이 뭐였더라? 나이? 순서를 외워야 합니다.
```

객체는 값에 '이름표'를 붙여 묶습니다.

```
const user = { name: "김민준", age: 20, city: "부산" };
user.age ← 이름으로 꺼내니 외울 필요가 없습니다
```

### 객체 만들기

섹션 1

```
const user = {
  name: "김민준",
  age: 20,
  city: "부산",
};

console.log(user);
```

출력 { name: '김민준', age: 20, city: '부산' }

생김새를 뜯어봅시다.

```
{ name : "김민준" , age : 20 }
```

^   ^   ^   ^   ^  
 |   |   |   |   |   ↳ 쌍따옴표로 구분  
 |   |   |   |   |   ↳ 값(value)  
 |   |   |   |   |   ↳ 콜론으로 이름과 값을 연결  
 |   |   |   |   |   ↳ 이름(key, 속성 이름)  
 ↳ 중괄호로 감쌌 ← 배열의 대괄호와 다릅니다

이름과 값의 한 쌍을 '속성(property)'이라고 부릅니다.

한 줄로 써도 됩니다. 속성이 적을 때만 그렇게 하세요.



```
const point = { x: 10, y: 20 };
console.log(point);
```

출력 { x: 10, y: 20 }

빈 객체로 시작할 수도 있습니다.

```
const empty = {};
console.log(empty);
```

출력 {}

자료형은 배열과 마찬가지로 object 입니다.

```
console.log(typeof user);
```

출력 object

배열인지 객체인지 구별하려면 `Array.isArray` 를 씁니다.

```
console.log(Array.isArray(user));
```

출력 false

#### ▫ 직접 해보기 1

자기 정보를 담은 객체(name, age)를 만들어 출력해 보세요.

## 점 표기법으로 꺼내기

섹션 2

코드

```
console.log(user.name);
```

```
console.log(user.age);
```

▶ 출력

▶ 김민준

▶ 20

배열의 [0] 자리에 이름이 들어간다고 생각하면 쉽습니다.

```
배열: 배열[0]
객체: 객체.이름
```

템플릿 리터럴 안에서도 그대로 씁니다.

```
console.log(`${user.name}님은 ${user.age}살, ${user.city} 거주`);
```

출력 김민준님은 20살, 부산 거주

없는 속성을 꺼내면 `undefined` 입니다. 에러가 아닙니다.

```
console.log(user.email);
```

출력 undefined

배열에서 없는 인덱스를 꺼냈을 때와 같습니다. 조용해서 더 위험합니다.

⇒ 직접 해보기 2

위에서 만든 객체의 `name` 을 꺼내 출력해 보세요.

## 대괄호 표기법

섹션 3

점 대신 대괄호와 따옴표로도 꺼낼 수 있습니다.

```
console.log(user["name"]);
```

출력 김민준

결과는 같은데 왜 두 가지가 있을까요? 대괄호는 '이름을 변수로 정할 수 있기' 때문입니다.

```
const key = "age";  
console.log(user[key]);
```

출력 20

점으로는 이게 안 됩니다.

```
console.log(user.key);
```

출력 undefined

`user` 에 `"key"` 라는 이름의 속성을 찾기 때문입니다.

그래서 이렇게 씁니다. 이름을 코드에 직접 적는다 → 점 표기법 (대부분 이쪽) 이름이 변수에 들어 있다 → 대괄호 표기법

반복문에서 여러 속성을 돌 때 유용합니다.

```
const keys = ["name", "age", "city"];  
for (const k of keys) {  
  console.log(`${k}: ${user[k]}`);  
}
```

```
출력      name: 김민준
          age: 20
          city: 부산
```

이름에 띄어쓰기나 특수문자가 있으면 대괄호만 가능합니다.

```
const weird = { "my name": "김민준", "1st": "첫째" };
console.log(weird["my name"]);
```

```
출력      김민준
```

```
console.log(weird.my name); ← SyntaxError
```

이런 이름은 되도록 쓰지 마세요. 카멜 표기법(myName)이 약속입니다.

▹ 직접 해보기

3

변수에 "city" 를 담고 대괄호로 user 의 city 를 꺼내 보세요.

## 값에는 아무거나 들어간다

섹션 4

```
const product = {
  name: "무선 이어폰",
  price: 89000,
  inStock: true, // 불리언 tags: ["전자기기", "음향"], // 배열 seller: {
```

객체 안에 객체

```
name: "봄날전자",
  rating: 4.8,
},
};

console.log(product.price);
```

```
출력      89000
```

```
console.log(product.inStock);
```

```
출력      true
```

배열이 값이면 인덱스로 꺼냅니다.

|                                                |        |
|------------------------------------------------|--------|
| 코드                                             | ▶ 출력   |
| <code>console.log(product.tags[0]);</code>     | ▶ 전자기기 |
| <code>console.log(product.tags.length);</code> | ▶ 2    |

객체가 값이면 점을 이어 붙입니다.

|                                                  |        |
|--------------------------------------------------|--------|
| 코드                                               | ▶ 출력   |
| <code>console.log(product.seller.name);</code>   | ▶ 봄날전자 |
| <code>console.log(product.seller.rating);</code> | ▶ 4.8  |

이 조합이 실무 데이터의 기본 모양입니다. 서버에서 받아오는 데이터가 대부분 이렇게 생겼습니다.

#### ▣ 직접 해보기 4

product의 판매자 평점을 꺼내 출력해 보세요.

## 중첩 객체에서 조심할 것

섹션 5

중간 단계가 없으면 에러가 납니다.

|                                                                  |
|------------------------------------------------------------------|
| <pre>const emptyUser = {};  console.log(emptyUser.seller);</pre> |
| 출력      undefined                                                |

여기까지는 괜찮습니다.

이런 실수를 합니다

```
console.log(emptyUser.seller.name);
```

```
TypeError: Cannot read properties of undefined (reading 'name') undefined
```

에서 name을 꺼내려 했기 때문입니다.

**해결 1** · 03단원에서 배운 && 로 먼저 확인하기

|                                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>if (emptyUser.seller &amp;&amp; emptyUser.seller.name) {   console.log(emptyUser.seller.name); } else {   console.log("판매자 정보 없음"); }</pre> |
|--------------------------------------------------------------------------------------------------------------------------------------------------|

출력 판매자 정보 없음

## 해결 2 · ?. — 옵셔널 체이닝 (더 짧고 요즘 방식)

```
console.log(emptyUser.seller?.name);
```

출력 undefined

?. 은 "앞의 값이 없으면 거기서 멈추고 undefined 를 내놔라"는 뜻입니다. 에러 없이 조용히 넘어갑니다.

```
console.log(product.seller?.name);
```

출력 봄날전자

값이 있으면 그냥 평소처럼 동작합니다.

기본값까지 주려면 02단원의 ?? 와 함께 씁니다.

```
console.log(emptyUser.seller?.name ?? "판매자 미정");
```

출력 판매자 미정

▸ 직접 해보기

5

빈 객체에서 ?. 를 써서 에러 없이 값을 꺼내 보세요.

07

## 자주 하는 실수

섹션 6

★ 아래에서 SyntaxError 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. 다른 에러 (ReferenceError, TypeError)는 주석을 풀어도 그 줄에서만 나고 그 앞의 출력은 그대로 다 나옵니다. 그런데 SyntaxError 는 다릅니다. 자바스크립트가 파일을 아예 못 읽어서 출력이 한 줄도 안 나옵니다. 여러분이 망가뜨린 것이 아닙니다. 실수로 풀었다면 다시 // 를 붙이면 그대로 돌아옵니다.

이런 실수를 합니다

대괄호 안에 따옴표를 빼먹기

```
console.log(user[name]);
```

name 이라는 '변수'를 찾습니다. 없으면 ReferenceError. user["name"] 처럼 따옴표를 붙이거나, user.name 을 쓰세요.

이런 실수를 합니다

#### 배열과 객체의 괄호를 헷갈리기

```
const wrong = [name: "김민준"];
```

SyntaxError.

이름표가 있으면 객체이므로 중괄호 {} 입니다.

이런 실수를 합니다

#### 마지막 쉼표 뒤에 값 없이 콜론

```
const wrong2 = { name: "김민준", age: };
```

SyntaxError.

값을 안 적을 거면 속성 자체를 지우세요. 참고로 마지막 속성 뒤의 쉼표(트레일링 콤마)는 있어도 괜찮습니다.

이런 실수를 합니다

#### 객체끼리 === 로 비교

```
console.log({ a: 1 } === { a: 1 });
```

출력          false

내용이 같아도 '다른 객체'라 false 입니다. 배열과 같은 이유입니다.

이런 실수를 합니다

#### 없는 속성인데 에러가 안 나서 못 알아차림

```
console.log(user.nmae);
```

출력          undefined

name 의 오타입니다. 에러가 안 나므로 undefined 가 보이면 속성 이름 오타부터 의심하세요.

## 정리

- 1 **const** 객체 = { 이름: 값, 이름: 값 }; — 중괄호와 콜론
- 2 객체.이름 으로 꺼낸다. 없으면 **undefined**.
- 3 이름이 변수에 들어 있으면 객체[변수] 로 꺼낸다.
- 4 값에는 숫자·문자열·배열·객체 무엇이든 들어간다.
- 5 중첩 객체는 ?. 를 쓰면 중간이 없어도 에러가 나지 않는다.

## 직접 해보기 정답

```

1  const me = { name: "김민준", age: 20 };
   console.log(me);
   // 출력: { name: '김민준', age: 20 }

2  console.log(me.name);           // 출력: 김민준

3  const k = "city";
   console.log(user[k]);           // 출력: 부산

4  console.log(product.seller.rating); // 출력: 4.8

5  const nothing = {};
   console.log(nothing.a?.b);      // 출력: undefined

```

## 객체 추가·수정·삭제

RUN node 개념02\_추가\_수정\_삭제.js

객체는 만든 뒤에도 속성을 넣고 빼고 고칠 수 있습니다. 배열의 push / splice 같은 메소드가 따로 없고, 그냥 = 로 합니다.

### 추가하기

섹션 1

```
const user = {
  name: "김민준",
};

console.log(user);
```

출력 { name: '김민준' }

없는 이름에 값을 넣으면 새 속성이 생깁니다.

```
user.age = 20;
console.log(user);
```

출력 { name: '김민준', age: 20 }

```
user.city = "부산";
console.log(user);
```

출력 { name: '김민준', age: 20, city: '부산' }

대괄호로도 됩니다. 이름이 변수에 들어 있을 때 씁니다.

```
const newKey = "email";
user[newKey] = "test@example.com";
console.log(user);
```

출력 { name: '김민준', age: 20, city: '부산', email: 'test@example.com' }

⇒ 직접 해보기

1

빈 객체를 만들고 title 과 price 속성을 넣어 출력해 보세요.



## 수정하기

섹션 2

이미 있는 이름에 값을 넣으면 덮어쓰기가 됩니다.

```
user.age = 21;
console.log(user.age);
```

출력      21

추가와 수정의 문법이 똑같습니다. 없는 이름이면 → 추가 있는 이름이면 → 수정 그래서 이름을 오타 내면 “수정한 줄 알았는데 새 속성이 생기는” 사고가 납니다.

```
user.agee = 99; // 오타
console.log(user.age);
```

출력      21

```
console.log(user.agee);
```

출력      99

age 는 그대로고 agee 라는 이상한 속성이 생겼습니다.

값을 계산해서 넣을 수도 있습니다.

```
const product = { price: 10000, count: 3 };
product.total = product.price * product.count;
console.log(product);
```

출력      { price: 10000, count: 3, total: 30000 }

▸ 직접 해보기 2

product 의 price 를 20000 으로 바꾸고 출력해 보세요.

## 삭제하기

섹션 3

```
const item = { name: "이어폰", price: 89000, oldPrice: 120000 };

delete item.oldPrice;
console.log(item);
```

출력      { name: '이어폰', price: 89000 }

배열에서는 delete 를 쓰면 안 됐지만(구멍이 생겨서), 객체는 순서가 아니라 이름으로 찾으므로 delete 를 써도 됩니다.

없는 속성을 지워도 에러가 안 납니다.

```
delete item.nothing;  
console.log(item);
```

출력 { name: '이어폰', price: 89000 }

실무에서는 지우기보다 null 을 넣는 경우가 더 많습니다. “속성 자체가 없다” 와 “속성은 있는데 값이 비었다” 는 다른 뜻이니까요.

```
item.price = null;  
console.log(item);
```

출력 { name: '이어폰', price: null }

▫ 직접 해보기 3

객체를 만들고 속성 하나를 delete 로 지워 보세요.

## 속성이 있는지 확인하기

섹션 4

```
const config = { theme: "dark", fontSize: 14 };
```

### 방법 1 · undefined 인지 확인

| 코드                                          | 출력    |
|---------------------------------------------|-------|
| console.log(config.theme !== undefined);    | true  |
| console.log(config.language !== undefined); | false |

### 방법 2 · in 연산자 — 더 분명합니다

| 코드                                 | 출력    |
|------------------------------------|-------|
| console.log("theme" in config);    | true  |
| console.log("language" in config); | false |

속성 이름을 따옴표로 감싼다는 점에 주의하세요.

두 방법의 차이는 “값이 undefined 인 속성”에서 드러납니다.

```
const tricky = { value: undefined };
console.log(tricky.value !== undefined);
```

출력      false

```
console.log("value" in tricky);
```

출력      true

속성은 분명히 있습니다. 값이 `undefined` 일 뿐입니다. 그래서 존재 확인은 `in` 이 정확합니다.

조건문에서 이렇게 씁니다.

```
if ("theme" in config) {
  console.log(`테마: ${config.theme}`);
}
```

출력      테마: dark

▸ 직접 해보기

4

config 에 "fontSize" 속성이 있는지 `in` 으로 확인해 보세요.

## const 인데 왜 바뀌나

섹션 5

배열과 같은 이유입니다.

```
const settings = { volume: 50 };

settings.volume = 80; // 속성 바꾸기 - 가능
console.log(settings);
```

출력      { volume: 80 }

이런 실수를 합니다

```
settings = { volume: 80 };
```

```
TypeError: Assignment to constant variable.
```

`const` 는 "settings 라는 이름이 다른 객체를 가리키지 못하게" 막는 것입니다. 객체 '안의 내용'은 막지 않습니다. 그래서 객체도 배열처럼 대부분 `const` 로 만듭니다.

이건 초보자가 반드시 한 번은 당하는 함정입니다.

```
const original = { name: "김민준", age: 20 };
const copy = original; // 복사한 것처럼 보이지만...

copy.age = 99;

console.log(copy.age);
```

출력 99

```
console.log(original.age);
```

출력 99

원본까지 바뀌었습니다!

왜 그럴까요? 객체를 변수에 담을 때 담기는 것은 ‘내용’이 아니라 ‘어디에 있는지’입니다. `copy = original`은 같은 객체를 가리키는 이름표를 하나 더 붙인 것뿐입니다. 집 주소를 적어 준 것이지 집을 새로 지어 준 게 아닙니다.

**복사하기** · 스프레드 문법을 쓰면 됩니다. (09단원에서 자세히 배웁니다)

```
const original2 = { name: "이서연", age: 22 };
const realCopy = { ...original2 };

realCopy.age = 99;

console.log(realCopy.age);
```

출력 99

```
console.log(original2.age);
```

출력 22

이번엔 원본이 지켜졌습니다.

**주의** · 이 방법은 ‘한 겹’만 복사합니다.

속성 값이 숫자·문자열이면 지금처럼 잘 됩니다. 하지만 속성 안에 객체가 또 들어 있으면 그 안쪽은 원본과 그대로 이어져 있습니다. 이 이야기는 개념05 섹션7과 09단원 개념04에서 다시 합니다.

숫자·문자열은 이런 일이 없습니다. 값이 그대로 복사됩니다.

```
let a = 10;
let b = a;
b = 99;
console.log(a);
```

출력 10

정리: 숫자, 문자열, 불리언 → 값이 복사된다 (원본 안전) 객체, 배열 → 위치가 복사된다 (원본 위험)

▸ 직접 해보기 5

객체를 { ... } 로 복사한 뒤 복사본만 바꿔 보고 원본이 그대로인지 확인해 보세요.

## 자주 하는 실수

섹션 7

이런 실수를 합니다

**속성 이름 오타로 새 속성이 생김 (섹션 2에서 봤습니다)**

값이 안 바뀌면 이름 오타부터 확인하세요.

이런 실수를 합니다

**배열 delete 습관을 객체에 그대로 적용하고 반대로도 함**

객체 → delete 써도 됨 배열 → delete 쓰면 안 됨 (splice 사용)

이런 실수를 합니다

**in 에 따옴표를 빼먹기**

```
console.log(theme-in-config);
```

theme 이라는 변수를 찾습니다. ReferenceError. "theme" 처럼 문자열로 써야 합니다.

이런 실수를 합니다

**복사한 줄 알고 원본을 망가뜨림 (섹션 6)**

이 실수는 에러도 안 나고 한참 뒤에 이상한 값으로 발견됩니다. 객체를 넘길 때는 항상 "원본이 바뀌어도 되는가" 를 먼저 생각하세요.

## 정리

- 1 객체.이름 = 값 — 없으면 추가, 있으면 수정. 문법이 같다.
- 2 delete 객체.이름 으로 삭제. 객체에는 써도 된다.
- 3 "이름" in 객체 로 존재를 확인한다. 따옴표 필수.
- 4 const 객체도 속성은 바꿀 수 있다.
- 5 copy = original 은 복사가 아니다. { ...original } 로 복사한다. 단, 한 겹만 복사된다. 안에 객체가 또 있으면 그쪽은 이어져 있다. (09단원 개념04)

## 직접 해보기 정답

- 1 

```
const book = {};  
book.title = "자바스크립트 입문";  
book.price = 25000;  
console.log(book);  
// 출력: { title: '자바스크립트 입문', price: 25000 }
```
- 2 

```
product.price = 20000;  
console.log(product);  
// 출력: { price: 20000, count: 3, total: 30000 }  
// total 은 자동으로 다시 계산되지 않습니다. 넣을 때 계산된 값이 그대로입니다.
```
- 3 

```
const temp = { a: 1, b: 2 };  
delete temp.b;  
console.log(temp);           // 출력: { a: 1 }
```
- 4 

```
console.log("fontSize" in config); // 출력: true
```
- 5 

```
const o = { n: 1 };  
const c = { ...o };  
c.n = 100;  
console.log(c.n);           // 출력: 100  
console.log(o.n);           // 출력: 1
```

## 메소드와 this

RUN node 개념03\_메소드와\_this.js

객체의 값에는 함수도 들어갈 수 있습니다. 객체 안에 들어 있는 함수를 '메소드'라고 부릅니다.

사실 우리는 이미 계속 써 왔습니다.

```
배열.push() ← 배열이라는 객체의 메소드
문자열.split() ← 문자열의 메소드
console.log() ← console 이라는 객체의 log 메소드
```

### 객체 안에 함수 넣기

섹션 1

```
const counter = {
  count: 0,
  increase: function () {
    console.log("증가시킵니다");
  },
};

counter.increase();
```

출력      증가시킵니다

값을 꺼낼 때처럼 점으로 접근하고, 뒤에 괄호를 붙여 실행합니다.

```
counter.count      ← 속성 (값)
counter.increase() ← 메소드 (실행)
```

괄호를 빼면 함수 자체가 나옵니다. (05단원에서 배웠죠)

```
console.log(typeof counter.increase);
```

출력      function

▸ 직접 해보기 1

greet 라는 메소드를 가진 객체를 만들어 "안녕하세요" 를 출력해 보세요.

`function`이라는 글자를 생략할 수 있습니다. 요즘은 이 형태를 씁니다.

```
const calculator = {
  add(a, b) {
    return a + b;
  },
  subtract(a, b) {
    return a - b;
  },
};

console.log(calculator.add(3, 4));
```

출력 7

```
console.log(calculator.subtract(10, 3));
```

출력 7

세 가지 다 같은 뜻입니다. `add: function(a, b) { ... }` ← 옛날 방식

`add(a, b) { ... }` ← 줄여 쓰기 (권장)

`add: (a, b) => { ... }` ← 화살표 (this 때문에 주의, 섹션 5)

▫ 직접 해보기 2

곱하기 메소드를 줄여 쓰기 형태로 추가해 보세요.

## this — 나를 부른 객체

메소드 안에서 같은 객체의 다른 속성을 쓰려면 `this`를 씁니다.

```
const user = {
  name: "김민준",
  age: 20,
  introduce() {
    console.log(`저는 ${this.name}이고 ${this.age}살입니다`);
  },
};

user.introduce();
```



출력          저는 김민준이고 20살입니다

this 는 “이 메소드를 부른 객체”를 가리킵니다. user.introduce() 라고 불렀으니 this 는 user 입니다.

외우기 쉬운 규칙 하나만 기억하세요.

```
this = 부를 때 점( . ) 왼쪽에 있는 것

user.introduce()
——
이것이 this
```

이 규칙 하나로 이 파일의 모든 경우가 설명됩니다. 점 왼쪽에 아무것도 없이 그냥 introduce() 로 부르면? 가리킬 것이 없어집니다. 그게 뒤에 나오는 실수의 정체입니다.

this 를 빼면 어떻게 될까요?

```
const user2 = {
  name: "이서연",
  introduceBad() {
```

이런 실수를 합니다

```
console.log(name);
```

```
ReferenceError: name
```

이라는 변수는 어디에도 없습니다. 객체 안이라고 해서 속성 이름을 그냥 쓸 수는 없습니다.

```
console.log(this.name);
},
};
user2.introduceBad();
```

출력          이서연

값을 고치는 메소드도 만들 수 있습니다.

```
const counter2 = {
  count: 0,
  increase() {
    this.count++;
  },
  show() {
    console.log(`현재 ${this.count}회`);
  },
};
```

```
counter2.increase();
counter2.increase();
counter2.show();
```

출력      현재 2회

▹ 직접 해보기 3

price 속성과, 그 값을 10% 올리는 메소드를 가진 객체를 만들어 보세요.

## this 대신 객체 이름을 써도 될까

섹션 4

됩니다. 하지만 문제가 있습니다.

```
const product = {
  name: "이어폰",
  showName() {
    console.log(product.name); // this 대신 객체 이름
  },
};

product.showName();
```

출력      이어폰

잘 됩니다. 그런데 이 객체를 복사해서 쓰는 순간 깨집니다.

```
const product2 = { ...product }; // 내용을 복사한 새 객체 (09단원에서 배웁니다)
product2.name = "블루투스 스피커";

product2.showName();
```

출력      이어폰

product2의 이름은 "블루투스 스피커" 인데 "이어폰" 이 나왔습니다. showName 안에 product 라는 '이름' 을 직접 가리키고 있어서, 복사본으로 불러도 원래 객체를 읽어 버리기 때문입니다.

this 를 쓰면 이름과 상관없이 "나를 부른 객체" 를 따라갑니다. 객체를 복사하거나 이름을 바꿔도 안전합니다. 그래서 메소드 안에서는 this 를 쓰는 것이 약속입니다.

## 화살표 함수는 자기 this 가 없다

섹션 5

이게 화살표 함수의 유일한 함정입니다.

```
const badUser = {
  name: "박지훈",
  introduce: () => {
    console.log(`저는 ${this.name}입니다`);
  },
};

badUser.introduce();
```

출력      저는 undefined입니다

왜 `undefined` 일까요? 화살표 함수는 자기만의 `this` 를 만들지 않습니다. 그래서 `this` 가 `badUser` 가 아니라 '바깥의 `this`' 를 그대로 씁니다. 지금은 파일 맨 바깥이라 거기에 `name` 이 없어서 `undefined` 입니다.

일반 함수(줄여 쓰기)로 바꾸면 정상입니다.

```
const goodUser = {
  name: "박지훈",
  introduce() {
    console.log(`저는 ${this.name}입니다`);
  },
};

goodUser.introduce();
```

출력      저는 박지훈입니다

규칙 하나만 기억하세요.

객체의 메소드는 화살표 함수로 만들지 않는다.

반대로 08단원에서 배울 콜백(배열 메소드에 넘기는 함수)에서는 화살표 함수가 오히려 안전합니다. 그때 다시 설명합니다.

▫ 직접 해보기

4

위 `badUser` 를 정상 동작하도록 고쳐 보세요.

## 우리가 이미 쓰던 것들도 메소드였다

섹션 6

```
const list = [3, 1, 2];
list.push(4); // 배열 객체의 메소드
console.log(list);
```

출력      [ 3, 1, 2, 4 ]

```
const text = "안녕하세요";
console.log(text.includes("안녕")); // 문자열의 메소드
```

출력            true

```
console.log(Math.round(3.7)); // Math 라는 객체의 메소드
```

출력            4

```
console.log(Math.PI); // Math 의 속성 (괄호 없음)
```

출력            3.141592653589793

괄호가 붙으면 메소드, 안 붙으면 속성입니다.

|           |             |
|-----------|-------------|
| 배열.length | 속성 (괄호 없음)  |
| 배열.push() | 메소드 (괄호 있음) |

이 구분을 헛갈리면 "is not a function" 에러가 납니다.

## 자주 하는 실수

섹션 7

이런 실수를 합니다

**메소드에 괄호를 안 붙임**

```
console.log(calculator.add);
```

출력            [Function: add]

실행 결과가 아니라 함수 자체가 나옵니다. add(1, 2) 로 써야 합니다.

이런 실수를 합니다

**메소드 안에서 this 를 빼먹기 (섹션 3에서 봤습니다)**

이런 실수를 합니다

**객체 메소드를 화살표 함수로 만들기 (섹션 5)**

이런 실수를 합니다

메소드를 꺼내서 따로 부르기

```
const introduceFn = goodUser.introduce;
introduceFn();
```

출력          저는 undefined입니다

에러가 나지 않습니다. 값만 조용히 틀리게 나옵니다. 그래서 더 위험합니다. `goodUser.introduce()` 가 아니라 그냥 `introduceFn()` 으로 불렀기 때문에 "나를 부른 객체" 가 없어서 `this` 가 `goodUser` 를 가리키지 못합니다. 메소드는 반드시 객체에 붙여서 부르세요.

```
goodUser.introduce();
```

출력          저는 박지훈입니다

이런 실수를 합니다

속성 뒤에 괄호를 붙임

```
console.log(list.length());
```

TypeError: list.length is not a function

## 정리

- 1 객체 안의 함수를 메소드라고 한다. 객체.메소드() 로 부른다.
- 2 줄여 쓰기: { **add**(a, b) { ... } }
- 3 메소드 안에서 같은 객체의 속성을 쓰려면 `this.속성` 이라고 쓴다.
- 4 `this` 는 "나를 부른 객체" 다.
- 5 객체의 메소드는 화살표 함수로 만들지 않는다. `this` 가 사라진다.
- 6 속성은 괄호 없이, 메소드는 괄호를 붙여서.

## 직접 해보기 정답

```
1) const hello = {
```

```
  greet() {
    console.log("안녕하세요");
  },
```

```
};  
hello.greet();           // 출력: 안녕하세요
```

2) `const calc = {`

```
  multiply(a, b) {  
    return a * b;  
  },  

```

```
};  
console.log(calc.multiply(3, 4));  // 출력: 12
```

3) `const item = {`

```
  price: 10000,  
  raise() {  
    this.price = Math.round(this.price * 1.1);  
  },  

```

```
};  
item.raise();  
console.log(item.price);  // 출력: 11000
```

4) introduce: () ⇒ { ... } 를

```
introduce() { ... } 로 바꾸면 됩니다.
```

## 객체 순회하기

RUN node 개념04\_객체\_순회하기.js

배열은 for...of 로 돌았습니다. 객체는 순서가 아니라 이름으로 되어 있어서 다른 방법을 씁니다.

|                  |                     |
|------------------|---------------------|
| for...in         | 이름을 하나씩 꺼낸다         |
| Object.keys()    | 이름들을 배열로 만든다        |
| Object.values()  | 값들을 배열로 만든다         |
| Object.entries() | [이름, 값] 쌍들을 배열로 만든다 |

### for...in — 이름을 하나씩

섹션 1

```
const user = {
  name: "김민준",
  age: 20,
  city: "부산",
};

for (const key in user) {
  console.log(key);
}
```

```
출력      name
          age
          city
```

나오는 것은 '이름'입니다. 값이 아닙니다. 값을 꺼내려면 대괄호 표기법을 씁니다. (개념01 섹션 3)

```
for (const key in user) {
  console.log(`${key}: ${user[key]}`);
}
```

```
출력      name: 김민준
          age: 20
          city: 부산
```

user.key 라고 쓰면 안 됩니다. key 라는 이름의 속성을 찾아 undefined 가 됩니다.

```
for (const key in user) {
  console.log(user.key);
}
```

```
출력      undefined
          undefined
          undefined
```

for...of 와 for...in 을 헛갈리지 마세요.

```
for...of → 배열의 '값'
for...in → 객체의 '이름'
```

⇒ 직접 해보기 1

객체를 만들고 for...in 으로 "이름: 값" 을 출력해 보세요.

## Object.keys — 이름들을 배열로

섹션 2

```
const keys = Object.keys(user);
console.log(keys);
```

```
출력      [ 'name', 'age', 'city' ]
```

배열이 되었으니 06단원에서 배운 것을 전부 쓸 수 있습니다.

| 코드                                              | ▶ 출력   |
|-------------------------------------------------|--------|
| <code>console.log(keys.length);</code>          | ▶ 3    |
| <code>console.log(keys.includes("age"));</code> | ▶ true |

```
console.log(keys.join(", "));
```

```
출력      name, age, city
```

속성이 몇 개인지 세는 표준 방법입니다. 객체에는 .length 가 없습니다.

| 코드                                                  | ▶ 출력        |
|-----------------------------------------------------|-------------|
| <code>console.log(user.length);</code>              | ▶ undefined |
| <code>console.log(Object.keys(user).length);</code> | ▶ 3         |

for...of 로 돌 수도 있습니다.

```
for (const key of Object.keys(user)) {
  console.log(`${key} 확인`);
}
```



```
출력      name 확인
          age 확인
          city 확인
```

#### ▹ 직접 해보기 2

객체의 속성 개수를 출력해 보세요.

## Object.values — 값들을 배열로

섹션 3

```
const values = Object.values(user);
console.log(values);
```

```
출력      [ '김민준', 20, '부산' ]
```

값만 필요할 때 편합니다.

```
const scores = { 국어: 90, 영어: 85, 수학: 70 };
const scoreList = Object.values(scores);
console.log(scoreList);
```

```
출력      [ 90, 85, 70 ]
```

배열이 되었으니 합계를 낼 수 있습니다.

```
let sum = 0;
for (const score of scoreList) {
  sum += score;
}
console.log("합계:", sum);
```

```
출력      합계: 245
```

```
console.log("평균:", sum / scoreList.length);
```

```
출력      평균: 81.66666666666667
```

```
console.log("평균:", (sum / scoreList.length).toFixed(1));
```

```
출력      평균: 81.7
```

#### ▹ 직접 해보기 3

scores 의 값 중 최고점을 구해 보세요.

```
const entries = Object.entries(user);
console.log(entries);
```

출력      [ [ 'name', '김민준' ], [ 'age', 20 ], [ 'city', '부산' ] ]

**이름, 값** · 짝이 담긴 배열들의 배열입니다.

```
console.log(entries[0]);
```

출력      [ 'name', '김민준' ]

코드

```
console.log(entries[0][0]);
```

```
console.log(entries[0][1]);
```

▶ 출력

▶ name

▶ 김민준

for...of 로 돌면서 인덱스로 꺼내 씁니다.

```
for (const entry of Object.entries(user)) {
  console.log(`${entry[0]} = ${entry[1]}`);
}
```

출력      name = 김민준  
            age = 20  
            city = 부산

entry[0], entry[1] 이 읽기 불편하죠? 09단원의 구조분해를 배우면 이렇게 쓸 수 있습니다.

```
for (const [key, value] of Object.entries(user)) {
  console.log(`${key} = ${value}`);
}
```

출력      name = 김민준  
            age = 20  
            city = 부산

훨씬 읽기 좋습니다. 지금은 이런 게 있다는 것만 봐 두세요.

▹ 직접 해보기 4

Object.entries 로 scores 를 "과목: 점수" 형태로 출력해 보세요.

## 무엇을 쓸까

섹션 5

이름만 필요하다 → `Object.keys` 값만 필요하다 → `Object.values` 둘 다 필요하다 → `Object.entries`  
(또는 `for...in`)

실무에서는 `for...in` 보다 `Object.keys` / `entries` 를 더 많이 씁니다. 결과가 배열이라 06단원의 메소드들과 08단원의 `filter` / `map` 을 이어 쓸 수 있기 때문입니다.

조건에 맞는 것만 골라내는 예

```
const stock = { 아메리카노: 10, 라떼: 0, 케이크: 3, 쿠키: 0 };

const soldOut = [];
for (const entry of Object.entries(stock)) {
```

`entry[0]` 은 이름, `entry[1]` 은 개수입니다. (섹션 4에서 봤습니다)

```
  if (entry[1] === 0) {
    soldOut.push(entry[0]);
  }
}

console.log("품절:", soldOut.join(", "));
```

출력      품절: 라떼, 쿠키

09단원의 구조분해를 배우면 `entry[0]`, `entry[1]` 대신 `for (const [name, count] of ...)` 처럼 이름을 바로 붙여 쓸 수 있습니다.

▹ 직접 해보기 5

`stock` 에서 재고가 1개 이상인 메뉴만 모아 보세요.

## 자주 하는 실수

섹션 6

이런 실수를 합니다

`for...in` 에서 값이 나올 거라 기대 (섹션 1에서 봤습니다)

이런 실수를 합니다

#### 객체에 length 쓰기

```
console.log(user.length);
```

출력 undefined

객체에는 length 가 없습니다. `Object.keys(객체).length` 를 쓰세요.

이런 실수를 합니다

#### `Object.keys` 를 `object.keys` 로 씀 → `ReferenceError`

```
console.log(object.keys(user));
```

ReferenceError: object is not defined object

라는 이름의 변수가 없어서 `.keys` 를 꺼내기도 전에 멈춥니다. 첫 글자가 대문자인 `Object` 입니다.

이런 실수를 합니다

#### 배열에 `for...in` 을 씀

```
const arr = ["a", "b"];
for (const i in arr) {
  console.log(i, typeof i);
}
```

출력 0 string  
1 string

인덱스가 나오는데, 심지어 숫자가 아니라 문자열입니다. 배열에는 `for...of` 를 쓰세요.

이런 실수를 합니다

#### 객체의 순서를 믿기

```
const mixed = { 10: "십", 2: "이", name: "이름" };
console.log(Object.keys(mixed));
```

출력 [ '2', '10', 'name' ]

숫자로 된 이름은 작은 수부터 앞으로 정렬됩니다. 순서가 중요한 데이터는 객체가 아니라 배열에 담으세요.

## 정리

- 1 for (const key in 객체) — 이름이 나온다. 값은 객체[key].
- 2 Object.keys(객체) 이름 배열
- 3 Object.values(객체) 값 배열
- 4 Object.entries(객체) [이름, 값] 짝의 배열
- 5 객체에는 length 가 없다. Object.keys(객체).length 를 쓴다.
- 6 배열은 for...of, 객체는 for...in / Object.keys.

## 직접 해보기 정답

```
1) const book = { title: "입문서", price: 25000 };
   for (const key in book) {
```

```
   console.log(`${key}: ${book[key]}`);
```

```
   }
   // 출력: title: 입문서
   // 출력: price: 25000
```

```
2 console.log(Object.keys(book).length); // 출력: 2
```

```
3 const list = Object.values(scores);
   let max = list[0];
   for (const s of list) {
```

```
   if (s > max) max = s;
```

```
   }
   console.log(max); // 출력: 90
```

```
4) for (const entry of Object.entries(scores)) {
```

```
   console.log(`${entry[0]}: ${entry[1]}`);
```

```
   }
   // 출력: 국어: 90 / 영어: 85 / 수학: 70
```

(09단원을 배우면 for (const [subject, score] of ...) 로 짧아집니다)

```
5) const available = [];  
for (const entry of Object.entries(stock)) {  
  
  if (entry[1] >= 1) available.push(entry[0]);  
  
}  
console.log(available);  
// 출력: [ '아메리카노', '케이크' ]
```

## 객체와 배열 섞어 쓰기

RUN node 개념05\_객체와\_배열\_섞어쓰기.js

실무 데이터는 거의 항상 이 모양입니다.

`{...}, {...}, {...}` ← 객체들이 담긴 배열

상품 목록, 회원 목록, 게시글 목록 — 전부 이렇게 생겼습니다. 서버에서 받아오는 데이터도 대부분 이 형태입니다.

### 객체 배열 만들기

섹션 1

```
const products = [
  { name: "아메리카노", price: 4000, stock: 10 },
  { name: "라떼", price: 4500, stock: 0 },
  { name: "케이크", price: 6000, stock: 3 },
];

console.log(products.length);
```

출력 3

배열이니까 인덱스로 꺼내고, 꺼낸 것은 객체니까 점으로 접근합니다.

```
console.log(products[0]);
```

출력 { name: '아메리카노', price: 4000, stock: 10 }

코드

```
console.log(products[0].name);
```

```
console.log(products[1].price);
```

▶ 출력

▶ 아메리카노

▶ 4500

순서대로 읽으세요.

```
products      → 배열
products[1]   → 그중 두 번째, 객체
products[1].price → 그 객체의 price
```

값을 바꿀 수도 있습니다.

```
products[1].stock = 5;
console.log(products[1]);
```

출력 { name: '라떼', price: 4500, stock: 5 }

▹ 직접 해보기 1

products 의 세 번째 상품 이름을 출력해 보세요.

## 반복하며 꺼내기

섹션 2

```
for (const product of products) {
  console.log(`${product.name} ${product.price}원`);
}
```

출력 아메리카노 4000원  
라떼 4500원  
케이크 6000원

번호가 필요하면 일반 for 문을 씁니다.

```
for (let i = 0; i < products.length; i++) {
  console.log(`${i + 1}. ${products[i].name}`);
}
```

출력 1. 아메리카노  
2. 라떼  
3. 케이크

▹ 직접 해보기 2

각 상품의 이름과 재고를 한 줄씩 출력해 보세요.

## 조건으로 찾고 거르기

섹션 3



## — 하나 찾기

```
let found = null;
for (const product of products) {
  if (product.name === "케이크") {
    found = product;
    break;
  }
}
console.log(found);
```

출력 { name: '케이크', price: 6000, stock: 3 }

```
console.log(found.price);
```

출력 6000

못 찾았을 때를 대비해 `null` 로 시작했습니다. `found` 를 바로 쓰기 전에 확인하는 습관을 들이세요.

```
if (found) {
  console.log(`${found.name} 있습니다`);
}
```

출력 케이크 있습니다

## — 여러 개 거르기

```
const cheap = [];
for (const product of products) {
  if (product.price <= 4500) {
    cheap.push(product);
  }
}
console.log(cheap.length);
```

출력 2

```
for (const p of cheap) {
  console.log(p.name);
}
```

출력 아메리카노  
라떼

08단원에서 배울 `find` 와 `filter` 를 쓰면 각각 한 줄이 됩니다.

```
console.log(products.find((p) => p.name === "케이크").price);
```

출력          6000

```
console.log(products.filter((p) => p.price <= 4500).length);
```

출력          2

⇒ 직접 해보기 3

재고가 0인 상품이 있는지 찾아 이름을 출력해 보세요. (섹션 1에서 라떼 재고를 5로 바꿨으니 없을 겁니다)

## 합계와 통계

섹션 4

```
const cart = [
  { name: "아메리카노", price: 4000, count: 2 },
  { name: "케이크", price: 6000, count: 1 },
  { name: "쿠키", price: 3000, count: 3 },
];

let totalPrice = 0;
let totalCount = 0;

for (const item of cart) {
  totalPrice += item.price * item.count;
  totalCount += item.count;
}

console.log(`${totalCount}개 / ${totalPrice}원`);
```

출력          6개 / 23000원

영수증을 만들어 봅시다.

```
for (const item of cart) {
  const sub = item.price * item.count;
  console.log(`${item.name} x${item.count} = ${sub}원`);
}
```

출력          아메리카노 x2 = 8000원  
                케이크 x1 = 6000원  
                쿠키 x3 = 9000원

```
console.log(`합계 ${totalPrice}원`);
```

출력      합계 23000원

▹ 직접 해보기

4

cart 에서 가장 비싼 상품(단가 기준)의 이름을 출력해 보세요.

## 객체 안에 배열

섹션 5

반대 방향도 혼합니다.

```
const student = {
  name: "김민준",
  scores: [90, 85, 70],
  hobbies: ["독서", "등산"],
};
```

```
console.log(student.scores);
```

출력      [ 90, 85, 70 ]

코드

▶ 출력

```
console.log(student.scores[0]);
```

▶ 90

```
console.log(student.scores.length);
```

▶ 3

```
console.log(student.hobbies.join(", "));
```

▶ 독서, 등산

반복해서 합계 구하기

```
let scoreSum = 0;
for (const score of student.scores) {
  scoreSum += score;
}
console.log(`${student.name} 총점 ${scoreSum}점`);
```

출력      김민준 총점 245점

▹ 직접 해보기

5

student 의 평균 점수를 소수 첫째 자리까지 출력해 보세요.

## 깊게 중첩된 데이터

섹션 6

배열 안에 객체, 그 객체 안에 또 배열이 있는 경우입니다.

```
const classes = [
  { name: "1반", students: ["김민준", "이서연"] },
  { name: "2반", students: ["박지훈", "최유진", "정하늘"] },
];
```

여기서부터 세 겹입니다. 먼저 모양을 그림으로 잡고 들어갑시다.

classes ————— 배열 (1겹)

```
└ [0] { name, students } — 객체      (2겹)
  │      └ students ————— 배열      (3겹)
  │          └ [0] "김민준"
  │              └ [1] "이서연"
  └ [1] { name, students }
      └ students
          └ [0] "박지훈" ...
```

꺼내는 법도 그림 그대로 따라 내려갑니다.

```
classes[1]          → { name: '2반', students: [...] }
classes[1].students → [ '박지훈', '최유진', '정하늘' ]
classes[1].students[0] → '박지훈'
```

그래서 아래 반복문이 두 겹인 것입니다. 바깥 반복문이 반(2겹)을, 안쪽 반복문이 그 반의 학생(3겹)을 돕니다.

반 이름과 인원 수

```
for (const c of classes) {
  console.log(`${c.name}: ${c.students.length}명`);
}
```

```
출력      1반: 2명
          2반: 3명
```

전체 학생 이름 (중첩 반복문)

```
for (const c of classes) {
  for (const s of c.students) {
    console.log(`${c.name} - ${s}`);
  }
}
```

```
출력      1반 - 김민준
          1반 - 이서연
          2반 - 박지훈
          2반 - 최유진
          2반 - 정하늘
```

전체 인원

```
let totalStudents = 0;
for (const c of classes) {
  totalStudents += c.students.length;
}
console.log("전체:", totalStudents);
```

```
출력      전체: 5
```

깊어질수록 헛갈립니다. 한 단계씩 짚어 보면서 확인하세요.

```
console.log(classes[1]);
```

```
출력      { name: '2반', students: [ '박지훈', '최유진', '정하늘' ] }
```

```
console.log(classes[1].students);
```

```
출력      [ '박지훈', '최유진', '정하늘' ]
```

```
console.log(classes[1].students[0]);
```

```
출력      박지훈
```

## 자주 하는 실수

섹션 7

이런 실수를 합니다

**배열에 점을 바로 붙이기**

```
console.log(products.name);
```

```
출력      undefined
```

products 는 배열입니다. 먼저 [0] 으로 객체를 꺼내야 합니다.

이런 실수를 합니다

#### 객체에 인덱스를 쓰기

```
const one = { name: "아메리카노" };  
console.log(one[0]);
```

출력 undefined

객체는 이름으로 꺼냅니다. one.name 입니다.

이런 실수를 합니다

#### 못 찾았는데 바로 쓰기

```
let notFound = null;  
for (const product of products) {  
  if (product.name === "녹차라떼") {  
    notFound = product;  
  }  
}
```

TypeError: Cannot read properties of null (reading 'price')

찾기 전에 반드시 확인하세요.

```
console.log(notFound?.price ?? "해당 상품 없음");
```

출력 해당 상품 없음

이런 실수를 합니다

#### 객체 배열을 복사했다고 생각하기

...배열 · 은 배열을 복사하는 문법입니다. (09단원에서 자세히 배웁니다)

```
const copy = [...products]; // 배열은 새로 만들어지지만  
copy[0].name = "바뀐이름"; // 안의 객체는 원본과 같은 것을 가리킵니다  
console.log(products[0].name);
```

출력 바뀐이름

원본까지 바뀌었습니다. 안쪽 객체까지 복사하려면 따로 처리해야 합니다.

## 정리

- 1 [ {...}, {...} ] 객체 배열이 실무 데이터의 기본 모양이다.
- 2 배열[인덱스].속성 순서로 꺼낸다.
- 3 for...of 로 돌면서 item.속성 을 쓴다.
- 4 찾기 결과는 null 로 시작하고, 쓰기 전에 반드시 확인한다.
- 5 헛갈리면 한 단계씩 console.log 로 찍어 본다.

## 직접 해보기 정답

```

1 console.log(products[2].name);           // 출력: 케이크
2 for (const p of products) {

console.log(`${p.name} ${p.stock}개`);

}

3) let soldOut = null;
for (const p of products) {

if (p.stock === 0) soldOut = p;

}
console.log(soldOut ? soldOut.name : "품절 상품 없음");
// 출력: 품절 상품 없음

4) let top = cart[0];
for (const item of cart) {

if (item.price > top.price) top = item;

}
console.log(top.name);                     // 출력: 케이크

```

```
5) let sum = 0;
   for (const s of student.scores) sum += s;
   console.log((sum / student.scores.length).toFixed(1));
   // 출력: 81.7
```



# 객체와 메소드

```
RUN node 연습문제.js
```

TODO 자리에 코드를 쓰고, '기대 출력'과 같은지 확인하세요. 1~17은 기본, 18은 응용, 19는 [도전], 20은 에러 확인입니다.

객체를 `console.log` 로 찍으면 `{ name: '값', age: 20 }` 모양으로 나옵니다.

## 문제 1 (개념01)

name 이 "이서연", age 가 22 인 객체 `user1` 을 만들어 출력하세요.

기대 출력    { name: '이서연', age: 22 }

여기에 코드를 쓰세요

## 문제 2 (개념01 점 표기법)

아래 객체에서 `price` 를 꺼내 출력하세요.

```
const book2 = { title: "자바스크립트 입문", price: 25000, author: "김작가" };
```

기대 출력    25000

여기에 코드를 쓰세요

### 문제 3 (개념01 대괄호 표기법)

아래 변수에 담긴 이름을 이용해 book2 에서 값을 꺼내 출력하세요. 점 표기법으로는 안 됩니다.

```
const key3 = "author";
```

기대 출력     김작가

여기에 코드를 쓰세요

### 문제 4 (개념01 중첩 객체)

아래 객체에서 판매자 이름을 꺼내 출력하세요.

```
const product4 = {  
  name: "무선 이어폰",  
  seller: { name: "봄날전자", rating: 4.8 },  
};
```

기대 출력     봄날전자

여기에 코드를 쓰세요

### 문제 5 (개념01 옵셔널 체이닝)

아래 빈 객체에서 seller.name 을 에러 없이 꺼내려 합니다. 값이 없으면 "판매자 미정" 이 나오도록 한 줄로 출력하세요. 힌트: ?. 와 ??

```
const empty5 = {};
```

기대 출력     판매자 미정

여기에 코드를 쓰세요

문제 6 (개념02 추가)

빈 객체 car6 을 만들고 brand 에 "현대", year 에 2024 를 넣어 출력하세요.

기대 출력 { brand: '현대', year: 2024 }

여기에 코드를 쓰세요

문제 7 (개념02 수정)

아래 객체의 stock 을 0 으로 바꾸고 출력하세요.

```
const item7 = { name: "케이크", stock: 3 };
```

기대 출력 { name: '케이크', stock: 0 }

여기에 코드를 쓰세요

문제 8 (개념02 삭제)

아래 객체에서 oldPrice 속성을 지우고 출력하세요.

```
const sale8 = { name: "이어폰", price: 89000, oldPrice: 120000 };
```

기대 출력 { name: '이어폰', price: 89000 }

여기에 코드를 쓰세요

## 문제 9 (개념02 in)

아래 객체에 "theme" 속성이 있는지, "language" 속성이 있는지 차례로 출력하세요.

```
const config9 = { theme: "dark", fontSize: 14 };
```

기대 출력     true  
                 false

여기에 코드를 쓰세요

## 문제 10 (개념02 복사)

아래 객체를 '진짜로' 복사해서 copy10 을 만들고, copy10 의 age 만 99 로 바꾼 뒤 두 객체를 차례로 출력하세요. 원본은 그대로여야 합니다.

```
const origin10 = { name: "박지훈", age: 30 };
```

힌트: copy10 = origin10 은 복사가 아닙니다. 이름표만 하나 더 붙인 것이라

copy10 을 바꾸면 origin10 도 같이 바뀝니다. (개념02 섹션 6)  
내용을 진짜로 복사하려면 개념02 섹션 6에 나온 { ...객체 } 를 쓰세요.

기대 출력     { name: '박지훈', age: 99 }  
                 { name: '박지훈', age: 30 }

여기에 코드를 쓰세요

**문제 11** (개념03 메소드)

name 속성과, "안녕하세요, 000입니다" 를 출력하는 introduce 메소드를 가진 객체 person11 을 만들고 메소드를 호출하세요. 메소드 안에서는 this 를 쓰세요. 화살표 함수로 만들면 안 됩니다.

기대 출력    안녕하세요, 최유진입니다

여기에 코드를 쓰세요

**문제 12** (개념03 this 로 값 바꾸기)

count 가 0 이고, increase 메소드로 1씩 늘리는 객체 counter12 를 만드세요. increase 를 세 번 부른 뒤 count 를 출력하세요.

기대 출력    3

여기에 코드를 쓰세요

**문제 13** (개념04 for...in)

아래 객체를 for...in 으로 돌면서 "이름: 값" 형태로 출력하세요.

```
const user13 = { name: "정하늘", age: 28, city: "대구" };
```

기대 출력     name: 정하늘  
                 age: 28  
                 city: 대구

여기에 코드를 쓰세요

## 문제 14 (개념04 Object.keys)

아래 객체의 속성 개수를 출력하세요.

```
const scores14 = { 국어: 90, 영어: 85, 수학: 70, 과학: 95 };
```

기대 출력     4

여기에 코드를 쓰세요

## 문제 15 (개념04 Object.values)

위 scores14 의 점수 합계를 출력하세요.

기대 출력     340

여기에 코드를 쓰세요

문제 16 (개념05 객체 배열)

아래는 객체가 담긴 배열입니다. 실무 데이터가 거의 이 모양입니다. 두 번째 상품의 이름과, 세 번째 상품의 가격을 차례로 출력하세요.

```
const products16 = [
  { name: "아메리카노", price: 4000, stock: 10 },
  { name: "라떼", price: 4500, stock: 0 },
  { name: "케이크", price: 6000, stock: 3 },
];
```

기대 출력

라떼  
6000

여기에 코드를 쓰세요

07

문제 17 (개념05 객체 배열 반복)

products16 을 for...of 로 돌면서 “이름 가격원” 을 한 줄씩 출력하세요. 그다음 재고가 0인 상품의 이름을 “품절: 000” 형태로 출력하세요.

기대 출력

아메리카노 4000원  
라떼 4500원  
케이크 6000원  
품절: 라떼

여기에 코드를 쓰세요

문제 18 [응용]

아래 장바구니에서 각 줄과 합계를 출력하세요. 각 줄의 금액은 price × count 입니다.

```
const cart16 = [
  { name: "아메리카노", price: 4000, count: 2 },
  { name: "케이크", price: 6000, count: 1 },
  { name: "쿠키", price: 3000, count: 3 },
];
```

기대 출력     아메리카노 x2 = 8000원  
                  케이크 x1 = 6000원  
                  쿠키 x3 = 9000원  
                  합계 23000원

여기에 코드를 쓰세요

## 문제 19 [도전]

아래 학생 목록에서 평균 점수가 가장 높은 학생의 이름과 평균을 출력하세요. 평균은 소수 첫째 자리까지 표시합니다.

```
const students17 = [
  { name: "김민준", scores: [90, 85, 70] },
  { name: "이서연", scores: [95, 88, 92] },
  { name: "박지훈", scores: [60, 75, 80] },
];
```

기대 출력     이서연 91.7

여기에 코드를 쓰세요

## 문제 20 (에러 확인 - 맨 마지막)

아래 두 줄의 주석을 풀고 실행해서 어떤 에러가 나는지 확인하세요. 확인했으면 반드시 다시 주석 처리하세요.



```
const empty18 = {};  
console.log(empty18.seller.name);
```

무슨 에러가 났나요? 에러 없이 처리하려면 어떤 기호를 쓰면 될까요?

답

## 객체와 메소드

RUN node 심화문제.js

### — 이걸 선택 과제입니다

연습문제를 다 푼 사람만 하세요. 안 풀어도 다음 단원에 지장 없습니다.

주의 · 09단원의 구조분해( `const { a } = 객체` )는 아직 안 배웠습니다.

entry[0] / entry[1] 이나 점 접근으로 푸세요.

### 심화1 스스로 관리하는 가게 객체

아래 store 객체에 메소드 세 개를 추가하세요.

sell(name, count) 재고를 count 만큼 줄인다.

재고가 모자라면 줄이지 말고 "재고 부족" 을 출력한다.

restock(name, count) 재고를 count 만큼 늘린다. report() 전체 재고를 "아메리카노 8개 / 라떼 0개" 형태로 출력한다.

힌트: 메소드 안에서 자기 items 를 쓰려면 this.items 입니다. (개념03)

report 는 Object.entries 로 돌면서 문자열을 이어 붙인 뒤 마지막에 출력하세요.  
(구조분해를 안 쓰므로 entry[0] 이 이름, entry[1] 이 개수입니다)

```
const store = {
  items: {
    아메리카노: 10,
    라떼: 3,
  },
}
```

여기에 메소드 세 개를 쓰세요

```
};
```

아래는 그대로 두고 실행하세요.

```
store.sell("아메리카노", 2);
store.sell("라떼", 5);
store.restock("라떼", 4);
store.report();
```

기대 출력      재고 부족  
아메리카노 8개 / 라떼 7개

## 심화 2 객체 배열 집계

아래 주문 목록에서 다음 세 가지를 구해 출력하세요.

① 총 매출 (가격 × 개수의 합) ② 가장 비싼 주문 한 건의 메뉴 이름 ③ 메뉴별 판매 개수를 담은 객체

힌트: ③은 빈 객체 {} 를 하나 만들고, 주문을 돌면서

"이미 그 이름이 있으면 더하고, 없으면 새로 넣는다" 를 반복합니다.  
개념02에서 배운 대괄호 접근( 객체[변수] )을 쓰세요.

```
const orders = [
  { menu: "아메리카노", price: 4000, count: 2 },
  { menu: "케이크", price: 6000, count: 1 },
  { menu: "아메리카노", price: 4000, count: 3 },
  { menu: "라떼", price: 4500, count: 2 },
];
```

기대 출력      총 매출: 35000원  
가장 비싼 주문: 아메리카노  
{ '아메리카노': 5, '케이크': 1, '라떼': 2 }

(한글 속성 이름은 출력할 때 따옴표가 붙습니다. 정상입니다)

여기에 코드를 쓰세요

### 심화 3 없을 수도 있는 값 안전하게 꺼내기

아래 세 사용자에서 "도시" 를 꺼내 출력하세요. 주소가 없거나 도시가 없으면 "정보 없음" 을 출력해야 합니다. 어떤 경우에도 에러가 나면 안 됩니다.

힌트: 개념01에서 배운 ?. 와 03단원의 || 를 함께 쓰면 한 줄로 됩니다.

```
const users = [
  { name: "김민준", address: { city: "부산" } },
  { name: "이서연", address: {} },
  { name: "박지훈" },
];
```

기대 출력     김민준: 부산  
                 이서연: 정보 없음  
                 박지훈: 정보 없음

여기에 코드를 쓰세요

#### 07단원 마무리 객체와 메소드

#### 자주 넘어지는 자리

**1** 메소드를 변수에 떼어 담으면 `this` 가 풀리는 것 — 에러가 안 나고 값만 틀립니다

#### 자주 나오는 질문

Q "왜 함수를 객체에 넣어요?"

A 배열 `.push()` 처럼 데이터와 그 데이터를 다루는 동작을 한 덩어리로 묶는 것

# 콜백과 내장메소드

OPEN 08\_콜백과\_내장메소드

```
function doTwice(callback) {  
  callback(); // 넘겨받은 함수를 실행  
  callback();  
  function sayHi() {
```

08

01 콜백이란

02 forEach: 하나씩 처리하기

03 map: 모양을 바꿔 새 배열 만들기

04 filter 와 find: 골라내기

05 reduce: 하나의 값으로 접기

06 sort: 정렬하기

- 연습문제

- 심화문제

## 콜백이란

RUN node 개념01\_콜백이란.js

왜 배우나? 06단원에서 배열을 반복문으로 돌렸습니다. 세기, 거르기, 바꾸기 — 할 때마다 for 문을 처음부터 다시 썼습니다. 매번 같은 뼈대인데도요. “도는 일” 은 자바스크립트에 맡기고 “무엇을 할지” 만 적을 수 있다면 어떨까요? 그러려면 ‘할 일’ 을 함수로 만들어 넘겨야 합니다. 그것이 콜백입니다. 이 단원의 forEach·map·filter·reduce·sort 가 전부 이 방식으로 동작합니다.

05단원에서 “함수도 값이다” 를 배웠습니다. 값이니까 다른 함수에 인자로 넘길 수 있었죠.

이렇게 ‘다른 함수에 넘겨져서 나중에 실행되는 함수’를 콜백이라고 합니다. call(부르다) + back(다시) — “이따가 다시 불러 줘” 라는 뜻입니다.

### 함수를 넘기고, 넘겨받은 쪽이 실행한다

섹션 1

```
function doTwice(callback) {
  callback(); // 넘겨받은 함수를 실행 callback();
}

function sayHi() {
  console.log("안녕!");
}

doTwice(sayHi);
```

출력      안녕!  
          안녕!

넘길 때 괄호를 붙이지 않는다는 점이 핵심입니다.

```
doTwice(sayHi)      함수를 넘김 (맞음)
doTwice(sayHi())   실행 결과인 undefined 를 넘김 (틀림)
```

여기서 sayHi 가 콜백입니다. doTwice 는 “언제 실행할지” 를 정하고, sayHi 는 “무엇을 할지” 를 정합니다. 역할이 나뉘어 있다는 게 콜백의 핵심입니다.

#### ⇒ 직접 해보기 1

“반갑습니다” 를 출력하는 함수를 만들어 doTwice 로 두 번 실행해 보세요.

## 그 자리에서 만들어 넘기기

섹션 2

한 번만 쓸 함수라면 이름을 붙이지 않고 그 자리에서 만듭니다.

```
doTwice(function () {
  console.log("이름 없는 함수입니다");
});
```

출력      이름 없는 함수입니다  
          이름 없는 함수입니다

화살표 함수로 쓰면 훨씬 짧습니다. (05단원 개념05)

```
doTwice(() => {
  console.log("화살표 함수입니다");
});
```

출력      화살표 함수입니다  
          화살표 함수입니다

한 줄이면 중괄호도 없앨 수 있습니다.

```
doTwice(() => console.log("한 줄 화살표"));
```

출력      한 줄 화살표  
          한 줄 화살표

앞으로 나올 배열 메소드들은 전부 이 모양입니다.

```
배열.forEach(() => { ... })
배열.map((x) => ...)
```

처음엔 괄호가 겹쳐 보여 어지럽지만, “함수를 인자로 넣고 있다” 는 것만 기억하세요.

▸ 직접 해보기 2

doTwice 에 화살표 함수로 “화이팅” 을 출력해 보세요.

## 콜백에 값을 넘겨 주기

섹션 3

콜백을 실행하는 쪽이 값을 넘겨 줄 수 있습니다. 이게 진짜 쓸모입니다.

```
function repeat(count, callback) {
  for (let i = 1; i <= count; i++) {
    callback(i); // 몇 번째인지 콜백에 넘겨 줍니다
  }
}

repeat(3, (n) => {
  console.log(`${n}번째 실행`);
});
```

```
출력      1번째 실행
          2번째 실행
          3번째 실행
```

콜백의 매개변수 n 은 어디서 왔을까요? repeat 안에서 callback(i) 라고 부를 때 i 가 넘어온 것입니다. 즉 '넘겨받는 쪽'이 아니라 '실행하는 쪽'이 값을 정합니다.

이름은 마음대로 지어도 됩니다.

```
repeat(2, (whatever) => {
  console.log(`이름은 아무거나: ${whatever}`);
});
```

```
출력      이름은 아무거나: 1
          이름은 아무거나: 2
```

안 받아도 됩니다. 필요 없으면 그냥 안 쓰면 그만입니다.

```
repeat(2, () => {
  console.log("값을 안 받아도 됩니다");
});
```

```
출력      값을 안 받아도 됩니다
          값을 안 받아도 됩니다
```

▹ 직접 해보기

3

repeat 로 1부터 4까지의 제곱을 출력해 보세요.

## 왜 굳이 이렇게 하나

섹션 4

“하는 일”과 “무엇을 할지”를 분리할 수 있기 때문입니다.



```
function processScores(scores, callback) {
  for (const score of scores) {
    callback(score);
  }
}

const scores = [90, 55, 78];
```

같은 processScores 인데 넘기는 함수에 따라 하는 일이 달라집니다.

```
processScores(scores, (s) => console.log(`점수: ${s}`));
```

```
출력      점수: 90
          점수: 55
          점수: 78
```

```
processScores(scores, (s) => console.log(s >= 60 ? "합격" : "불합격"));
```

```
출력      합격
          불합격
          합격
```

“배열을 하나씩 도는 코드”를 매번 새로 쓰지 않아도 됩니다. 자바스크립트는 아예 이런 함수들을 배열에 기본으로 넣어 두었습니다. 그것이 forEach, map, filter, find, reduce, sort 입니다. 다음 파일부터 배웁니다.

미리 맛보기 — 위 코드는 사실 이렇게 씁니다.

```
scores.forEach((s) => console.log(`점수: ${s}`));
```

```
출력      점수: 90
          점수: 55
          점수: 78
```

## 콜백에는 화살표 함수를 쓴다

섹션 5

07단원에서 “객체의 메소드는 화살표 함수로 만들지 말라”고 했습니다. 콜백은 반대입니다. 화살표 함수가 더 안전하고 짧습니다.

```
const user = {
  name: "김민준",
  hobbies: ["독서", "등산"],
```

메소드는 일반 함수 (this 가 user 를 가리켜야 하니까)

```
showHobbies() {
```

콜백은 화살표 함수 (바깥의 this 를 그대로 쓰니까)

```
this.hobbies.forEach((hobby) => {  
    console.log(`${this.name}의 취미: ${hobby}`);  
});  
},  
};  
  
user.showHobbies();
```

```
출력      김민준의 취미: 독서  
          김민준의 취미: 등산
```

말로만 하면 안 믿기니 직접 확인해 봅시다. 콜백만 일반 함수로 바꾼 것입니다.

```
const badUser = {  
    name: "김민준",  
    hobbies: ["독서", "등산"],  
  
    showHobbies() {  
        this.hobbies.forEach(function (hobby) {  
            console.log(`${this.name}의 취미: ${hobby}`);  
        });  
    },  
};  
  
badUser.showHobbies();
```

```
출력      undefined의 취미: 독서  
          undefined의 취미: 등산
```

07단원과 반대라서 헷갈리시죠? 사실 이유는 하나입니다.

화살표 함수의 this = 그 화살표 함수를 감싸고 있는 코드의 this

07단원에서는 화살표 함수를 '객체 안'에 바로 놓았습니다.

그것을 감싼 것은 파일 맨 바깥이고, 거기에는 name 이 없어서 undefined 였습니다.

여기서는 화살표 함수를 'showHobbies 메소드 안'에 놓았습니다.

그것을 감싼 것은 showHobbies 이고, 그 안의 this 는 user 라서 잘 됩니다.

반대로 일반 함수(function)는 '자기만의 this'를 새로 만들어 버립니다. 그래서 위 badUser 처럼 바깥의 this 와 끊겨 undefined 가 나옵니다.

같은 성질이 한 번은 독이 되고 한 번은 약이 된 것입니다. 결론만 쓰면 이렇습니다.

객체의 메소드 → 일반 함수 (줄여 쓰기)  
콜백 → 화살표 함수

## 자주 하는 실수

섹션 6

이런 실수를 합니다

넘길 때 괄호를 붙임

```
doTwice(sayHi());
```

`sayHi()` 가 먼저 실행되어 "안녕!" 이 한 번 찍히고, 그 결과인 `undefined` 가 넘어가서 `TypeError` 가 납니다.

이런 실수를 합니다

콜백 안에서 `return` 하면 바깥 함수가 끝날 거라 기대

```
function findFirst(numbers) {
  numbers.forEach((n) => {
    if (n > 10) {
      return n; // 이 return 은 콜백만 끝냅니다
    }
  });
  return "못 찾음";
}
console.log(findFirst([5, 20, 30]));
```

출력      못 찾음

콜백의 `return` 은 콜백 자신만 끝냅니다. `findFirst` 는 계속 진행됩니다. 이럴 때 쓰는 것이 개념04에서 배울 `find` 입니다.

이런 실수를 합니다

콜백 자리에 함수가 아닌 것을 넣기

```
doTwice("안녕");
```

```
TypeError: callback is not a function
```

이런 실수를 합니다

화살표 함수에서 중괄호를 쓰고 `return` 을 빠뜨림

```
const double = (n) => {  
  n * 2;  
};  
console.log(double(5));
```

출력            undefined

중괄호를 쓰면 `return` 이 필요합니다.  $(n) \Rightarrow n * 2$  로 쓰거나 `return` 을 넣으세요.

## 정리

- 1 다른 함수에 넘겨져 나중에 실행되는 함수를 콜백이라고 한다.
- 2 넘길 때 괄호를 붙이지 않는다. 붙이면 '결과'가 넘어간다.
- 3 콜백의 매개변수는 '실행하는 쪽'이 넘겨 준다.
- 4 콜백 덕분에 "도는 방법"과 "할 일"을 나눌 수 있다.
- 5 콜백은 화살표 함수로 쓴다. (객체 메소드는 반대)
- 6 콜백 안의 `return` 은 콜백만 끝낸다.

## 직접 해보기 정답

```
1) function sayHello() {
```

```
  console.log("반갑습니다");
```

```
}  
doTwice(sayHello);
```

```
2 doTwice(() => console.log("화이팅"));
```

```
3 repeat(4, (n) => console.log(n * n));  
// 출력: 1 / 4 / 9 / 16 (한 줄씩)
```

# forEach: 하나씩 처리하기

RUN node 개념02\_forEach.js

배열의 값을 하나씩 꺼내 콜백에 넘겨 주는 메소드입니다. 06단원에서 for...of 로 하던 일을 대신합니다.

왜 굳이 갈아타나? for...of 도 충분히 좋습니다. 다만 forEach 는 인덱스를 자동으로 넘겨 주고, 무엇보다 다음 파일부터 배울 map·filter·reduce 와 생김새가 똑같습니다. 여기서 모양에 익숙해지면 나머지가 전부 쉬워집니다.

```
배열.forEach((값) => { 할 일 });
```

## for...of 를 forEach 로

섹션 1

```
const fruits = ["사과", "바나나", "포도"];
```

### — for...of 버전

```
for (const fruit of fruits) {  
  console.log(fruit);  
}
```

출력      사과  
            바나나  
            포도

### — forEach 버전

```
fruits.forEach((fruit) => {  
  console.log(fruit);  
});
```

출력      사과  
            바나나  
            포도

한 줄이면 중괄호를 없앨 수 있습니다.

```
fruits.forEach((fruit) => console.log(fruit));
```

출력      사과  
            바나나  
            포도

동작을 따라가 보면 `forEach` 가 배열을 처음부터 끝까지 돌면서 매 바퀴 콜백을 부르고, 그때 그 자리의 값을 넘겨 줍니다. `fruit` 라는 이름은 우리가 지은 것입니다. `item`, `f` 무엇이든 됩니다.

#### ▹ 직접 해보기 1

숫자 배열 `[10, 20, 30]` 을 `forEach` 로 출력해 보세요.

## 인덱스도 받을 수 있다

섹션 2

콜백의 두 번째 매개변수로 인덱스가 넘어옵니다.

```
fruits.forEach((fruit, index) => {  
  console.log(`${index}: ${fruit}`);  
});
```

출력      0: 사과  
            1: 바나나  
            2: 포도

번호를 1부터 붙이려면 `index + 1` 을 씁니다.

```
fruits.forEach((fruit, index) => {  
  console.log(`${index + 1}번 ${fruit}`);  
});
```

출력      1번 사과  
            2번 바나나  
            3번 포도

세 번째로는 배열 전체가 넘어옵니다. 거의 안 씁니다.

```
fruits.forEach((fruit, index, array) => {  
  console.log(`${fruit} (전체 ${array.length}개 중 ${index + 1}번째)`);  
});
```

출력      사과 (전체 3개 중 1번째)  
            바나나 (전체 3개 중 2번째)  
            포도 (전체 3개 중 3번째)

필요한 만큼만 받으면 됩니다. 순서는 항상 (값, 인덱스, 배열) 로 고정입니다.

## ▹ 직접 해보기 2

아래 배열을 "1. 국어" 형태로 출력해 보세요. `const subjects = ["국어", "영어", "수학"];`

## 객체 배열에 쓰기

섹션 3

```
const cart = [
  { name: "아메리카노", price: 4000, count: 2 },
  { name: "케이크", price: 6000, count: 1 },
];

cart.forEach((item) => {
  console.log(`${item.name} x${item.count} = ${item.price * item.count}원`);
});
```

출력      아메리카노 x2 = 8000원  
            케이크 x1 = 6000원

합계를 구하려면 밖에 변수를 두고 더합니다.

```
let total = 0;
cart.forEach((item) => {
  total += item.price * item.count;
});
console.log(`합계 ${total}원`);
```

출력      합계 14000원

(합계는 개념05의 reduce 로 더 깔끔하게 쓸 수 있습니다)

## ▹ 직접 해보기 3

cart 의 상품 이름만 한 줄씩 출력해 보세요.

## forEach 는 아무것도 돌려주지 않는다

섹션 4

```
const result = fruits.forEach((fruit) => fruit + "!");
console.log(result);
```

출력      undefined

forEach 의 결과는 언제나 `undefined` 입니다. "각 값으로 무언가를 하는" 메소드이지 "새 배열을 만드는" 메소드가 아닙니다.

새 배열이 필요하다면 다음 파일의 map 을 씁니다.

원본을 바꾸지도 않습니다.

```
const numbers = [1, 2, 3];
numbers.forEach((n) => {
  n = n * 2; // 콜백 안의 n 만 바뀝니다
});
console.log(numbers);
```

출력 [ 1, 2, 3 ]

원본을 바꾸려면 인덱스로 직접 넣어야 합니다. (권장하지 않습니다)

```
const numbers2 = [1, 2, 3];
numbers2.forEach((n, i) => {
  numbers2[i] = n * 2;
});
console.log(numbers2);
```

출력 [ 2, 4, 6 ]

이렇게 쓰지 말고 map 을 쓰세요.

## forEach 는 중간에 멈출 수 없다

섹션 5

break 를 쓰면 에러입니다.

```
fruits.forEach((fruit) => {
  if (fruit === "바나나") break;
```

이런 실수를 합니다

```
});
```

SyntaxError: Illegal break statement break

는 반복문의 문법인데, forEach 는 반복문이 아니라 메소드입니다.

return 은 이번 바퀴만 건너뛹니다. continue 처럼 동작합니다.

```
fruits.forEach((fruit) => {
  if (fruit === "바나나") {
    return; // 이번 바퀴만 건너뛸
  }
  console.log(fruit);
});
```



|    |          |
|----|----------|
| 출력 | 사과<br>포도 |
|----|----------|

그래서 중간에 멈춰야 하면 for 문이나 for...of 를 쓰세요.

```
for (const fruit of fruits) {
  if (fruit === "바나나") {
    break;
  }
  console.log("for...of:", fruit);
}
```

|    |              |
|----|--------------|
| 출력 | for...of: 사과 |
|----|--------------|

정리: 전부 다 돌아도 된다 → forEach (읽기 편함) 중간에 멈춰야 한다 → for / for...of 조건에 맞는 첫 개를 찾는다 → find (개념04)

👉 직접 해보기 4

forEach 로 짝수만 출력해 보세요. (홀수는 return 으로 건너뛰기)

for 문과 비교해서 언제 무엇을

섹션 6

```
const list = ["a", "b", "c"];
```

forEach — 값과 인덱스가 자동으로 넘어옴. 읽기 편함.

```
list.forEach((v, i) => console.log(`forEach ${i} ${v}`));
```

|    |                                           |
|----|-------------------------------------------|
| 출력 | forEach 0 a<br>forEach 1 b<br>forEach 2 c |
|----|-------------------------------------------|

for...of — break / continue 를 쓸 수 있음

```
for (const v of list) {
  console.log(`for...of ${v}`);
}
```

|    |                                        |
|----|----------------------------------------|
| 출력 | for...of a<br>for...of b<br>for...of c |
|----|----------------------------------------|

for — 거꾸로 돌기, 건너뛰기 등 자유로움

```
for (let i = list.length - 1; i >= 0; i--) {
  console.log(`for(거꾸로) ${list[i]}`);
}
```

```
출력      for(거꾸로) c
          for(거꾸로) b
          for(거꾸로) a
```

실무에서는 `forEach` 와 `for...of` 를 가장 많이 씁니다.

## 자주 하는 실수

섹션 7

★ 아래에서 `SyntaxError` 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. 다른 에러 (`ReferenceError`, `TypeError`)는 주석을 풀어도 그 줄에서만 나고 그 앞의 출력은 그대로 다 나옵니다. 그런데 `SyntaxError` 는 다릅니다. 자바스크립트가 파일을 아예 못 읽어서 출력이 한 줄도 안 나옵니다. 여러분이 망가뜨린 것이 아닙니다. 실수로 풀었다면 다시 `//` 를 붙이면 그대로 돌아옵니다.

이런 실수를 합니다

**forEach의 결과를 변수에 담기 (섹션 4에서 봤습니다)**

```
const doubled = numbers.forEach((n) => n * 2); // undefined
```

새 배열이 필요하면 `map` 입니다.

이런 실수를 합니다

**콜백에 괄호를 붙여 넘기기**

```
fruits.forEach(console.log());
```

`console.log()` 가 먼저 실행되어 빈 줄이 하나 찍히고, 그 결과인 `undefined` 가 콜백 자리로 넘어갑니다. `undefined` 는 함수가 아니므로 `TypeError: undefined is not a function` 으로 프로그램이 멈춥니다.

참고로 이건 됩니다. 함수 자체를 넘기는 것이니까요.

```
fruits.forEach(console.log);
```

```
출력      사과 0 [ '사과', '바나나', '포도' ]
          바나나 1 [ '사과', '바나나', '포도' ]
          포도 2 [ '사과', '바나나', '포도' ]
```

값, 인덱스, 배열 세 개가 전부 넘어가서 지저분합니다. 그래서 보통 `(f) => console.log(f)` 처럼 감싸서 씁니다.

이런 실수를 합니다

### break 를 쓰기 (섹션 5)

이런 실수를 합니다

### 화살표 함수 매개변수 순서를 바꿔 쓰기

```
fruits.forEach((index, fruit) => {
  console.log(index, fruit);
});
```

출력          사과 0  
                바나나 1  
                포도 2

이름만 바꿨을 뿐 넘어오는 순서는 (값, 인덱스) 로 고정입니다. 이름에 속아 값과 인덱스가 뒤바뀐 코드를 쓰게 됩니다.

## 정리

- 1 배열.forEach((값, 인덱스) => { ... })
- 2 넘어오는 순서는 (값, 인덱스, 배열) 로 고정. 이름은 마음대로.
- 3 forEach 는 undefined 를 돌려준다. 새 배열이 필요하면 map.
- 4 break 를 쓸 수 없다. return 은 continue 처럼 동작한다.
- 5 중간에 멈춰야 하면 for / for...of 를 쓴다.

## 직접 해보기 정답

- 1 `[10, 20, 30].forEach((n) => console.log(n));`
- 2 `const subjects = ["국어", "영어", "수학"];  
subjects.forEach((s, i) => console.log(`${i + 1}. ${s}`));  
// 출력: 1. 국어 / 2. 영어 / 3. 수학`
- 3 `cart.forEach((item) => console.log(item.name));  
// 출력: 아메리카노 / 케이크`
- 4 `[1, 2, 3, 4, 5].forEach((n) => {`

```
if (n % 2 === 1) return;  
console.log(n);
```

```
});  
// 출력: 2 / 4
```

## map: 모양을 바꿔 새 배열 만들기

RUN node 개념03\_map.js

forEach 는 하나씩 처리만 하고 아무것도 안 돌려줬습니다. map 은 각 값을 바꿔서 '새 배열'을 만들어 돌려줍니다.

```
배열.map((값) => 바꾼값)
```

개수는 그대로입니다. 3개를 넣으면 3개가 나옵니다.

### forEach 와 map 의 차이

섹션 1

```
const numbers = [1, 2, 3];
```

forEach · 결과가 없습니다

```
const r1 = numbers.forEach((n) => n * 2);
console.log(r1);
```

출력 undefined

map · 새 배열이 나옵니다

```
const r2 = numbers.map((n) => n * 2);
console.log(r2);
```

출력 [ 2, 4, 6 ]

원본은 그대로입니다.

```
console.log(numbers);
```

출력 [ 1, 2, 3 ]

06단원에서 반복문으로 쓰던 코드를 비교해 보세요.

```
const doubled = [];
for (const n of numbers) {
  doubled.push(n * 2);
}
console.log(doubled);
```

출력 [ 2, 4, 6 ]

map 이 이 세 줄을 한 줄로 만들어 준 것입니다.

▹ 직접 해보기 1

[1, 2, 3] 의 각 값을 10배로 만든 새 배열을 만들어 보세요.

## 반드시 값을 돌려줘야 한다

섹션 2

map 의 콜백은 '바꾼 값'을 return 해야 합니다.

```
const prices = [1000, 2000, 3000];
```

화살표 한 줄 · return 이 자동으로 됩니다

```
console.log(prices.map((p) => p * 1.1));
```

출력 [ 1100, 2200, 3300.0000000000005 ]

세 번째 값에 이상한 꼬리가 붙었습니다. 소수 오차입니다. (02단원 개념01) 돈 계산은 Math.round 로 다듬습니다.

```
console.log(prices.map((p) => Math.round(p * 1.1)));
```

출력 [ 1100, 2200, 3300 ]

— 중괄호를 쓰면 return 필수

```
console.log(
  prices.map((p) => {
    return Math.round(p * 1.1);
  })
);
```

출력 [ 1100, 2200, 3300 ]

return 을 빠뜨리면 undefined 로 가득 찬 배열이 나옵니다.

```
console.log(
  prices.map((p) => {
    Math.round(p * 1.1); // return 이 없습니다
  })
);
```

출력 [ undefined, undefined, undefined ]

map 을 쓸 때 가장 흔한 실수입니다. `undefined` 가 보이면 `return` 부터 확인하세요.

▹ 직접 해보기 2

가격 배열의 각 값에 500원을 더한 새 배열을 만들어 보세요.

## 자료형을 바꿔도 된다

섹션 3

숫자 배열 → 문자열 배열

```
const scores = [90, 85, 70];
console.log(scores.map((s) => `${s}점`));
```

출력 [ '90점', '85점', '70점' ]

숫자 배열 → 불리언 배열

```
console.log(scores.map((s) => s >= 80));
```

출력 [ true, true, false ]

문자열 배열 → 숫자 배열 (입력창에서 온 값 처리에 자주 씁니다)

```
const inputs = ["10", "20", "30"];
const nums = inputs.map((v) => Number(v));
console.log(nums);
```

출력 [ 10, 20, 30 ]

```
console.log(typeof nums[0]);
```

출력 number

숫자 배열 → 객체 배열

```
console.log(scores.map((s) => ({ score: s })));
```

출력 [ { score: 90 }, { score: 85 }, { score: 70 } ]

객체를 바로 돌려줄 때는 중괄호를 소괄호로 감싸야 합니다. 안 그러면 자바스크립트가 중괄호를 '함수 몸통'으로 오해합니다. (05단원 개념05)

⇒ 직접 해보기 3

["1", "2", "3"] 을 숫자 배열로 바꿔 보세요.

## 객체 배열에서 값만 뽑기

섹션 4

실무에서 가장 많이 쓰는 형태입니다.

```
const users = [
  { name: "김민준", age: 20 },
  { name: "이서연", age: 22 },
  { name: "박지훈", age: 28 },
];
```

이름만 뽑기

```
const names = users.map((user) => user.name);
console.log(names);
```

출력 [ '김민준', '이서연', '박지훈' ]

뽑았으니 06단원의 메소드를 이어 쓸 수 있습니다.

```
console.log(names.join(", "));
```

출력 김민준, 이서연, 박지훈

```
console.log(names.includes("이서연"));
```

출력 true

나이만 뽑기

```
console.log(users.map((user) => user.age));
```

출력 [ 20, 22, 28 ]

모양을 바꿔서 새 객체 배열 만들기



```
const cards = users.map((user) => ({
  title: user.name,
  isAdult: user.age >= 19,
}));
console.log(cards);
```

```
출력      [
  { title: '김민준', isAdult: true },
  { title: '이서연', isAdult: true },
  { title: '박지훈', isAdult: true }
]
```

▹ 직접 해보기

4

users 에서 이름만 뽑아 " / " 로 이어 출력해 보세요.

## 인덱스도 받을 수 있다

섹션 5

```
const menu = ["아메리카노", "라떼", "케이크"];

const numbered = menu.map((item, index) => `${index + 1}. ${item}`);
console.log(numbered);
```

```
출력      [ '1. 아메리카노', '2. 라떼', '3. 케이크' ]
```

join 과 함께 쓰면 화면에 그대로 뿌릴 목록이 됩니다.

```
console.log(numbered.join("\n"));
```

```
출력      1. 아메리카노
          2. 라떼
          3. 케이크
```

한 번에 쓰면 이렇게 됩니다. 10단원에서 화면을 만들 때 이 패턴을 계속 씁니다.

```
console.log(menu.map((item, i) => `${i + 1}. ${item}`).join("\n"));
```

```
출력      1. 아메리카노
          2. 라떼
          3. 케이크
```

▹ 직접 해보기

5

아래 배열에 번호를 붙인 새 배열을 만들어 보세요. `const fruits = ["사과", "포도"]`; 결과는 `[ '1 사과', '2 포도' ]` 형태면 됩니다.

새 배열이 필요하다 → map 그냥 출력하거나 처리만 → forEach

결과를 안 쓸 거면서 map 을 쓰면, 쓸데없는 배열이 만들어집니다. 동작은 하지만 읽는 사람이 "이 결과를 어디에 쓰지?" 하고 헛갈립니다.

**나쁜 예** · 결과를 안 쓰는데 map

```
menu.map((item) => console.log("나쁜 예:", item));
```

출력      나쁜 예: 아메리카노  
            나쁜 예: 라떼  
            나쁜 예: 케이크

— 좋은 예

```
menu.forEach((item) => console.log("좋은 예:", item));
```

출력      좋은 예: 아메리카노  
            좋은 예: 라떼  
            좋은 예: 케이크

## 자주 하는 실수

이런 실수를 합니다

중괄호를 쓰고 **return** 을 빠뜨림 (섹션 2에서 봤습니다)

이런 실수를 합니다

객체를 돌려줄 때 소괄호를 안 감쌘

```
const bad = users.map((u) => { name: u.name });
```

중괄호를 함수 몸통으로 해석해서 **undefined** 가 나옵니다. ({ name: u.name }) 처럼 소괄호로 감싸세요.

```
console.log(users.map((u) => ({ name: u.name })));
```

출력      [ { name: '김민준' }, { name: '이서연' }, { name: '박지훈' } ]

이런 실수를 합니다

개수가 줄어들 거라 기대

```
console.log(scores.map((s) => (s >= 80 ? s : null)));
```

출력 [ 90, 85, null ]

map 은 개수를 바꾸지 않습니다. 걸러 내려면 filter 입니다. (개념04)

이런 실수를 합니다

원본이 바뀔 거라 기대

```
const original = [1, 2, 3];
original.map((n) => n * 10);
console.log(original);
```

출력 [ 1, 2, 3 ]

map 은 새 배열을 '돌려줄' 뿐입니다. 받아서 써야 합니다.

```
const changed = original.map((n) => n * 10);
console.log(changed);
```

출력 [ 10, 20, 30 ]

## 정리

- 1 배열.map((값) => 바꾼값) — 새 배열을 돌려준다. 개수는 그대로.
- 2 콜백은 반드시 값을 돌려줘야 한다. 종괄호를 쓰면 `return` 필수.
- 3 자료형을 바꿔도 된다. 숫자 → 문자열, 객체 → 값 등.
- 4 객체를 돌려줄 때는 `{ ... }` 처럼 소괄호로 감싼다.
- 5 원본은 바뀌지 않는다. 결과를 변수에 받아야 한다.
- 6 결과를 안 쓸 거면 map 이 아니라 `forEach`.

## 직접 해보기 정답

- 1 `console.log([1, 2, 3].map((n) => n * 10));`  
`// 출력: [ 10, 20, 30 ]`
- 2 `console.log(prices.map((p) => p + 500));`  
`// 출력: [ 1500, 2500, 3500 ]`
- 3 `console.log(["1", "2", "3"].map((v) => Number(v)));`  
`// 출력: [ 1, 2, 3 ]`
- 4 `console.log(users.map((u) => u.name).join(" / "));`  
`// 출력: 김민준 / 이서연 / 박지훈`
- 5 `const fs = ["사과", "포도"];`  
`console.log(fs.map((f, i) => `${i + 1} ${f}`));`  
`// 출력: [ '1 사과', '2 포도' ]`

## filter 와 find: 골라내기

```
RUN node 개념04_filter와_find.js
```

map 은 개수를 그대로 두고 모양만 바꿨습니다. 이번에는 조건에 맞는 것만 골라냅니다.

```
filter  조건에 맞는 것을 '전부' 모아 새 배열로
find   조건에 맞는 '첫 번째 하나'만
```

콜백이 `true` 를 돌려주면 통과, `false` 면 탈락입니다.

### filter — 조건에 맞는 것 전부

섹션 1

```
const scores = [90, 55, 78, 40, 88];

const passed = scores.filter((score) => score >= 60);
console.log(passed);
```

```
출력      [ 90, 78, 88 ]
```

원본은 그대로입니다.

```
console.log(scores);
```

```
출력      [ 90, 55, 78, 40, 88 ]
```

06단원에서 반복문으로 쓰던 코드와 비교해 보세요.

```
const passed2 = [];
for (const score of scores) {
  if (score >= 60) {
    passed2.push(score);
  }
}
console.log(passed2);
```

```
출력      [ 90, 78, 88 ]
```

filter 가 이 다섯 줄을 한 줄로 만들어 준 것입니다.

하나도 안 맞으면 빈 배열이 나옵니다. `null` 이 아닙니다.

```
console.log(scores.filter((score) => score > 100));
```

출력 []

```
console.log(scores.filter((score) => score > 100).length);
```

출력 0

개수만 필요하면 .length 를 붙입니다.

```
console.log(`합격자 ${scores.filter((s) => s >= 60).length}명`);
```

출력 합격자 3명

▸ 직접 해보기 1

[12, 7, 30, 5] 에서 10 이상인 값만 골라 보세요.

## filter 는 true/false 만 본다

섹션 2

콜백이 돌려주는 값이 true 면 남고 false 면 빠집니다.

```
const words = ["사과", "", "바나나", "", "포도"];
```

빈 문자열 걸러내기 — 빈 문자열은 falsy 입니다 (01단원)

```
console.log(words.filter((w) => w !== ""));
```

출력 [ '사과', '바나나', '포도' ]

값 자체를 돌려줘도 됩니다. falsy 면 자동으로 걸러집니다.

```
console.log(words.filter((w) => w));
```

출력 [ '사과', '바나나', '포도' ]

숫자 배열에서 0 과 NaN 을 한 번에 걸러내는 데도 쓰입니다.

```
const mixed = [1, 0, 2, NaN, 3];
console.log(mixed.filter((n) => n));
```

출력 [ 1, 2, 3 ]

주의 · filter 는 값을 바꾸지 않습니다. 걸러내기만 합니다.

```
console.log(scores.filter((s) => s * 2));
```

출력 [ 90, 55, 78, 40, 88 ]

s \* 2 는 전부 0이 아니라 truthy 라서 모두 통과했습니다. 값을 바꾸려면 map 입니다.

#### ▹ 직접 해보기 2

["a", "", "b", ""] 에서 빈 문자열을 걸러 보세요.

## 객체 배열에서 filter

섹션 3

```
const products = [
  { name: "아메리카노", price: 4000, stock: 10 },
  { name: "라떼", price: 4500, stock: 0 },
  { name: "케이크", price: 6000, stock: 3 },
  { name: "쿠키", price: 3000, stock: 0 },
];
```

재고가 있는 것만

```
const available = products.filter((p) => p.stock > 0);
console.log(available.length);
```

출력 2

```
available.forEach((p) => console.log(p.name));
```

출력 아메리카노  
케이크

품절인 것만

```
console.log(products.filter((p) => p.stock === 0).map((p) => p.name));
```

출력 [ '라떼', '쿠키' ]

filter 로 고르고 map 으로 이름만 뽑았습니다. 이렇게 이어 쓰는 것을 체이닝이라고 합니다.

조건 두 개

```
console.log(products.filter((p) => p.stock > 0 && p.price <= 5000).map((p) =>
p.name));
```

출력 [ '아메리카노' ]

#### ▹ 직접 해보기 3

products 에서 5000원 이상인 상품 이름만 뽑아 보세요.

filter 는 배열을 돌려주지만, find 는 값 하나를 돌려줍니다.

```
const firstPass = scores.find((score) => score >= 60);
console.log(firstPass);
```

출력      90

filter 와 비교

| 코드                                                                     | 출력               |
|------------------------------------------------------------------------|------------------|
| <code>console.log(scores.filter((score) =&gt; score &gt;= 60));</code> | ▶ [ 90, 78, 88 ] |
| <code>console.log(scores.find((score) =&gt; score &gt;= 60));</code>   | ▶ 90             |

못 찾으면 undefined 입니다. 빈 배열이 아닙니다.

```
console.log(scores.find((score) => score > 100));
```

출력      undefined

객체 배열에서 하나 찾을 때 가장 많이 씁니다.

```
const cake = products.find((p) => p.name === "케이크");
console.log(cake);
```

출력      { name: '케이크', price: 6000, stock: 3 }

```
console.log(cake.price);
```

출력      6000

없는 것을 찾으면 undefined 이므로 바로 쓰면 에러가 납니다.

```
const notFound = products.find((p) => p.name === "녹차");
console.log(notFound);
```

출력      undefined

이런 실수를 합니다

```
console.log(notFound.price);
```

TypeError: Cannot read properties of undefined (reading 'price')



안전하게 쓰려면 ?. 를 붙입니다. (07단원 개념01)

```
console.log(notFound?.price ?? "해당 상품 없음");
```

출력      해당 상품 없음

▹ 직접 해보기

4

products 에서 이름이 "쿠키" 인 상품의 가격을 출력해 보세요.

## findIndex — 몇 번째인지

섹션 5

```
console.log(products.findIndex((p) => p.name === "케이크"));
```

출력      2

없으면 -1 입니다. (06단원 indexOf 와 같습니다)

```
console.log(products.findIndex((p) => p.name === "녹차"));
```

출력      -1

indexOf 와의 차이:

|               |               |
|---------------|---------------|
| indexOf(값)    | 정확히 같은 값을 찾는다 |
| findIndex(조건) | 조건에 맞는 것을 찾는다 |

객체 배열에서는 indexOf 가 거의 쓸모없습니다. 객체는 === 로 비교되니까요.

찾아서 지우기

```
const list = products.slice(); // 원본을 지키려고 복사 (06단원 개념03)
const idx = list.findIndex((p) => p.name === "라떼");
if (idx !== -1) {
  list.splice(idx, 1);
}
console.log(list.map((p) => p.name));
```

출력      [ '아메리카노', '케이크', '쿠키' ]

▹ 직접 해보기

5

products 에서 재고가 0인 첫 상품의 위치를 출력해 보세요.

some : 하나라도 조건에 맞으면 `true`

```
console.log(products.some((p) => p.stock === 0));
```

출력      `true`

“품질인 게 하나라도 있나?”

every : 전부 조건에 맞아야 `true`

```
console.log(products.every((p) => p.stock === 0));
```

출력      `false`

“전부 품질인가?”

```
console.log(products.every((p) => p.price >= 3000));
```

출력      `true`

“전부 3000원 이상인가?”

filter 로도 할 수 있지만 some / every 가 뜻이 더 분명합니다.

```
console.log(products.filter((p) => p.stock === 0).length > 0);
```

출력      `true`

some 이 더 읽기 좋습니다. 게다가 조건에 맞는 걸 찾는 순간 멈추므로 빠릅니다.

조건문에 그대로 씁니다.

```
if (products.some((p) => p.stock === 0)) {
  console.log("품질 상품이 있습니다");
}
```

출력      품질 상품이 있습니다

▹ 직접 해보기

6

scores 에 90점 이상이 하나라도 있는지 확인해 보세요.

## 무엇을 쓸까

섹션 7

조건에 맞는 것 전부, 배열로 → filter 조건에 맞는 첫 개, 값 하나 → find 조건에 맞는 첫 개의 위치 →  
 findIndex 하나라도 있는지 (true/false) → some 전부 그런지 (true/false) → every 모양을 바꾸기 →  
 map 그냥 하나씩 처리 → forEach

“무엇이 나오는가” 를 기준으로 고르면 헷갈리지 않습니다.

filter → 배열      find → 값 하나  
 some/every → 불리언    map → 배열(같은 개수)

## 자주 하는 실수

섹션 8

이런 실수를 합니다

**filter** 결과를 값 하나로 착각

```
const one = products.filter((p) => p.name === "케이크");
console.log(one);
```

출력      [ { name: '케이크', price: 6000, stock: 3 } ]

console.log(one.price);    → undefined 가 나옵니다.

배열이라서 one[0].price 라고 써야 값이 나옵니다.

```
console.log(one[0].price);
```

출력      6000

하나만 찾을 거면 처음부터 find 를 쓰세요.

이런 실수를 합니다

**find** 결과를 확인 없이 사용 (섹션 4에서 봤습니다)

이런 실수를 합니다

조건식 대신 '계산 결과'를 돌려줌

```
console.log(scores.filter((s) => s + 1));
```

출력 [ 90, 55, 78, 40, 88 ]

$s + 1$ 은 전부 0이 아닌 숫자라 truthy 입니다. 그래서 하나도 안 걸러집니다. filter는 "이 값을 남길까?"를 묻는 것이지 "이 값을 어떻게 바꿀까?"가 아닙니다. 값을 바꾸려면 map을 쓰세요. (섹션 2의  $\text{filter}((w) \Rightarrow w)$ 는 "이 값이 truthy 인가?"를 묻는 것이라 다릅니다)

이런 실수를 합니다

중괄호를 쓰고 return을 빠뜨림

```
console.log(
  scores.filter((s) => {
    s >= 60;
  })
);
```

출력 []

아무것도 안 돌려주니 전부 undefined(falsy)로 취급되어 다 걸러집니다.

## 정리

- 1 filter(조건) — 맞는 것 전부를 새 배열로. 없으면 빈 배열.
- 2 find(조건) — 맞는 첫 개 하나. 없으면 undefined.
- 3 findIndex(조건) — 위치. 없으면 -1.
- 4 some / every — 하나라도 / 전부. 결과는 true/false.
- 5 콜백이 돌려준 값이 truthy 면 남고 falsy 면 빠진다. 값을 '바꾸는' 게 아니다.
- 6 filter → map 처럼 이어 쓰는 것을 체이닝이라고 한다.

## 직접 해보기 정답

- 1 `console.log([12, 7, 30, 5].filter((n) => n >= 10));`  
`// 출력: [ 12, 30 ]`
- 2 `console.log(["a", "", "b", ""].filter((w) => w !== ""));`  
`// 출력: [ 'a', 'b' ]`
- 3 `console.log(products.filter((p) => p.price >= 5000).map((p) => p.name));`  
`// 출력: [ '케이크' ]`
- 4 `console.log(products.find((p) => p.name === "쿠키").price);`  
`// 출력: 3000`  
`// 안전하게: products.find((p) => p.name === "쿠키")?.price`
- 5 `console.log(products.findIndex((p) => p.stock === 0));`  
`// 출력: 1`
- 6 `console.log(scores.some((s) => s >= 90));`  
`// 출력: true`

## reduce: 하나의 값으로 접기

RUN node 개념05\_reduce.js

배열 전체를 '값 하나'로 줄일 때 씁니다. 합계가 대표적입니다.

```
배열.reduce((누적값, 현재값) => 새누적값, 시작값)
      ^^^^^  ^^^^^      ^^^^^
      지금까지 이번 바퀴의 값      누적값의 시작
```

이 파일은 08단원에서 가장 어렵습니다. 천천히 보세요. 어렵다면 for 문으로 먼저 쓰고 나중에 바뀌도 됩니다.

### 합계를 세 가지 방법으로

섹션 1

```
const numbers = [10, 20, 30, 40];
```

방법 1 · for...of — 06단원에서 배운 것

```
let sum1 = 0;
for (const n of numbers) {
  sum1 += n;
}
console.log(sum1);
```

출력 100

방법 2 · forEach — 08단원 개념02

```
let sum2 = 0;
numbers.forEach((n) => {
  sum2 += n;
});
console.log(sum2);
```

출력 100

**방법 3** · reduce — 밖에 변수를 두지 않아도 됩니다

```
const sum3 = numbers.reduce((acc, n) => acc + n, 0);
console.log(sum3);
```

출력            100

acc 는 accumulator(누적값)의 줄임말입니다. total 이라고 이름 지어도 됩니다.

▫ 직접 해보기 **1**

[1, 2, 3, 4, 5] 의 합을 reduce 로 구해 보세요.

**어떻게 도는지 한 바퀴씩 보기**

섹션 2

```
const small = [1, 2, 3];

small.reduce((acc, n) => {
  console.log(`acc=${acc}, n=${n}, 결과=${acc + n}`);
  return acc + n;
}, 0);
```

출력            acc=0, n=1, 결과=1  
                  acc=1, n=2, 결과=3  
                  acc=3, n=3, 결과=6

읽는 법 1바퀴: acc 는 시작값 0, n 은 1 → 돌려준 1이 다음 acc 가 됩니다 2바퀴: acc 는 1, n 은 2 → 3

3바퀴: acc 는 3, n 은 3 → 6 끝나면 마지막 acc 인 6이 reduce 의 결과가 됩니다

핵심: 콜백이 return 한 값이 '다음 바퀴의 acc' 가 됩니다.

 여기서 멈추고 직접 해 보세요. 위 코드의 배열을 [10, 20, 30] 으로 바꿔서 실행해 보세요. acc 와 n 이 어떻게 바뀌는지 눈으로 보는 것이 reduce 를 이해하는 가장 빠른 길입니다. (결과는 acc=0,n=10 → acc=10,n=20 → acc=30,n=30 이 됩니다)

return 을 빠뜨리면 acc 가 undefined 가 되어 무너집니다.

```
console.log(
  small.reduce((acc, n) => {
    acc + n; // return 이 없습니다
  }, 0)
);
```

출력            undefined

reduce 를 쓸 때 가장 흔한 실수입니다.

시작값(두 번째 인자)을 안 주면 배열의 첫 값이 시작값이 됩니다.

```
console.log([10, 20, 30].reduce((acc, n) => acc + n));
```

출력      60

이 경우엔 결과가 같습니다. 첫 값 10이 acc 가 되고 20부터 돌기 때문입니다.

하지만 빈 배열에서는 에러가 납니다.

이런 실수를 합니다

```
console.log([].reduce((acc, n) => acc + n));
```

TypeError: Reduce of empty array with no initial value

시작값을 주면 안전합니다.

```
console.log([].reduce((acc, n) => acc + n, 0));
```

출력      0

그리고 자료형이 다른 결과를 만들 때는 반드시 시작값이 필요합니다.

```
console.log(["a", "b", "c"].reduce((acc, s) => acc + s, ""));
```

출력      abc

규칙: 시작값은 항상 쓰세요. 숫자면 0, 글자면 "", 배열이면 [], 객체면 {}.

▫ 직접 해보기

2

["안", "녕", "하", "세", "요"] 를 reduce 로 이어붙여 보세요.



## 객체 배열의 합계

섹션 4

```
const cart = [
  { name: "아메리카노", price: 4000, count: 2 },
  { name: "케이크", price: 6000, count: 1 },
  { name: "쿠키", price: 3000, count: 3 },
];

const total = cart.reduce((acc, item) => acc + item.price * item.count, 0);
console.log(total);
```

출력          23000

개수 합계

```
const countTotal = cart.reduce((acc, item) => acc + item.count, 0);
console.log(countTotal);
```

출력          6

map 과 reduce 를 이어 쓸 수도 있습니다. 뜻이 더 잘 보일 때가 있습니다.

```
const total2 = cart.map((item) => item.price * item.count).reduce((acc, n) => acc + n, 0);
console.log(total2);
```

출력          23000

▸ 직접 해보기

3

cart 에서 단가(price)의 합만 구해 보세요.

## 최댓값 구하기

섹션 5

```
const scores = [90, 55, 78, 88];

const max = scores.reduce((acc, n) => (n > acc ? n : acc), scores[0]);
console.log(max);
```

출력          90

시작값이 0 이 아니라 scores[0] 이라는 점에 주의하세요. 섹션 3에서 “숫자면 0” 이라고 했지만 그건 ‘합계’ 일 때 이야기입니다. 최댓값을 0부터 비교하면 전부 음수인 배열에서 0이 답으로 나옵니다. (06단원 개념05) 시작값은 “아무것도 없을 때의 답” 으로 정하세요. 합계 → 0 (아무것도 없으면 0원) 최댓값 → 첫 번째 값 (비교할 기준이 하나는 있어야 하므로)

삼항 연산자를 풀어 쓰면 이렇습니다.

```
const max2 = scores.reduce((acc, n) => {
  if (n > acc) {
    return n;
  }
  return acc;
}, scores[0]);
console.log(max2);
```

출력 90

사실 최댓값은 `Math.max` 가 더 간단합니다. (09단원 스프레드와 함께)

```
console.log(Math.max(...scores));
```

출력 90

객체 배열에서 “가장 비싼 것” 을 찾을 때는 `reduce` 가 유용합니다.

```
const expensive = cart.reduce((acc, item) => (item.price > acc.price ? item : acc),
cart[0]);
console.log(expensive.name);
```

출력 케이크

▹ 직접 해보기

4

scores 의 최솟값을 `reduce` 로 구해 보세요.

## 개수 세기 — 결과가 객체인 경우

섹션 6

`reduce` 의 결과가 꼭 숫자일 필요는 없습니다. 객체도 됩니다.

```
const votes = ["짜장", "짬뽕", "짜장", "볶음밥", "짜장", "짬뽕"];

const counts = votes.reduce((acc, item) => {
```

`acc[item]` 이 아직 없으면 0에서 시작

```
  acc[item] = (acc[item] ?? 0) + 1;
  return acc; // 매 바퀴 acc 를 돌려줘야 합니다
}, {});

console.log(counts);
```

출력 { '짜장': 3, '짬뽕': 2, '볶음밥': 1 }

시작값이 빈 객체 {} 라는 점이 핵심입니다. 매 바퀴 acc 에 값을 넣고, 그 acc 를 그대로 돌려줍니다.

반복문으로 쓰면 이렇습니다. 처음엔 이쪽이 이해하기 쉬울 겁니다.

```
const counts2 = {};
for (const item of votes) {
  counts2[item] = (counts2[item] ?? 0) + 1;
}
console.log(counts2);
```

출력 { '짜장': 3, '짬뽕': 2, '볶음밥': 1 }

▹ 직접 해보기 5

["A", "B", "A", "A"] 의 개수를 세어 보세요.

## 언제 reduce 를 쓸까

섹션 7

배열 → 값 하나 (숫자, 문자열, 객체) → reduce 배열 → 배열(같은 개수) → map 배열 → 배열(줄어들) → filter

다만 reduce 는 읽기가 어렵습니다. 억지로 쓰지 마세요. 합계 정도는 reduce 가 깔끔하지만, 복잡해지면 for 문이 더 읽기 좋습니다.

나쁜 예 · reduce 로 filter 흉내내기

```
console.log(scores.reduce((acc, n) => (n >= 80 ? acc.concat(n) : acc), []));
```

출력 [ 90, 88 ]

동작은 하지만 아무도 이렇게 안 씁니다.

```
console.log(scores.filter((n) => n >= 80));
```

출력 [ 90, 88 ]

filter 를 쓰세요.

## 자주 하는 실수

섹션 8

이런 실수를 합니다

return 을 빠뜨림 (섹션 2에서 봤습니다)

이런 실수를 합니다  
시작값을 안 줌 (섹션 3)

이런 실수를 합니다  
acc 와 현재값의 순서를 바꿔 쓰기

```
console.log([10, 20, 30].reduce((n, acc) => n + acc, 0));
```

출력          60

이번엔 더하기라 결과가 같지만, 빼기였다면 완전히 달라집니다. 순서는 항상 (누적값, 현재값) 으로 고정입니다. 이름에 속지 마세요.

이런 실수를 합니다  
객체를 누적할 때 acc 를 안 돌려줌

```
votes.reduce((acc, item) => { acc[item] = 1; }, {});
```

return acc 가 없어서 다음 바퀴의 acc 가 undefined 가 됩니다. TypeError: Cannot set properties of undefined

이런 실수를 합니다  
빈 배열에 시작값 없이 reduce

이런 실수를 합니다

```
{}.reduce((a, b) => a + b);
```

TypeError.

시작값을 주면 해결됩니다.

## 정리

- 1 배열.reduce((acc, 값) => 새acc, 시작값) — 배열을 값 하나로 줄인다.
- 2 콜백이 return 한 값이 다음 바퀴의 acc 가 된다. return 필수.
- 3 시작값은 항상 쓴다. 숫자 0, 글자 "", 배열 [], 객체 {}.
- 4 결과가 객체나 배열이어도 된다. (개수 세기)
- 5 어렵게 느껴지면 for 문으로 쓰고, 익숙해지면 바꿔라.

## 직접 해보기 정답

```

1 console.log([1, 2, 3, 4, 5].reduce((acc, n) => acc + n, 0));
  // 출력: 15

2 console.log(["안", "녕", "하", "세", "요"].reduce((acc, s) => acc + s, ""));
  // 출력: 안녕하세요
  // 배열.join("") 을 쓰면 더 간단합니다.

3 console.log(cart.reduce((acc, item) => acc + item.price, 0));
  // 출력: 13000

4 console.log(scores.reduce((acc, n) => (n < acc ? n : acc), scores[0]));
  // 출력: 55

5 const r = ["A", "B", "A", "A"].reduce((acc, v) => {

```

```

acc[v] = (acc[v] ?? 0) + 1;
return acc;

```

```

}, {});
console.log(r);      // 출력: { A: 3, B: 1 }

```

## sort: 정렬하기

RUN node 개념06\_sort.js

06단원에서 `sort()` 를 그냥 쓰면 숫자가 이상하게 정렬된다고 했습니다. 이제 콜백을 배웠으니 제대로 정렬할 수 있습니다.

```
배열.sort((a, b) => a - b)  오름차순
배열.sort((a, b) => b - a)  내림차순
```

### 그냥 `sort()` 의 함정

섹션 1

```
const numbers = [10, 9, 100, 1];
numbers.sort();
console.log(numbers);
```

출력 [ 1, 10, 100, 9 ]

`sort` 는 기본적으로 값을 문자열로 바뀌어서 사전 순으로 비교합니다. "10" 과 "9" 를 비교하면 첫 글자 "1" 이 "9" 보다 앞이라 10이 먼저 옵니다.

문자열 배열은 기본 `sort()` 로도 잘 됩니다.

```
const names = ["박지훈", "김민준", "이서연"];
names.sort();
console.log(names);
```

출력 [ '김민준', '박지훈', '이서연' ]

### 비교 함수 — 두 개씩 건주는 규칙

섹션 2

`sort` 에 콜백을 넘기면 "이 둘 중 누가 앞이나" 를 우리가 정할 수 있습니다. 콜백은 두 값 `a`, `b` 를 받아 숫자를 돌려줍니다.

```
음수를 돌려주면 → a 가 앞
양수를 돌려주면 → b 가 앞
0 을 돌려주면   → 순서 유지
```

그래서 오름차순은 `a - b` 입니다.

a 가 작으면 a - b 는 음수 → a 가 앞으로 (작은 게 앞)

```
const nums = [10, 9, 100, 1];
nums.sort((a, b) => a - b);
console.log(nums);
```

출력 [ 1, 9, 10, 100 ]

내림차순은 순서를 뒤집습니다.

```
const nums2 = [10, 9, 100, 1];
nums2.sort((a, b) => b - a);
console.log(nums2);
```

출력 [ 100, 10, 9, 1 ]

외우는 요령:

a - b → 작은 것부터 (오름차순, ascending)  
b - a → 큰 것부터 (내림차순, descending)

▹ 직접 해보기 1

[5, 30, 7, 2] 를 오름차순으로 정렬해 보세요.

08

## sort 는 원본을 바꾼다

섹션 3

```
const original = [3, 1, 2];
const sorted = original.sort((a, b) => a - b);

console.log(sorted);
```

출력 [ 1, 2, 3 ]

```
console.log(original);
```

출력 [ 1, 2, 3 ]

원본까지 정렬되었습니다. sorted 와 original 은 같은 배열입니다.

원본을 지키려면 복사한 뒤 정렬합니다.

```
const original2 = [3, 1, 2];
const sorted2 = original2.slice().sort((a, b) => a - b);

console.log(sorted2);
```

출력      [ 1, 2, 3 ]

```
console.log(original2);
```

출력      [ 3, 1, 2 ]

09단원에서 배운 '스프레드' 로도 복사할 수 있습니다. 이런 게 있다는 것만 봐 두세요. 이 단원에서는 이미 배운 `slice()` 로 통일해서 쓰겠습니다.

```
const original3 = [3, 1, 2];
console.log([...original3].sort((a, b) => a - b));
```

출력      [ 1, 2, 3 ]

```
console.log(original3);
```

출력      [ 3, 1, 2 ]

최근에는 원본을 안 바꾸는 `toSorted` 도 생겼습니다.

```
const original4 = [3, 1, 2];
console.log(original4.toSorted((a, b) => a - b));
```

출력      [ 1, 2, 3 ]

```
console.log(original4);
```

출력      [ 3, 1, 2 ]

#### ▫ 직접 해보기 2

아래 배열의 원본은 그대로 두고 정렬된 복사본을 만들어 보세요. `const origin = [3, 1, 2];` (힌트: 06단원에서 배운 `slice()` 로 먼저 복사합니다)



## 객체 배열 정렬

섹션 4

```
const products = [
  { name: "케이크", price: 6000 },
  { name: "아메리카노", price: 4000 },
  { name: "쿠키", price: 3000 },
];
```

가격 낮은 순

```
const byPrice = products.slice().sort((a, b) => a.price - b.price);
console.log(byPrice.map((p) => p.name));
```

출력 [ '쿠키', '아메리카노', '케이크' ]

가격 높은 순

```
const byPriceDesc = products.slice().sort((a, b) => b.price - a.price);
console.log(byPriceDesc.map((p) => p.name));
```

출력 [ '케이크', '아메리카노', '쿠키' ]

비교할 값을 a.price, b.price 로 꺼내는 것만 다릅니다. 나머지는 숫자 정렬과 똑같습니다.

#### ▫ 직접 해보기 3

products 를 이름의 가나다순으로 정렬해 보세요. (섹션 5 참고)

## 문자열 정렬

섹션 5

문자열은 빠기를 할 수 없습니다.

```
const words = ["banana", "apple", "cherry"];
console.log(words.slice().sort((a, b) => a - b));
```

출력 [ 'banana', 'apple', 'cherry' ]

a - b 가 전부 NaN 이라 순서가 안 바뀌었습니다.

문자열은 비교 연산자로 건줍니다.

```
console.log(words.slice().sort((a, b) => (a > b ? 1 : -1)));
```

출력 [ 'apple', 'banana', 'cherry' ]

또는 localeCompare 를 씁니다. 한글·영어 모두 잘 처리합니다.

```
console.log(words.slice().sort((a, b) => a.localeCompare(b)));
```

출력 [ 'apple', 'banana', 'cherry' ]

```
const korean = ["하늘", "가방", "나무"];  
console.log(korean.slice().sort((a, b) => a.localeCompare(b)));
```

출력 [ '가방', '나무', '하늘' ]

객체 배열의 문자열 속성으로 정렬할 때도 마찬가지입니다.

```
const sortedByName = products.slice().sort((a, b) => a.name.localeCompare(b.name));  
console.log(sortedByName.map((p) => p.name));
```

출력 [ '아메리카노', '케이크', '쿠키' ]

▹ 직접 해보기 4

["다", "가", "나"] 를 가나다순으로 정렬해 보세요.

## 정렬 + 자르기 — 상위 N개 뽑기

섹션 6

```
const scores = [88, 95, 62, 100, 74];
```

상위 3개

```
const top3 = scores.slice().sort((a, b) => b - a).slice(0, 3);  
console.log(top3);
```

출력 [ 100, 95, 88 ]

객체 배열에서 가장 비싼 상품 2개

```
const top2Products = products.slice()  
  .sort((a, b) => b.price - a.price)  
  .slice(0, 2)  
  .map((p) => p.name);  
console.log(top2Products);
```

출력 [ '케이크', '아메리카노' ]

이렇게 메소드를 줄줄이 이어 쓰는 것을 체이닝이라고 합니다. 한 줄이 길어지면 위처럼 점(.) 앞에서 줄을 나누면 읽기 좋습니다.

## 자주 하는 실수

섹션 7

이런 실수를 합니다

숫자 배열에 `sort()` 를 그냥 씀 (섹션 1)

이런 실수를 합니다

비교 함수에서 `true/false` 를 돌려줌

```
const bad = [3, 1, 2];
console.log(bad.slice().sort((a, b) => a > b));
```

출력 [ 3, 1, 2 ]

정렬이 전혀 되지 않았습니다. 비교 함수는 음수 / 양수 / 0 을 돌려줘야 하는데,  $a > b$  는 `true` 아니면 `false` 이고 이것은 각각 1과 0으로 바뀝니다. “a 가 앞” 을 뜻하는 음수가 나올 방법이 없으니 자리를 바꿀 수 없는 것입니다. 반드시  $a - b$  를 쓰세요.

이런 실수를 합니다

원본이 안 바뀔 거라 기대 (섹션 3)

특히 화면에 보여 줄 목록을 정렬했다가 원본 데이터 순서를 망가뜨리는 사고가 잦습니다.

이런 실수를 합니다

문자열에  $a - b$  를 씀 (섹션 5)

이런 실수를 합니다

`sort` 결과를 다시 담아야 한다고 생각하기

```
const arr = [3, 1, 2];
arr.sort((a, b) => a - b);
console.log(arr);
```

출력 [ 1, 2, 3 ]

원본이 바뀌므로 다시 담지 않아도 됩니다. (담아도 같은 배열입니다)

## 정리

- 1 숫자 정렬은 `sort((a, b) => a - b)`. 그냥 `sort()` 는 사전 순이라 틀린다.
- 2 `a - b` 오름차순 / `b - a` 내림차순
- 3 객체는 `a.속성 - b.속성` 으로 비교한다.
- 4 문자열은 `a.localeCompare(b)` 를 쓴다.
- 5 `sort` 는 원본을 바꾼다. `배열.slice().sort()` 또는 `toSorted()` 를 쓸 것.
- 6 정렬 후 `slice(0, N)` 으로 상위 N개를 뽑는다.

## 직접 해보기 정답

- 1 

```
console.log([5, 30, 7, 2].sort((a, b) => a - b));  
// 출력: [ 2, 5, 7, 30 ]
```
- 2 

```
const origin = [3, 1, 2];  
const copy = origin.slice().sort((a, b) => a - b);  
console.log(copy);    // 출력: [ 1, 2, 3 ]  
console.log(origin);  // 출력: [ 3, 1, 2 ]
```
- 3 

```
console.log(products.slice().sort((a, b) => a.name.localeCompare(b.name)).map((p) =>  
p.name));  
// 출력: [ '아메리카노', '케이크', '쿠키' ]
```
- 4 

```
console.log(["다", "가", "나"].sort((a, b) => a.localeCompare(b)));  
// 출력: [ '가', '나', '다' ]
```

08단원 · 연습문제

## 콜백과 내장 메소드

RUN node 연습문제.js

TODO 자리에 코드를 쓰고, '기대 출력'과 같은지 확인하세요. 1~17은 기본, 18은 응용, 19는 [도전], 20은 에러 확인입니다.

이 단원은 어렵습니다. 막히면 06단원처럼 for 문으로 먼저 풀어 보고 그다음 메소드로 바꿔 보세요. 그게 가장 빨리 익히는 방법입니다.

### 문제 1 (개념01 콜백)

아래 runTwice 는 이미 만들어져 있습니다. "출석" 을 출력하는 함수를 만들어 runTwice 에 넘기세요. 넘길 때 괄호를 붙이지 않도록 주의하세요.

```
function runTwice(callback) {
  callback();
  callback();
}
```

기대 출력      출석  
                  출석

여기에 코드를 쓰세요

### 문제 2 (개념01 화살표 콜백)

runTwice 에 화살표 함수를 그 자리에서 만들어 넘겨 "화이팅" 을 출력하세요.

기대 출력      화이팅  
                  화이팅

여기에 코드를 쓰세요

### 문제 3 (개념02 forEach)

아래 배열을 forEach 로 한 줄씩 출력하세요.

```
const nums3 = [10, 20, 30];
```

기대 출력     10  
                 20  
                 30

여기에 코드를 쓰세요

### 문제 4 (개념02 인덱스)

아래 배열을 "1. 국어" 형태로 출력하세요. 번호는 1부터입니다.

```
const subjects4 = ["국어", "영어", "수학"];
```

기대 출력     1. 국어  
                 2. 영어  
                 3. 수학

여기에 코드를 쓰세요

## 문제 5 (개념02 객체 배열)

아래 배열을 forEach 로 "이름 가격원" 형태로 출력하세요.

```
const menu5 = [
  { name: "아메리카노", price: 4000 },
  { name: "케이크", price: 6000 },
];
```

기대 출력    아메리카노 4000원  
              케이크 6000원

여기에 코드를 쓰세요

## 문제 6 (개념03 map)

아래 배열의 각 값을 2배로 만든 새 배열을 출력하세요.

```
const nums6 = [1, 2, 3];
```

기대 출력

— 2, 4, 6

여기에 코드를 쓰세요

## 문제 7 (개념03 map 자료형 바꾸기)

아래 점수 배열을 ["90점", "85점", "70점"] 형태로 바꿔 출력하세요.

```
const scores7 = [90, 85, 70];
```

기대 출력

— '90점', '85점', '70점'

여기에 코드를 쓰세요

## 문제 8 (개념03 map 속성 뽑기)

아래 배열에서 이름만 뽑은 배열을 출력하세요.

```
const users8 = [
  { name: "김민준", age: 20 },
  { name: "이서연", age: 22 },
  { name: "박지훈", age: 28 },
];
```

기대 출력

— '김민준', '이서연', '박지훈'

여기에 코드를 쓰세요

## 문제 9 (개념04 filter)

아래 점수 중 60점 이상만 골라 출력하세요.

```
const scores9 = [90, 55, 78, 40, 88];
```

기대 출력



— 90, 78, 88

여기에 코드를 쓰세요

## 문제 10 (개념04 filter + map)

아래 상품 중 재고가 0인 것의 '이름만' 배열로 출력하세요.

```
const products10 = [
  { name: "아메리카노", price: 4000, stock: 10 },
  { name: "라떼", price: 4500, stock: 0 },
  { name: "케이크", price: 6000, stock: 3 },
  { name: "쿠키", price: 3000, stock: 0 },
];
```

기대 출력

— '라떼', '쿠키'

여기에 코드를 쓰세요

## 문제 11 (개념04 find)

products10 에서 이름이 "케이크" 인 상품의 가격을 출력하세요.

기대 출력     6000

여기에 코드를 쓰세요

## 문제 12 (개념04 findIndex)

products10 에서 재고가 0인 첫 상품의 위치를 출력하세요.

기대 출력     1

여기에 코드를 쓰세요

## 문제 13 (개념04 some / every)

1 품질인 상품이 하나라도 있는가

2 모든 상품이 5000원 이상인가

기대 출력     true  
                 false

여기에 코드를 쓰세요

## 문제 14 (개념05 reduce)

아래 배열의 합계를 reduce 로 구해 출력하세요. 시작값을 꼭 넣으세요.

```
const nums14 = [10, 20, 30, 40];
```

기대 출력     100

여기에 코드를 쓰세요

## 문제 15 (개념05 reduce 객체 배열)

아래 장바구니의 총액(가격 × 개수의 합)을 reduce 로 구해 출력하세요.

```
const cart15 = [
  { name: "아메리카노", price: 4000, count: 2 },
  { name: "케이크", price: 6000, count: 1 },
  { name: "쿠키", price: 3000, count: 3 },
];
```

기대 출력     23000

여기에 코드를 쓰세요

## 문제 16 (개념06 sort)

아래 배열을 작은 수부터 정렬해 출력하세요.

```
const nums16 = [10, 9, 100, 1];
```

기대 출력

— 1, 9, 10, 100

여기에 코드를 쓰세요

## 문제 17 (개념06 sort 객체 배열)

products10 을 가격이 낮은 순으로 정렬해 이름만 배열로 출력하세요. 그다음 원본이 그대로인지 확인하려고 products10 의 이름 배열도 출력하세요. (원본이 바뀌었다면 두 줄이 같게 나올 것입니다)

기대 출력

— '쿠키', '아메리카노', '라떼', '케이크'

— '아메리카노', '라떼', '케이크', '쿠키'

여기에 코드를 쓰세요

## 문제 18 [응용]

products10 에서 재고가 있는 상품만 골라, 그 상품들의 "가격 × 재고" 를 모두 더한 값을 출력하세요. filter 와 reduce 를 이어서 쓰면 됩니다.

기대 출력      재고 자산 58000원

여기에 코드를 쓰세요

## 문제 19 [도전]

아래 투표 결과에서 항목별 표 수를 세어 객체로 만들어 출력하세요. reduce 의 시작값을 빈 객체 {} 로 두면 됩니다.

```
const votes19 = ["짜장", "짬뽕", "짜장", "볶음밥", "짜장", "짬뽕"];
```

기대 출력      { '짜장': 3, '짬뽕': 2, '볶음밥': 1 }

여기에 코드를 쓰세요

## 문제 20 (에러 확인 - 맨 마지막)

아래 두 줄의 주석을 풀고 실행해서 어떤 에러가 나는지 확인하세요. 확인했으면 반드시 다시 주석 처리하세요.

```
const scoreObject = { 국어: 90, 영어: 85 };
scoreObject.forEach((v) => console.log(v));
```

무슨 에러가 났나요? 객체를 하나씩 돌려면 무엇을 써야 할까요?

답

## 콜백과 내장 메소드

RUN node 심화문제.js

### — 이걸 선택 과제입니다

연습문제를 다 푼 사람만 하세요. 안 풀어도 다음 단원에 지장 없습니다.

이 단원은 이 자료에서 가장 무거운 곳입니다. 심화 1을 풀고 나면 07단원 심화 2를 다시 열어 보세요. 같은 일을 반복문으로 했었습니다.

### 심화 1 reduce 로 분류하기

상품 배열을 카테고리별로 몇 개인지 센 객체를 만드세요. 반드시 reduce 한 번으로 끝내세요. for 문이나 forEach 를 쓰면 안 됩니다.

힌트: reduce 의 시작값을 빈 객체 {} 로 둡니다. (개념05)

콜백 안에서 acc[키] 를 확인해 있으면 +1, 없으면 1 을 넣습니다.  
그리고 반드시 acc 를 return 해야 합니다. 안 하면 다음 바퀴에 undefined 가 옵니다.

```
const products = [
  { name: "아메리카노", category: "음료" },
  { name: "라떼", category: "음료" },
  { name: "케이크", category: "디저트" },
  { name: "쿠키", category: "디저트" },
  { name: "샌드위치", category: "식사" },
];
```

기대 출력 { '음료': 2, '디저트': 2, '식사': 1 }

(한글 속성 이름은 출력할 때 따옴표가 붙습니다. 정상입니다)

여기에 코드를 쓰세요

## 심화 2 메소드 이어 붙이기

아래 학생들 중 60점 이상인 사람만 골라, 점수가 높은 순으로 줄을 세우고, "이름(점수)" 형태로 바꿔서, 심표로 이어 한 줄로 출력하세요.

조건: 중간 결과를 변수에 담지 말고 한 줄로 이어 쓰세요.

filter → sort → map → join 순서입니다.

힌트: sort 는 원본을 바꿉니다. 그런데 filter 가 이미 '새 배열' 을 만들었으니

그 뒤에 오는 sort 는 원본을 건드리지 않습니다. 마지막에 원본을 찍어 확인하세요.

```
const students = [
  { name: "김민준", score: 90 },
  { name: "이서연", score: 55 },
  { name: "박지훈", score: 78 },
  { name: "최유진", score: 95 },
];
```

기대 출력     최유진(95), 김민준(90), 박지훈(78)  
원본 첫 번째: 김민준

여기에 코드를 쓰세요

## 심화 3 기준이 두 개인 정렬

아래 상품을 '가격 낮은 순' 으로 정렬하되, 가격이 같으면 '재고가 많은 순' 으로 정렬하세요.

힌트: 비교 함수 안에서 먼저 가격을 비교하고,

가격 차이가 0 이면(같으면) 그때 재고를 비교해 돌려주면 됩니다.  
원본은 건드리지 말고 slice() 로 복사한 뒤 정렬하세요. (개념06)

결과는 "이름 가격원 재고N개" 형태로 한 줄씩 출력하세요.

```
const items = [
  { name: "케이크", price: 6000, stock: 2 },
  { name: "아메리카노", price: 4000, stock: 5 },
  { name: "라떼", price: 4000, stock: 3 },
  { name: "쿠키", price: 3000, stock: 5 },
];
```

기대 출력      쿠키 3000원 재고5개  
                  아메리카노 4000원 재고5개  
                  라떼 4000원 재고3개  
                  케이크 6000원 재고2개

여기에 코드를 쓰세요

## 08단원 마무리      콜백과 내장메소드

### 자주 넘어지는 자리

- 1    sort 비교 함수는 **true/false** 가 아니라 **음수/양수**를 돌려줘야 한다는 것
- 2    `forEach(console.log())` 처럼 괄호를 붙이면 안 되는 것



부 록 가

# 연습문제·종합연습 정답

먼저 스스로 풀어 본 다음에 보세요. 정답과 다르게 썼더라도 기대 출력이 같으면 맞은 것입니다.  
다만 “왜 그렇게 되는지” 설명은 꼭 읽으세요.

## 01단원 연습문제 정답 변수와 자료형

RUN node 연습문제\_정답.js

먼저 스스로 풀어 본 다음에 보세요. 정답과 다르게 썼더라도 기대 출력이 같으면 맞은 것입니다. 다만 “왜 그렇게 되는지” 설명을 꼭 읽으세요.

### 문제 1

```
console.log("자바스크립트 시작!");
```

출력      자바스크립트 시작!

### 문제 2

심표로 나열하면 띄어쓰기로 구분되어 한 줄에 찍힙니다.

```
console.log("삼각김밥", 1200, "원");
```

출력      삼각김밥 1200 원

다른 방법 · + 로 이어 붙일 수도 있지만 띄어쓰기를 직접 넣어야 합니다.

```
console.log("삼각김밥 " + 1200 + " 원");
```

### 문제 3

```
const cafeName = "봄날카페";
console.log(cafeName);
```

출력      봄날카페

### 문제 4

```
let stock = 10;
console.log(stock);
```

|    |    |
|----|----|
| 출력 | 10 |
|----|----|

값을 바꿀 때는 `let` 을 다시 쓰지 않습니다. 이름 = 새값 만 씁니다.

```
stock = 7;
console.log(stock);
```

|    |   |
|----|---|
| 출력 | 7 |
|----|---|

### 문제 5

```
const bread = 3200;
const milk = 1500;
console.log(bread + milk);
```

|    |      |
|----|------|
| 출력 | 4700 |
|----|------|

### 문제 6

값과 `typeof` 를 같이 찍으면 자료형을 눈으로 확인할 수 있습니다.

| 코드                                           | 출력             |
|----------------------------------------------|----------------|
| <code>console.log(42, typeof 42);</code>     | ▶ 42 number    |
| <code>console.log("42", typeof "42");</code> | ▶ 42 string    |
| <code>console.log(true, typeof true);</code> | ▶ true boolean |

42 와 "42" 는 화면에 똑같이 보이지만 자료형이 다릅니다.

### 문제 7

```
const city = "부산";
console.log(city + " 여행 가자");
```

|    |          |
|----|----------|
| 출력 | 부산 여행 가자 |
|----|----------|

찍어쓰기는 자동으로 안 들어갑니다. 따옴표 안에 직접 넣어야 합니다.

## 문제 8

```
console.log("대한민국만세".length);
```

출력          6

다른 방법 · 변수에 담아서 써도 됩니다.

```
const word = "대한민국만세";
console.log(word.length);
```

## 문제 9

```
const myScore = 85;
const isPass = myScore > 60;
console.log(isPass);
```

출력          true

비교의 결과는 `true` / `false` 인 불리언입니다.

## 문제 10

`null` 은 “일부러 비워 뒀다”는 뜻이라 나중에 바꿉니다. 그래서 `let` 입니다.

```
let selectedMenu = null;
console.log(selectedMenu);
```

출력          null

```
selectedMenu = "김밥";
console.log(selectedMenu);
```

출력          김밥

## 문제 11

```
const input1 = "3000";
const input2 = "4500";
```

그냥 더하면 이어붙기 때문에 30004500 이 나옵니다. 반드시 Number 로 바꾼 뒤 더해야 합니다.

```
console.log(Number(input1) + Number(input2));
```

출력 7500

## 문제 12

```
const height = "180cm";
```

Number("180cm") 은 NaN 입니다. cm 때문에 전체를 숫자로 못 바꿉니다. 앞에서부터 읽히는 만큼만 가져오는 parseInt 를 씁니다.

```
console.log(parseInt(height));
```

출력 180

## 문제 13

| 코드                         | 출력    |
|----------------------------|-------|
| console.log(Boolean(0));   | false |
| console.log(Boolean(""));  | false |
| console.log(Boolean("0")); | true  |

"0" 은 글자가 한 개 들어 있는 문자열이라 true 입니다. falsy 6개: 0, "", null, undefined, NaN, false

## 문제 14

```
const bigNumber2 = 987654321;
```

숫자에는 .length 가 없어서 undefined 가 나옵니다. String 으로 바꿔 문자열의 글자 수를 세면 자릿수가 됩니다.

```
console.log(String(bigNumber2).length);
```

출력 9

## 문제 15

```
const before = "10";

console.log(before + 5);
```

출력      105

```
console.log(Number(before) + 5);
```

출력      15

왜 다른가: before 는 문자열 "10" 입니다. + 는 한쪽이라도 문자열이면 '더하기'가 아니라 '이어붙이기'가 됩니다. 그래서 "10" + 5 는 "105" 가 됩니다. Number 로 숫자 10 으로 바꾸면 그때부터 진짜 덧셈이 됩니다.

## 문제 16

```
const itemName = "우유";
const itemPrice = "1500"; // 문자열!
const itemCount = 3;
```

문자열 \* 숫자는 자동으로 숫자 계산이 되지만, 의도를 분명히 하려고 Number 로 직접 바꿉니다.

```
const itemTotal = Number(itemPrice) * itemCount;

console.log(itemName + " " + itemCount + "개 = " + itemTotal + "원");
```

출력      우유 3개 = 4500원

## 문제 17

```
const fixedValue = "고정";
fixedValue = "변경";
```

이 코드는 검사에서 걸립니다

왜:

`const` 로 만든 변수는 "이 값은 안 바뀝니다"라고 약속한 것입니다.  
약속을 어기고 새 값을 넣으려 하면 자바스크립트가 막습니다.  
값을 바꿀 생각이라면 처음부터 `let` 으로 만들어야 합니다.

`TypeError: Assignment to constant variable.`

## 02단원 연습문제 정답 연산자

RUN node 연습문제\_정답.js

### 문제 1

```
const applePrice = 1200;
const appleCount = 3;
console.log(applePrice * appleCount);
```

출력 3600

### 문제 2

```
console.log(25 % 4);
```

출력 1

25 ÷ 4 = 6 ... 나머지 1

### 문제 3

```
const totalSec = 350;
const min3 = Math.floor(totalSec / 60);
const sec3 = totalSec % 60;
console.log(`${min3}분 ${sec3}초`);
```

출력 5분 50초

다른 방법 · 백틱 안에서 바로 계산해도 됩니다. 다만 읽기가 나빠집니다.

```
console.log(`${Math.floor(totalSec / 60)}분 ${totalSec % 60}초`);
```

### 문제 4

```
const rawValue = 12345.678;
console.log(rawValue.toFixed(1));
```



출력 12345.7

주의: toFixed 의 결과는 문자열입니다. 계산에 다시 쓰려면 Number( ) 로 되돌리세요.

## 문제 5

누적 변수는 반드시 0으로 시작합니다. 안 그러면 NaN 이 됩니다.

```
let sumTotal = 0;
sumTotal += 1200;
sumTotal += 800;
sumTotal += 500;
console.log(sumTotal);
```

출력 2500

## 문제 6

```
let visitorCount = 99;
visitorCount++;
console.log(visitorCount);
```

출력 100

## 문제 7

```
const checkAge = 17;
console.log(checkAge >= 19);
```

출력 false

‘이상’은 >= 입니다. > 로 쓰면 19살이 빠집니다.

## 문제 8

| 코드                      | 출력    |
|-------------------------|-------|
| console.log("7" === 7); | false |
| console.log("7" == 7);  | true  |

=== 는 자료형까지 비교합니다. 문자열 "7" 과 숫자 7 은 자료형이 다르므로 false. == 는 자료형을 맞춰 준 뒤 비교해서 true 가 됩니다. 그래서 == 를 쓰면 안 됩니다.

## 문제 9

| 코드                                         | 출력      |
|--------------------------------------------|---------|
| <code>console.log("80" &gt; "100");</code> | ▶ true  |
| <code>console.log(80 &gt; 100);</code>     | ▶ false |

문자열끼리 비교하면 사전 순으로 한 글자씩 봅니다. "8" 과 "1" 을 비교해서 "8" 이 뒤에 있으므로 "80" 이 크다고 나옵니다. 입력창에서 받은 값을 크기 비교할 때는 반드시 Number( ) 로 바꿔야 합니다.

## 문제 10

|                                                                                                         |      |
|---------------------------------------------------------------------------------------------------------|------|
| <code>const score2 = 75;</code><br><code>console.log(score2 &gt;= 70 &amp;&amp; score2 &lt; 80);</code> |      |
| 출력                                                                                                      | true |

70 <= score2 < 80 처럼 쓰면 안 됩니다. && 로 이어야 합니다.

## 문제 11

|                                                                                    |    |
|------------------------------------------------------------------------------------|----|
| <code>const inputNick = "";</code><br><code>console.log(inputNick    "익명");</code> |    |
| 출력                                                                                 | 익명 |

빈 문자열은 falsy 라서 오른쪽 값이 나옵니다.

## 문제 12

|                                                                                |      |
|--------------------------------------------------------------------------------|------|
| <code>const isRaining = false;</code><br><code>console.log(!isRaining);</code> |      |
| 출력                                                                             | true |

## 문제 13

```
const name3 = "이서연";
const age3 = 22;
console.log(`안녕하세요, ${name3}님! ${age3}살이시군요.`);
```

출력          안녕하세요, 이서연님! 22살이시군요.

다른 방법 · + 로도 되지만 훨씬 지저분합니다.

```
console.log("안녕하세요, " + name3 + "님! " + age3 + "살이시군요.");
```

## 문제 14

```
const numA = 3;
const numB = 4;
console.log(`${numA} 곱하기 ${numB} 는 ${numA * numB}`);
```

출력          3 곱하기 4 는 12

## 문제 15

```
const itemName15 = "샌드위치";
const itemPrice15 = 4800;
const itemCount15 = 2;

const itemTotal15 = itemPrice15 * itemCount15;
console.log(`${itemName15} ${itemCount15}개 ${itemTotal15}원`);
```

출력          샌드위치 2개 9600원

## 문제 16

```
const menu16 = "아메리카노";
const price16 = 4500;
const count16 = 2;
```

백틱 안에서 엔터를 치면 그대로 줄이 바뀝니다. 들여쓰기한 공백도 출력에 포함되므로 왼쪽 끝에 붙여 씁니다.

```
console.log(`=== 주문서 ===
${menu16} x ${count16}
합계 ${price16 * count16}원`);
```

```
출력      === 주문서 ===
          아메리카노 x 2
          합계 9000원
```

## 문제 17

```
const clickCount = 0;
clickCount++;
```

이 코드는 검사에서 걸립니다

왜:

clickCount++ 는 clickCount = clickCount + 1 을 줄여 쓴 것입니다.  
즉 겹보기와 달리 '값을 다시 넣는' 동작입니다.  
const 는 값을 다시 넣을 수 없으므로 에러가 납니다.  
1씩 늘어나는 변수는 처음부터 let 으로 만들어야 합니다.

`TypeError: Assignment to constant variable.`

## 03단원 연습문제 정답 조건문

RUN node 연습문제\_정답.js

### 문제 1

```
const temp1 = 33;
if (temp1 >= 33) {
  console.log("폭염 주의보");
}
```

출력      폭염 주의보

### 문제 2

```
const balance1 = 0;
if (balance1 === 0) {
  console.log("잔액이 부족합니다");
  console.log("충전해 주세요");
}
```

출력      잔액이 부족합니다  
충전해 주세요

중괄호 안에 여러 줄을 넣을 수 있습니다.

### 문제 3

```
const memo1 = "";
if (!memo1) {
  console.log("메모 없음");
}
```

출력      메모 없음

memo1 === "" 라고 써도 맞습니다. 뜻이 더 분명해서 이쪽이 나을 때도 많습니다.

## 문제 4

```
const age4 = 15;
if (age4 >= 19) {
  console.log("성인");
} else {
  console.log("미성년자");
}
```

출력      미성년자

## 문제 5

```
const score5 = 72;
if (score5 >= 90) {
  console.log("A");
} else if (score5 >= 80) {
  console.log("B");
} else if (score5 >= 70) {
  console.log("C");
} else {
  console.log("F");
}
```

출력      C

큰 수부터 내려오는 순서입니다. 70을 먼저 쓰면 90점도 C가 됩니다.

## 문제 6

```
const total6 = 30000;
if (total6 >= 50000) {
  console.log("배송비 0원");
} else if (total6 >= 30000) {
  console.log("배송비 1500원");
} else {
  console.log("배송비 3000원");
}
```

출력      배송비 1500원

## 문제 7

```
const age7 = 70;
const member7 = true;
```

두 조건은 서로 무관하므로 if 를 따로 씁니다.

```
if (age7 >= 65) {
  console.log("경로 할인");
}
if (member7) {
  console.log("회원 할인");
}
```

```
출력      경로 할인
          회원 할인
```

else if 로 이으면 "경로 할인" 하나만 나옵니다.

## 문제 8

```
const member8 = true;
const total8 = 60000;
if (member8 && total8 >= 50000) {
  console.log("무료 배송");
}
```

```
출력      무료 배송
```

## 문제 9

```
const day9 = "일";
if (day9 === "토" || day9 === "일") {
  console.log("주말입니다");
}
```

```
출력      주말입니다
```

day9 === "토" || "일" 이라고 쓰면 안 됩니다. 비교를 양쪽에 다 써야 합니다.

## 문제 10

```
const nickname10 = null;
```

&& 는 왼쪽이 `false` 면 오른쪽을 아예 계산하지 않습니다. 그래서 `nickname10` 이 `null` 이어도 `.length` 를 건드리지 않아 에러가 안 납니다.

```
if (nickname10 && nickname10.length > 0) {  
  console.log(nickname10);  
} else {  
  console.log("닉네임 없음");  
}
```

출력      닉네임 없음

## 문제 11

```
const profile11 = "";  
console.log(profile11 || "기본이미지.png");
```

출력      기본이미지.png

## 문제 12

```
const menu12 = "케이크";  
switch (menu12) {  
  case "아메리카노":  
    console.log("4000원");  
    break;  
  case "라떼":  
    console.log("4500원");  
    break;  
  case "케이크":  
    console.log("6000원");  
    break;  
  default:  
    console.log("메뉴에 없습니다");  
}
```

출력      6000원



## 문제 13

```
const grade13 = "C";
switch (grade13) {
  case "A":
  case "B":
    console.log("합격");
    break;
  case "C":
  case "D":
    console.log("재시험");
    break;
}
```

출력      재시험

case "C": 아래에 코드가 없으니 case "D": 로 흘러내려 같은 코드를 실행합니다.

## 문제 14

```
const age14 = 20;
console.log(age14 >= 19 ? "성인" : "미성년자");
```

출력      성인

## 문제 15

```
const stock15 = 0;
console.log(stock15 === 0 ? "품절" : `재고 ${stock15}개`);
```

출력      품절

**다른 방법** · 변수에 담아서 쓰면 더 읽기 쉽습니다.

```
const stockText = stock15 === 0 ? "품절" : `재고 ${stock15}개`;
console.log(stockText);
```

## 문제 16

```
const price16 = 1200;
const count16 = 5;
const member16 = true;

const rawTotal16 = price16 * count16; // 6000
```

회원이면 10% 할인 = 90% 만 낸다

```
const finalTotal16 = member16 ? Math.round(rawTotal16 * 0.9) : rawTotal16;

console.log(`합계 ${finalTotal16}원`);
```

출력          합계 5400원

이 계산은 마침 딱 떨어지지만, 할인율에 따라서는 5400.000000000001 같은 값이 나옵니다. (02단원 개념01 소수 오차) 돈 계산은 마지막에 `Math.round` 로 다듬는 습관을 들이세요.

## 문제 17

```
const age17 = 25;
const member17 = true;
const total17 = 120000;

if (member17 && total17 >= 100000) {
  console.log("VIP");
} else if (member17) {
  console.log("일반회원");
} else {
  console.log("비회원");
}
```

출력          VIP

순서가 중요합니다. `member17` 만 검사하는 조건을 위에 두면 VIP 인 사람도 “일반회원” 에서 걸려 버립니다. `age17` 은 조건에 쓰이지 않습니다. 주어진 값을 다 써야 하는 건 아닙니다.

## 문제 18

```
const nullMemo = null;  
console.log(nullMemo.length);
```

이 코드는 검사에서 걸립니다

왜:

null 은 "값이 없음" 이라 안에서 꺼낼 게 아무것도 없습니다.

없는 것에서 .length 를 꺼내려 하니 에러가 납니다.

해결:

먼저 값이 있는지 확인하고, 있을 때만 꺼냅니다.

```
if (nullMemo && nullMemo.length > 0) { ... }
```

&& 는 왼쪽이 false 면 오른쪽을 계산조차 하지 않기 때문에 안전합니다.

TypeError: Cannot read properties of null (reading 'length')

# 04단원 연습문제 정답 반복문

RUN node 연습문제\_정답.js

## 문제 1

```
for (let i = 1; i <= 5; i++) {  
  console.log(i);  
}
```

출력

|   |
|---|
| 1 |
| 2 |
| 3 |
| 4 |
| 5 |

1부터 시작하면 조건은 <= 입니다.

## 문제 2

반복 횟수만 필요하고 i 를 안 쓰는 경우입니다. 0부터 < 3 이 자연스럽습니다.

```
for (let i = 0; i < 3; i++) {  
  console.log("안녕하세요");  
}
```

출력

|       |
|-------|
| 안녕하세요 |
| 안녕하세요 |
| 안녕하세요 |

## 문제 3

```
for (let i = 5; i >= 1; i--) {  
  console.log(i);  
}
```

```
출력      5
          4
          3
          2
          1
```

감소시킬 때는 조건도 방향을 뒤집어 `>=` 로 씁니다.

## 문제 4

```
let sum4 = 0;
for (let i = 1; i <= 100; i++) {
  sum4 += i;
}
console.log(sum4);
```

```
출력      5050
```

누적 변수를 반복문 안에서 만들면 매 바퀴 0으로 되돌아갑니다.

## 문제 5

```
for (let i = 2; i <= 10; i += 2) {
  console.log(i);
}
```

```
출력      2
          4
          6
          8
          10
```

**다른 방법** · 1씩 돌면서 홀수를 건너뛰어도 됩니다.

```
for (let i = 1; i <= 10; i++) {
  if (i % 2 === 1) continue;
  console.log(i);
}
```

## 문제 6

```
let bar6 = "";
for (let i = 0; i < 10; i++) {
  bar6 += "=";
}
console.log(bar6);
```

출력            =====

글자를 누적할 때는 빈 문자열 "" 로 시작합니다.

## 문제 7

```
let n7 = 1;
while (n7 <= 5) {
  console.log(n7);
  n7++; // 이걸 빠뜨리면 무한 루프입니다
}
```

출력            1  
                 2  
                 3  
                 4  
                 5

## 문제 8

```
let money8 = 10000;
while (money8 >= 2500) {
  money8 -= 2500;
  console.log(money8);
}
```

출력            7500  
                 5000  
                 2500  
                 0

0이 된 뒤에는 0 >= 2500 이 false 라서 멈춥니다.

## 문제 9

```
do {
  console.log("일단 한 번은 실행됩니다");
} while (1 > 10);
```

출력      일단 한 번은 실행됩니다

do...while 은 조건을 나중에 검사하므로 최소 한 번은 실행됩니다. 마지막 while 뒤에 세미콜론이 필요합니다.

## 문제 10

```
for (let i = 1; i <= 100; i++) {
  if (i % 7 === 0) {
    console.log(i);
    break; // 찾았으니 나머지 93번은 볼 필요가 없습니다
  }
}
```

출력      7

## 문제 11

```
for (let i = 1; i <= 10; i++) {
  if (i % 2 === 0) {
    continue; // 짝수는 건너뛰
  }
  console.log(i);
}
```

출력      1  
          3  
          5  
          7  
          9

## 문제 12

```
let sum12 = 0;
for (let i = 1; i <= 20; i++) {
  if (i % 3 !== 0) {
    continue;
  }
  sum12 += i;
}
console.log(sum12);
```

출력          63

3+6+9+12+15+18 = 63

## 문제 13

```
const dan13 = 3;
for (let n = 1; n <= 9; n++) {
  console.log(`${dan13} x ${n} = ${dan13 * n}`);
}
```

출력          3 x 1 = 3  
              3 x 2 = 6  
              3 x 3 = 9  
              3 x 4 = 12  
              3 x 5 = 15  
              3 x 6 = 18  
              3 x 7 = 21  
              3 x 8 = 24  
              3 x 9 = 27

## 문제 14

```
for (let row = 1; row <= 5; row++) {
  let line = ""; // 줄마다 새로 비웁니다 - 바깥 반복문 '안'에서 만들기
  for (let col = 1; col <= row; col++) {
    line += "*";
  }
  console.log(line);
}
```



```
출력      *
          **
          ***
          ****
          *****
```

안쪽 조건이 `col <= row` 라서 줄마다 개수가 달라집니다.

## 문제 15

```
let money15 = 100;
let day15 = 1;

while (money15 <= 10000) {
  money15 *= 2;
  day15++;
}

console.log(`${day15}일째에 ${money15}원`);
```

```
출력      8일째에 12800원
```

100 → 200 → 400 → 800 → 1600 → 3200 → 6400 → 12800 12800 이 되는 순간 조건이 `false` 가 되어 멈춥니다.

## 문제 16

```
for (let row = 1; row <= 4; row++) {
  let line = "";
  for (let col = 1; col <= row; col++) {
    line += col; // 숫자를 문자열에 이어붙이면 글자가 됩니다
  }
  console.log(line);
}
```

```
출력      1
          12
          123
          1234
```

`line` 이 문자열("")로 시작했기 때문에 `+` 가 이어붙이기로 동작합니다. `line` 을 0 으로 시작했다면 숫자 덧셈이 되어 완전히 다른 결과가 나옵니다.

## 문제 17

```
const idx = 0;
for (idx = 0; idx < 3; idx++) { console.log(idx); }
```

이 코드는 검사에서 걸립니다

왜:

for 문의 카운터는 한 바퀴마다 `idx++` 로 값이 바뀝니다.

`idx++` 는 결국 `idx = idx + 1` 이라 '값을 다시 넣는' 동작입니다.

`const` 는 값을 다시 넣을 수 없으므로 에러가 납니다.

그래서 `for (let i = ...)` 처럼 반드시 `let` 을 씁니다.

참고: 06단원에서 배울 `for...of` 는 매 바퀴 새 변수를 만들기 때문에 `const` 를 써도 됩니다. 지금은 "카운터는 `let`" 으로 기억하세요.

`TypeError: Assignment to constant variable.`

## 05단원 연습문제 정답 함수

RUN node 연습문제\_정답.js

### 문제 1

```
function printWelcome() {  
  console.log("환영합니다!");  
}  
printWelcome();
```

출력      환영합니다!

만들기만 하면 아무 일도 안 일어납니다. 괄호를 붙여 호출해야 실행됩니다.

### 문제 2

```
function printStar() {  
  console.log("*****");  
}  
  
function printTitle() {  
  console.log("MENU");  
}  
  
function printHeader() {  
  printStar();  
  printTitle();  
  printStar();  
}  
  
printHeader();
```

출력      \*\*\*\*\*  
          MENU  
          \*\*\*\*\*

작은 함수를 조립해 큰 함수를 만드는 것이 프로그램을 짜는 기본 방식입니다.

## 문제 3

```
function callName(name) {  
  console.log(`${name}님, 안녕하세요`);  
}  
callName("이서연");
```

출력      이서연님, 안녕하세요

## 문제 4

```
function printArea(width, height) {  
  console.log(width * height);  
}  
printArea(8, 5);
```

출력      40

## 문제 5

```
function order(menu, count = 1) {  
  console.log(`${menu} ${count}개`);  
}  
order("라떼");
```

출력      라떼 1개

```
order("케이크", 3);
```

출력      케이크 3개

기본값이 있는 매개변수는 반드시 뒤쪽에 둡니다.

## 문제 6

```
function getSum(a, b) {  
  return a + b;  
}  
console.log(getSum(3, 7));
```

출력      10

함수 안에서 `console.log` 를 하면 값을 다시 쓸 수 없습니다. `return` 으로 돌려줘야 계산에 재사용할 수 있습니다.

## 문제 7

```
function getTotal(price, count) {
  return price * count;
}
```

```
const total7 = getTotal(4500, 4);
console.log(`합계 ${total7}원`);
```

출력      합계 18000원

05

## 문제 8

```
function divide(a, b) {
  if (b === 0) {
    return "0으로 나눌 수 없습니다"; // 여기서 함수가 끝납니다
  }
  return a / b;
}
```

```
console.log(divide(10, 0));
```

출력      0으로 나눌 수 없습니다

```
console.log(divide(10, 2));
```

출력      5

걸러낼 조건을 위에서 먼저 `return` 으로 끝내는 것을 '조기 반환'이라고 합니다. `else` 가 필요 없어서 들여쓰기가 얹아집니다.

## 문제 9

```
function isPass(score) {  
  return score >= 60;  
}  
console.log(isPass(45));
```

출력        false

```
console.log(isPass(80));
```

출력        true

이렇게 쓰지 마세요. 불필요합니다.

```
if (score >= 60) { return true; } else { return false; }
```

score >= 60 자체가 이미 true / false 입니다.

## 문제 10

```
const minus = function (a, b) {  
  return a - b;  
};  
console.log(minus(10, 3));
```

출력        7

함수 표현식은 '값을 변수에 넣는 문장'이라 끝에 세미콜론을 붙입니다.

## 문제 11

```
function calculate(a, b, operation) {  
  return operation(a, b);  
}  
  
function multiply(a, b) {  
  return a * b;  
}  
  
console.log(calculate(6, 7, multiply));
```

출력 42

multiply 라고만 씁니다. `multiply()` 라고 쓰면 실행 결과(NaN)가 넘어갑니다.

## 문제 12

```
const arrowDouble = (n) => {
  return n * 2;
};
console.log(arrowDouble(7));
```

출력 14

줄여 쓰면 · `const arrowDouble = (n) => n * 2;`

## 문제 13

```
const square = (n) => n * n;
console.log(square(9));
```

출력 81

중괄호를 없애면 “이 식의 결과를 돌려준다”는 뜻이 됩니다. 중괄호와 `return` 은 반드시 같이 지워야 합니다.

## 문제 14

```
const getDiscounted = (price, rate) => {
  const discount = price * rate;
  return price - discount;
};
console.log(getDiscounted(10000, 0.3));
```

출력 7000

중괄호를 쓰면 `return` 을 반드시 써야 합니다. 안 쓰면 `undefined` 가 나옵니다.

## 문제 15

```
const taxRate = 0.1; // 함수 밖 - 전역
function printTax(price) {
  console.log(price * taxRate); // 밖의 변수를 그대로 씁니다
}

printTax(10000);
```

출력          1000

안에서 밖은 보이고, 밖에서 안은 안 보입니다.

## 문제 16

```
function getRawTotal(price, count) {
  return price * count;
}

function applyDiscount(total, isMember) {
  if (!isMember) {
    return total;
  }
  return Math.round(total * 0.9);
}

const raw16 = getRawTotal(1200, 5); // 6000
const final16 = applyDiscount(raw16, true); // 5400 console.log(`합계 ${final16}원`);
```

출력          합계 5400원

함수를 작게 나누면 각각을 따로 확인할 수 있어 버그를 찾기 쉽습니다.



## 문제 17

```
function getSumRange(start, end) {
  let sum = 0;
  for (let i = start; i <= end; i++) {
    sum += i;
  }
  return sum; // 반복이 '끝난 뒤'에 돌려줘야 합니다
}
```

```
console.log(getSumRange(1, 10));
```

```
출력      55
```

```
console.log(getSumRange(5, 10));
```

```
출력      45
```

`return` 을 `for` 문 안에 두면 첫 바퀴에서 함수가 끝나 버립니다.

## 문제 18

```
function makeSecret() {
  const secret = "비밀번호";
}
makeSecret();
console.log(secret);
```

이 코드는 검사에서 걸립니다

왜:

함수 안 중괄호에서 만든 변수는 그 중괄호 안에서만 삽니다.  
함수가 끝나면 사라지기 때문에 밖에서는 존재하지 않습니다.

해결:

`return` 으로 돌려받으면 됩니다.

```
function makeSecret() {
  const secret = "비밀번호";
  return secret;
}
const result = makeSecret();
console.log(result); // 출력: 비밀번호
```

ReferenceError: secret is not defined

## 06단원 연습문제 정답 배열

RUN node 연습문제\_정답.js

### 문제 1

```
const colors1 = ["빨강", "파랑", "노랑"];
console.log(colors1);
```

출력 [ '빨강', '파랑', '노랑' ]

### 문제 2

```
const animals2 = ["강아지", "고양이", "토끼"];
console.log(animals2[1]);
```

출력 고양이

인덱스는 0부터입니다. “두 번째”는 [1] 입니다.

### 문제 3

```
const week3 = ["월", "화", "수", "목", "금"];
console.log(week3.length);
```

출력 5

```
console.log(week3[week3.length - 1]);
```

출력 금

마지막 인덱스는 length - 1 입니다. week3.at(-1) 로도 됩니다.

## 문제 4

```
const menu4 = ["김밥", "라면", "돈까스"];
menu4[1] = "우동";
console.log(menu4);
```

출력      [ '김밥', '우동', '돈까스' ]

const 로 만들어도 배열 '안의 내용'은 바꿀 수 있습니다.

## 문제 5

```
const basket5 = [];
basket5.push("사과");
basket5.push("바나나", "포도");
console.log(basket5);
```

출력      [ '사과', '바나나', '포도' ]

basket5[0] = "사과" 처럼 직접 넣지 말고 push 를 쓰세요.

## 문제 6

```
const stack6 = ["첫째", "둘째", "셋째"];
const last6 = stack6.pop();
console.log(last6);
```

출력      셋째

```
console.log(stack6);
```

출력      [ '첫째', '둘째' ]

pop 은 '빼낸 값'을 돌려줍니다. push 는 '넣은 뒤의 개수'를 돌려줍니다.

## 문제 7

```
const days7 = ["월", "수", "목"];
days7.splice(1, 0, "화");
console.log(days7);
```

```
출력      [ '월', '화', '수', '목' ]
```

두 번째 인자 0 은 "하나도 지우지 말라"는 뜻입니다.

## 문제 8

```
const nums8 = [10, 20, 30, 40, 50];  
console.log(nums8.slice(1, 4));
```

```
출력      [ 20, 30, 40 ]
```

```
console.log(nums8);
```

```
출력      [ 10, 20, 30, 40, 50 ]
```

slice 는 끝 인덱스를 포함하지 않습니다. 1, 2, 3 만 가져옵니다. 원본은 그대로입니다. splice 였다면 원본이 잘려 나갔을 겁니다.

## 문제 9

```
const a9 = [1, 2];  
const b9 = [3, 4];  
console.log(a9.concat(b9));
```

```
출력      [ 1, 2, 3, 4 ]
```

a9 + b9 로 쓰면 "1,23,4" 라는 문자열이 나옵니다.

## 문제 10

```
const fruits10 = ["사과", "바나나", "포도"];  
console.log(fruits10.join(", "));
```

```
출력      사과, 바나나, 포도
```

join 의 결과는 문자열입니다. 따옴표와 대괄호가 사라집니다.

## 문제 11

```
const text11 = "월-화-수";
console.log(text11.split("-"));
```

출력            [ '월', '화', '수' ]

split 은 문자열의 메소드입니다. 배열 → 문자열은 join, 문자열 → 배열은 split.

## 문제 12

```
const menu12 = ["아메리카노", "라떼", "케이크"];
console.log(menu12.includes("라떼"));
```

출력            true

```
console.log(menu12.includes("녹차"));
```

출력            false

## 문제 13

```
const list13 = ["사과", "바나나", "포도"];
console.log(list13.indexOf("포도"));
```

출력            2

```
console.log(list13.indexOf("귤"));
```

출력            -1

없으면 0이 아니라 -1 입니다. 0은 '첫 번째'라는 진짜 위치니까요.

## 문제 14

```
const scores14 = [90, 85, 70];
for (const score of scores14) {
  console.log(score);
}
```

```
출력      90
           85
           70
```

**다른 방법** · 인덱스가 필요하면 일반 for 문을 씁니다.

```
for (let i = 0; i < scores14.length; i++) {
  console.log(scores14[i]);
}
```

## 문제 15

```
const prices15 = [1200, 4500, 3000];

let sum15 = 0; // 누적 변수는 반복문 밖에서 0으로 시작
for (const price of prices15) {
  sum15 += price;
}

console.log(sum15);
```

```
출력      8700
```

```
console.log(sum15 / prices15.length);
```

```
출력      2900
```

## 문제 16

```
const ages16 = [15, 22, 34, 17, 40];

const adults16 = [];
for (const age of ages16) {
  if (age >= 19) {
    adults16.push(age);
  }
}

console.log(adults16);
```

```
출력      [ 22, 34, 40 ]
```

08단원에서 배울 filter 를 쓰면 한 줄이 됩니다.

```
console.log(ages16.filter((age) => age >= 19));
```

## 문제 17

```
const cart17 = ["아메리카노", "케이크", "쿠키"];
const prices17 = [4500, 6000, 3000];
```

두 배열을 같은 인덱스로 짝지어야 하므로 for...of 가 아니라 for 문을 씁니다.

```
let total17 = 0;
for (let i = 0; i < cart17.length; i++) {
  console.log(`${i + 1}. ${cart17[i]} ${prices17[i]}원`);
  total17 += prices17[i];
}
```

```
출력      1. 아메리카노 4500원
          2. 케이크 6000원
          3. 쿠키 3000원
```

```
console.log(`합계 ${total17}원`);
```

```
출력      합계 13500원
```

## 문제 18

```
const temps18 = [23, 31, 18, 27];
```

최댓값은 반드시 배열의 첫 번째 값으로 시작합니다. 0으로 시작하면 음수에서 틀립니다.

```
let maxTemp = temps18[0];
let maxIndex = 0;

for (let i = 0; i < temps18.length; i++) {
  if (temps18[i] > maxTemp) {
    maxTemp = temps18[i];
    maxIndex = i;
  }
}

console.log(`최고 기온 ${maxTemp}도 (${maxIndex + 1}번째)`);
```

```
출력      최고 기온 31도 (2번째)
```

인덱스는 0부터이므로 “몇 번째”로 말하려면 +1 을 합니다.

## 문제 19

```
const notArray = "안녕";  
notArray.push("!");
```

이 코드는 검사에서 걸립니다

왜:

push 는 배열이 가진 메소드입니다. 문자열에는 없습니다.  
문자열은 한 번 만들어지면 내용을 바꿀 수 없기 때문입니다.  
글자를 덧붙이려면 새 문자열을 만들어야 합니다.

```
let text = "안녕";  
text += "!";           // 출력: 안녕!
```

참고: 문자열과 배열은 length, includes, indexOf 처럼 겹치는 것도 있어서  
헷갈리기 쉽습니다. push / pop / splice 는 배열에만 있습니다.

`TypeError: notArray.push is not a function`



## 07단원 연습문제 정답 객체와 메소드

RUN node 연습문제\_정답.js

### 문제 1

```
const user1 = { name: "이서연", age: 22 };  
console.log(user1);
```

출력 { name: '이서연', age: 22 }

### 문제 2

```
const book2 = { title: "자바스크립트 입문", price: 25000, author: "김작가" };  
console.log(book2.price);
```

출력 25000

### 문제 3

```
const key3 = "author";  
console.log(book2[key3]);
```

출력 김작가

book2.key3 이라고 쓰면 "key3" 라는 이름의 속성을 찾아 `undefined` 가 됩니다. 이름이 변수에 들어 있을 때는 대괄호 표기법을 씁니다.

### 문제 4

```
const product4 = {  
  name: "무선 이어폰",  
  seller: { name: "봄날전자", rating: 4.8 },  
};  
console.log(product4.seller.name);
```

출력 봄날전자

객체 안의 객체는 점을 이어 붙입니다.

## 문제 5

```
const empty5 = {};  
console.log(empty5.seller?.name ?? "판매자 미정");
```

출력      판매자 미정

?. 은 앞이 없으면 거기서 멈추고 `undefined` 를 냅니다. 에러가 나지 않습니다. ?? 는 왼쪽이 `null` 이나 `undefined` 일 때만 오른쪽 값을 씁니다.

## 문제 6

```
const car6 = {};  
car6.brand = "현대";  
car6.year = 2024;  
console.log(car6);
```

출력      { brand: '현대', year: 2024 }

없는 이름에 값을 넣으면 새 속성이 생깁니다.

## 문제 7

```
const item7 = { name: "케이크", stock: 3 };  
item7.stock = 0;  
console.log(item7);
```

출력      { name: '케이크', stock: 0 }

추가와 수정의 문법이 같습니다. 이름을 오타 내면 새 속성이 생기니 주의하세요.

## 문제 8

```
const sale8 = { name: "이어폰", price: 89000, oldPrice: 120000 };  
delete sale8.oldPrice;  
console.log(sale8);
```

출력      { name: '이어폰', price: 89000 }

배열에는 delete 를 쓰면 안 되지만 객체에는 써도 됩니다.

## 문제 9

```
const config9 = { theme: "dark", fontSize: 14 };
console.log("theme" in config9);
```

출력          true

```
console.log("language" in config9);
```

출력          false

in 의 왼쪽은 반드시 따옴표로 감싼 문자열입니다.

## 문제 10

```
const origin10 = { name: "박지훈", age: 30 };
```

```
const copy10 = origin10; ← 이걸 복사가 아닙니다. 원본까지 바꿉니다.
```

```
const copy10 = { ...origin10 };

copy10.age = 99;

console.log(copy10);
```

출력          { name: '박지훈', age: 99 }

```
console.log(origin10);
```

출력          { name: '박지훈', age: 30 }

## 문제 11

```
const person11 = {
  name: "최유진",
  introduce() {
    console.log(`안녕하세요, ${this.name}입니다`);
  },
};

person11.introduce();
```

출력      안녕하세요, 최유진입니다

화살표 함수로 만들면 this 가 person11 을 가리키지 않아 undefined 가 나옵니다.

## 문제 12

```
const counter12 = {
  count: 0,
  increase() {
    this.count++;
  },
};

counter12.increase();
counter12.increase();
counter12.increase();

console.log(counter12.count);
```

출력      3

## 문제 13

```
const user13 = { name: "정하늘", age: 28, city: "대구" };

for (const key in user13) {
  console.log(`${key}: ${user13[key]}`);
}
```

```
출력      name: 정하늘
          age: 28
          city: 대구
```

for...in 은 '이름'을 줍니다. 값은 user13[key] 로 꺼냅니다. user13.key 라고 쓰면 undefined 가 세 번 나옵니다.

## 문제 14

```
const scores14 = { 국어: 90, 영어: 85, 수학: 70, 과학: 95 };
console.log(Object.keys(scores14).length);
```

```
출력      4
```

객체에는 .length 가 없습니다. Object.keys 로 배열을 만든 뒤 세야 합니다.

## 문제 15

```
let sum15 = 0;
for (const score of Object.values(scores14)) {
  sum15 += score;
}
console.log(sum15);
```

```
출력      340
```

## 문제 16

```
const products16 = [
  { name: "아메리카노", price: 4000, stock: 10 },
  { name: "라떼", price: 4500, stock: 0 },
  { name: "케이크", price: 6000, stock: 3 },
];

console.log(products16[1].name);
```

```
출력      라떼
```

```
console.log(products16[2].price);
```

```
출력      6000
```

배열[인덱스].속성 순서로 꺼냅니다. products16 은 배열이므로 products16.name 이라고 쓰면 undefined 입니다.

## 문제 17

```
for (const product of products16) {  
  console.log(`${product.name} ${product.price}원`);  
}
```

출력            아메리카노 4000원  
                 라떼 4500원  
                 케이크 6000원

재고가 0인 상품 찾기

```
let soldOut17 = null;  
for (const product of products16) {  
  if (product.stock === 0) {  
    soldOut17 = product;  
    break; // 찾았으니 더 볼 필요가 없습니다  
  }  
}  
  
console.log(`품절: ${soldOut17.name}`);
```

출력            품절: 라떼

못 찾았을 수도 있으므로 null 로 시작했습니다. 실제로는 쓰기 전에 확인하는 습관을 들이세요.

```
console.log(soldOut17 ? `품절: ${soldOut17.name}` : "품절 상품 없음");
```

08단원에서 배운 find 를 쓰면 이 반복문이 한 줄이 됩니다.

## 문제 18

```
const cart16 = [
  { name: "아메리카노", price: 4000, count: 2 },
  { name: "케이크", price: 6000, count: 1 },
  { name: "쿠키", price: 3000, count: 3 },
];

let total16 = 0;
for (const item of cart16) {
  const sub = item.price * item.count;
  console.log(`${item.name} x${item.count} = ${sub}원`);
  total16 += sub;
}
```

```
출력      아메리카노 x2 = 8000원
          케이크 x1 = 6000원
          쿠키 x3 = 9000원
```

```
console.log(`합계 ${total16}원`);
```

```
출력      합계 23000원
```

## 문제 19

```
const students17 = [
  { name: "김민준", scores: [90, 85, 70] },
  { name: "이서연", scores: [95, 88, 92] },
  { name: "박지훈", scores: [60, 75, 80] },
];

let topName = "";
let topAverage = 0;

for (const student of students17) {
```

학생 한 명의 평균 구하기

```
let sum = 0;
for (const score of student.scores) {
  sum += score;
}
const average = sum / student.scores.length;
```

지금까지 최고보다 높으면 갈아치우기

```
if (average > topAverage) {
  topAverage = average;
  topName = student.name;
}
}

console.log(`${topName} ${topAverage.toFixed(1)}`);
```

출력 이서연 91.7

중첩 반복문입니다. 바깥은 학생, 안쪽은 그 학생의 점수입니다. 평균을 비교할 때는 toFixed 로 다듬기 '전'의 숫자로 비교해야 합니다. toFixed 의 결과는 문자열이라 크기 비교가 사전 순이 되어 버립니다.

## 문제 20

```
const empty18 = {};
console.log(empty18.seller.name);
```

이 코드는 검사에서 걸립니다

왜:

empty18.seller 는 undefined 입니다. 여기까지는 에러가 안 납니다.  
그런데 undefined 에서 다시 .name 을 꺼내려 하면 에러가 납니다.  
"없는 것 안에서 무언가를 꺼낼 수는 없다" 는 뜻입니다.

해결:

```
console.log(empty18.seller?.name); // 출력: undefined
console.log(empty18.seller?.name ?? "없음"); // 출력: 없음
또는 03단원에서 배운 && 로 먼저 확인해도 됩니다.
if (empty18.seller && empty18.seller.name) { ... }
```

TypeError: Cannot read properties of undefined (reading 'name')



## 08단원 연습문제 정답 콜백과 내장메소드

RUN node 연습문제\_정답.js

```
function runTwice(callback) {
  callback();
  callback();
}
```

### 문제 1

```
function printAttendance() {
  console.log("출석");
}
runTwice(printAttendance);
```

|    |    |
|----|----|
| 출력 | 출석 |
|    | 출석 |

runTwice(printAttendance()) 라고 쓰면 "출석" 이 한 번만 찍히고 그 결과인 `undefined` 가 넘어가  
TypeError 가 납니다.

### 문제 2

```
runTwice(() => console.log("화이팅"));
```

|    |     |
|----|-----|
| 출력 | 화이팅 |
|    | 화이팅 |

한 번만 쓸 함수는 이름을 붙이지 않고 그 자리에서 만듭니다.

### 문제 3

```
const nums3 = [10, 20, 30];
nums3.forEach((n) => console.log(n));
```

|    |    |
|----|----|
| 출력 | 10 |
|    | 20 |
|    | 30 |

## 문제 4

```
const subjects4 = ["국어", "영어", "수학"];
subjects4.forEach((subject, index) => {
  console.log(`${index + 1}. ${subject}`);
});
```

|    |       |
|----|-------|
| 출력 | 1. 국어 |
|    | 2. 영어 |
|    | 3. 수학 |

콜백에 넘어오는 순서는 (값, 인덱스) 로 고정입니다.

## 문제 5

```
const menu5 = [
  { name: "아메리카노", price: 4000 },
  { name: "케이크", price: 6000 },
];

menu5.forEach((item) => {
  console.log(`${item.name} ${item.price}원`);
});
```

|    |             |
|----|-------------|
| 출력 | 아메리카노 4000원 |
|    | 케이크 6000원   |

## 문제 6

```
const nums6 = [1, 2, 3];
console.log(nums6.map((n) => n * 2));
```

|    |             |
|----|-------------|
| 출력 | [ 2, 4, 6 ] |
|----|-------------|

forEach 를 쓰면 `undefined` 가 나옵니다. 새 배열이 필요하다면 map 입니다.

## 문제 7

```
const scores7 = [90, 85, 70];
console.log(scores7.map((s) => `${s}점`));
```

```
출력      [ '90점', '85점', '70점' ]
```

map 은 자료형을 바꿔도 됩니다. 숫자 배열이 문자열 배열이 되었습니다.

## 문제 8

```
const users8 = [
  { name: "김민준", age: 20 },
  { name: "이서연", age: 22 },
  { name: "박지훈", age: 28 },
];

console.log(users8.map((user) => user.name));
```

```
출력      [ '김민준', '이서연', '박지훈' ]
```

## 문제 9

```
const scores9 = [90, 55, 78, 40, 88];
console.log(scores9.filter((score) => score >= 60));
```

```
출력      [ 90, 78, 88 ]
```

filter 의 콜백은 조건식(true/false)을 돌려줘야 합니다.

## 문제 10

```
const products10 = [
  { name: "아메리카노", price: 4000, stock: 10 },
  { name: "라떼", price: 4500, stock: 0 },
  { name: "케이크", price: 6000, stock: 3 },
  { name: "쿠키", price: 3000, stock: 0 },
];

console.log(products10.filter((p) => p.stock === 0).map((p) => p.name));
```

출력 [ '라떼', '쿠키' ]

filter 로 고르고 map 으로 모양을 바꿉니다. 이렇게 이어 쓰는 것이 체이닝입니다.

## 문제 11

```
console.log(products10.find((p) => p.name === "케이크").price);
```

출력 6000

filter 를 쓰면 배열이 나와서 [0].price 라고 써야 합니다. 하나만 찾을 거면 처음부터 find 를 쓰세요.

없을 수도 있는 값이라면 ?. 를 붙이는 게 안전합니다. 아래 주석을 풀면 "없음" 이 찍힙니다. 확인했으면 다시 주석 처리하세요.

```
console.log(products10.find((p) => p.name === "녹차").price ?? "없음");
```

## 문제 12

```
console.log(products10.findIndex((p) => p.stock === 0));
```

출력 1

못 찾으면 -1 입니다.

## 문제 13

```
console.log(products10.some((p) => p.stock === 0));
```

출력 true

```
console.log(products10.every((p) => p.price >= 5000));
```

출력 false

some 은 "하나라도", every 는 "전부" 입니다.

## 문제 14

```
const nums14 = [10, 20, 30, 40];
console.log(nums14.reduce((acc, n) => acc + n, 0));
```

출력          100

마지막 0 이 시작값입니다. 빠뜨리면 빈 배열에서 에러가 납니다.

## 문제 15

```
const cart15 = [
  { name: "아메리카노", price: 4000, count: 2 },
  { name: "케이크", price: 6000, count: 1 },
  { name: "쿠키", price: 3000, count: 3 },
];

console.log(cart15.reduce((acc, item) => acc + item.price * item.count, 0));
```

출력          23000

다른 방법 · map 으로 금액을 만든 뒤 더해도 됩니다.

```
console.log(cart15.map((i) => i.price * i.count).reduce((a, b) => a + b, 0));
```

## 문제 16

```
const nums16 = [10, 9, 100, 1];
console.log(nums16.sort((a, b) => a - b));
```

출력          [ 1, 9, 10, 100 ]

그냥 sort() 를 쓰면 [ 1, 10, 100, 9 ] 가 됩니다. 사전 순으로 비교하기 때문입니다.

## 문제 17

원본을 지켜야 하므로 slice() 로 복사한 뒤 정렬합니다. (06단원 개념03)

```
console.log(products10.slice().sort((a, b) => a.price - b.price).map((p) => p.name));
```

출력 [ '쿠키', '아메리카노', '라떼', '케이크' ]

```
console.log(products10.map((p) => p.name));
```

출력 [ '아메리카노', '라떼', '케이크', '쿠키' ]

원본은 그대로입니다. 복사 없이 정렬했다면 원본 순서가 망가졌을 겁니다.

## 문제 18

```
const assets18 = products10
  .filter((p) => p.stock > 0)
  .reduce((acc, p) => acc + p.price * p.stock, 0);

console.log(`재고 자산 ${assets18}원`);
```

출력 재고 자산 58000원

아메리카노  $4000 \times 10 = 40000$  케이크  $6000 \times 3 = 18000$  라떼·쿠키는 재고가 0이라 filter 에서 빠졌습니다.

## 문제 19

```
const votes19 = ["짜장", "짬뽕", "짜장", "볶음밥", "짜장", "짬뽕"];

const counts19 = votes19.reduce((acc, item) => {
  acc[item] = (acc[item] ?? 0) + 1;
  return acc; // 이 return 을 빠뜨리면 다음 바퀴의 acc 가 undefined 가 됩니다
}, {});

console.log(counts19);
```

출력 { '짜장': 3, '짬뽕': 2, '볶음밥': 1 }

**다른 방법** · 반복문으로 쓰면 이해하기 쉽습니다.

```
const counts = {};
for (const item of votes19) {
  counts[item] = (counts[item] ?? 0) + 1;
}
```

## 문제 20

```
const scoreObject = { 국어: 90, 영어: 85 };
scoreObject.forEach((v) => console.log(v));
```

이 코드는 검사에서 걸립니다

왜:

forEach 는 '배열'이 가진 메소드입니다. 객체에는 없습니다.

객체는 순서가 아니라 이름으로 되어 있어서 "하나씩 순서대로" 라는 개념이  
다릅니다.

해결:

07단원에서 배운 방법을 씁니다.

```
for (const key in scoreObject) { ... }
```

```
Object.keys(scoreObject).forEach((key) => { ... });
```

```
Object.values(scoreObject).forEach((value) => { ... });
```

```
Object.entries(scoreObject).forEach(([key, value]) => { ... });
```

(([key, value]) 처럼 괄호 안에서 바로 쪼개는 것은 09단원에서 배웁니다)

Object.keys / values / entries 는 '배열'을 돌려주므로

그 뒤에는 forEach, map, filter 를 마음껏 이어 쓸 수 있습니다.

`TypeError: scoreObject.forEach is not a function`





부 록 나

# 심화문제 정답

심화문제는 선택 과제입니다. 풀지 않고 넘어가도 다음 단원에 지장이 없습니다.

## 01단원 심화문제 정답 변수와 자료형

RUN node 심화문제\_정답.js

답만 보지 말고 '왜 그런지' 를 꼭 읽으세요. 심화문제는 그게 전부입니다.

### 심화 1

| 코드                                         | 출력  |
|--------------------------------------------|-----|
| <code>console.log(Number(""));</code>      | 0   |
| <code>console.log(Number(" "));</code>     | 0   |
| <code>console.log(Number("12abc"));</code> | NaN |
| <code>console.log(Number(null));</code>    | 0   |

왜 이렇게 되나 Number 는 "이 값 전체가 숫자로 읽히는가" 를 봅니다. 빈 문자열과 공백은 "아무것도 없다" 로 보고 0 으로 칩니다. "12abc" 는 뒤에 글자가 붙어 있어서 전체를 숫자로 못 읽습니다 → **NaN** **null** 은 "비어 있음" 이라 0 입니다.

여기서 진짜 위험한 것 빈 입력칸에서 값을 받아 Number 로 바꾸면 0 이 됩니다. "아무것도 안 넣었다" 와 "0을 넣었다" 가 구분이 안 됩니다. 나중에 입력값을 다룰 때(10단원) 이것 때문에 버그가 납니다.

### 심화 2

| 코드                                           | 출력  |
|----------------------------------------------|-----|
| <code>console.log(parseInt(""));</code>      | NaN |
| <code>console.log(parseInt(" "));</code>     | NaN |
| <code>console.log(parseInt("12abc"));</code> | 12  |
| <code>console.log(parseInt(null));</code>    | NaN |

Number 와 정반대입니다. 네 개 중 네 개가 다 다릅니다.

Number : 전체가 숫자여야 한다. 아니면 **NaN**. 빈 값은 0. parseInt : 앞에서부터 읽히는 만큼만. 하나도 못 읽으면 **NaN**.

그래서 쓰는 곳이 다릅니다.

|        |                                         |
|--------|-----------------------------------------|
| "1200" | → 둘 다 1200. 아무거나 써도 됩니다.                |
| "30px" | → parseInt 를 써야 30 이 나옵니다. (CSS 값 다를 때) |

빈 입력칸 → Number 는 0, parseInt 는 NaN.

|         |                           |
|---------|---------------------------|
| "안 넣었다" | 를 잡아내려면 parseInt 쪽이 낫습니다. |
|---------|---------------------------|

### 심화 3

|                                                                                                                                 |                                     |
|---------------------------------------------------------------------------------------------------------------------------------|-------------------------------------|
| <pre>console.log(   Boolean(0),   Boolean(""),   Boolean(null),   Boolean(undefined),   Boolean(NaN),   Boolean(false) );</pre> |                                     |
| 출력                                                                                                                              | false false false false false false |
| <pre>console.log(Boolean("0"), Boolean(" "), Boolean("false"));</pre>                                                           |                                     |
| 출력                                                                                                                              | true true true                      |

왜 아래 셋이 true 인가 문자열은 '내용' 을 보지 않습니다. '비어 있는가' 만 봅니다. 따옴표 안에 뭐라도 하나 들어 있으면 true 입니다.

"0" → 글자 '0' 이 들어 있음 → true " " → 공백도 글자입니다 → true "false" → 글자 다섯 개 → true  
빈 문자열 "" 만 false 입니다.

이게 왜 중요한가 나중에 화면 입력칸에서 값을 받으면 전부 문자열로 옵니다. (10만원) 사용자가 공백만 눌렀는데 if 가 true 가 되어 통과해 버립니다. 그래서 실무에서는 공백을 지운 뒤에 검사합니다. (10만원의 trim)

## 02단원 심화문제 정답 연산자

RUN node 심화문제\_정답.js

### 심화 1

```
console.log(0.1 + 0.2);
```

출력 0.30000000000000004

코드

▶ 출력

```
console.log(0.1 + 0.2 === 0.3);
```

▶ false

```
console.log(Math.round((0.1 + 0.2) * 100) / 100);
```

▶ 0.3

```
console.log(Math.round((0.1 + 0.2) * 100) / 100 === 0.3);
```

출력 true

왜 이렇게 하나  $0.30000000000000004 \times 100 = 30.000000000000004$  `Math.round` 로 반올림하면 딱 30 (정수)  $30 / 100 = 0.3$  (정수를 나눈 것이라 오차가 안 생깁니다)

핵심은 “소수끼리 직접 비교하지 않는다” 입니다. 정수로 만들어 비교하거나, 아예 처음부터 정수로 다룹니다.

실무에서 돈을 다루는 방법 1200.50 원 대신 120050 (전 단위 정수) 로 저장하고, 화면에 보여 줄 때만 100 으로 나눕니다. 그러면 오차가 아예 안 생깁니다.

### 심화 2

코드

▶ 출력

```
console.log(0 == "");
```

▶ true

```
console.log(0 === "");
```

▶ false

```
console.log(null == undefined);
```

▶ true

```
console.log(null === undefined);
```

▶ false

```
console.log(NaN === NaN);
```

▶ false

1번이 왜 `true` 인가 `==` 는 자료형이 다르면 '한쪽을 바꿔서' 비교합니다. 빈 문자열 `""` 을 숫자로 바꾸면 `0` 입니다. (01단원 심화 1에서 확인했죠) 그래서 `0 == 0` 이 되어 `true` 가 됩니다.

`===` 는 자료형부터 봅니다. `number` 와 `string` 은 애초에 다르니 `false` 입니다.

3번이 `true` 인 것은 언어가 그렇게 정한 것입니다. "값이 없다" 를 뜻하는 둘을 같은 것으로 쳐 줍니다. 이것만은 `==` 가 편해서 실무에서도 가끔 씁니다.

```
if (value == null) → null 이거나 undefined 면
```

5번 — `NaN` 은 자기 자신과도 같지 않습니다 `NaN` 은 "숫자가 아님" 이라는 뜻이지 특정 값이 아닙니다. "계산 실패" 가 두 번 있었다고 그 둘이 같은 실패는 아니죠. 그래서 `NaN` 인지 확인할 때는 `===` 를 쓸 수 없습니다. `Number.isNaN(값)` 을 씁니다.

```
console.log(Number.isNaN(NaN));
```

```
출력      true
```

## 심화 3

```
const count = 0;
const nickname = "";

console.log(`${count} || 100`);
```

```
출력      [100]
```

코드

```
console.log(`${count} ?? 100`);
console.log(`${nickname} || "손님"`);
console.log(`${nickname} ?? "손님"`)`);
```

▶ 출력

▶ [0]

▶ [손님]

▶ []

차이는 딱 하나입니다.

```
|| 왼쪽이 falsy 면 오른쪽  (0, "", null, undefined, NaN, false 전부)
?? 왼쪽이 null/undefined 면 오른쪽  (그 둘만)
```

그래서 `0` 과 빈 문자열에서 갈립니다.

어느 쪽을 써야 하나 장바구니 개수, 좋아요 수, 점수처럼 `0` 이 '진짜 값' 이면 `??` 를 씁니다. `||` 를 쓰면 `0` 인데도 기본값으로 바뀌어 버립니다. 찾기 어려운 버그입니다.

```
const 개수 = 담은개수 || 1;   ← 0개 담았는데 1개로 바뀜 (버그)  
const 개수 = 담은개수 ?? 1;   ← 0개는 0개 그대로 (맞음)
```

반대로 "빈 칸이면 기본 이름" 처럼 빈 문자열도 걸러야 하면 || 가 맞습니다.

# 03단원 심화문제 정답 조건문

```
RUN node 심화문제_정답.js
```

## 심화 1

```
const score1 = 95;
```

이런 실수를 합니다  
그대로 두었습니다. 왜 D가 나오는지 보세요.

```
if (score1 >= 60) {  
  console.log("틀린 결과: D");  
} else if (score1 >= 70) {  
  console.log("틀린 결과: C");  
} else if (score1 >= 80) {  
  console.log("틀린 결과: B");  
} else if (score1 >= 90) {  
  console.log("틀린 결과: A");  
}
```

출력          틀린 결과: D

왜 D 인가 else if 는 '위에서부터 순서대로' 확인하고, 맞는 것을 찾으면 거기서 끝냅니다. 95 는 첫 줄의 60 이상 조건에 이미 맞습니다. 그래서 아래 줄들은 아예 확인조차 하지 않습니다.  
이게 else if 의 핵심 성질입니다. "먼저 쓴 것이 이깁니다."

고친 코드 · 좁은 조건(큰 수)부터 위에 씁니다.

```
if (score1 >= 90) {  
  console.log("올바른 결과: A");  
} else if (score1 >= 80) {  
  console.log("올바른 결과: B");  
} else if (score1 >= 70) {  
  console.log("올바른 결과: C");  
} else if (score1 >= 60) {  
  console.log("올바른 결과: D");  
}
```

출력          올바른 결과: A

규칙 하나만 기억하세요. 범위를 나눌 때는 한 방향으로 정렬합니다. 90 → 80 → 70 → 60. 거꾸로 60 → 70 → 80 → 90 도 안 됩니다. 위 코드가 그 예입니다.

## 심화 2

```
const month1 = 1;
const month2 = 7;
const month3 = 10;

switch (month1) {
  case 12:
  case 1:
  case 2:
    console.log("겨울");
    break;
  case 3:
  case 4:
  case 5:
    console.log("봄");
    break;
  case 6:
  case 7:
  case 8:
    console.log("여름");
    break;
  default:
    console.log("가을");
}
```

출력      겨울

```
switch (month2) {
  case 12:
  case 1:
  case 2:
    console.log("겨울");
    break;
  case 3:
  case 4:
  case 5:
    console.log("봄");
    break;
  case 6:
  case 7:
  case 8:
```



## 04단원 심화문제 정답 반복문

RUN node 심화문제\_정답.js

### 심화 1

```
for (let dan = 2; dan <= 4; dan++) {
  let line = `${dan}단:`; // * 빈 문자열이 아니라 "2단:" 으로 시작합니다
  for (let i = 1; i <= 9; i++) {
    line += ` ${dan * i}`; // 앞에 공백을 하나 붙여 이어 갑니다
  }

  console.log(line);
}
```

```
출력      2단: 2 4 6 8 10 12 14 16 18
          3단: 3 6 9 12 15 18 21 24 27
          4단: 4 8 12 16 20 24 28 32 36
```

핵심 두 가지 ① line 을 '바깥' 반복문 안에 만들었습니다.

안쪽에 만들면 곱할 때마다 새로 비워져서 마지막 값만 남습니다.  
바깥보다 더 밖에 만들면 세 단이 한 줄에 다 이어집니다.

② console.log 를 안쪽에 두면 27줄이 나옵니다. 바깥에 두어야 3줄입니다.

"어느 반복문 안에 두느냐" 가 중첩 반복문의 전부입니다.

### 심화 2

```
let result = "";
let count = 0;

for (let i = 1; i <= 100; i++) {
  if (i % 3 !== 0) {
    continue; // 3의 배수가 아니면 건너뛰다
  }
  if (i % 5 !== 0) {
    continue; // 5의 배수가 아니면 건너뛰다
  }
}
```

여기까지 왔다면 둘 다 만족한 것입니다

```

    result += `${i} `;
    count++;
  }

  console.log(result.trim());

```

출력      15 30 45 60 75 90

```

console.log(`총 ${count}개`);

```

출력      총 6개

continue 방식 vs if 방식

```

if (i % 3 === 0 && i % 5 === 0) { ... } ← 이렇게 써도 결과는 같습니다

```

그런데 조건이 다섯 개쯤 되면 && 로 이은 줄이 화면을 넘어갑니다. continue 로 하나씩 걸러내면 조건마다 한 줄씩이라 읽기 쉽고, 나중에 조건을 하나 더 넣을 때도 줄만 추가하면 됩니다.

05단원에서 배울 '조기 반환(return)' 과 똑같은 생각입니다. "아니면 빨리 빠져나간다" 가 코드를 납작하게 만듭니다.

trim() 을 쓴 이유 마지막 숫자 뒤에도 공백이 붙어 있습니다. 눈에는 안 보이지만 있습니다. trim() 은 앞뒤 공백을 지웁니다. 10단원에서 다시 만납니다.

## 심화 3

```

const number = 12345;

let rest = number; // 원본을 그대로 두려고 복사해서 씁니다
let reversed = 0;
let digits = 0;

while (rest > 0) {
  const last = rest % 10; // ① 맨 뒷자리 꺼내기
  reversed = reversed * 10 + last; // 자리를 한 칸 밀고 새 자리를 더한다
  rest = Math.floor(rest / 10); // ② 맨 뒷자리 떼어 버리기
  digits++;
}

console.log(reversed);

```

출력      54321

|                                           |     |
|-------------------------------------------|-----|
| <code>console.log(`\${digits}자리`);</code> |     |
| 출력                                        | 5자리 |

한 바퀴씩 따라가 보면 이렇습니다.

| rest  | last | reversed | 계산            | reversed         |
|-------|------|----------|---------------|------------------|
| 12345 | 5    | 0        | $\ast 10 + 5$ | 5 1234           |
| 1234  | 4    | 5        | $\ast 10 + 4$ | 54 123           |
| 123   | 3    | 54       | $\ast 10 + 3$ | 543 12           |
| 12    | 2    | 543      | $\ast 10 + 2$ | 5432 1           |
| 1     | 1    | 5432     | $\ast 10 + 1$ | 54321            |
| 0     |      |          | -             | rest 가 0 이라 반복 끝 |

`reversed * 10` 이 하는 일 기존 숫자를 왼쪽으로 한 칸 밀어 오른쪽에 빈 자리를 만듭니다. 거기에 새로 꺼낸 자리를 채워 넣는 것입니다.

왜 문자열을 안 쓰나 `String(number).split('').reverse().join('')` 로 하면 한 줄입니다. (06·08단원에서 배웁니다) 그런데 `%` 와 `Math.floor` 로 자릿수를 다루는 방법은 나중에 진법 변환, 자릿수 합 구하기 같은 데서 계속 나옵니다. 반복문으로 숫자를 분해하는 감각을 잡는 것이 이 문제의 목적입니다.

## 05단원 심화문제 정답 함수

RUN node 심화문제\_정답.js

### 심화 1

```
function getSubtotal(price, count) {
  return price * count;
}

function applyDiscount(total, isMember) {
  if (!isMember) {
    return total; // 조기 반환 - 회원이 아니면 여기서 끝
  }
  return Math.round(total * 0.9);
}

function addTax(total) {
  return Math.round(total * 1.1);
}

console.log(getSubtotal(1200, 5));
```

출력 6000

```
console.log(applyDiscount(getSubtotal(1200, 5), true));
```

출력 5400

```
console.log(addTax(applyDiscount(getSubtotal(1200, 5), true)));
```

출력 5940

안쪽부터 바깥쪽으로 읽습니다.

```
getSubtotal(1200, 5)      → 6000
applyDiscount(6000, true) → 5400
addTax(5400)             → 5940
```

왜 이렇게 나누나 함수 하나가 한 가지 일만 하면, 그중 하나가 바뀌어도 거기만 고치면 됩니다. 부가세율이 바뀌면 addTax 만 고칩니다. 나머지는 건드릴 필요가 없습니다.

다만 너무 깊게 겹치면 읽기 어렵습니다. 실무에서는 이렇게 나눠 쓰기도 합니다.

```
const subtotal = getSubtotal(1200, 5);
const discounted = applyDiscount(subtotal, true);
const final = addTax(discounted);
```

중간 이름이 설명 역할을 해서 오히려 읽기 좋습니다.

## 심화 2

```
function makeGreeter(greeting) {
  return (name) => `${greeting}, ${name}님`;
}

const hello = makeGreeter("안녕하세요");
const nice = makeGreeter("반갑습니다");

console.log(hello("김민준"));
```

출력      안녕하세요, 김민준님

```
console.log(nice("이서연"));
```

출력      반갑습니다, 이서연님

```
console.log(hello("박지훈"));
```

출력      안녕하세요, 박지훈님

무슨 일이 일어난 건가 makeGreeter("안녕하세요") 를 부르면 계산 결과가 아니라 '함수' 가 나옵니다. 그 함수는 자기가 태어날 때 받았던 greeting 을 계속 기억합니다.

hello 와 nice 는 생김새가 같은 함수지만 기억하는 인사말이 다릅니다. 그래서 hello("박지훈") 은 몇 번을 불러도 "안녕하세요" 를 씁니다.

이게 왜 쓸모 있나 "설정을 미리 넣어 둔 함수" 를 만들 수 있습니다. 08단원에서 콜백을 배우면 이 방식이 자주 나옵니다. React 에서도 이벤트 핸들러를 만들 때 이 모양을 씁니다.

```
const 삭제하기 = (id) => () => remove(id);
```

버튼마다 다른 id 를 미리 넣어 둔 함수를 만들어 주는 것입니다.

이것을 클로저(closure)라고 부릅니다. 이름은 몰라도 됩니다. "안쪽 함수가 바깥 값을 기억한다" 만 기억하세요.

## 심화 3

```
let message = "바깥";

function test() {
  console.log(message);
  let message2 = "안쪽";
  console.log(message2);
}

test();
```

|    |    |
|----|----|
| 출력 | 바깥 |
|    | 안쪽 |

```
console.log(message);
```

|    |    |
|----|----|
| 출력 | 바깥 |
|----|----|

```
function test2() {
  let message = "안에서 새로 만든 것";
  console.log(message);
}

test2();
```

|    |             |
|----|-------------|
| 출력 | 안에서 새로 만든 것 |
|----|-------------|

```
console.log(message);
```

|    |    |
|----|----|
| 출력 | 바깥 |
|----|----|

왜 바깥 message 가 안 바뀌었나 test2 안의 `let message` 는 '바깥 것을 고친 것' 이 아니라 '같은 이름의 새 변수를 안쪽에 하나 더 만든 것' 입니다. 함수가 끝나면 안쪽 것은 사라지고, 바깥 것은 처음부터 손댄 적이 없습니다.

안에서 밖은 보이지만(test 에서 message 를 읽었죠), 안에서 새로 만들면 그건 별개입니다. 이것을 '가려짐 (shadowing)' 이라고 합니다.

진짜 바깥 것을 바꾸고 싶었다면 `let` 을 빼야 합니다.

```
function test3() {
```

```
message = "진짜로 바꿈"; ← let 없이 쓰면 바꿀 것을 고칩니다
```

```
}
```

다만 이렇게 바꿀 값을 함수가 몰래 바꾸면 나중에 “누가 이 값을 바꿨지?” 를 찾기 어려워집니다. 필요한 값은 매개변수로 받고, 결과는 `return` 으로 돌려주는 편이 안전합니다.

## 06단원 심화문제 정답 배열

RUN node 심화문제\_정답.js

### 심화 1

```
const numbers = [3, 1, 4, 1, 5, 9, 2, 5, 3];

const unique = [];
for (const n of numbers) {
  if (!unique.includes(n)) {
    unique.push(n);
  }
}

console.log(unique);
```

출력 [ 3, 1, 4, 5, 9, 2 ]

```
console.log(`원본 ${numbers.length}개 → 중복 제거 ${unique.length}개`);
```

출력 원본 9개 → 중복 제거 6개

(node 는 배열 항목이 7개가 넘으면 여러 줄로 나눠 찍습니다. 6개라 한 줄입니다)

왜 이 방법이 순서를 지키나 원본을 앞에서부터 훑으면서 처음 만난 것만 넣기 때문입니다. 3을 넣고, 1을 넣고, 4를 넣고... 두 번째 1은 이미 있으니 건너뛸니다.

원본이 안 바뀐 이유 unique 라는 '새 배열' 에만 push 했습니다. numbers 는 읽기만 했습니다. 원본을 지키려면 "새 그릇에 담는다" 를 기억하세요.

**참고** · 09단원에서 Set 을 쓰면 한 줄입니다.

하지만 includes 로 하나씩 확인하는 이 방식이  
"이미 있는지 확인하고 넣는다" 라는 생각을 잡아 줍니다.

### 심화 2

```
const a = ["사과", "바나나", "포도", "딸기"];
const b = ["바나나", "딸기", "수박"];
```



```
const both = [];
const onlyA = [];
const onlyB = [];

for (const fruit of a) {
  if (b.includes(fruit)) {
    both.push(fruit); // a 에 있는데 b 에도 있다 → 교집합
  } else {
    onlyA.push(fruit); // a 에 있는데 b 에는 없다
  }
}

for (const fruit of b) {
  if (!a.includes(fruit)) {
    onlyB.push(fruit); // b 에 있는데 a 에는 없다
  }
}

console.log("둘 다:", both);
```

출력      둘 다: [ '바나나', '딸기' ]

```
console.log("a에만:", onlyA);
```

출력      a에만: [ '사과', '포도' ]

```
console.log("b에만:", onlyB);
```

출력      b에만: [ '수박' ]

반복문을 두 번 도는 이유 첫 번째 반복은 a 기준입니다. a 의 각 항목이 b 에 있는지만 봅니다. 그러면 b 에만 있는 것(수박)은 아예 만나지도 못합니다. 그래서 b 를 한 번 더 돌아야 합니다.

교집합은 어느 쪽 기준으로 만들어도 같습니다. 그래서 첫 번째에서만 만듭니다.

실무에서 어디에 쓰나 “장바구니에 이미 담긴 상품” / “새로 추가된 항목” / “삭제된 항목” 을 찾을 때 정확히 이 형태를 씁니다.

### 심화 3

```
const items = [1, 2, 3, 4, 5];
```

## 방법 1 · slice 로 잘라 이어 붙이기

```
const rotated = items.slice(-2).concat(items.slice(0, -2));  
  
console.log(rotated);
```

출력            [ 4, 5, 1, 2, 3 ]

```
console.log("원본:", items);
```

출력            원본: [ 1, 2, 3, 4, 5 ]

slice(-2) 뒤에서 2개 → [4, 5] slice(0, -2) 처음부터 뒤에서 2개 전까지 → [1, 2, 3] concat 으로 이으면  
→ [4, 5, 1, 2, 3]

slice 는 원본을 안 건드립니다. 그래서 items 가 그대로입니다. 만약 splice 를 썼다면 원본이 잘려 나갔을  
겁니다. (개념02의 그 함정)

## 방법 2 · 복사본에서 pop → unshift 를 2번

```
const copy = items.slice(); // 먼저 복사 (원본 보호) for (let i = 0; i < 2; i++) {  
  const last = copy.pop(); // 맨 뒤를 빼서  
  copy.unshift(last); // 맨 앞에 넣는다  
}  
  
console.log(copy);
```

출력            [ 4, 5, 1, 2, 3 ]

한 바퀴씩 보면

**1,2,3,4,5** · → 5를 빼서 앞에 → [5,1,2,3,4]

**5,1,2,3,4** · → 4를 빼서 앞에 → [4,5,1,2,3]

방법 1이 짧지만, 방법 2는 “한 칸씩 미는 것” 이 눈에 보입니다. 밀 칸 수가 변수로 바뀌어도 for 의 조건만  
고치면 됩니다.

**주의** · copy = items 라고 썼다면 둘이 같은 배열이라 원본도 바뀝니다.

```
slice() 로 복사해야 합니다. 09단원에서 이 이야기를 더 합니다.
```

## 07단원 심화문제 정답 객체와 메소드

RUN node 심화문제\_정답.js

### 심화 1

```
const store = {
  items: {
    아메리카노: 10,
    라떼: 3,
  },

  sell(name, count) {
    if (this.items[name] < count) {
      console.log("재고 부족");
      return; // 조기 반환 - 재고를 건드리지 않고 끝냅니다
    }
    this.items[name] -= count;
  },

  restock(name, count) {
    this.items[name] += count;
  },

  report() {
    let line = "";

    for (const entry of Object.entries(this.items)) {
```

entry[0] 은 이름, entry[1] 은 개수입니다 (개념04 섹션 4)

```
      line += `${entry[0]} ${entry[1]}개 / `;
    }
```

마지막 " / " 를 빼려고 뒤에서 3글자를 잘라냅니다

```
    console.log(line.slice(0, -3));
  },
};

store.sell("아메리카노", 2);
```

출력      재고 부족

```
store.sell("라떼", 5);
store.restock("라떼", 4);
store.report();
```

출력      아메리카노 8개 / 라떼 7개

잠깐 — 출력 순서가 이상하지 않나요? “재고 부족” 은 라떼를 5개 팔려 할 때(재고 3개) 나온 것입니다. 아메리카노 2개는 정상으로 팔려서 10 → 8 이 되었고, 라떼는 팔리지 않은 채 3 → 4개 입고 → 7 이 되었습니다.

this 를 왜 쓰나 store.items 라고 직접 써도 지금은 동작합니다. 그런데 나중에 가게가 두 개가 되어 객체를 복사하면, 복사본의 메소드가 여전히 원본 store 를 보게 됩니다. (개념03 섹션 4의 그 함정) this 는 “나를 부른 그 객체” 라서 복사본에서도 제대로 동작합니다.

slice(0, -3) 은 무엇인가 “처음부터 뒤에서 3번째 전까지” 입니다. 마지막 “ / ” 를 지웁니다. 08단원의 join(“ / ”) 을 쓰면 이런 뒤풀리가 필요 없습니다.

## 심화 2

```
const orders = [
  { menu: "아메리카노", price: 4000, count: 2 },
  { menu: "케이크", price: 6000, count: 1 },
  { menu: "아메리카노", price: 4000, count: 3 },
  { menu: "라떼", price: 4500, count: 2 },
];
```

### ① 총 매출

```
let total = 0;
for (const order of orders) {
  total += order.price * order.count;
}
console.log(`총 매출: ${total}원`);
```

출력      총 매출: 35000원

### ② 가장 비싼 주문 한 건

```
let topOrder = orders[0];
for (const order of orders) {
  if (order.price * order.count > topOrder.price * topOrder.count) {
    topOrder = order;
  }
}
console.log(`가장 비싼 주문: ${topOrder.menu}`);
```

출력      가장 비싼 주문: 아메리카노

## ③ 메뉴별 판매 개수

```
const countByMenu = {};
for (const order of orders) {
  if (countByMenu[order.menu]) {
    countByMenu[order.menu] += order.count; // 이미 있으면 더한다
  } else {
    countByMenu[order.menu] = order.count; // 없으면 새로 넣는다
  }
}
console.log(countByMenu);
```

출력      { '아메리카노': 5, '케이크': 1, '라떼': 2 }

한글 속성 이름은 찍을 때 따옴표가 붙습니다. 영어 이름이면 안 붙습니다.

②에서 왜 orders[0] 으로 시작하나 0 으로 시작하면 “가장 비싼 것” 은 찾을 수 있지만, 그 주문의 menu 를 꺼낼 수가 없습니다. 객체 전체를 들고 있어야 합니다. 첫 항목을 후보로 두고, 더 큰 것을 만나면 바꿔치기하는 것이 정석입니다.

아메리카노 3개 = 12000원이 가장 큼니다. (케이크 6000, 라떼 9000, 아메 2개 8000) 총 매출 = 8000 + 6000 + 12000 + 9000 = 35000원

③의 대괄호 접근이 핵심입니다 countByMenu.order.menu 라고 쓰면 안 됩니다. 그건 “order 라는 속성” 을 찾습니다. countByMenu[order.menu] 라고 써야 “아메리카노 라는 속성” 이 됩니다. 변수에 담긴 이름으로 찾을 때는 반드시 대괄호입니다. (개념01 섹션 4)

08단원에서 reduce 를 배우면 ①과 ③이 각각 한 줄이 됩니다. 지금은 반복문으로 “무슨 일이 일어나는지” 를 보는 게 목적입니다.

## 심화 3

```
const users = [
  { name: "김민준", address: { city: "부산" } },
  { name: "이서연", address: {} },
  { name: "박지훈" },
];

for (const user of users) {
  console.log(`${user.name}: ${user.address?.city || "정보 없음"}`);
}
```

|    |            |
|----|------------|
| 출력 | 김민준: 부산    |
|    | 이서연: 정보 없음 |
|    | 박지훈: 정보 없음 |

?. 가 없으면 어떻게 되나 박지훈에게는 address 가 아예 없습니다. `undefined` 입니다. `user.address.city` 라고 쓰면 → `TypeError: Cannot read properties of undefined (reading 'city')` 여기서 프로그램이 통째로 멈춥니다.

?. 는 "왼쪽이 없으면 더 들어가지 말고 `undefined` 를 내놔라" 입니다. 에러 대신 `undefined` 가 나오니 프로그램이 살아남습니다.

|| 가 하는 일 이서연은 address 는 있는데 city 가 없어서 `undefined` 입니다. 박지훈은 ?. 때문에 `undefined` 입니다. 둘 다 falsy 라서 || 가 "정보 없음" 으로 바꿔 줍니다.

?. 는 "에러를 막는 것", || 는 "빈 값을 채우는 것" 입니다. 역할이 다르니 둘 다 필요합니다.

서버에서 받은 데이터에는 이런 구멍이 항상 있습니다. (12단원) 이 한 줄 패턴을 손에 익혀 두세요.

## 08단원 심화문제 정답 콜백과 내장메소드

RUN node 심화문제\_정답.js

### 심화 1

```
const products = [
  { name: "아메리카노", category: "음료" },
  { name: "라떼", category: "음료" },
  { name: "케이크", category: "디저트" },
  { name: "쿠키", category: "디저트" },
  { name: "샌드위치", category: "식사" },
];

const countByCategory = products.reduce((acc, product) => {
  if (acc[product.category]) {
    acc[product.category] += 1;
  } else {
    acc[product.category] = 1;
  }
  return acc; // * 이걸 빼면 다음 바퀴의 acc 가 undefined 가 됩니다
}, {});

console.log(countByCategory);
```

출력 { '음료': 2, '디저트': 2, '식사': 1 }

한 바퀴씩 따라가 보면

| 바퀴 | product   | acc 시작            | 하는 일        | acc 끝             |
|----|-----------|-------------------|-------------|-------------------|
| 1  | 아메리카노(음료) | {}                | 음료 없음 → 1   | { 음료: 1 }         |
| 2  | 라떼(음료)    | { 음료: 1 }         | 음료 있음 → +1  | { 음료: 2 }         |
| 3  | 케이크(디저트)  | { 음료: 2 }         | 디저트 없음 → 1  | { 음료: 2, 디저트: 1 } |
| 4  | 쿠키(디저트)   | { 음료: 2, 디저트: 1 } | 디저트 있음 → +1 | { 음료: 2, 디저트: 2 } |
| 5  | 샌드위치(식사)  | ...               | 식사 없음 → 1   | { ..., 식사: 1 }    |

reduce 의 시작값이 왜 {} 인가 합계를 구할 때는 0 으로 시작했습니다. 여기서는 '객체를 쌓아 가는' 것이라 빈 객체로 시작합니다. "무엇을 만들 것인가" 가 시작값을 정합니다.

07단원 심화 2와 비교해 보세요 거기서는 for 문 + 빈 객체로 똑같은 일을 했습니다. reduce 는 "빈 객체를 만들고 돌면서 채운다" 를 한 덩어리로 묶은 것뿐입니다. 새로운 마법이 아닙니다. 이걸 알면 reduce 가 훨씬 덜 무섭습니다.

더 짧게 쓰는 법 (실무에서 자주 보입니다)

```
acc[key] = (acc[key] || 0) + 1;
```

“있으면 그 값, 없으면 0” 을 || 로 처리한 것입니다. (02단원)

## 심화 2

```
const students = [
  { name: "김민준", score: 90 },
  { name: "이서연", score: 55 },
  { name: "박지훈", score: 78 },
  { name: "최유진", score: 95 },
];

console.log(
  students
    .filter((s) => s.score >= 60)
    .sort((a, b) => b.score - a.score)
    .map((s) => `${s.name}(${s.score})`)
    .join(", ")
);
```

출력          최유진(95), 김민준(90), 박지훈(78)

```
console.log("원본 첫 번째:", students[0].name);
```

출력          원본 첫 번째: 김민준

순서가 중요합니다 filter 를 먼저 해야 정렬할 것이 줄어듭니다. map 을 먼저 하면 문자열이 되어 버려서 score 로 정렬할 수 없습니다. “거르고 → 줄 세우고 → 모양 바꾸고 → 이어 붙인다” 가 자연스러운 순서입니다.

왜 원본이 안 바뀌었나 sort 는 원본을 바꾸는 메소드입니다. (개념06) 그런데 여기서 sort 가 받은 것은 students 가 아니라 filter 가 방금 만든 ‘새 배열’ 입니다. 그래서 students 는 무사합니다.

만약 students.sort(...) 로 시작했다면 원본이 뒤섞였을 겁니다. 그때는 students.slice().sort(...) 로 복사부터 해야 합니다.

b.score - a.score 인 이유 빼기 순서가 방향입니다. a - b 면 오름차순, b - a 면 내림차순. “높은 순” 이니 b 를 앞에 씁니다.



## 심화 3

```
const items = [
  { name: "케이크", price: 6000, stock: 2 },
  { name: "아메리카노", price: 4000, stock: 5 },
  { name: "라떼", price: 4000, stock: 3 },
  { name: "쿠키", price: 3000, stock: 5 },
];

const sorted = items.slice().sort((a, b) => {
  const priceDiff = a.price - b.price; // 가격 오름차순 if (priceDiff !== 0) {
    return priceDiff; // 가격이 다르면 그것으로 결정
  }

  return b.stock - a.stock; // 가격이 같을 때만 재고 내림차순
});

sorted.forEach((item) => {
  console.log(`${item.name} ${item.price}원 재고${item.stock}개`);
});
```

```
출력      쿠키 3000원 재고5개
          아메리카노 4000원 재고5개
          라떼 4000원 재고3개
          케이크 6000원 재고2개
```

기준이 두 개일 때의 정석입니다 ① 첫 기준으로 비교한다 ② 결과가 0 이 아니면(순서가 정해졌으면) 그걸 그대로 돌려준다 ③ 0 이면(같으면) 두 번째 기준으로 다시 비교해 돌려준다

기준이 세 개여도 같습니다. if 를 하나 더 넣으면 됩니다.

한 줄로 쓰는 법도 있습니다 .sort((a, b) => a.price - b.price || b.stock - a.stock)

앞이 0 이면 falsy 라서 || 가 뒤를 내놓습니다. (02단원 단축 평가) 짧지만 처음 보면 무슨 뜻인지 알기 어렵습니다. 실무 코드에서 이 모양을 보면 “아, 두 번째 기준이구나” 하고 읽으시면 됩니다.

왜 slice() 로 복사했나 sort 는 원본을 바꿉니다. 여기서는 items 를 그대로 두려고 복사했습니다. 심화 2와 달리 앞에 filter 가 없어서, 복사를 안 하면 원본이 뒤섞입니다.